U-Boot (v5) Developer Guide

ID: RK-KF-YF-C65

Release Version: V1.1.0

Release Date: 2026-06-29

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document mainly guides readers on how to carry out project development based on U-Boot (v5).

Supported Platforms

Chipset
RK3576, RK3588, RK3538, RK3572

Intended Audience

This document (this guide) is mainly intended for:

- Technical support engineers
- Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Joseph Chen	2026-04-17	Initial version
V1.1.0	Joseph Chen	2026-06-29	Correct patch information related to FIT standalone signing

Contents

U-Boot (v5) Developer Guide

1. Chapter-1 Basic Introduction
    - 1.1 Feature
    - 1.2 DM
    - 1.3 Security
    - 1.4 Boot-order
    - 1.5 Driver-probe
    - 1.6 Shell
    - 1.7 Boot-Command
    - 1.8 SPL/U-Boot-proper
    - 1.9 Build-Output
    - 1.10 Environment-Variables
    - 1.11 U-Boot DTS
    - 1.12 Relocation
2. Chapter-2 RK Architecture
    - 2.1 Preface
    - 2.2 Platform Files
    - 2.3 Platform Configuration
    - 2.4 Boot Flow
    - 2.5 Memory Layout
    - 2.6 Storage Layout
    - 2.7 Aliases
    - 2.8 AMP
    - 2.9 Atags
    - 2.10 Bootdev
    - 2.11 Bidram/sysmem
    - 2.12 Fuse/OTP
    - 2.13 Hotkey
    - 2.14 Image Decompress
    - 2.15 Image kernel
    - 2.16 Image U-Boot
    - 2.17 Interrupt
    - 2.18 Kernel-DTB
    - 2.19 MMU Cache
    - 2.20 Make.sh
    - 2.21 HW-ID DTB
    - 2.22 Partition Table
    - 2.23 Relocation
    - 2.24 Reset
    - 2.25 Sd/Udisk
    - 2.26 Stacktrace
    - 2.27 TimeCost
    - 2.28 Timestamp
    - 2.29 Vendor storage
3. Chapter-3 Build and Flash
    - 3.1 Preparation
    - 3.2 Firmware Build
    - 3.3 Firmware Flashing
    - 3.4 Firmware Size Modification
4. Chapter-4 System Modules
    - 4.1 AArch32
    - 4.2 ANDROID AB

4.2.1 Configuration
4.2.2 Partition Table
4.3 ANDROID BCB
4.4 AVB Secure Boot
4.4.1 Feature
4.4.2 Configuration
4.4.3 Reference
4.5 Cmdline
4.5.1 Data Sources
4.5.2 Meaning of Data
4.6 DTBO/DTO
4.6.1 Principle Introduction
4.6.2 Enable DTO
4.6.3 DTO Result
4.7 ENV
4.7.1 Framework Support
4.7.2 Related Interfaces
4.7.3 Advanced Interfaces
4.7.4 Storage Location
4.7.5 Common Options
4.7.6 fw_printenv Tool
4.8 Fastboot
4.8.1 Configuration Options
4.8.2 Trigger Methods
4.8.3 Fastboot commands
4.8.3.1 Command Quick Reference
4.8.3.2 Command Usage
4.9 FileSystem
4.9.1 Framework Support
4.9.2 Related Interfaces
4.9.3 Command Examples
4.10 HW-ID DTB
4.10.1 Design Principle
4.10.2 Hardware Reference
4.10.3 DTB Naming
4.10.4 DTB Packaging
4.10.5 Enable Feature
4.10.6 Load Result
4.11 SD and USB Disk
4.11.1 Mechanism Principle
4.11.2 Firmware Creation
4.11.3 SD Configuration
4.11.4 USB Configuration
4.11.5 Feature Validation
4.11.6 Notes
5. Chapter-5 Driver Modules
5.1 AMP
5.1.1 Implementation Idea
5.1.2 Framework Support
5.1.3 Enable Feature
5.2 Button
5.2.1 Framework Support
5.2.2 Related Interfaces
5.3 Charge
5.3.1 Framework Support

5.3.2 Package Images
5.3.3 DTS Configuration
5.3.4 System Suspend
5.3.5 Replace Images
5.3.6 Charging LEDs
5.4 Clock
5.4.1 Framework Support
5.4.2 Related Interfaces
5.4.3 Clock Initialization
5.4.4 CPU Frequency Boost
5.4.5 Clock Tree
5.5 Crypto
5.5.1 Framework Support
5.5.2 Related Interfaces
5.6 Display
5.6.1 Framework Support
5.6.2 Related Interfaces
5.6.3 DTS Configuration
5.6.4 defconfig
5.6.5 LOGO Partition
5.6.6 Common Issue Analysis
5.7 Efuse/Otp
5.7.1 Framework Support
5.7.2 Related Interfaces
5.7.3 DTS Configuration
5.7.4 Usage Examples
5.7.5 Others
5.8 Ethernet
5.8.1 Framework Support
5.8.2 Related Interfaces
5.8.3 DTS Configuration
5.8.4 Usage Examples
5.8.5 Network Troubleshooting
5.9 Gpio
5.9.1 Framework Support
5.9.2 DM Interfaces
5.9.3 Legacy Interfaces
5.10 Interrupt
5.10.1 Framework Support
5.10.2 Related Interfaces
5.11 I2C
5.11.1 Framework Support
5.11.2 Related Interfaces
5.12 Led
5.12.1 Framework Support
5.12.2 Related Interfaces
5.12.3 DTS Node
5.13 Mtd
5.13.1 Framework Support
5.13.2 Related Interfaces
5.13.3 Usage Examples
5.14 Mtd_blk
5.14.1 Framework Support
5.14.2 Related Interfaces
5.15 Optee Client

5.15.1 Framework Support
5.15.2 Firmware Description
5.15.3 Interface Description
5.15.3.1 Applicability
5.15.3.2 Return Values
5.15.3.3 optee_base_write_security_data
5.15.3.4 optee_base_read_security_data
5.15.3.5 optee_read_vbootkey_hash
5.15.3.6 optee_write_vbootkey_hash
5.15.3.7 optee_read_vbootkey_enable_flag
5.15.3.8 optee_read_attribute_hash
5.15.3.9 optee_write_attribute_hash
5.15.3.10 optee_write_oem_otp_key
5.15.3.11 optee_oem_otp_key_is_written
5.15.3.12 optee_set_oem_hr_otp_read_lock
5.15.3.13 optee_oem_otp_key_cipher
5.15.3.14 optee_read_oem_ns_otp
5.15.3.15 optee_write_oem_ns_otp
5.15.3.16 optee_write_ta_encryption_key
5.15.3.17 optee_ta_encryption_key_is_written
5.15.3.18 optee_write_oem_encrypt_data
5.15.3.19 optee_oem_encrypt_data_is_written
5.15.3.20 optee_check_security_level_flag
5.15.3.21 optee_write_oem_huk
5.15.4 Shared Memory
5.15.5 Test Command
5.15.6 Common Error Logs
5.16 PCIe
5.16.1 Development Notes
5.16.2 Framework Support
5.16.3 DTS Configuration
5.16.4 Usage Examples
5.16.4.1 PCIe CMD
5.16.4.2 NVMe
5.16.4.3 RK3588 RC DMA
5.16.4.4 RK3568 RC DMA
5.16.5 Common Issue Analysis
5.17 Pinctrl
5.17.1 Framework Support
5.17.2 Related Interfaces
5.18 Pmic/Regulator
5.18.1 Framework Support
5.18.2 Related Interfaces
5.18.3 Initial Voltage
5.18.4 Skip Initialization
5.19 Reset
5.19.1 Framework Support
5.19.2 Related Interfaces
5.19.3 DTS Configuration
5.20 Rng
5.20.1 Framework Support
5.20.2 Related Interfaces
5.20.3 DTS Configuration
5.21 Spi
5.21.1 Framework Support

5.21.2 Related Interfaces
5.21.3 DTS Configuration
5.21.4 Usage Example
5.21.5 Test Command
5.21.6 Common Issue Analysis
5.22 Storage
5.22.1 Framework Support
5.22.2 Related Interfaces
5.22.3 Type Distinction
5.22.4 DTS Configuration
5.22.5 Dual Storage Extension
5.22.6 Common Issue Analysis
5.23 Thermal
5.23.1 Framework Support
5.23.2 Related Interfaces
5.23.3 DTS Configuration
5.24 Uart
5.24.1 Change Only U-Boot UART
5.24.2 Global Replacement
5.24.3 Disable Console
5.24.4 Related Interfaces
5.25 USB
5.25.1 Framework Support
5.25.2 Board-Level Configuration
5.25.3 DTS Configuration
5.25.4 Related Commands
5.26 Vendor Storage
5.26.1 Principle Overview
5.26.2 Framework Support
5.26.3 Related Interfaces
5.26.4 Self-Test
5.27 Watchdog
5.27.1 Framework Support
5.27.2 Related Interfaces
6. Chapter-6 Advanced Principles
6.1 kernel-DTB
6.1.1 Design Background
6.1.2 Live device tree
6.1.3 Mechanism Implementation
6.1.4 U-Boot
6.2 Kernel Parameters Passed by U-Boot
6.2.1 cmdline
6.2.2 Memory Size
6.2.3 Other Methods
6.3 AB System
6.3.1 AB Data Format
6.3.2 AB Boot Modes
6.3.2.1 successful-boot
6.3.2.2 reset-retry
6.3.2.3 Mode Comparison
6.3.3 Boot Flow
6.3.4 Upgrade and Exceptions
6.3.5 Verification Method
6.3.5.1 successful-boot
6.3.5.2 reset-retry

6.3.6 References
6.4 AVB Secure Boot
6.4.1 References
6.4.2 Terminology
6.4.3 Introduction
6.4.4 Encryption Example
6.4.5 AVB
6.4.5.1 AVB Features
6.4.5.2 Generate key, signature, and certificate
6.4.5.3 AVB lock
6.4.5.4 AVB unlock
6.4.5.5 kernel Configuration
6.4.5.6 Android SDK
6.4.5.7 New cmdline content
6.4.6 Firmware Flashing
6.4.7 Pre-loader verified
6.4.8 U-Boot verified
6.4.9 System Verification Boot
6.4.10 Linux AVB
6.4.10.1 Operation Flow
6.4.10.2 Verification Flow
6.5 SD Boot and Upgrade
6.5.1 Introduction
6.5.2 Categories
6.5.2.1 Regular Card
6.5.2.2 Upgrade Card
6.5.2.3 Boot Card
6.5.2.4 Repair Card
6.5.3 Firmware Flags
6.5.4 Boot Flow
6.5.4.1 pre-loader boot
6.5.4.2 U-Boot boot
6.5.4.3 Recovery and PCBA
6.5.5 Notes
7. Chapter-7 Configuration Trimming
8. Chapter-8 Debug Methods
8.1 DEBUG
8.2 Initcall
8.3 io command
8.4 iomem command
8.5 i2c command
8.6 gpio command
8.7 fdt command
8.8 mmc command
8.9 Timestamp
8.10 dm tree
8.11 dm uclass
8.12 stacktrace.sh
8.13 System Hang
8.14 Modify DDR Size
8.15 Jump Information
8.16 Boot Information
9. Chapter-9 Test Cases
10. Chapter-10 SPL
10.1 Firmware Boot

10.1.1 FIT Firmware
10.1.2 RKFW Firmware
10.1.3 Storage Priority
10.2 Build and Packaging
10.2.1 Code Build
10.3 System Modules
10.3.1 Partition Table
10.3.2 A/B system
10.3.3 Boot Priority
10.3.4 ATAGS
10.3.5 kernel boot
10.3.6 pinctrl
10.4 Driver Modules
10.4.1 MMC
10.4.2 MTD block
10.4.3 OTP
11. Chapter-11 TPL
12. Chapter-12 FIT
12.1 Preface
12.2 Introduction
12.2.1 Basic Introduction
12.2.2 Example Introduction
12.2.3 itb Structure
12.3 Platform Configuration
12.3.1 Chip Support
12.3.2 Code Configuration
12.3.3 Image Files
12.3.4 its File
12.3.5 Related Tools
12.4 Non-Secure Boot
12.4.1 uboot.img
12.4.2 boot.img
12.5 Secure Boot
12.5.1 Principle
12.5.1.1 Verification Flow
12.5.1.2 Key Storage
12.5.1.3 Key Usage
12.5.1.4 Signature Storage
12.5.1.5 Anti-Rollback
12.5.2 Preparation
12.5.2.1 Key
12.5.2.2 Configuration
12.5.2.3 Firmware
12.5.3 Build and Packaging
12.5.4 Verification Rules
12.5.5 Boot Logs
12.6 Standalone Signing
12.6.1 Signing Flow
12.6.2 Signing Script
12.6.3 Signing Configuration
12.7 Remote Signing
12.7.1 Implementation Idea
12.7.2 Data to Be Signed
12.7.3 Detailed Steps
12.8 Firmware Unpacking

- 12.9 Firmware Replacement
- 12.10 Secure Verification Step-by-Step

13. Chapter-13 Fast Boot
14. Chapter-14 Platform Definition
    - 14.1 ATF/OP-TEE
    - 14.2 Clock
    - 14.3 Defconfig
    - 14.4 DFU
    - 14.5 OP-TEE
15. Chapter-15 Remarks
    - 15.1 Configuration Changes
    - 15.2 Documentation Usage
    - 15.3 Debug And Release
16. Chapter-16 Tools
    - 16.1 boot_merger
    - 16.2 resource_tool
    - 16.3 mkimage
    - 16.4 stacktrace.sh
    - 16.5 mkbootimg
    - 16.6 unpack_bootimg
    - 16.7 repack-bootimg
    - 16.8 pack_resource.sh
17. Chapter-17 Appendix
    - 17.1 Terminology

1. Chapter-1 Basic Introduction

1.1 Feature

The main features currently supported by U-Boot v5 include:

- Supports booting Android AOSP firmware
- Supports booting Linux distro firmware
- Supports the DDR + SPL pre-loader boot flow
- Supports display devices such as LVDS, EDP, MIPI, HDMI, CVBS, and RGB
- Supports booting from storage devices such as eMMC, NAND Flash, SPI NAND Flash, SPI NOR Flash, SD card, and USB drive
- Supports FAT, EXT2, and EXT4 file systems
- Supports GPT partition tables
- Supports boot logos, charging animations, low-battery management, and power management
- Supports I2C, PMIC, CHARGE, FUEL GAUGE, USB, GPIO, PWM, GMAC, eMMC, NAND, Interrupt, and more
- Supports using Vendor Storage to save user data and configuration
- Supports flashing eMMC through two USB gadget modes: RockUSB and Google Fastboot
- Supports USB devices such as Mass Storage, Ethernet, and HID
- Supports dynamically selecting the kernel DTB based on hardware status

1.2 DM

DM (Driver Model) is the standard device-driver development model in U-Boot, and it is very similar to the device-driver model in the kernel. Version v5 also develops its functional modules based on the DM framework. Readers are advised to read the DM documentation first to understand the DM architecture and implementation.

README:

```
./doc/develop/driver-model/design.rst
```

```
Terminology
-----------

Uclass - a group of devices which operate in the same way. A uclass provides
        a way of accessing individual devices within the group, but always
        using the same interface. For example a GPIO uclass provides
        operations for get/set value. An I2C uclass may have 10 I2C ports,
        4 with one driver, and 6 with another.

Driver - some code which talks to a peripheral and presents a higher-level
        interface to it.

Device - an instance of a driver, tied to a particular port or peripheral.
```

In short:

- uclass: device class
- driver: driver
- device: device

Core code:

```
./drivers/core/
```

1.3 Security

In the ARM TrustZone security architecture, U-Boot belongs to the Non-Secure World and cannot directly access secure resources such as secure memory or secure OTP (efuse). It must access them indirectly through Trust firmware. On RK platforms, the CPU execution mode of U-Boot is as follows:

```
32-bit platform: Non-Secure PL1
64-bit platform: EL2 (non-secure)
```

1.4 Boot-order

```
// ddr/ATF/OPTEE: closed-source; SPL/U-Boot: open-source
BOOTROM => ddr bin => SPL => Trust(ATF/OPTEE) => U-Boot => KERNEL
```

1.5 Driver-probe

Although U-Boot introduces the device-driver development model, during initialization it does not automatically trigger probe for registered device drivers as the kernel does. A driver's probe must be triggered explicitly by the user. The interfaces are as follows:

```
int uclass_get_device(enum uclass_id id, int index, struct udevice **devp);
int uclass_get_device_by_name(enum uclass_id id, const char *name,
```

```
                        struct udevice **devp);
int uclass_get_device_by_seq(enum uclass_id id, int seq, struct udevice **devp);
int uclass_get_device_by_of_offset(enum uclass_id id, int node, struct udevice **devp);
int uclass_get_device_by_ofnode(enum uclass_id id, ofnode node, struct udevice **devp);
int uclass_get_device_by_phandle_id(enum uclass_id id,
                                    int phandle_id, struct udevice **devp);
int uclass_get_device_by_phandle(enum uclass_id id,
                                 struct udevice *parent, struct udevice **devp);
int uclass_get_device_by_driver(enum uclass_id id,
                                const struct driver *drv, struct udevice **devp);
int uclass_get_device_tail(struct udevice *dev, int ret, struct udevice **devp);
......
```

Core call behind the above interfaces:

```
int device_probe(struct udevice *dev);  /* Users are strongly advised to understand the internal implementation. */
```

1.6 Shell

The U-Boot shell is called the CLI (command line interface). Users can customize commands according to their needs. In addition to being invoked from the shell, commands can also be invoked programmatically through `run_command()` and `run_command_list()`.

```
int run_command(const char *cmd, int flag);
int run_command_list(const char *cmd, int len, int flag);
```

1.7 Boot-Command

U-Boot ultimately boots the kernel through the boot command defined by `CONFIG_BOOTCOMMAND`. Before executing `CONFIG_BOOTCOMMAND`, it also executes the preboot command `CONFIG_PREBOOT`, which is usually defined as empty.

1.8 SPL/U-Boot-proper

By using **different build conditions**, U-Boot can generate different loader stages from the same codebase, such as SPL and U-Boot proper.

SPL (Secondary Program Loader) is a loader stage that runs earlier than U-Boot:

- SPL: runs in DDR and is responsible for low-level system initialization and loading later-stage firmware;
- U-Boot proper: runs in DDR, which is what we usually refer to as "U-Boot", and is responsible for booting the kernel;

> Note: The term U-Boot proper is mainly used to distinguish it from SPL. For convenience, in the following chapters we simply refer to U-Boot proper as U-Boot.

Boot flow:

```
BOOTROM => ddr bin => SPL => Trust(ATF/OPTEE) => U-Boot => KERNEL
```

1.9 Build-Output

After U-Boot is built successfully, several important files are generated in the root directory. TPL/SPL output files are generated only when TPL/SPL build is enabled:

```
// U-Boot stage
./u-boot.map          // MAP file
./u-boot.sym          // SYMBOL file
./u-boot              // ELF file, similar to the kernel's vmlinux (important!)
./u-boot.dtb          // U-Boot's own DTB file
./u-boot.bin          // Executable binary, which will be packaged into uboot.img for flashing

// SPL stage
./spl/u-boot-spl.map    // MAP file
./spl/u-boot-spl.sym    // SYMBOL file
./spl/u-boot-spl        // ELF file, similar to the kernel's vmlinux (important!)
./spl/u-boot-spl.dtb    // SPL's own DTB file
./spl/u-boot-spl.bin    // Executable binary, which will be packaged into the loader for flashing
```

If TPL is enabled on the platform, additional outputs such as `./tpl/u-boot-tpl`, `./tpl/u-boot-tpl.bin`, and `./tpl/u-boot-tpl.dtb` are also generated.

1.10 Environment-Variables

ENV (Environment Variables) is a global data management and passing mechanism supported by U-Boot. Conceptually, it is a hash mapping table that manages user data as key-value entries.

ENV is usually used to define platform configuration parameters such as firmware load addresses, network configuration (`ipaddr`, `serverip`), `bootcmd`, and `bootargs`. Users can print them on the command line with the `printenv` command.

- Users can choose whether to save ENV data to local storage
- ENV data is only for use by U-Boot, and cannot be directly passed to the kernel or parsed directly by the kernel
- User space can access ENV data through the `fw_printenv` tool provided by U-Boot

On RK platforms, the storage address and size of ENV data are defined as follows (unit: bytes):

```
config ENV_OFFSET
    hex "Environment offset"
    depends on ENV_IS_IN_EEPROM || ENV_IS_IN_MMC || ENV_IS_IN_NAND || \
            ENV_IS_IN_SPI_FLASH || ENV_IS_IN_BLK || ENV_IS_IN_FRAGMENT
    default 0x0 if ENV_IS_IN_FRAGMENT
    default 0x3f8000
    help
```

```
    Offset from the start of the device (or partition).

config ENV_SIZE
    hex "Environment Size"
    default 0x8000
    help
      Size of the environment storage area
```

1.11 U-Boot DTS

U-Boot has its own DTS files. During compilation, the corresponding DTB file is generated automatically and appended to the end of `u-boot.bin`. Relevant directories:

```
arch/arm/dts/                   // rkxxx-u-boot.dtsi
dts/upstream/src/arm64/rockchip/    // base dtsi/dts files
```

The specific DTS file used by each platform is specified through `CONFIG_DEFAULT_DEVICE_TREE` in the defconfig.

1.12 Relocation

Usually during boot, U-Boot is loaded by the previous-stage bootloader to a low DRAM address. After U-Boot finishes the flow in `board_f.c`, it relocates itself to a reserved address near the end of memory. This is called relocation, and the address is determined by the U-Boot memory layout. It then continues to execute the flow in `board_r.c`. This can be identified through the boot logs:

```
U-Boot v5-gabfd1c5e3d-210202 #cjh (Mar 08 2021 - 16:57:31 +0800)

Model: Rockchip RK3588 Evaluation Board
PreSerial: 2, raw, 0xfe660000
DRAM:  2 GiB
Sysmem: init
// Relocated to an address with offset 0x7d304000 from the DDR base. If it is 0, relocation was not performed.
Relocation Offset: 7d304000, fdt: 7b9f8ed8
Using default environment
......
```

2. Chapter-2 RK Architecture

This chapter introduces some important basic information and features on the RK platform.

2.1 Preface

Whenever enabling or disabling `CONFIG_` options is mentioned in this document, it refers to enabling or disabling them through `make menuconfig`.

> Except for some special `CONFIG_` options defined directly in `.h` files.

Do not enable or disable `CONFIG_` options directly in defconfig, to avoid generating an ineffective `.config` due to configuration dependencies defined in Kconfig.

When updating defconfig, use `make savedefconfig`.

2.2 Platform Files

Platform directories:

```
./arch/arm/include/asm/arch-rockchip/
./arch/arm/mach-rockchip/
./board/rockchip/
./include/configs/
```

Defconfig directory:

```
./configs/
```

Core common board-level file:

```
./arch/arm/mach-rockchip/board.c
```

2.3 Platform Configuration

Configuration files

The configuration options and parameters for each platform are usually located in the following places:

```
// Common files for all platforms (developers usually do not need to modify them)
./arch/arm/mach-rockchip/Kconfig
./include/configs/rockchip-common.h

// Platform-specific files, RK3588 as an example
./include/configs/rk3588_common.h
./include/configs/evb_rk3588.h
./configs/rk3588_defconfig
```

2.4 Boot Flow

The U-Boot boot flow on RK platforms is as follows. Only some important steps are listed:

```
start.S
  // Assembly environment
  => IRQ/FIQ/lowlevel/vbar/errata/cp15/gic   // ARM architecture-related low-level initialization
  => _main
    => stack                        // Prepare the stack required by the C environment
    // [Phase 1] C environment initialization, launching a series of function calls
    => board_init_f: init_sequence_f[]
      initf_malloc
      arch_cpu_init                 // [SoC low-level initialization]
      serial_init                   // UART initialization
      dram_init                     // [Get DDR capacity information]
      reserve_mmu                   // Reserve memory from the end of DDR toward lower addresses
      reserve_video
      reserve_uboot
      reserve_malloc
      reserve_global_data
      reserve_fdt
      reserve_stacks
      dram_init_banksize
      sysmem_init
      setup_reloc                   // Determine the relocation address of U-Boot itself
    // Assembly environment
    => relocate_code                // U-Boot code relocation implemented in assembly
    // [Phase 2] C environment initialization, launching a series of function calls
    => board_init_r: init_sequence_r[]
      initr_caches                  // Enable MMU and I/D cache
      initr_malloc
      bidram_initr
      sysmem_initr
      initr_of_live                 // Initialize of_live
      initr_dm                      // Initialize the DM framework
      board_init                    // [Platform initialization, the most critical part]
        board_debug_uart_init       // UART iomux and clock configuration
        init_kernel_dtb             // [Switch to Kernel DTB]!
        clks_probe                  // Initialize system clocks
        regulators_enable_boot_on   // Initialize system power rails
        io_domain_init              // io-domain initialization
        set_armclk_rate             // __weak, ARM frequency boost (implemented only when needed)
        dvfs_init                   // Frequency and voltage scaling for wide-temperature chips
```

```
    rk_board_init           // __weak, implemented by each platform
  console_init_r
  board_late_init           // [Platform late initialization]
    rockchip_set_ethaddr      // Set MAC address
    rockchip_set_serialno     // Set serial number
    setup_boot_mode           // Parse "reboot xxx" command,
                              // identify key events, loader flashing mode, recovery
    charge_display          // U-Boot charging
    rockchip_show_logo        // Display boot logo
    soc_clk_dump            // Dump clock tree
    rk_board_late_init      // __weak, implemented by each platform
  run_main_loop             // [Enter command-line mode or execute boot command]
```

2.5 Memory Layout

U-Boot is loaded by the previous-stage loader to the `CONFIG_TEXT_BASE` address. During initialization it detects the total system memory size. On 32-bit platforms, at most 4 GB is considered available, although this does not affect how the kernel detects memory size. On 64-bit platforms, all memory is considered available. Then a series of `reserve_xxx()` interfaces reserve memory from the end of RAM toward lower addresses, and finally U-Boot relocates itself to one of those reserved regions. The overall memory layout is shown below, using ARM64 as an example in the normal case:

Name	Start Addr Offset	Size	Usage	Secure
ATF	0x00000000	1M	ARM Trusted Firmware	Yes
SHM	0x00100000	1M	SHM, Pstore	No
OP-TEE	0x08400000	2M~30M	See the TEE development guide	Yes
FDT	fdt_addr_r	-	kernel dtb	No
KERNEL	kernel_addr_r	-	kernel image	No
RAMDISK	ramdisk_addr_r	-	ramdisk image	No
.....	-	-	-	-
FASTBOOT	-	-	Fastboot buffer	No
.....	-	-	-	-
SP	-	-	stack	No
FDT	-	sizeof(dtb)	U-Boot dtb	No
GD	-	sizeof(gd)	-	No
Board	-	sizeof(bd_t)	-	No
MALLOC	-	CONFIG_SYS_MALLOC_LEN	System heap space	No
U-Boot	-	sizeof(mon)	U-Boot image	No
Video FB	-	fb size	32M	No
TLB Table	RAM_TOP-64K	32K	MMU page table	No

> The `Start Addr Offset` column above indicates the address offset relative to the DDR base.
>
> Fastboot address and size are determined by `CONFIG_FASTBOOT_BUF_ADDR` and `CONFIG_FASTBOOT_BUF_SIZE`.

- Video FB/U-Boot/Malloc/Board/Gd/Fdt/Sp are allocated from top to bottom according to actual size requirements.
- On 64-bit platforms, ATF is required by ARMv8 and OP-TEE is optional. On 32-bit platforms, only OP-TEE is present.
- `kernel fdt` / `kernel` / `ramdisk` are the firmware load addresses used by U-Boot and are defined by `ENV_MEM_LAYOUT_SETTINGS`.
- The Fastboot buffer address and size are defined in defconfig.
- The space occupied by OP-TEE depends on actual requirements, with a maximum of 30 MB.

2.6 Storage Layout

The storage layout for the RK Linux solution is as follows. In the Android solution, only the definitions of `boot` and `rootfs` differ from the Linux platform; the rest are basically the same and can be used as a reference.

Partition	Start Sector		Number of Sectors		Partition Size		Requirements
MBR	0	00000000	1	00000001	512	0.5KB	
Primary GPT	1	00000001	63	0000003F	32256	31.5KB	
loader1	**64**	**00000040**	**7104**	**00001bc0**	**4096000**	**2.5MB**	**preloader (miniloader or U-Boot SPL)**
Vendor Storage	7168	00001c00	512	00000200	262144	256KB	SN, MAC and etc.
Reserved Space	7680	00001e00	384	00000180	196608	192KB	Not used
reserved1	8064	00001f80	128	00000080	65536	64KB	legacy DRM key
U-Boot ENV	8128	00001fc0	64	00000040	32768	32KB	
reserved2	8192	00002000	8192	00002000	4194304	4MB	legacy parameter
loader2	**16384**	**00004000**	**8192**	**00002000**	**4194304**	**4MB**	**U-Boot or UEFI**
trust	**24576**	**00006000**	**8192**	**00002000**	**4194304**	**4MB**	**trusted-os like ATF, OP-TEE**
boot (bootable must be set)	**32768**	**00008000**	**229376**	**00038000**	**117440512**	**112MB**	**kernel, dtb, extlinux.conf, ramdisk**
rootfs	**262144**	**00040000**	**-**	**-**	**-**	**-MB**	**Linux system**
Secondary GPT	16777183	00FFFFDF	33	00000021	16896	16.5KB	

> Image source: http://opensource.rock-chips.com/wiki_Partitions

2.7 Aliases

Some special aliases in U-Boot differ from those defined in the kernel DTS.

In U-Boot, eMMC and SD are both treated as `mmc` devices and distinguished by index 0 and 1. SD has higher boot priority than eMMC.

```
mmc0: eMMC
mmc1: SD
```

2.8 AMP

U-Boot on RK platforms supports AMP (Asymmetric Multi-Processing) firmware boot.

> For more details, see the Driver Modules chapter.

2.9 Atags

The RK platform boot flow:

```
BOOTROM => ddr-bin => SPL => TRUST(ATF/OPTEE) => U-Boot => KERNEL
```

Different firmware stages on RK platforms can pass certain configuration information through the ATAGS mechanism.

- Scope: `ddr-bin`, SPL, ATF, OP-TEE, U-Boot, but not the kernel.
- Contents: UART configuration, storage type, memory occupied by ATF and OP-TEE, DDR capacity, and so on.

Code implementation:

```
./arch/arm/include/asm/arch-rockchip/atags.h
./arch/arm/mach-rockchip/atags.c
```

2.10 Bootdev

Get the handle of the current boot storage:

```
struct blk_desc *plat_bootdev(void);
```

For usage, search the existing codebase.

2.11 Bidram/sysmem

U-Boot can use all system memory and reserves the memory it needs from high addresses to low addresses. After reservation, a relatively large amount of memory usually remains. U-Boot itself has no mechanism to manage this remaining space, so RK platforms introduce the `bidram` and `sysmem` memory block mechanisms to manage it.

Together with U-Boot's existing `malloc` mechanism, RK platforms therefore manage all system memory through `sysmem + bidram + malloc`, preventing memory conflicts and similar issues.

```
low-addr                                                high-addr
|-------------------------------------------------------|----------------|
|           no management                               | system used    |
|-------------------------------------------------------|----------------|
0x0                                                     N GB
```

- `bidram`: manages memory regions that are unavailable during the U-Boot and kernel stages and must be excluded, such as regions occupied by ATF and OP-TEE.
- `sysmem`: manages memory regions visible and usable by the kernel, such as those used by FDT, ramdisk, kernel, and Fastboot.

Related code:

```
./lib/sysmem.c
./lib/bidram.c
./include/memblk.h
./arch/arm/mach-rockchip/memblk.c
```

Below are the memory management tables for `bidram` and `sysmem`. They are dumped when memory block initialization or allocation fails. A brief introduction follows.

`bidram` memory information table:

```
bidram_dump_all:
   ----------------------------------------------------------------
   // <1> This shows the total DDR capacity obtained by U-Boot from the previous-stage loader: 2 GB
   memory.rgn[0].addr    = 0x00000000 - 0x80000000 (size: 0x80000000)

   memory.total         = 0x80000000 (2048 MiB. 0 KiB)
   ----------------------------------------------------------------
   // <2> This shows the reserved memory regions for various firmware; these regions are invisible to the kernel
   reserved.rgn[0].name   = "ATF"
            .addr   = 0x00000000 - 0x00100000 (size: 0x00100000)
   reserved.rgn[1].name   = "SHM"
            .addr   = 0x00100000 - 0x00200000 (size: 0x00100000)
   reserved.rgn[2].name   = "OP-TEE"
            .addr   = 0x08400000 - 0x0a200000 (size: 0x01e00000)

   reserved.total       = 0x02000000 (32 MiB. 0 KiB)
   ----------------------------------------------------------------
   // <3> This is the core algorithm's consolidation of the reservations in <2>, for example merging adjacent regions
   LMB.reserved[0].addr   = 0x00000000 - 0x00200000 (size: 0x00200000)
   LMB.reserved[1].addr   = 0x08400000 - 0x0a200000 (size: 0x01e00000)

   reserved.core.total    = 0x02000000 (32 MiB. 0 KiB)
   ----------------------------------------------------------------
```

`sysmem` memory information table:

```
sysmem_dump_all:
```

```
    ----------------------------------------------------------------------
    // <1> This is the total memory that sysmem can manage, i.e. usable DDR outside bidram<3>, visible to the kernel
    memory.rgn[0].addr    = 0x00200000 - 0x08400000 (size: 0x08200000)
    memory.rgn[1].addr    = 0x0a200000 - 0x80000000 (size: 0x75e00000)

    memory.total          = 0x7e000000 (2016 MiB. 0 KiB)
    ----------------------------------------------------------------------
    // <2> This shows the memory blocks allocated by each firmware component
    allocated.rgn[0].name  = "U-Boot"
                .addr  = 0x71dd6140 - 0x80000000 (size: 0x0e229ec0)
    allocated.rgn[1].name  = "STACK"      <Overflow!> // Indicates stack overflow
                .addr  = 0x71bd6140 - 0x71dd6140 (size: 0x00200000)
    allocated.rgn[2].name  = "FDT"
                .addr  = 0x08300000 - 0x08316204 (size: 0x00016204)
    allocated.rgn[3].name  = "KERNEL"     <Overflow!> // Indicates memory block overflow
                .addr  = 0x00280000 - 0x014ce204 (size: 0x0124e204)
    allocated.rgn[4].name  = "RAMDISK"
                .addr  = 0x0a200000 - 0x0a3e6804 (size: 0x001e6804)
    // <3> Size of malloc_r/f
    malloc_r: 192 MiB, malloc_f: 16 KiB

    allocated.total       = 0x0f874acc (248 MiB. 466 KiB)
    ----------------------------------------------------------------------
    // <4> This is the core algorithm's consolidation of the allocations in <2>, showing occupied memory blocks
    LMB.reserved[0].addr   = 0x00280000 - 0x014ce204 (size: 0x0124e204)
    LMB.reserved[1].addr   = 0x08300000 - 0x08316204 (size: 0x00016204)
    LMB.reserved[2].addr   = 0x0a200000 - 0x0a3e6804 (size: 0x001e6804)
    LMB.reserved[3].addr   = 0x71bd6140 - 0x80000000 (size: 0x0e429ec0)

    reserved.core.total    = 0x0f874acc (248 MiB. 466 KiB)
    ----------------------------------------------------------------------
```

Below are some common error logs. When these exceptions occur, analyze them together with the `bidram` and `sysmem` memory dump information above.

```
// The requested memory region has already been occupied by other firmware, causing overlap. This means the current memory planning is unreasonable.
Sysmem Error: "KERNEL" (0x00200000 - 0x02200000) alloc is overlap with existence "RAMDISK" (0x00100000 - 0x01200000)

// The requested region cannot be allocated for some special reason (analyze sysmem and bidram information)
Sysmem Error: Failed to alloc "KERNEL" expect at 0x00200000 - 0x02200000 but at 0x00400000 - 0x04200000

// sysmem-managed space starts at 0x200000, so a region starting at 0x100000 can never be allocated
Sysmem Error: Failed to alloc "KERNEL" at 0x00100000 - 0x02200000

// Repeated allocation of the "RAMDISK" memory block
Sysmem Error: Failed to double alloc for existence "RAMDISK"
```

2.12 Fuse/OTP

To make secure-boot debugging easier on RK platforms, simply signing the firmware is enough to enable secure-boot mode, without necessarily burning efuse/otp. The miniloader appends extra cmdline content through U-Boot to the kernel to indicate whether efuse/otp enablement has been burned:

- `"fuse.programmed=1"`: secure-boot is enabled, and efuse/otp has been burned.
- `"fuse.programmed=0"`: secure-boot is enabled, but efuse/otp has not been burned.
- No `fuse.programmed` in cmdline: secure-boot is not enabled (SPL does not pass it).

2.13 Hotkey

RK platforms provide UART key combinations that trigger certain events for debugging and flashing. If they do not trigger, try several times. They are ineffective when secure-boot is enabled. **Hold the following during power-on:**

- `ctrl+c`: enter U-Boot command-line mode
- `ctrl+d`: enter loader flashing mode
- `ctrl+b`: enter maskrom flashing mode
- `ctrl+f`: enter fastboot mode
- `ctrl+m`: print bidram/system information
- `ctrl+i`: enable kernel `initcall_debug`
- `ctrl+p`: print cmdline information
- `ctrl+s`: enter U-Boot command-line mode after `"Starting kernel..."`

2.14 Image Decompress

- On 64-bit platforms, the machine usually flashes `Image`, which U-Boot loads to the target runtime address. RK U-Boot also supports decompressing 64-bit kernels compressed with LZ4. Users must enable:

  ```
  CONFIG_LZ4=y
  ```

  The load addresses before and after decompression for a 64-bit LZ4-compressed kernel must be defined in each platform's `rkxxx_common.h`:

  ```
  #define ENV_MEM_LAYOUT_SETTINGS \
    "scriptaddr=0x00500000\0" \
    "pxefile_addr_r=0x00600000\0" \
    "fdt_addr_r=0x01f00000\0" \
    "kernel_addr_no_bl32_r=0x00280000\0" \
    "kernel_addr_r=0x00680000\0" \      // Address for the LZ4 decompressed kernel
    "kernel_addr_c=0x02480000\0" \      // Address for the LZ4 compressed kernel
    "ramdisk_addr_r=0x04000000\0"
  ```

- On 32-bit platforms, the machine usually flashes `zImage`, which U-Boot loads to `kernel_addr_r`, and then the kernel performs self-decompression. RK U-Boot can also support `Image`, loading it directly to the target runtime address.

  Currently, each platform's `rkxxx_common.h` defines only `kernel_addr_r` and not `kernel_addr_c`. Users do not need to change the configuration, because U-Boot determines whether the current image is `zImage` or `Image` and handles the two addresses dynamically. However, users must disable:

  ```
  CONFIG_SKIP_RELOCATE_UBOOT
  ```

  32-bit kernel load address definition:

  ```
  #define ENV_MEM_LAYOUT_SETTINGS \
     "scriptaddr=0x60000000\0" \
     "pxefile_addr_r=0x60100000\0" \
     "fdt_addr_r=0x68300000\0" \
     "kernel_addr_r=0x62008000\0" \      // Address for the compressed zImage kernel
     "ramdisk_addr_r=0x6a200000\0"
  ```

2.15 Image kernel

RK U-Boot supports booting three kernel firmware formats:

- Android format

  The image magic is `"ANDROID!"`:

  ```
  00000000   41 4E 44 52  4F 49 44 21  24 10 74 00  00 80 40 60  ANDROID!$.t...@`
  00000010   F9 31 CD 00  00 00 00 62  00 00 00 00  00 00 F0 60  .1.....b.......`
  ```

  `boot.img = kernel + ramdisk + resource + <dtb>`

  `recovery.img = kernel + ramdisk(for recovery) + resource + <recovery_dtbo> + <dtb>`

  > Note: `recovery_dtbo` was added starting from Android 9.0. `dtb` was added starting from Android 10.0.

- FIT format

  A highly flexible firmware format supported by U-Boot mainline. Firmware such as U-Boot, Trust, and MCU can be packaged together into `uboot.img`.

  The magic of `uboot.img` is `"d0 0d fe ed"`. You can inspect the header with `fdtdump uboot.img`.

  ```
  00000000  d0 0d fe ed 00 00 06 00  00 00 00 58 00 00 04 c4  |...........X....|
  00000010  00 00 00 28 00 00 00 11  00 00 00 10 00 00 00 00  |...(............|
  ```

  > For more about FIT, see the FIT chapter.

- Distro format

  A common firmware packaging format currently used by open-source Linux: `ramdisk`, `dtb`, and

`kernel` are packaged into one image. This image usually exists in a file system such as `ext2`, `ext4`, or `fat`, and U-Boot accesses its contents through that file system. For more details:

```
boot/bootstd-uclass.c
cmd/bootstd.c
boot/bootflow.c
boot/bootflow_internal.h
boot/bootflow_menu.c
cmd/bootflow.c
......
```

- Boot priority: `android > FIT > distro`. Each firmware type has a corresponding boot command. These three commands are executed in order of priority until one successfully boots the firmware. If all fail, U-Boot stays in command-line mode.

  Boot priority definition:

```
#define CONFIG_BOOTCOMMAND        \
   "boot_android ${devtype} ${devnum};"    \
   "boot_fit;"           \
   "bootflow scan -bl;"
```

2.16 Image U-Boot

- FIT format

  A highly flexible firmware format supported by U-Boot mainline. U-Boot, Trust, and MCU firmware can be packaged together into `uboot.img`.

  The magic of `uboot.img` is `"d0 0d fe ed"`. Use `fdtdump uboot.img` to inspect the header.

```
00000000  d0 0d fe ed 00 00 06 00  00 00 00 58 00 00 00 04 c4  |...........X....|
00000010  00 00 00 28 00 00 00 11  00 00 00 10 00 00 00 00 00  |...(............|
```

> For more about FIT, see the FIT chapter.

- Backup packaging

  To handle cases such as unexpected power loss during OTA upgrades that may corrupt firmware, `uboot.img` is usually packaged with multiple backups.

Firmware	Size Per Copy	Number of Copies
FIT uboot.img	3 MB	2

  - `CONFIG_SPL_FIT_IMAGE_KB`: size per copy (KB)
  - `CONFIG_SPL_FIT_IMAGE_MULTIPLE`: number of packaged copies

  Notes:

  - SPL determines the positions and count of backup entries in `uboot.img` according to `CONFIG_SPL_FIT_IMAGE_KB` and `CONFIG_SPL_FIT_IMAGE_MULTIPLE`.
  - If developers modify either option, `make.sh` must be given the additional argument `--spl-`

`new`, so that the generated loader contains the newly built `u-boot-spl.bin`. Otherwise, the default `u-boot-spl.bin` from `rkbin` is packaged, and it still assumes `3 MB * 2` when locating backup entries in `uboot.img`.

2.17 Interrupt

Native U-Boot code does not fully support interrupts. RK platforms extend this area and support both GIC-V2 and GIC-V3.

> For more details, see the Driver Modules chapter.

2.18 Kernel-DTB

Native U-Boot only supports using its own DTB. RK platforms add support for the kernel DTB mechanism, meaning the kernel DTB is used to initialize peripherals. The main purpose is to accommodate board-level peripheral differences such as power, clock, display, and so on.

Their roles:

- U-Boot DTB: initializes storage, the debug UART, and similar devices.
- Kernel DTB: initializes devices other than storage and the debug UART.

During initialization, U-Boot first uses the U-Boot DTB to initialize storage and the debug UART, then loads the Kernel DTB from storage and switches to that DTB to continue initializing the remaining peripherals. The Kernel DTB logic is implemented in `init_kernel_dtb()`.

Developers generally do not need to modify the U-Boot DTB, unless changing the debug UART. In the SDKs released for each platform, the provided defconfigs already enable the kernel DTB mechanism. Therefore, peripheral DTS changes should usually be made in the kernel DTB.

About the U-Boot DTB:

DTS directory:

```
./arch/arm/dts/rkxxx-u-boot.dtsi
```

After enabling the kernel DTB mechanism, during compilation the nodes in the U-Boot DTS that carry the `bootph-all` property are filtered out. Then the properties specified by `CONFIG_OF_REMOVE_PROPS` in defconfig are removed. The final `u-boot.dtb` is generated and appended to the end of `u-boot.bin`.

After building U-Boot, users can inspect the DTB content with:

```
fdtdump ./u-boot.dtb | less
```

> For more details, see the Advanced Principles chapter.

2.19 MMU Cache

RK platforms enable MMU, Dcache, and Icache by default. The MMU uses 1:1 linear mapping, and Dcache uses the write-back policy. Related interfaces:

```
// Icache interfaces:
void icache_enable (void);
void icache_disable (void);
void invalidate_icache_all(void);

// Dcache interfaces:
void dcache_disable (void);
void dcache_enable(void);
void flush_dcache_range(unsigned long start, unsigned long stop);
void flush_cache(unsigned long start, unsigned long size);
void flush_dcache_all(void);
void invalidate_dcache_range(unsigned long start, unsigned long stop);
void invalidate_dcache_all(void);
// Remap the Dcache attributes of a memory region
void mmu_set_region_dcache_behaviour(phys_addr_t start, size_t size,
                                     enum dcache_option option);
```

2.20 Make.sh

`make.sh` is both a build script and a packaging/debugging tool. It can be used for disassembly and firmware packaging.

```
// Help
./make.sh --help

// Firmware packaging
./make.sh itb              // Package u-boot.itb (on 64-bit platforms, only ATF and U-Boot are packaged, OP-TEE is not)
./make.sh env              // Generate the fw_printenv tool

// Disassembly
./make.sh elf-[x] [type]    // Disassemble: use the -[x] parameter, [type] can select SPL or TPL
./make.sh elf              // Disassemble the u-boot file, using -D by default
./make.sh elf-S            // Disassemble the u-boot file, using -S
./make.sh elf-d            // Disassemble the u-boot file, using -d
./make.sh elf spl          // Disassemble tpl/u-boot-tpl, using -D by default
./make.sh elf tpl          // Disassemble spl/u-boot-tpl, using -D by default
./make.sh <addr>           // Query the function name and code location for an address
./make.sh map              // Open u-boot.map
./make.sh sym              // Open u-boot.sym
```

2.21 HW-ID DTB

RK U-Boot can select and load, from multiple DTB files, the DTB that matches the actual hardware status according to GPIO or ADC state.

> For more details, see the System Modules chapter.

2.22 Partition Table

RK U-Boot supports the standard GPT format. The partition table file used for flashing is `parameter.txt`:

```
FIRMWARE_VER:8.1
MACHINE_MODEL:RK3399
MACHINE_ID:007
MANUFACTURER: RK3399
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3399
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
CMDLINE:mtdparts=rk29xxnand:0x00002000@0x00004000(uboot),0x00002000@0x00006000(trust),0
x00002000@0x00008000(misc),0x00008000@0x0000a000(resource),0x00010000@0x00012000(kernel
),0x00010000@0x00022000(boot),0x00020000@0x00032000(recovery),0x00038000@0x00052000(bac
kup),0x00002000@0x0008a000(security),0x00100000@0x0008c000(cache),0x00500000@0x0018c000
(system),0x00008000@0x0068c000(metadata),0x00100000@0x00694000(vendor),0x00100000@0x007
96000(oem),0x00000400@0x00896000(frp),-@0x00896400(userdata:grow)
```

2.23 Relocation

After completing the flow in `board_f.c`, U-Boot relocates itself to an address near the end of memory. The exact address depends on the U-Boot memory layout. By default on RK platforms:

- 32-bit platforms: no relocation if `CONFIG_SKIP_RELOCATE_UBOOT=y`; otherwise relocation is performed.
- 64-bit platforms: relocation is enabled by default.

2.24 Reset

- Like the kernel, U-Boot reset ultimately needs to trap into Trust to complete.
- U-Boot command-line mode can support `reboot xxx` commands like the kernel, depending on the definitions in the kernel DTS.

2.25 Sd/Udisk

RK U-Boot supports booting or upgrading firmware from SD cards and USB disks:

- SD boot/upgrade is supported starting from the BootROM stage.
- USB disk boot/upgrade is supported starting from the U-Boot stage.

> For more details, see the System Modules chapter.

2.26 Stacktrace

Native U-Boot does not support call stack backtracing. RK platforms add this feature. Currently there are three ways to trigger call stack printing:

- automatically when the system crashes
- by manually calling `dump_stack()`
- by enabling `CONFIG_IRQ_TIMER_DUMP`

For example, a system abort:

```
"Synchronous Abort" handler, esr 0x96000010

// abort reason, pc, lr, sp
* Reason:       Exception from a Data abort, from current exception level
* PC       =   000000000028f430
* LR       =   00000000002608d0
* SP       =   00000000f3dceb30

...

// Focus on PC and LR
Call trace:
 PC:   [< 0028f430 >]
 LR:   [< 002608d0 >]

// Function call relationship
Stack:
      [< 0028f430 >]
      [< 0028da24 >]
      [< 00211600 >]
      [< 002117b0 >]
      [< 00202910 >]
      [< 00202aa8 >]
      [< 0027698c >]
      [< 002151ec >]
      [< 00201b2c >]

// Instruct users how to convert the stack trace
Copy info from "Call trace..." to a file(eg. dump.txt), and run
command in your U-Boot project: ./scripts/stacktrace.sh dump.txt
```

According to the instructions above, after copying the call stack information into any text file such as `dump.txt`, run:

```
cjh@Ubuntu:~/u-boot$ ./scripts/stacktrace.sh dump.txt

// Symbol table source
SYMBOL File: ./u-boot.sym

// Highlight the code locations corresponding to PC and LR
Call trace:
 PC:   [< 0028f430 >]  strncpy+0xc/0x20      ./lib/string.c:98
 LR:   [< 002608d0 >]  on_serialno+0x10/0x1c ./drivers/usb/gadget/g_dnl.c:217

// Converted real function names
Stack:
      [< 0028f430 >]  strncpy+0xc/0x20
      [< 0028da24 >]  hdelete_r+0xcc/0xf0
      [< 00211600 >]  _do_env_set.isra.0+0x70/0x1b8
      [< 002117b0 >]  env_set+0x3c/0x58
      [< 00202910 >]  rockchip_set_serialno+0x54/0x140
      [< 00202aa8 >]  board_late_init+0x5c/0xa0
      [< 0027698c >]  initcall_run_list+0x58/0x94
      [< 002151ec >]  board_init_r+0x20/0x24
      [< 00201b2c >]  relocation_return+0x4/0x0
```

Notes:

- There are three conversion commands. Choose the correct one according to the guidance shown after the stack trace is printed.

  ```
  ./scripts/stacktrace.sh ./dump.txt      // Parse stack trace from U-Boot
  ./scripts/stacktrace.sh ./dump.txt tpl   // Parse stack trace from tpl
  ./scripts/stacktrace.sh ./dump.txt spl   // Parse stack trace from spl
  ```

  > For the result to be meaningful, the firmware running on the current machine must match the current source tree. Otherwise the conversion result will be wrong.

2.27 TimeCost

After U-Boot initialization finishes, it prints the total time spent in this stage by default:

```
### Booting Android Image at 0x02007800 ...
Kernel load addr 0x02008000 size 8062 KiB
### Flattened Device Tree blob at 08300000
   Booting using the fdt blob at 0x8300000
   XIP Kernel Image ... OK
  'reserved-memory' dma-unusable@fe000000: addr=fe000000 size=1000000
  'reserved-memory' ramoops@00000000: addr=8000000 size=f0000
   Using Device Tree in place at 08300000, end 08316ed1
Adding bank: 0x00000000 - 0x08400000 (size: 0x08400000)
Adding bank: 0x09200000 - 0x80000000 (size: 0x76e00000)
Total: 812.613 ms    // Total time spent in the U-Boot stage

Starting kernel ...
```

Users can enable `debug()` and `DEBUG` in `lib/initcall.c` to get per-stage timing output like the following. Function addresses can be resolved by disassembling with `./make.sh`.

```
U-Boot v5-00019-g9b55ed0-dirty (Dec 26 2019 - 14:45:33 +0800)

                        #   5212 us # 137.868 ms
initcall: 0020de1f
                        #      1 us # 142.636 ms
initcall: 0020e015
Model: Evb-RK3288
                        #   1646 us # 149.48 ms
initcall: 0020dd61
PreSerial: 2
                        #   1213 us # 155.28 ms
initcall: 0020ddcd
DRAM:                       #    606 us # 160.401 ms
initcall: 00203719
        // 187 us below is the time consumed by initcall: 00203719
        // 165.355 ms below is the cumulative U-Boot boot time up to initcall: 00203719
                        #    187 us # 165.355 ms
initcall: 0020de81
                        #      2 us # 169.938 ms
initcall: 0020dc29
                        #      1 us # 174.703 ms
initcall: 0020dc3d
                        #      1 us # 179.469 ms
initcall: 0020ddad
                        #      2 us # 184.237 ms
initcall: 0020de27
                        #      1 us # 189.2 ms
......
```

2.28 Timestamp

Kernel logs include timestamps by default, making time-related analysis easier. U-Boot logs do not include timestamps by default. If needed, users can enable `CONFIG_BOOTSTAGE_PRINTF_TIMESTAMP`, as shown below:

```
[    0.324987] U-Boot v5-00019-g9b55ed0-dirty (Dec 26 2019 - 14:31:44 +0800)

[    0.327215] Model: Evb-RK3288
[    0.330039] PreSerial: 2
[    0.332526] DRAM:  2 GiB
[    0.336454] Relocation Offset: 00000000, fdt: 7be22c38
[    0.346981] Using default environment

[    0.351075] dwmmc@ff0c0000: 1, dwmmc@ff0f0000: 0
[    0.394136] Bootdev(atags): mmc 0
[    0.394272] MMC0: High Speed, 52Mhz
[    0.395276] PartType: EFI
[    0.400347] Android 9.0, Build 2019.6
[    0.402070] boot mode: None
[    0.405213] Found DTB in boot part
[    0.407833] DTB: rk-kernel.dtb
[    0.418211] ANDROID: fdt overlay OK
[    0.432128] I2c0 speed: 400000Hz
[    0.435916] PMIC:  RK808
[    0.439113] vdd_arm 1100000 uV
[    0.444148] vdd_gpu 1100000 uV
......

[    1.005018] ## Booting Android Image at 0x02007800 ...
[    1.009917] Kernel load addr 0x02008000 size 8062 KiB
[    1.014981] ## Flattened Device Tree blob at 08300000
[    1.019970]    Booting using the fdt blob at 0x8300000
[    1.025185]    XIP Kernel Image ... OK
[    1.035469]   'reserved-memory' dma-unusable@fe000000: addr=fe000000 size=1000000
[    1.037448]   'reserved-memory' ramoops@00000000: addr=8000000 size=f0000
[    1.044412]    Using Device Tree in place at 08300000, end 08316ed1
[    1.064363] Adding bank: 0x00000000 - 0x08400000 (size: 0x08400000)
[    1.064976] Adding bank: 0x09200000 - 0x80000000 (size: 0x76e00000)
[    1.075259] Total: 812.613 ms

[    1.075279] Starting kernel ...
......
```

> Note: the timestamp is relative elapsed time, not absolute time.

2.29 Vendor storage

RK U-Boot provides a Vendor Storage area for users to save SN, MAC, and similar information. The storage offsets are as follows (see `vendor.c` for details):

```
#define EMMC_VENDOR_PART_OFFSET    (1024 * 7)
/* --- Spi Nand/SLC/MLC large capacity case define --- */
#define NAND_VENDOR_PART_OFFSET    0
/* --- Spi/Spi Nand/SLC/MLC small capacity case define --- */
#define FLASH_VENDOR_PART_OFFSET   8
......
```

Users generally do not need to care about or modify the storage offsets; they mainly need the read/write interfaces:

```
int vendor_storage_read(u16 id, void *pbuf, u16 size)
int vendor_storage_write(u16 id, void *pbuf, u16 size)
```

3. Chapter-3 Build and Flash

3.1 Preparation

- Download `rkbin`

  This is a tool-package repository used to store Rockchip closed-source binaries, scripts, and packaging tools. During U-Boot compilation, related files are indexed from this repository and packaged into Loader, Trust, and `uboot.img` firmware. `rkbin` and the U-Boot project must be kept at the same directory level.

- Download GCC

  The GCC compiler used is `gcc-arm-10.3-2021.07`, placed under the `prebuilts` directory. `prebuilts` and U-Boot should remain at the same directory level, as follows:

  ```
  // 32-bit:
  prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf
  // 64-bit:
  prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu
  ```

- Select a defconfig: **Please refer to the Platform Definition chapter**.

- Introduction to config fragments

  Due to differentiated product requirements on a single platform, one defconfig is no longer sufficient. Therefore, starting from `rk3588`, config fragments are supported, meaning an overlay is applied on top of the defconfig.

  For example, `rk3588-emmc-tb.config` specifies `CONFIG_BASE_DEFCONFIG="rk3588_defconfig"`. When the command `./make.sh rk3588-emmc` is executed, `.config` is first generated from `rk3588_defconfig`, and then the configuration in `rk3588-emmc-tb.config` is overlaid onto `.config`. This command is equivalent to:

  ```
  make rk3588_defconfig rk3588-emmc.config && make
  ```

  If you want to update a config fragment file, just use `./scripts/sync-fragment.sh`. For example:

  ```
  ./scripts/sync-fragment.sh configs/rk3588-emmc.config
  ```

  Command effect: synchronize the configuration differences between the current `.config` and `rk3588_defconfig` into the target fragment file.

3.2 Firmware Build

Build command:

```
./make.sh [board]        // [board]: configs/[board]_defconfig file.
```

First build: For both 32-bit and 64-bit platforms, when building for the first time or when you want to reselect the defconfig, the build command must specify `[board]`. For example:

```
./make.sh rk3588         // build for rk3588_defconfig
./make.sh rk3572         // build for rk3572_defconfig
./make.sh rk3572 --spl-new  // build for rk3572_defconfig with u-boot-spl.bin in Loader
```

Subsequent build: For both 32-bit and 64-bit platforms, if you want to rebuild based on the current `.config`, the build command does not need to specify `[board]`:

```
./make.sh
```

Build SPL from source: If you want the generated Loader file to include the newly built `u-boot-spl.bin`, you need to specify `--spl-new`.

```
./make.sh --spl-new
```

Build open-source USBPLUG: Build with `usbplug_defconfig`, then copy `usbplug.bin` into the `rkbin` repository. For example:

```
./make.sh rk3576-usbplug                    // build USBPLUG
cp usbplug.bin ../rkbin/bin/rk35/rk3576_usbplug_vX.XX.bin // update usbplug.bin in rkbin
./make.sh rk3576                            // build again to get a loader containing USBPLUG
```

> Notes:
>
> 1. If unexpected issues occur during compilation and cause build failures, try `make distclean` and build again.
> 2. To see which platforms support open-source `usbplug`, refer to `CH14-Platform_Definition_EN.md`. For unsupported platforms, refer to `rockchip-usbplug_defconfig` or contact an RK engineer.

Firmware output: After compilation, flashable firmware such as `uboot.img` and `loader` is packaged into the U-Boot root directory:

```
./uboot.img
./rk3572_loader_v1.01.101.bin
```

> Firmware packaging tool: see the Tools chapter.

3.3 Firmware Flashing

Flashing tools:

For Windows/Linux, it is recommended to use the SDK-released flashing tool version or the latest version.

Flashing modes:

There are two flashing modes on RK platforms: Maskrom mode and Loader mode (U-Boot).

1. Methods to enter Loader flashing mode:

- Hold `Volume+` while powering on
- Hold `Ctrl+D` on the host while powering on
- Enter the following in the U-Boot command line: `download` or `rockusb 0 $devtype $devnum`

2. Methods to enter Maskrom flashing mode:

- Hold `Ctrl+B` on the host while powering on
- Enter `rbrom` in the U-Boot command line

3.4 Firmware Size Modification

Please refer to the chapter: RK Architecture => Image U-Boot.

4. Chapter-4 System Modules

4.1 AArch32

ARMv8 64-bit chips support switching from AArch64 to AArch32 mode for execution, which is compatible with ARMv7. The code must be compiled in 32-bit mode.

Users can use the following macro to determine whether the current mode is ARMv8 AArch32:

```
CONFIG_ARM64_BOOT_AARCH32=y
```

4.2 ANDROID AB

The so-called A/B System means that the system firmware is divided into two copies, named slot-a and slot-b. The system can boot from either slot. If one slot fails to boot, it can still boot from the other. At the same time, during upgrade, the firmware can be copied directly to the other slot without entering the system upgrade mode. For detailed principles and flow, refer to the Advanced Principles chapter.

At present, both the RK pre-loader and U-Boot support the A/B system.

4.2.1 Configuration

The A/B System depends on LIBAVB:

```
// Libraries required by A/B
CONFIG_LIBAVB=y
CONFIG_LIBAVB_AB=y
CONFIG_LIBAVB_ATX=y
CONFIG_LIBAVB_USER=y
// Enable A/B functionality
CONFIG_ANDROID_AB=y
```

4.2.2 Partition Table

The A/B System has requirements for the partition table: partitions that support A/B must have suffixes `_a` and `_b`. Example `parameter.txt`:

```
FIRMWARE_VER:8.1
MACHINE_MODEL:RK3326
MACHINE_ID:007
MANUFACTURER: RK3326
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3326
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
CMDLINE:
mtdparts=rk29xxnand:0x00002000@0x00004000(uboot_a),0x00002000@0x00006000(uboot_b),0x00002000@0x00008000(trust_a),0x00002000@0x0000a000(trust_b),0x00001000@0x0000c000(misc),0x00001000@0x0000d000(vbmeta_a),0x00001000@0x0000e000(vbmeta_b),0x00020000@0x0000e000(boot_a),0x00020000@0x0002e000(boot_b),0x00100000@0x0004e000(system_a),0x00300000@0x0032e000(system_b),0x00100000@0x0062e000(vendor_a),0x00100000@0x0072e000(vendor_b),0x00002000@0x0082e000(oem_a),0x00002000@0x00830000(oem_b),0x0010000@0x00832000(factory),0x00008000@0x842000(factory_bootloader),0x00080000@0x008ca000(oem),-@0x0094a000(userdata)
```

4.3 ANDROID BCB

BCB (Bootloader Control Block) is an Android mechanism designed for interacting with the bootloader to control the system boot flow. The data structure is defined at offset 16 KB or 0 in the `misc` partition.

Data structure:

```
struct android_bootloader_message {
    char command[32];
    char status[32];
    char recovery[768];

    /* The 'recovery' field used to be 1024 bytes.  It has only ever
     * been used to store the recovery command line, so 768 bytes
     * should be plenty.  We carve off the last 256 bytes to store the
     * stage string (for multistage packages) and possible future
     * expansion. */
    char stage[32];

    /* The 'reserved' field used to be 224 bytes when it was initially
     * carved off from the 1024-byte recovery field. Bump it up to
     * 1184-byte so that the entire bootloader_message struct rounds up
     * to 2048-byte. */
    char slot_suffix[32];
    char reserved[1184 - 32];
};
```

`command`: boot command. Currently the following three are supported:

Parameter	Function
bootonce-bootloader	Boot into U-Boot fastboot

Parameter	Function
boot-recovery	Boot into recovery
boot-fastboot	Boot into recovery fastboot mode (`fastbootd`)

4.4 AVB Secure Boot

AVB stands for Android Verified Boot, a firmware verification flow designed by Google, mainly used to verify firmware such as boot and system. Rockchip Secure Boot draws on both the verification method used in secure communication and AVB to implement a complete Secure Boot solution.

4.4.1 Feature

- Security verification
- Integrity verification
- Anti-rollback protection
- Persistent partition support
- Chained partitions support. The signing private key may be the same as the one used for boot/system, or it may be kept separately by the OEM, but it must still be signed by the PRK

4.4.2 Configuration

Enabling AVB requires OP-TEE support. Taking `rk3572` as an example:

```
CONFIG_TEE=y
CONFIG_OPTEE=y
CONFIG_ROCKCHIP_OPTEE_V3=y
CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION=y  // Security data is stored in the security partition by default, and RPMB can also be used
```

OP-TEE has two versions. Refer to the relevant `defconfig` for the version used by a given SoC:

- `CONFIG_ROCKCHIP_OPTEE_V2`
- `CONFIG_ROCKCHIP_OPTEE_V3`

AVB-related configuration:

```
// Build the AVB and A/B system libraries
CONFIG_LIBAVB=y
CONFIG_LIBAVB_AB=y
CONFIG_LIBAVB_ATX=y
CONFIG_LIBAVB_USER=y
CONFIG_AVB_VERIFY=y
// Enable A/B system and AVB verify
CONFIG_ANDROID_AB=y
CONFIG_AVB_VERIFY=y
```

```
CONFIG_LIBAVB_VBMETA_PUBLIC_KEY_VALIDATE=y
// Enable strict unlock verification. A certificate is then required to unlock the device.
CONFIG_LIBAVB_ATH_UNLOCK_SUPPORT=y
// Enable the following if CPUID is needed as the challenge number
CONFIG_MISC=y
CONFIG_ROCKCHIP_EFUSE=y
CONFIG_ROCKCHIP_OTP=y
```

4.4.3 Reference

Because AVB involves many details, refer to the Advanced Principles chapter for the remaining principles and configuration.

4.5 Cmdline

`cmdline` is one of the important ways U-Boot passes parameters to the kernel, for example to pass boot media type, device state, and so on. Currently cmdline parameters come from multiple sources. U-Boot concatenates them, filters duplicate items, and then passes them to the kernel. In the U-Boot stage the cmdline is stored in the `bootargs` environment variable.

U-Boot ultimately passes the cmdline by modifying `/chosen/bootargs` in the kernel DTB.

4.5.1 Data Sources

- `/chosen/bootargs` in the kernel DTS, for example:

```
chosen {
    bootargs = "earlyprintk=uart8250,mmio32,0xff30000 swiotlb=1 console=ttyFIQ0
            androidboot.baseband=N/A androidboot.veritymode=enforcing
            androidboot.hardware=rk30board androidboot.console=ttyFIQ0
            init=/init kpti=0";
};
```

- U-Boot dynamically appends content according to current runtime state, for example:

```
storagemedia=emmc androidboot.mode=emmc ......
```

- The headers of `boot.img` / `recovery.img` usually also contain cmdline fields.

4.5.2 Meaning of Data

Below are the meanings of commonly used cmdline parameters on RK platforms. For more details, refer to the kernel document `Documentation/admin-guide/kernel-parameters.txt`.

- `sdfwupdate`: SD upgrade card flag, required by recovery
- `root=PARTUUID`: specify the UUID of the rootfs(system) partition, supported only by GPT

- `skip_initramfs`: the kernel does not use the ramdisk loaded by U-Boot, but the ramdisk in rootfs(system)
- `storagemedia`: storage boot type
- `console`: kernel console configuration
- `earlycon`: specify UART and its configuration before the serial node is established
- `loop.max_part`: maximum partition count supported by each loop device
- `rootwait`: used when the file system is temporarily unavailable, for example when eMMC has not finished initialization. Without `rootwait`, you may encounter `mount rootfs failed`; with this parameter, U-Boot can wait for the driver to finish loading, then read rootfs from the storage device and mount it, thereby avoiding boot failure
- `ro/rw`: mount rootfs read-only or read-write
- `firmware_class.path`: firmware path, such as wifi, bt, gpu
- `dm="lroot none 0, 0 4096 linear 98:3 0, 4096 4096 linear 98:32" root=/dev/dm-0`: Will boot to a rw dm-linear target of 8192 sectors split across two block devices identified by their major:minor numbers. After boot, udev will rename this target to `/dev/mapper/lroot` (depending on the rules). No UUID was assigned. Reference: https://android.googlesource.com/kernel/common/+/android-3.18/Documentation/device-mapper/boot.txt
- `androidboot.slot_suffix`: specify which slot the kernel should boot from in an A/B system
- `androidboot.serialno`: provide the serial number to the kernel and upper layers, such as adb
- `androidboot.verifiedbootstate`: reports the firmware verification state to upper layers:
    1. `green`: locked state and verification key not set by end user
    2. `yellow`: locked state and verification key set by end user
    3. `orange`: unlocked state
- `androidboot.hardware`: boot device identifier, such as `rk30board`
- `androidboot.verifymode`: actual mode/state used for partition verification
- `androidboot.selinux`: SELinux is a MAC (Mandatory Access Control) security system based on the domain-type model. There are three modes:
    1. `enforcing`: SELinux is running and actively enforcing domain/type restrictions
    2. `permissive`: SELinux is running but only outputs warning messages without actually restricting domain/type access. This mode can be used for SELinux debugging
    3. `disabled`: SELinux is not operational
- `androidboot.mode`: Android boot mode:
    1. `normal`: normal boot
    2. `charger`: when the device is powered on via a charger after being powered off, `androidboot.mode` is set to `charger`. This value is written to the `bootargs` environment variable after U-Boot detects the charging state
- `androidboot.wificountrycode`: Wi-Fi country code such as `US`, `CN`
- `androidboot.baseband`: baseband setting, RK does not support it, so set to `N/A`
- `androidboot.console`: Android console output configuration
- `androidboot.vbmeta.device=PARTUUID`: vbmeta storage location

- `androidboot.vbmeta.hash_alg`: vbmeta hash algorithm, such as `sha512`
- `androidboot.vbmeta.size`: vbmeta size
- `androidboot.vbmeta.digest`: vbmeta digest passed to the kernel. The kernel loads vbmeta, calculates the digest, and compares it with this value
- `androidboot.vbmeta.device_state`: AVB 2.0 lock/unlock state

4.6 DTBO/DTO

To make this section easier to understand, first refer to the Appendix chapter to clarify the following terms: DTB, DTBO, DTC, DTO, DTS, FDT.

Their relationships can be described as follows:

- DTS is a file used to describe FDT
- DTS can be compiled by DTC into DTB/DTBO
- DTB and DTBO can be merged through DTO operations into a new DTB

Users often casually use "DTBO" to refer to the "DTO" action itself. To avoid confusion, the following convention is used here:

- DTO: verb, meaning the overlay action
- DTBO: noun, meaning the secondary DTB used for overlay

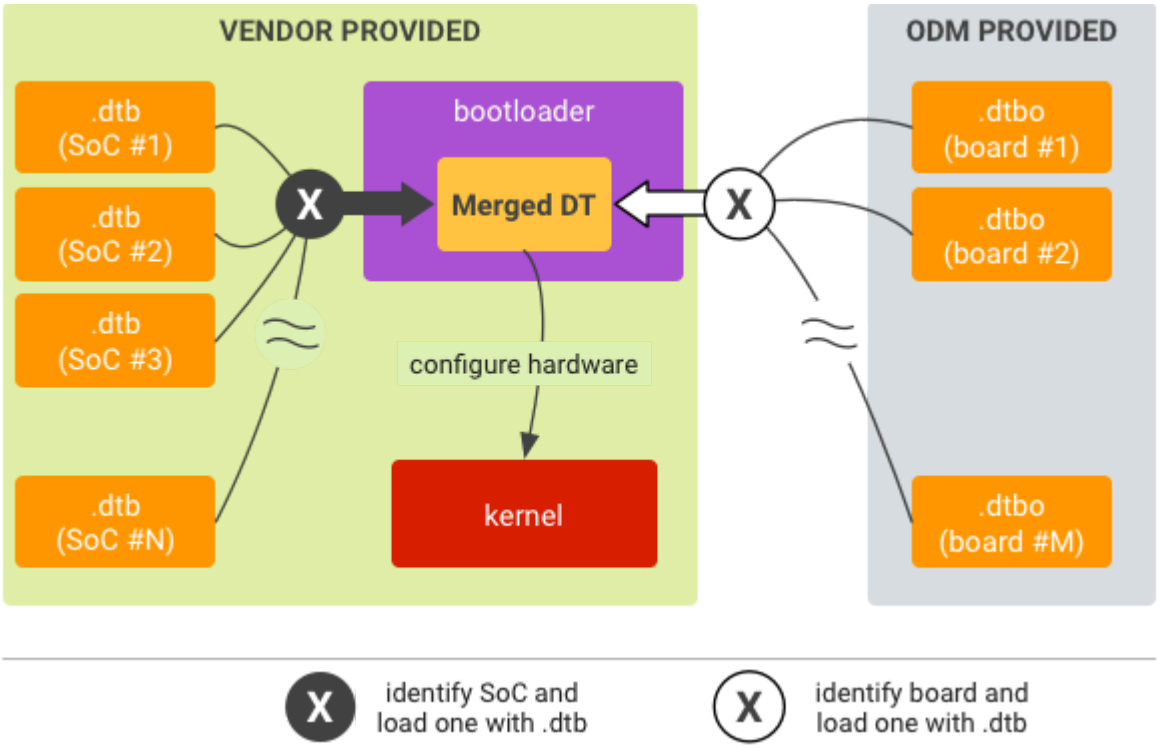
For more knowledge, refer to:

https://source.android.google.cn/devices/architecture/dto

4.6.1 Principle Introduction

DTO (Device Tree Overlay) was introduced after Android P and is mandatory. It allows a secondary device tree blob (DTBO) to be overlaid onto an existing primary device tree blob. DTO can maintain the SoC device tree and dynamically overlay device-specific device tree content, thereby adding nodes and modifying properties of existing nodes.

The primary DTB is usually provided by the vendor, while the secondary DTBO can be provided by ODM/OEM vendors. The bootloader merges them before passing the result to the kernel.



> Image source: https://source.android.google.cn/devices/architecture/dto

Note: DTB/DTBO compilation for DTO differs from ordinary DTB compilation and has special syntax requirements:

When using `dtc` to compile `.dts`, you must add the `-@` option so that the generated `.dtbo` contains the `__symbols__` node. The `__symbols__` node contains a list of all labeled nodes, which the DTO library can use as references.

Example:

1. Compile the main `.dts`:

```
dtc -@ -O dtb -o my_main_dt.dtb my_main_dt.dts
```

2. Compile the overlay `.dts`:

```
dtc -@ -O dtb -o my_overlay_dt.dtbo my_overlay_dt.dts
```

4.6.2 Enable DTO

1. Enable configuration:

```
CONFIG_CMD_DTIMG=y
CONFIG_OF_LIBFDT_OVERLAY=y
```

2. Implement `board_select_fdt_index()`. This is a `__weak` function that users can re-implement according to actual needs. It is used to choose which DTBO is used among multiple DTBOs to perform the DTO operation. The default weak implementation returns index `0`.

```
/*
```

```
 * Default return index 0.
 */
__weak int board_select_fdt_index(ulong dt_table_hdr)
{
    /*
     * User can use "dt_for_each_entry(entry, hdr, idx)" to iterate
     * over all dt entry of DT image and pick up which they want.
     *
     * Example:
     *  struct dt_table_entry *entry;
     *  int index;
     *
     *  dt_for_each_entry(entry, dt_table_hdr, index) {
     *
     *      .... (use entry)
     *  }
     *
     *  return index;
     */
    return 0;
}
```

4.6.3 DTO Result

1. After DTO finishes, the result can be seen in the U-Boot boot log:

```
// Success
ANDROID: fdt overlay OK

// Failure
ANDROID: fdt overlay failed, ret=-19
```

In most cases failure is caused by incompatibility between the primary and secondary device tree blobs, so users need to understand their generation syntax and compatibility.

2. After DTO succeeds, the following is appended to the kernel cmdline to indicate which DTBO was used:

```
androidboot.dtbo_idx=1  // idx starts from 0, meaning DTBO with idx=1 was selected
```

3. After DTO succeeds, users can inspect the DTB content with the `fdt` command in the U-Boot command line to confirm whether the changes took effect.

4.7 ENV

4.7.1 Framework Support

ENV is a very important data management mechanism in the U-Boot framework. It uses a hash table to manage mappings between keys and values and supports add/delete/modify/query operations. Usually the keys and values it manages are collectively called environment variables. U-Boot supports storing ENV data in various media such as NOWHERE/eMMC/FLASH/EEPROM/NAND/SPI_FLASH/UBI...

Configuration:

```
// Default: ENV stored in memory
CONFIG_ENV_IS_NOWHERE

// ENV stored in different media
CONFIG_ENV_IS_IN_EEPROM
CONFIG_ENV_IS_IN_FAT
CONFIG_ENV_IS_IN_EXT4
CONFIG_ENV_IS_IN_FLASH
CONFIG_ENV_IS_IN_MMC
CONFIG_ENV_IS_IN_NAND
CONFIG_ENV_IS_IN_NVRAM
CONFIG_ENV_IS_IN_REMOTE
CONFIG_ENV_IS_IN_SPI_FLASH
CONFIG_ENV_IS_IN_FRAGMENT

// Any storage medium connected to the BLK framework (except mmc), recommended on RK
CONFIG_ENV_IS_IN_BLK
```

Framework code:

```
./env/nowhere.c
./env/blk.c
./env/mmc.c
./env/nand.c
./env/eeprom.c
./env/embedded.c
./env/ext4.c
./env/fat.c
./env/flash.c
......
```

4.7.2 Related Interfaces

```
// Get environment variables
char *env_get(const char *varname);
ulong env_get_ulong(const char *name, int base, ulong default_val);
ulong env_get_hex(const char *varname, ulong default_val);

// Modify or create environment variables. value=NULL is equivalent to delete
```

```
int env_set(const char *varname, const char *value);
int env_set_ulong(const char *varname, ulong value);
int env_set_hex(const char *varname, ulong value);

// Load all ENV information from storage
int env_load(void);

// Save all current ENV information to storage
int env_save(void);
```

- `env_load()`: users do not need to call it; the U-Boot framework calls it automatically at the appropriate stage
- `env_save()`: users call it when needed to save all ENV information to the medium specified by `CONFIG_ENV_IS_XXX`

4.7.3 Advanced Interfaces

RK provides two advanced unified ENV interfaces that support create, append, and replace operations. They are mainly intended for handling `bootargs`, but they are also applicable to other environment variables.

```
/**
 * env_update() - update sub value of an environment variable
 *
 * This add/append/replace the sub value of an environment variable.
 *
 * @varname: Variable to adjust
 * @valude: Value to add/append/replace
 * @return 0 if OK, 1 on error
 */
int env_update(const char *varname, const char *varvalue);

/**
 * env_update_filter() - update sub value of an environment variable but
 * ignore some key word
 *
 * This add/append/replace/igore the sub value of an environment variable.
 *
 * @varname: Variable to adjust
 * @valude: Value to add/append/replace
 * @ignore: Value to be ignored that in varvalue
 * @return 0 if OK, 1 on error
 */
int env_update_filter(const char *varname, const char *varvalue, const char *ignore);
```

Rules of `env_update()`:

- Create: if `varname` does not exist, create both `varname` and `varvalue`
- Append: if `varname` exists and `varvalue` does not exist, append `varvalue`
- Replace: if both exist, replace the existing one with the new one. For example, if

`storagemedia=emmc` already exists and the new `varvalue` is `storagemedia=rknand`, the result will be `storagemedia=rknand`

`env_update_filter()` is an extended version of `env_update()` that also strips a keyword from `varvalue` while updating.

Special note: both `env_update()` and `env_update_filter()` split ENV content by spaces and `=`, so the operation unit is either a single word or a `"key=value"` pair. Long strings cannot be handled as one whole unit.

- Single word: `sdfwupdate`, etc.
- `"key=value"` pair: `storagemedia=emmc`, `init=/init`, `androidboot.console=ttyFIQ0`, etc.
- These two interfaces cannot handle long string units. For example, `console=ttyFIQ0 androidboot.baseband=N/A androidboot.selinux=permissive` cannot be operated as one whole unit.

4.7.4 Storage Location

`env_save()` can save ENV into storage. On RK platforms, the ENV offset and size are defined as follows:

```
config ENV_OFFSET
   hex "Environment offset"
   depends on ENV_IS_IN_EEPROM || ENV_IS_IN_MMC || ENV_IS_IN_NAND || \
         ENV_IS_IN_SPI_FLASH || ENV_IS_IN_BLK || ENV_IS_IN_FRAGMENT
   default 0x0 if ENV_IS_IN_FRAGMENT
   default 0x3f8000
   help
     Offset from the start of the device (or partition).

     This offset may be interpreted differently depending on the chosen
     ENV_IS_IN_* options. For example, for ENV_IS_IN_MMC=y, this offset may
     be negative to indicate an offset backwards from the end of the
     partition. See the relevant help messages for more details.

config ENV_SIZE
   hex "Environment Size"
   default 0x8000
   help
     Size of the environment storage area
```

- Normally, `ENV_OFFSET` and `ENV_SIZE` are not recommended to be modified.

4.7.5 Common Options

Common storage media include eMMC/sdmmc/Nandflash/Norflash. Native U-Boot Nand/Nor ENV drivers use the MTD framework, but RK-supported storage is connected through the BLK framework, so those native ENV drivers cannot be used directly.

Therefore RK provides `CONFIG_ENV_IS_IN_BLK` for storage attached to the BLK framework:

- For eMMC/sdmmc, still use `CONFIG_ENV_IS_IN_MMC`
- For Nand/Nor, use `CONFIG_ENV_IS_IN_BLK`

Before using it, read the `CONFIG_ENV_IS_IN_BLK` definition in Kconfig.

```
// Already assigned by default, no need to modify
CONFIG_ENV_OFFSET
CONFIG_ENV_SIZE

// Usually not needed
CONFIG_ENV_OFFSET_REDUND (optional)
```

Regardless of which `CONFIG_ENV_IS_IN_XXX` is selected, read the Kconfig description first.

4.7.6 fw_printenv Tool

`fw_printenv` is the ENV tool that U-Boot provides for Linux. With it, users can access and modify ENV content from Linux. It requires that the ENV region be located in a kernel-visible partition, preferably a dedicated partition. In essence, it accesses the ENV region through storage nodes exposed by the kernel.

Tool generation:

```
./make.sh env
```

Generated output:

```
./tools/env/fw_printenv     // ENV read/write tool
./tools/env/fw_env.config   // ENV configuration file
./tools/env/README          // Tool documentation
```

For usage, refer to the README.

4.8 Fastboot

Fastboot is an Android-provided way to interact with U-Boot over USB. It is generally used to obtain device information and flash firmware.

4.8.1 Configuration Options

```
// Enable
CONFIG_FASTBOOT
CONFIG_FASTBOOT_FLASH
CONFIG_USB_FUNCTION_FASTBOOT
CONFIG_FASTBOOT_FLASH
CONFIG_FASTBOOT_OEM_BOARD
CONFIG_FASTBOOT_OEM_BOARD_UNLOCK

// Parameters
CONFIG_FASTBOOT_BUF_ADDR
CONFIG_FASTBOOT_BUF_SIZE
CONFIG_FASTBOOT_FLASH_MMC_DEV
CONFIG_FASTBOOT_USB_DEV
```

4.8.2 Trigger Methods

Fastboot uses the Google adb VID/PID by default. It can be triggered in the following ways:

- Kernel command line: `reboot fastboot`
- U-Boot command line: `fastboot usb 0`
- Press and hold the key combination during boot: `ctrl+f`

4.8.3 Fastboot commands

In U-Boot, enter fastboot with:

```
fastboot usb 0
```

4.8.3.1 Command Quick Reference

```
fastboot flash < partition > [ < filename > ]
fastboot erase < partition >
fastboot getvar < variable > | all
fastboot set_active < slot >
fastboot reboot
fastboot reboot-bootloader
fastboot flashing unlock
fastboot flashing lock
fastboot stage [ < filename > ]
fastboot get_staged [ < filename > ]
fastboot oem format
fastboot oem board:unlock
fastboot oem board:lock
fastboot oem board:at-get-unlock-challenge
fastboot oem board:fuse at-perm-attr
```

```
fastboot oem board:fuse at-rsa-perm-attr
fastboot oem board:fuse at-bootloader-vboot-key
fastboot oem board:init-ab-metadata
```

4.8.3.2 Command Usage

1. `fastboot flash <partition> [<filename>]`

Function: flash a partition.

Example: `fastboot flash boot boot.img`

2. `fastboot erase <partition>`

Function: erase a partition.

Example: `fastboot erase boot`

3. `fastboot getvar <variable> | all`

Function: get device information.

Example: `fastboot getvar all` (get all device information)

The `variable` parameter also supports the following values:

```
version                     /* fastboot version */
version-bootloader              /* U-Boot version */
version-baseband                /* baseband version */
filesize                    /* size of the most recently downloaded file */
downloadsize/max-download-size      /* maximum single transfer size supported by fastboot */
serialno                    /* product name/model */
product                     /* baseband version */
platform                    /* hardware platform */
slot-count                  /* number of slots */
has-slot: < partition >         /* check whether the partition name exists in the slot */
current-slot                /* currently booted slot */
slot-suffixes               /* slots available on the current device */
slot-successful: < _a | _b >        /* check whether a slot has booted successfully */
slot-unbootable: < _a | _b >        /* check whether a slot is marked unbootable */
slot-retry-count: < _a | _b >       /* check the retry count of a slot */
logical-block-size              /* logical block size */
erase-block-size                /* erase block size */
vboot-state                 /* verified boot state */
unlocked                    /* device lock state */
flash-unlocked                  /* flash lock state */
partition-type : < partition >      /* partition type */
partition-size : < partition >      /* partition size */
snapshot-update-status          /* snapshot update state, used by virtual A/B */
avb-state                   /* AVB state */
```

4. `fastboot set_active <slot>`

Function: set the slot to boot next.

Example: `fastboot set_active _a`

5. `fastboot reboot`

Function: reboot the device and boot normally.

6. `fastboot reboot-bootloader`

Function: reboot the device and enter fastboot mode.

7. `fastboot flashing unlock`

Function: unlock the device and allow flashing firmware.

8. `fastboot flashing lock`

Function: lock the device and forbid flashing.

9. `fastboot stage [<filename>]`

Function: download data into device memory at `CONFIG_FASTBOOT_BUF_ADDR`.

Example: `fastboot stage permanent_attributes.bin`

10. `fastboot get_staged [<filename>]`

Function: fetch staged data from the device.

Example: `fastboot get_staged raw_unlock_challenge.bin`

11. `fastboot oem board:fuse at-perm-attr`

Function: burn `permanent_attributes.bin` and its hash.

Example:

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
```

12. `fastboot oem board:lock`

Function: lock the device.

13. `fastboot oem board:unlock`

Function: unlock the device. Authenticated unlock is currently supported.

Example:

```
fastboot oem board:at-get-vboot-unlock-challenge
fastboot get_staged raw_unlock_challenge.bin

./make_unlock.sh (see `make_unlock.sh` reference)
fastboot stage unlock_credential.bin
fastboot oem board:unlock
```

14. `fastboot oem board:fuse at-bootloader-vboot-key`

Function: burn the bootloader key hash.

Example:

```
fastboot stage bootloader-pub-key.bin
fastboot oem board:fuse at-bootloader-vboot-key
```

15. `fastboot oem format`

Function: reformat partitions according to `$partitions`.

16. `fastboot oem board:at-get-unlock-challenge`

Function: authenticated unlock, need to obtain unlock challenge data.

4.9 FileSystem

4.9.1 Framework Support

FAT and EXT2/4 are common file system formats. FAT usually uses the DOS (MBR) partition table, and typical devices include SD cards and USB disks.

These two file systems are commonly accessed in U-Boot.

FAT configuration:

```
CONFIG_DOS_PARTITION=y
CONFIG_FS_FAT=y
CONFIG_FAT_WRITE=y
CONFIG_FS_FAT_MAX_CLUSTSIZE=65536
CONFIG_CMD_FAT=y
CONFIG_CMD_FS_GENERIC=y
```

FAT commands:

```
fatinfo fatload fatls fatsize fatwrite
```

EXT2/4 configuration:

```
CONFIG_CMD_EXT2=y
CONFIG_CMD_EXT4=y
CONFIG_CMD_FS_GENERIC=y
```

EXT2/4 commands:

```
ext2load ext2ls ext4load ext4ls ext4size
```

4.9.2 Related Interfaces

FAT header file `./include/fat.h`:

```
int file_fat_detectfs(void);
int fat_exists(const char *filename);
```

```
int fat_size(const char *filename, loff_t *size);
int file_fat_read_at(const char *filename, loff_t pos, void *buffer,
          loff_t maxsize, loff_t *actread);
int file_fat_read(const char *filename, void *buffer, int maxsize);
int fat_set_blk_dev(struct blk_desc *rbdd, disk_partition_t *info);
int fat_register_device(struct blk_desc *dev_desc, int part_no);

int file_fat_write(const char *filename, void *buf, loff_t offset, loff_t len,
         loff_t *actwrite);
int fat_read_file(const char *filename, void *buf, loff_t offset, loff_t len,
        loff_t *actread);
int fat_opendir(const char *filename, struct fs_dir_stream **dirsp);
int fat_readdir(struct fs_dir_stream *dirs, struct fs_dirent **dentp);
void fat_closedir(struct fs_dir_stream *dirs);
void fat_close(void);
```

EXT2/4 header file `include/ext4fs.h`:

```
int ext4fs_init(void);
void ext4fs_deinit(void);
int ext4fs_filename_unlink(char *filename);
int ext4fs_write(const char *fname, unsigned char *buffer,
       unsigned long sizebytes);
int ext4_write_file(const char *filename, void *buf, loff_t offset, loff_t len,
          loff_t *actwrite);
#endif
#if defined(CONFIG_CMD_EXT4_SPARSE_WRITE)
int ext4_unsparse(struct blk_desc *desc, const u8 *buf, ulong start);
#endif
struct ext_filesystem *get_fs(void);
int ext4fs_open(const char *filename, loff_t *len);
int ext4fs_read(char *buf, loff_t offset, loff_t len, loff_t *actread);
int ext4fs_mount(unsigned part_length);
void ext4fs_close(void);
void ext4fs_reinit_global(void);
int ext4fs_ls(const char *dirname);
int ext4fs_exists(const char *filename);
int ext4fs_size(const char *filename, loff_t *size);
void ext4fs_free_node(struct ext2fs_node *node, struct ext2fs_node *currroot);
int ext4fs_devread(lbaint_t sector, int byte_offset, int byte_len, char *buf);
void ext4fs_set_blk_dev(struct blk_desc *rbdd, disk_partition_t *info);
long int read_allocated_block(struct ext2_inode *inode, int fileblock);
int ext4fs_probe(struct blk_desc *fs_dev_desc,
       disk_partition_t *fs_partition);
int ext4_read_file(const char *filename, void *buf, loff_t offset, loff_t len,
         loff_t *actread);
int ext4_read_superblock(char *buffer);
int ext4fs_uuid(char *uuid_str);
#endif
```

4.9.3 Command Examples

```
// Confirm that the SD card is recognized (for a USB disk, use usb commands, usually usb 0)
=> mmc dev 1
switch to partitions #0, OK
mmc1 is current device

// Show information
=> fatinfo mmc 1
Interface:  MMC
  Device 1: Vendor: Man 000003 Snr e81ec501 Rev: 1.9 Prod: SC16G
  Type: Removable Hard Disk
  Capacity: 15193.5 MB = 14.8 GB (31116288 x 512)
Filesystem: FAT32 "NO NAME    "

// List files
=> fatls mmc 1
            System Volume Information/
        23   hello.txt
        23   linux.txt

2 file(s), 1 dir(s)

// Read the size of hello.txt (result is saved to the filesize variable by default)
=> fatsize mmc 1 hello.txt
=> echo $filesize
0x17

// Read hello.txt to address 0x2000000
=> fatload mmc 1 0x2000000 hello.txt
reading hello.txt
23 bytes read in 2 ms (10.7 KiB/s)
// View the content of the loaded hello.txt
=> md.l 0x2000000
02000000: 6c6c6568 65682d6f 2d6f6c6c 6c6c6568    hello-hello-hell
02000010: 65682d6f ff6f6c6c ffffffff ffffffff    o-hello.........

// Create a new file: hello-copy.txt. Write the content from address 0x2000000~0x2000017 into hello-copy.txt
=> fatwrite mmc 1 0x2000000 hello-copy.txt 0x17
writing hello-copy.txt
23 bytes written
// See the new file: hello-copy.txt
=> fatls mmc 1
            System Volume Information/
        23   hello.txt
        23   linux.txt
        23   hello-copy.txt

3 file(s), 1 dir(s)
```

Note: ext2/4 commands are used similarly to FAT commands, so they are not described separately here.

4.10 HW-ID DTB

RK U-Boot supports detecting GPIO or ADC hardware states and dynamically loading different kernel DTBs. This is temporarily referred to as the HW-ID DTB (Hardware ID DTB) feature.

4.10.1 Design Principle

Hardware revisions and components such as displays and Wi-Fi modules are often updated. If each hardware version required a separate software build, maintenance would be inconvenient. Therefore the HW_ID mechanism allows one software image to support multiple hardware versions.

For different hardware versions, software must provide corresponding DTB files and unique ADC/GPIO values to characterize the current hardware version.

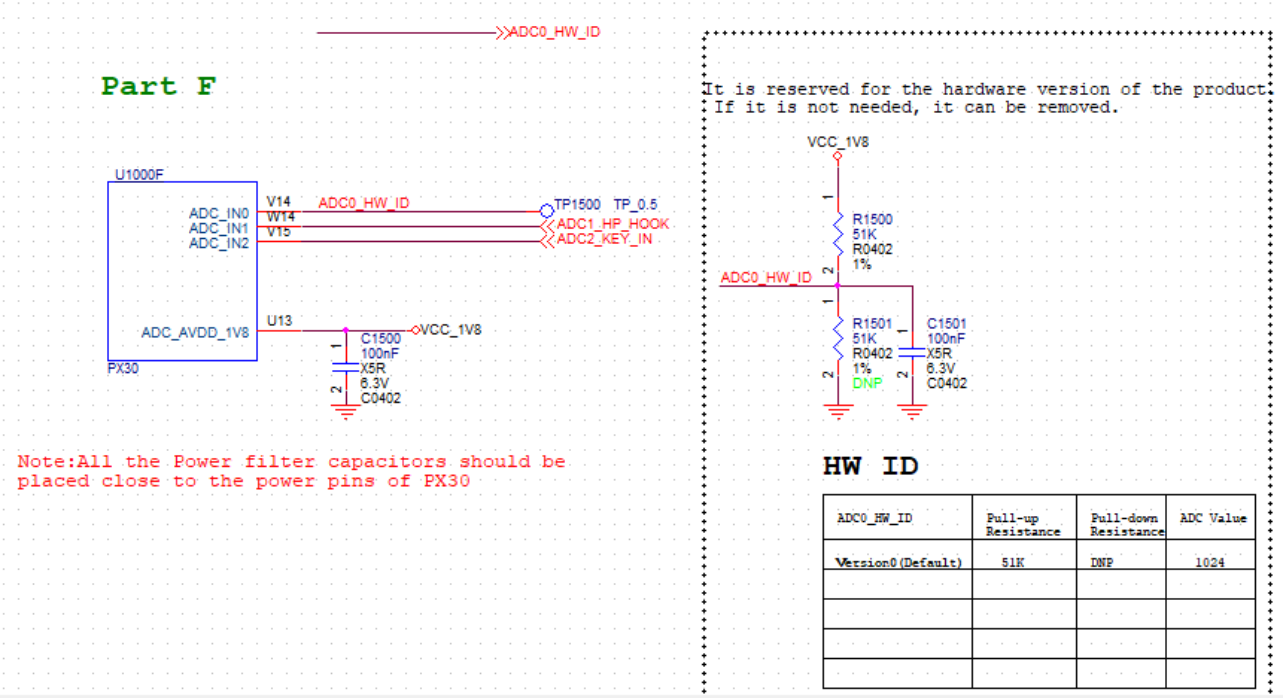
Users package all these DTB files for different hardware versions into the same `resource.img`. When U-Boot boots the kernel, it detects the hardware ID and picks the matching DTB from `resource.img`.

4.10.2 Hardware Reference

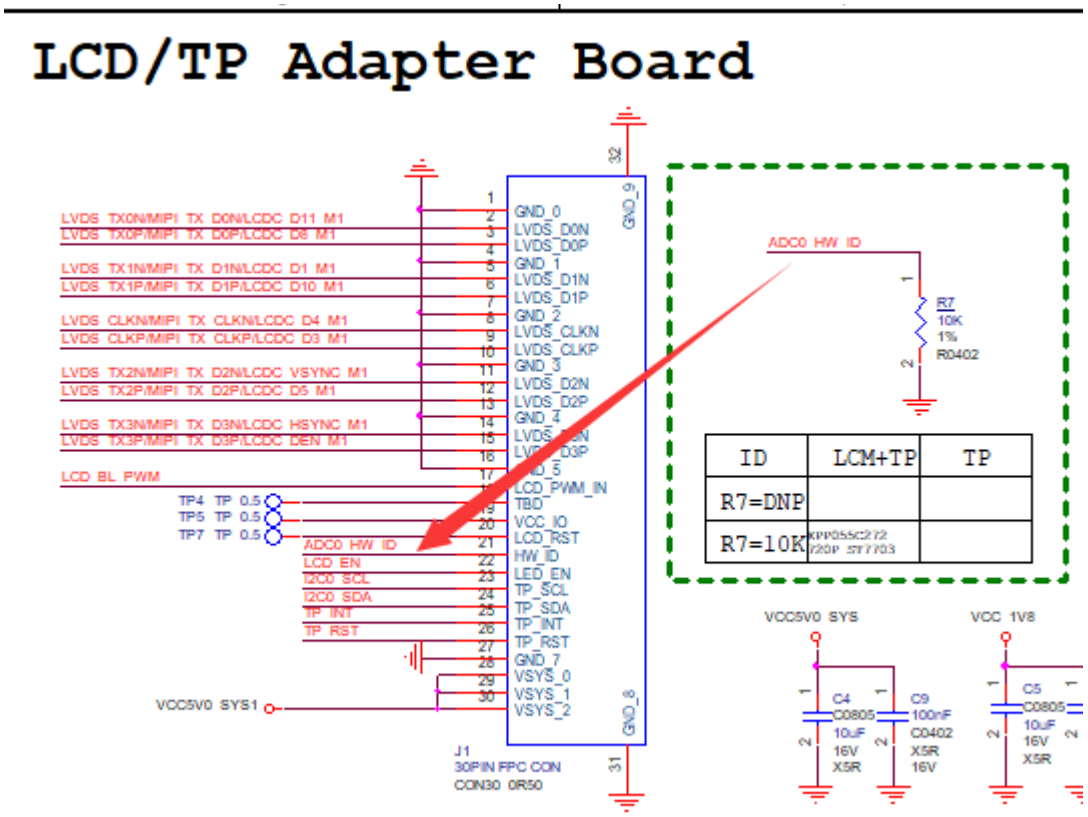
Currently two methods are supported for identifying the hardware version: ADC and GPIO.

ADC reference design

Reserved voltage-divider resistors on RK3326-EVB/PX30-EVB produce different ADC values, which can be used to distinguish hardware versions:



The matching MIPI display daughterboard provides another pull-down resistor:



Different MIPI panels use different resistor values, which work together with the EVB mainboard to determine a unique ADC parameter value.

The V1 version ADC calculation method: the ADC parameter maximum value is 1024, corresponding to the ADC_IN0 pin being pulled up directly to the supply voltage of 1.8V. The MIPI display daughterboard has a 10K pull-down resistor. When connected to the EVB board, ADC = 1024 * 10K / (10K + 51K) = 167.8.

GPIO reference design

There is currently no standard GPIO hardware reference design. Users can customize it themselves.

4.10.3 DTB Naming

Users need to reflect ADC/GPIO unique hardware ID information in the DTB filename.

ADC as HW_ID DTB:

- Filename ends with `.dtb`
- HW_ID format: `#[controller]_ch[channel]=[adcval]`, which is regarded as one complete unit

  `[controller]`: ADC controller node name in the DTS.

  `[channel]`: ADC channel.

  `[adcval]`: ADC center value; the actual valid range is `adcval +/- 30`.
- Each unit must be lowercase and contain no spaces
- Multiple units are separated by `#`, with at most 10 units

Examples:

```
rk3326-evb-lp3-v10#saradc_ch2=111#saradc_ch1=810.dtb
rk3326-evb-lp3-v10#_saradc_ch2=569.dtb
```

GPIO as HW_ID DTB:

- Filename ends with `.dtb`
- HW_ID format: `#gpio[pin]=[level]`

  `[pin]`: GPIO pin, e.g., `0a2` represents gpio0a2.

  `[level]`: GPIO pin level.
- Each unit must be lowercase and contain no spaces
- Multiple units are separated by `#`, with at most 10 units

Example:

```
rk3326-evb-lp3-v10#gpio0a2=0#gpio0c3=1.dtb
```

4.10.4 DTB Packaging

Kernel repository: `scripts/mkmultidtb.py`. This script can package multiple DTBs into the same `resource.img`.

Users need to edit the script and add the DTB files to the `DTBS` dictionary together with corresponding ADC/GPIO configuration information.

```
...
DTBS = {}
DTBS['PX30-EVB'] = OrderedDict([('rk3326-evb-lp3-v10', '#_saradc_ch0=166'),
            ('px30-evb-ddr3-lvds-v10', '#_saradc_ch0=512')])
...
```

The above example shows that running `scripts/mkmultidtb.py PX30-EVB` generates a `resource.img` containing 3 DTBs:

- `rk-kernel.dtb`: the default RK DTB, not included in the dictionary above. When no DTB matches, this is used by default. The packaging script uses the first DTB in the DTBS dictionary as the default DTB.
- `rk3326-evb-lp3-v10#_saradc_ch0=166.dtb`: an RK3326 DTB file with ADC information.
- `px30-evb-ddr3-lvds-v10#_saradc_ch0=512.dtb`: a PX30 DTB file with ADC information.

If you manually run `scripts/mkmultidtb.py`, you also need to manually replace the generated `resource.img` into `boot.img`. For convenience, users can add the following call so that the kernel build automatically includes the multi-DTB `resource.img`:

```
diff --git a/scripts/mkimg b/scripts/mkimg
index 1d895602935..5e1355b9da8 100755
--- a/scripts/mkimg
+++ b/scripts/mkimg
@@ -248,6 +248,7 @@ if [ "${srctree}" != "${objtree}" ]; then
 fi
 scripts/resource_tool ${DTB_PATH} ${LOGO} ${LOGO_KERNEL} >/dev/null
 echo "  Image:  resource.img (with ${DTB} ${LOGO} ${LOGO_KERNEL}) is ready"
+./scripts/mkmultidtb.py PX30-EVB

 if [ -f "${BOOT_IMG}" ]; then
        if file -L -p -b ${BOOT_IMG} | grep -q 'Device Tree Blob' ; then
```

4.10.5 Enable Feature

Configuration:

```
CONFIG_ROCKCHIP_HWID_DTB=y
```

Driver code:

```
./arch/arm/mach-rockchip/resource.c // implementation: rockchip_read_hwid_dtb()
```

DTS configuration:

If GPIO is used for hardware identification, the corresponding pinctrl and gpio nodes must be kept in `rkxx-u-boot.dtsi`. ADC is enabled by default.

For example, using gpio0 and gpio1 for identification:

```
...

&pinctrl {
    bootph-all;  // Add this property to keep the node in the U-Boot DTB. Same below.
};

&gpio0 {
    bootph-all;
};

&gpio1 {
    bootph-all;
};

...
```

4.10.6 Load Result

```
.....
mmc0(part 0) is current device
boot mode: None
DTB: rk3326-evb-lp3-v10#_saradc_ch0=166.dtb // prints the matched DTB, otherwise defaults to "rk-kernel.dtb"
Using kernel dtb
.....
```

4.11 SD and USB Disk

This section mainly introduces firmware boot and upgrade from SD cards and USB disks on RK platforms.

4.11.1 Mechanism Principle

After boot cards and upgrade cards are created, fixed tags are written at a fixed storage offset in the firmware header to indicate whether the card is a boot card or an upgrade card. When U-Boot recognizes this tag, it enters the corresponding boot or upgrade flow.

- Boot card: the card contains one complete firmware package, and U-Boot directly uses it to boot the system
- Upgrade card: the card contains two firmware sets. When creating the card, the PC tool writes one set containing only the partition images required to enter recovery mode (firmware A), and another full `update.img` package (firmware B). U-Boot uses firmware A to boot into recovery, and recovery then uses firmware B to complete the upgrade

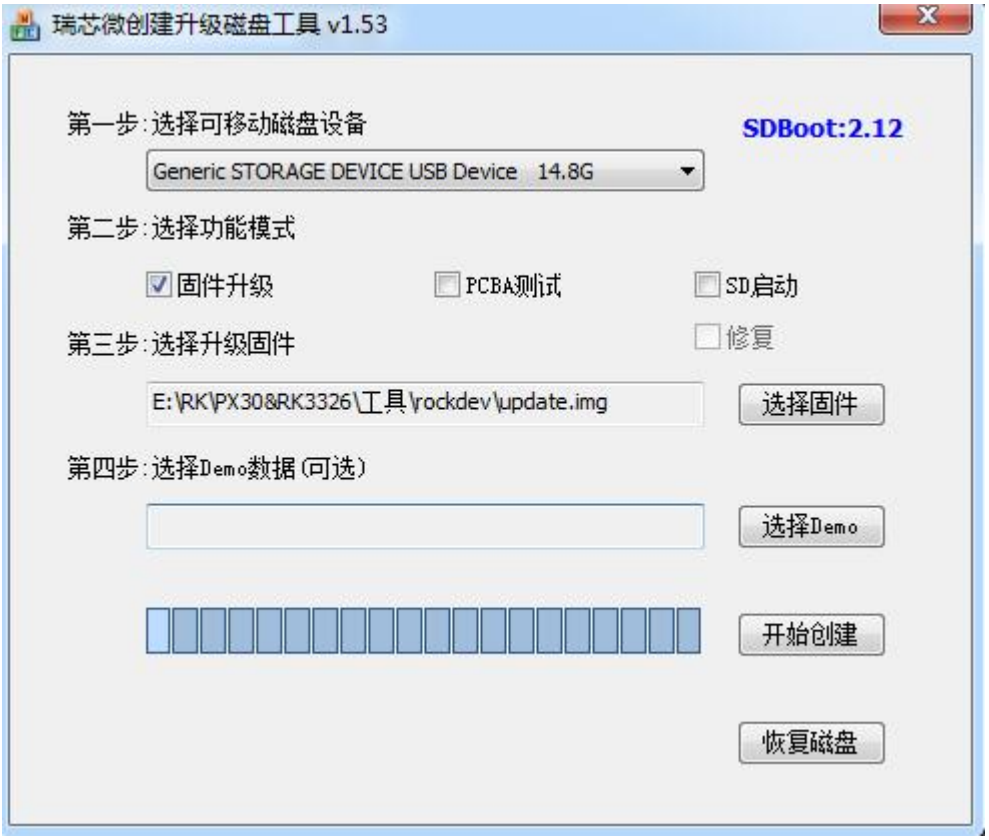
Special note:

- SD card boot/upgrade is supported starting from BootROM
- USB disk boot/upgrade is supported only from the U-Boot stage, so U-Boot itself must already work normally

4.11.2 Firmware Creation

The process for creating SD or USB disk boot cards and upgrade cards is identical and requires only two steps:

- Use the tool under `RKTools/linux/Linux_Pack_Firmware/rockdev/` in the SDK to generate `update.img`
- Use `SDDiskTool` to write `update.img` to the SD card or USB disk:
    - Select the removable disk
    - Select `Firmware Upgrade` or `SD Boot`
    - Click `Start Creating`



4.11.3 SD Configuration

SD boot/upgrade is enabled by default in the SDK-released U-Boot for each platform. No extra configuration is needed.

4.11.4 USB Configuration

USB disk boot/upgrade is not enabled by default in SDK-released U-Boot, because native U-Boot USB scanning is time-consuming. Users can enable it as needed:

1. Flash a complete upgrade firmware package to local storage and verify it boots normally
2. Plug in the USB disk, boot into the U-Boot command line, and run `usb start` and `usb info` to confirm the USB disk is recognized
3. Copy the kernel DTB used in step 1 and rename it to `kern.dtb`, then place it in `./dts/` of U-Boot. It will be packaged into `uboot.img`.

   > The purpose of `kern.dtb`: when the kernel DTB in the local storage partition is corrupted, U-Boot uses `kern.dtb` to ensure USB can be initialized properly.

4. Enable USB boot/upgrade in U-Boot:

   ```
   CONFIG_ROCKCHIP_USB_BOOT=y
   ```

   Then rebuild and flash `uboot.img`.

   > If the process reports that the U-Boot firmware is too large to be packaged, it is because step 3 added `kern.dtb`. Please trim some unused U-Boot configurations first.

4.11.5 Feature Validation

To confirm that SD or USB disk boot/upgrade is working:

Users can erase key partitions such as `kernel`, `resource`, `boot`, `recovery` in local storage (eMMC, Nand, etc.) and verify that inserting the SD/USB disk still allows the system to enter the kernel.

4.11.6 Notes

- USB disk initialization calls `usb start`, which is relatively time-consuming
- To support GPT partition tables on boot/upgrade cards, `SDDiskTool` version must be `>= v1.59`
- To support the A/B system on boot/upgrade cards, `SDDiskTool` version must be `>= v1.61`
- If the SDK in use is relatively old, besides adding the patches above separately, it is recommended to confirm with the recovery engineer whether recovery also needs related patches

5. Chapter-5 Driver Modules

5.1 AMP

5.1.1 Implementation Idea

The native U-Boot framework does not provide AMP (Asymmetric Multi-Processing) support. RK implements its own AMP mechanism, allowing different CPUs to run different firmware.

Implementation overview:

1. Firmware packaging

   All AMP firmware except Linux is described in an `its` file, including CPU runtime state and firmware metadata, and is finally packaged into a FIT-format `amp.img` flashed into the `amp` partition.

   At boot, U-Boot loads `amp.img` and performs `sha256` integrity verification. Trust then sets the runtime state for each CPU and dispatches execution to the corresponding entry address.

2. Boot order

   The CPU running U-Boot is called the primary core. After it finishes state switching and firmware jumps for the other cores, it handles itself last.

3. Resource management

   U-Boot does not coordinate resources among the firmware images in an AMP solution, including memory and interrupts. Developers must ensure this themselves.

4. Trust cooperation

   AMP requires Trust support. If the user-specified CPU state is the default state, the SDK Trust already supports it. If a non-default state is used, extra Trust support is required, although some platform SDKs already support it by default.

> Default CPU states:
>
> 32-bit chip default: `arch = "arm", thumb = <0>, hyp = 0;`
>
> 64-bit chip default: `arch = "arm64",  thumb = <0>, hyp = 1;`

5. Linux + AMP combination
    1. For compatibility, Linux-related firmware in the combined solution remains consistent with traditional SMP firmware, so the firmware form is: Linux-related firmware + `amp.img`
    2. Developers can add a `linux` node in the AMP its file to specify the CPU state used for Linux
    3. If the primary core runs Linux or no firmware is specified, after booting other AMP firmware the primary core starts Linux using the traditional SMP flow, driven by U-Boot
    4. If Linux runs on a non-primary core, Linux is started first and then the other AMP firmware is started. If the desired primary Linux core is not CPU0, special Trust support is required
    5. Linux firmware load addresses are determined only by U-Boot configuration, for example

`rk3568_common.h`

5.1.2 Framework Support

Configuration:

```
CONFIG_AMP
CONFIG_ROCKCHIP_AMP
```

Framework code:

```
./drivers/cpu/rockchip_amp.c
```

its template:

```
./drivers/cpu/amp.its
```

Packaging tool:

```
./tools/mkimage  // Automatically generated after a full U-Boot build
```

5.1.3 Enable Feature

1. To build `amp.img`, create an `its` file based on `drivers/cpu/amp.its`:

   The following its example: CPU1/2/3 run AMP, CPU0 runs Linux, and CPU3 is the primary core. Boot order: CPU0 => CPU1/2 => CPU3.

```
/dts-v1/;
/ {
    description = "FIT source file for rockchip AMP";
    #address-cells = <1>;

    // All AMP firmware (excluding Linux) must be specified in the images node;
    images {
        amp1 {
            description  = "bare-metal-core1";       // Required: description
            data         = /incbin/("./amp1.bin");    // Required: amp1 firmware
            type         = "firmware";  // Required: do not change
            compression  = "none";      // Required: do not change
            arch         = "arm";       // Required: "arm64" for 64-bit, "arm" for 32-bit
            cpu          = <0x100>;     // Required: CPU hardware ID (mpidr)
            thumb        = <0>;         // Required: 0: arm or thumb2; 1: pure thumb
            hyp          = <0>;         // Required: 0: el1/svc; 1: el2/hyp
            load         = <0x01800000>;// Required: firmware load and run address
            udelay       = <1000000>;   // Optional: delay after booting the current CPU before booting the next
            hash {                      // Required: do not change
                algo = "sha256";
            };
```

```
        };

        amp2 {
            description  = "bare-metal-core2";
            data         = /incbin/("./amp2.bin");
            type         = "firmware";
            compression  = "none";
            arch         = "arm";
            cpu          = <0x200>;
            thumb        = <0>;
            hyp          = <0>;
            load         = <0x03800000>;
            udelay       = <1000000>;
            hash {
                algo = "sha256";
            };
        };

        amp3 {
            description  = "bare-metal-core3";
            data         = /incbin/("./amp3.bin");
            type         = "firmware";
            compression  = "none";
            arch         = "arm";
            cpu          = <0x300>;
            thumb        = <0>;
            hyp          = <0>;
            load         = <0x05800000>;
            udelay       = <1000000>;
            hash {
                algo = "sha256";
            };
        };
    };

    configurations {
        default = "conf";
        conf {
            description = "Rockchip AMP images";
            rollback-index = <0x0>;
            // Specify the firmware to be loaded and its load/boot order,
            // but the primary core is not constrained by this order.
            loadables = "amp1", "amp2", "amp3";

            signature {
                algo = "sha256,rsa2048";
                padding = "pss";
                key-name-hint = "dev";
                sign-images = "loadables";
            };

            // Specify the CPU runtime state for Linux:
            // (1) Only the udelay attribute is optional;
            // (2) The boot address is not configurable and is determined
            //     by the U-Boot platform configuration file, e.g.: rk3568_common.h;
```

```
        linux {
            description  = "linux-os";
            arch         = "arm64";
            cpu          = <0x000>;    // CPU0 runs Linux
            thumb        = <0>;
            hyp          = <0>;
            udelay       = <1000000>;
        };
    };
  };
};
```

Description:

- description: description information.
- type: default is "firmware".
- compression: default is "none".
- data: firmware path, relative to the amp.its location.
- arch: CPU 32/64 mode. ARMv7 can only be set to "arm"; ARMv8 can be set to "arm64" or "arm", representing AArch64 or AArch32 respectively.
- cpu: CPU hardware ID, i.e., mpidr (Multiprocessor Affinity Register), the lower 32 bits. For example:

```
cpus {
    #address-cells = <2>;
    #size-cells = <0>;

    cpu0: cpu@0 {
        device_type = "cpu";
        compatible = "arm,cortex-a55";
        reg = <0x0 0x0>;    // mpidr
        ......
    };

    cpu1: cpu@100 {
        device_type = "cpu";
        compatible = "arm,cortex-a55";
        reg = <0x0 0x100>;  // mpidr
        ......
    };
    ......
};
```

- thumb: CPU instruction mode. Set to 1 for pure thumb, otherwise 0.
- hyp: CPU hypervisor mode.
- load: firmware load and run address.
- udelay: delay after boot completes (optional), in us. After booting the current CPU, wait for the specified delay before booting the next CPU.
- loadables: AMP firmware to be loaded and its load/boot order. The primary CPU is always booted last and is not constrained by this order.

- linux node: used for Linux + AMP combination. Refer to the "Implementation Idea" section for details.

2. Package firmware:

```
// 0xe00 is the firmware header size and is not recommended to be changed
./tools/mkimage -f ./drivers/cpu/amp.its -E -p 0xe00 amp.img
```

> A full U-Boot build is required to auto-generate the mkimage tool.

3. Add an `amp` partition in the partition table

Add an "amp" partition in the `parameter.txt` partition table file, then flash `amp.img`.

> U-Boot directly loads the entire amp partition content into memory. It is recommended to configure the amp partition size according to actual needs.

4. Bring up

The U-Boot framework automatically brings up all AMP firmware at the appropriate time. The following boot log shows CPU3 as the primary core running AMP firmware and CPU0/1/2 running Linux firmware:

```
......
// Primary core loads amp3 firmware
### Loading loadables from FIT Image at 7bdbcf80 ...
  Trying 'amp3' loadables subimage
    Description:  rtthread
    Type:         Firmware
    Compression:  uncompressed
    Data Start:   0x7bdbdd80
    Data Size:    311296 Bytes = 304 KiB
    Architecture: ARM
    Load Address: 0x01800000
    Hash algo:    sha256
    Hash value:   d08db937e4d7bd4125056239154bb30d44a2fcca9e70aa8dea448fabda4838d5
  Verifying Hash Integrity ... sha256+ OK
  Loading loadables from 0x7bdbdd80 to 0x01800000
### Booting FIT Image FIT: No fit blob
FIT: No FIT image
ANDROID: reboot reason: "(none)"
optee api revision: 2.0
TEEC: Waring: Could not find security partition
Not AVB images, AVB skip
ANDROID: Hash OK

// Primary core loads Linux firmware
Booting IMAGE kernel at 0x03880000 with fdt at 0x0a100000...

Fdt Ramdisk skip relocation
### Booting Android Image at 0x0387f800 ...
Kernel load addr 0x03880000 size 21655 KiB
### Flattened Device Tree blob at 0x0a100000
  Booting using the fdt blob at 0x0a100000
  XIP Kernel Image from 0x03880000 to 0x03880000 ... OK
 'reserved-memory' ramoops@110000: addr=110000 size=f0000
```

```
  Using Device Tree in place at 000000000a100000, end 000000000a12322a
vp1 adjust cursor plane from 0 to 1
vp0, plane_mask:0x2a, primary-id:5, curser-id:1
vp1 adjust cursor plane from 1 to 0
vp1, plane_mask:0x15, primary-id:4, curser-id:0
vp2, plane_mask:0x0, primary-id:0, curser-id:-1
Adding bank(fixed): 0x03880000 - 0x80000000 (size: 0x7c780000)

// At this point, all firmware has been loaded. The following begins
// starting each CPU according to the priority rules of the solution.

// Primary core starts CPU0 to run Linux (CPU1/2 are subsequently started by Linux)
AMP: Brought up cpu[0] with state 0x12, entry 0x03880000 ...OK
// Primary core starts itself (CPU3) to run AMP firmware
AMP: Brought up cpu[300, self] with state 0x10, entry 0x01800000 ...OK

// Linux running on CPU0:
[    0.000000] Booting Linux on physical CPU 0x0000000000 [0x412fd050]
[    0.000000] Linux version 4.19.193 (stevenliu@stevenliu) (gcc version 6.3.1 20170404 (Linaro GCC 6.3-2017.05), GNU ld (Linaro_Binutils-2017.05) 2.27.0.20161019) #5 SMP Mon Sep 13 16:22:51 CST 2021
[    0.000000] Machine model: Rockchip RK3568 EVB1 DDR4 V10 Board
......
```

The above print messages will vary depending on the user's its configuration. Refer to actual output.

5.2 Button

5.2.1 Framework Support

Button drivers use the `button-uclass` framework and standard interfaces.

Configuration:

```
CONFIG_BUTTON=y
CONFIG_BUTTON_ADC=y
CONFIG_BUTTON_GPIO=y
CONFIG_BUTTON_RK8XX=y
```

Framework code:

```
drivers/button/button-uclass.c
```

Driver code:

```
drivers/button/button-adc.c
drivers/button/button-gpio.c
drivers/button/button-rk8xx.c
```

`pwrkey` is recognized only by interrupt. Other GPIO buttons are recognized by polling.

5.2.2 Related Interfaces

Interfaces:

```
bool button_is_on(u32 code)
enum button_state_t button_get_state(struct udevice *dev);
```

`code` definition:

```
/include/dt-bindings/input/linux-event-codes.h
```

Return values:

```
enum button_state_t {
    BUTTON_OFF = 0,
    BUTTON_ON = 1,
    BUTTON_ON_HOLD = 2,
    BUTTON_COUNT,
};
```

The default duration of `KEY_PRESS_LONG_DOWN` is 2000 ms and is currently used only for long-press events of `pwrkey` in U-Boot charging.

```
#define KEY_LONG_DOWN_MS    2000
```

5.3 Charge

5.3.1 Framework Support

Native U-Boot does not support charging. RK implements its own charging framework.

Charging involves multiple modules: Display, PMIC, fuel gauge, charging animation, pwrkey, LED, low-power CPU suspend, timer, and so on.

Supported fuel gauges include:

```
RK809/RK816/RK817/RK818/cw201/...
```

Configuration:

```
// Framework
CONFIG_DM_CHARGE_DISPLAY
CONFIG_CHARGE_ANIMATION
CONFIG_DM_FUEL_GAUGE
// Drivers
CONFIG_POWER_FG_CW201X
CONFIG_POWER_FG_RK818
CONFIG_POWER_FG_RK817
CONFIG_POWER_FG_RK816
......
```

Charging framework:

```
./drivers/power/charge-display-uclass.c
```

Charging animation driver:

```
// Responsible for managing the entire charging process, including
// getting battery level, charge type, button events, and initiating
// low-power suspend, etc.
./drivers/power/charge_animation.c
```

Fuel-gauge framework:

```
./drivers/power/fuel_gauge/fuel_gauge_uclass.c
```

Fuel-gauge drivers:

```
./drivers/power/fuel_gauge/fg_rk818.c
./drivers/power/fuel_gauge/fg_rk817.c   // reused by rk809
./drivers/power/fuel_gauge/fg_rk816.c
......
```

Logic flow:

```
charge-display-uclass.c
   => charge_animation.c
      => fuel_gauge_uclass.c
         => fg_rkxx.c
```

5.3.2 Package Images

Charging images are located in `./tools/images/` and must be packed into `resource.img` before the charging framework can display them.

The `resource.img` built from the kernel does not include charging images by default, so they must be packed separately in U-Boot:

```
$ ls tools/images/
battery_0.bmp  battery_1.bmp  battery_2.bmp  battery_3.bmp  battery_4.bmp  battery_5.bmp
battery_fail.bmp
```

Pack command:

```
./scripts/pack_resource.sh <input resource.img>
```

Pack information:

```
./scripts/pack_resource.sh  /home/cjh/3399/kernel/resource.img

Pack ./tools/images/ & /home/guest/3399/kernel/resource.img to resource.img ...
Unpacking old image(/home/guest/3399/kernel/resource.img):
rk-kernel.dtb logo.bmp logo_kernel.bmp
Pack to resource.img succeeded!
Packed resources:
rk-kernel.dtb battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp battery_5.bmp battery_fail.bmp
logo.bmp logo_kernel.bmp battery_0.bmp

resource.img is packed ready
```

After success, a `resource.img` containing the images will be generated in the U-Boot root directory. Confirm the content using the `hd` command:

```
hd resource.img | less

00000000  52 53 43 45 00 00 00 00  01 01 01 00 0a 00 00 00  |RSCE............|
00000010  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
......
*
00000400  45 4e 54 52 62 61 74 74  65 72 79 5f 31 2e 62 6d  |ENTRbattery_1.bm| // image 1
00000410  70 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |p...............|
00000420  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000500  00 00 00 00 4d 00 00 00  9c 18 00 00 00 00 00 00  |....M...........|
00000510  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000600  45 4e 54 52 62 61 74 74  65 72 79 5f 32 2e 62 6d  |ENTRbattery_2.bm| // image 2
00000610  70 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |p...............|
00000620  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
......
```

5.3.3 DTS Configuration

DTS charge node:

```
charge-animation {
    compatible = "rockchip,uboot-charge";
```

```
    status = "okay";

    rockchip,uboot-charge-on = <0>;          // Enable/disable U-Boot charging
    rockchip,android-charge-on = <1>;        // Enable/disable Android charging

    rockchip,uboot-exit-charge-level = <5>;    // Minimum battery level to allow boot during U-Boot charging
    rockchip,uboot-exit-charge-voltage = <3650>;// Minimum voltage to allow boot during U-Boot charging
    rockchip,screen-on-voltage = <3400>;       // Minimum voltage to turn on the screen during U-Boot
charging

    rockchip,uboot-low-power-voltage = <3350>;  // Minimum voltage to unconditionally force charging
mode

    rockchip,system-suspend = <1>;           // Whether to enter Trust low-power standby when the screen is
off (requires ATF support)
    rockchip,auto-off-screen-interval = <20>;   // Auto screen-off timeout in seconds, default 15s
    rockchip,auto-wakeup-interval = <10>;      // Suspend auto-wakeup time in seconds. If value is 0 or this
                                // property is absent, suspend auto-wakeup is disabled,
                                // typically used for stress testing
    rockchip,auto-wakeup-screen-invert = <1>;   // Whether to turn on/off screen during suspend auto-
wakeup
};
```

5.3.4 System Suspend

`pwrkey` behavior:

- Short press toggles the screen on/off. When the screen is off, the system enters low-power mode
- Long press can boot into the system

Two low-power modes are supported through `rockchip,system-suspend = <VAL>`:

- `VAL = 0`: CPU WFI mode. Peripherals are not handled; only the CPU enters low-power mode
- `VAL = 1`: system suspend mode, which requires ATF/OPTEE support. Similar to kernel deep standby, the entire SoC enters standby

Minimum ATF/OPTEE version required to support U-Boot low-power standby: **refer to the Platform Definition chapter**.

5.3.5 Replace Images

1. Replace images in `./tools/images/` with 8-bit or 24-bit BMP files, use `ls | sort` to confirm the image order is from low battery to high battery, then use `pack_resource.sh` to pack the images into `resource.img`
2. Modify the image-to-battery-level mapping in `./drivers/power/charge_animation.c`

```
/*
 * IF you want to use your own charge images, please:
 *
 * 1. Update the following 'image[]' to point to your own images;
 * 2. You must set the failed image as last one and soc = -1 !!!
```

```
 */
static const struct charge_image image[] = {
    { .name = "battery_0.bmp", .soc = 5, .period = 600 },
    { .name = "battery_1.bmp", .soc = 20, .period = 600 },
    { .name = "battery_2.bmp", .soc = 40, .period = 600 },
    { .name = "battery_3.bmp", .soc = 60, .period = 600 },
    { .name = "battery_4.bmp", .soc = 80, .period = 600 },
    { .name = "battery_5.bmp", .soc = 100, .period = 600 },
    { .name = "battery_fail.bmp", .soc = -1, .period = 1000 },
};

// @name: image name;
// @soc: battery level corresponding to the image;
// @period: image refresh time (unit: ms);
// Note: The last image must be the fail image, and "soc=-1" must not be changed !!
```

5.3.6 Charging LEDs

Actual products have varying LED control requirements, so the charging framework only supports 2 LEDs: charging LED and full-charge LED:

- Charging LED: the LED toggles only when the battery level changes during charging
- Full-charge LED: the LED is lit only when the battery level reaches 100%

The above 2 LEDs are just a demo. Users should modify the code according to their own requirements.

Configuration options:

```
LED_CHARGING_NAME
LED_CHARGING_FULL_NAME
```

These two configuration options are used to specify the LED `label` property. Refer to the Led section.

5.4 Clock

5.4.1 Framework Support

Clock drivers use the `clk-uclass` framework and standard interfaces.

Configuration:

```
CONFIG_CLK
```

Framework code:

```
./drivers/clk/clk-uclass.c
```

Platform driver code:

```
./drivers/clk/rockchip/...
```

5.4.2 Related Interfaces

```
// Request clock
int clk_get_by_index(struct udevice *dev, int index, struct clk *clk);
int clk_get_by_name(struct udevice *dev, const char *name, struct clk *clk);

// Enable/disable clock
int clk_enable(struct clk *clk);
int clk_disable(struct clk *clk);

// Configure/get frequency
ulong (*get_rate)(struct clk *clk);
ulong (*set_rate)(struct clk *clk, ulong rate);

// Configure/get phase
int (*get_phase)(struct clk *clk);
int (*set_phase)(struct clk *clk, int degrees);
```

5.4.3 Clock Initialization

There are three classes of clock initialization. The cru node information is listed first for convenience:

```
cru: clock-controller@ff2b0000 {
    compatible = "rockchip,px30-cru";
    ......
    assigned-clocks =
        <&pmucru PLL_GPLL>, <&pmucru PCLK_PMU_PRE>,
        <&pmucru SCLK_WIFI_PMU>, <&cru ARMCLK>,
        <&cru ACLK_BUS_PRE>, <&cru ACLK_PERI_PRE>,
        <&cru HCLK_BUS_PRE>, <&cru HCLK_PERI_PRE>,
        <&cru PCLK_BUS_PRE>, <&cru SCLK_GPU>;
    assigned-clock-rates =
        <1200000000>, <100000000>,
        <26000000>, <600000000>,
        <200000000>, <200000000>,
        <150000000>, <150000000>,
        <100000000>, <200000000>;
    ......
}
```

Class 1: Basic platform clock initialization: `rkclk_init()`

Each platform's cru driver probe calls `rkclk_init()` to complete PLL/CPU/bus frequency initialization. Frequencies are defined in `cru_rkxxx.h`. For example RK3399:

```
#define APLL_HZ      (600 * MHz)
#define GPLL_HZ      (800 * MHz)
```

```
#define CPLL_HZ       (384 * MHz)
#define NPLL_HZ       (600 * MHz)
#define PPLL_HZ       (676 * MHz)
#define PMU_PCLK_HZ    ( 48 * MHz)
#define ACLKM_CORE_HZ   (300 * MHz)
#define ATCLK_CORE_HZ   (300 * MHz)
#define PCLK_DBG_HZ    (100 * MHz)
#define PERIHP_ACLK_HZ  (150 * MHz)
#define PERIHP_HCLK_HZ  ( 75 * MHz)
#define PERIHP_PCLK_HZ  (37500 * KHz)
#define PERILP0_ACLK_HZ (300 * MHz)
#define PERILP0_HCLK_HZ (100 * MHz)
#define PERILP0_PCLK_HZ ( 50 * MHz)
#define PERILP1_HCLK_HZ (100 * MHz)
#define PERILP1_PCLK_HZ ( 50 * MHz)
......
```

Class 2: Secondary platform clock initialization: `clk_set_defaults()`

Each platform's cru driver probe may call `clk_set_defaults()` to parse and configure the frequencies specified by `assigned-clocks/assigned-clock-parents/assigned-clock-rates` in the cru node (i.e., re-configure frequencies), but excluding the ARM frequency. The ARM frequency is only re-configured when `set_armclk_rate()` is implemented. Refer to the CPU Frequency Boost section below for details.

In addition to the cru, peripherals with requirements can also proactively call `clk_set_defaults()` in their own probe, such as vop and gmac.

Class 3: Per-module clock initialization: `clk_set_rate()`

Most peripheral modules call `clk_set_rate()` to configure their own frequencies.

5.4.4 CPU Frequency Boost

Current CPU frequency-boost support on each platform: **refer to the Platform Definition chapter**. Based on the implementation mechanism, there are three categories:

Category 1: CPU uses APLL

CPU boot frequency boost implementation flow:

- Step 1: Specify the ARM target frequency in the `assigned-clocks` of the cru node
- Step 2: The cru driver probe calls `clk_set_defaults()` to obtain (but not configure) the ARM target frequency from Step 1
- Step 3: Implement `set_armclk_rate()` to set the ARM target frequency obtained from Step 2. Some platforms with requirements have already implemented this by default. Other platforms can refer to existing implementations and add it as needed, for example: `arch\arm\mach-rockchip\px30\px30.c`

```
int set_armclk_rate(void)
{
   struct px30_clk_priv *priv;
   struct clk clk;
   int ret;
```

```
    ret = rockchip_get_clk(&clk.dev);
    if (ret) {
        printf("Failed to get clk dev\n");
        return ret;
    }
    clk.id = ARMCLK;
    priv = dev_get_priv(clk.dev);
    ret = clk_set_rate(&clk, priv->armclk_hz);
    if (ret < 0) {
        printf("Failed to set armclk %lu\n", priv->armclk_hz);
        return ret;
    }
    priv->set_armclk_rate = true;

    return 0;
}
```

- Step 4: Refer to the CPU opp-table frequency-voltage table, and add `regulator-init-microvolt = <...>` in the ARM regulator node to specify the init voltage, ensuring the target frequency and voltage match

Category 2: CPU uses SCMI CLK

For example RK356X, boot frequency boost requires using the SCMI interface to set CPU clock-related parameters.

CPU boot frequency boost implementation flow:

- Step 1: Specify the ARM target frequency in the scmi node's `rockchip,clk-init`
- Step 2: Confirm that U-Boot has SCMI enabled, CONFIG_CLK_SCMI
- Step 3: Implement `set_armclk_rate()` to set the ARM target frequency obtained from the SCMI DTS node. Some platforms with requirements have already implemented this by default. Other platforms can refer to existing implementations and add it as needed, for example: `arch\arm\mach-rockchip\rk3568\rk3568.c`

```
#ifdef CONFIG_CLK_SCMI
#include <dm.h>
/*
 * armclk: 1104M:
 *  rockchip,clk-init = <1104000000>,
 *  vdd_cpu : regulator-init-microvolt = <825000>;
 * armclk: 1416M(by default):
 *  rockchip,clk-init = <1416000000>,
 *  vdd_cpu : regulator-init-microvolt = <900000>;
 * armclk: 1608M:
 *  rockchip,clk-init = <1608000000>,
 *  vdd_cpu : regulator-init-microvolt = <975000>;
 */

int set_armclk_rate(void)
{
    struct clk clk;
    u32 *rates = NULL;
```

```
    int ret, size, num_rates;

    ret = rockchip_get_scmi_clk(&clk.dev);
    if (ret) {
        printf("Failed to get scmi clk dev\n");
        return ret;
    }

    size = dev_read_size(clk.dev, "rockchip,clk-init");
    if (size < 0)
        return 0;

    num_rates = size / sizeof(u32);
    rates = calloc(num_rates, sizeof(u32));
    if (!rates)
        return -ENOMEM;

    ret = dev_read_u32_array(clk.dev, "rockchip,clk-init",
                rates, num_rates);
    if (ret) {
        printf("Cannot get rockchip,clk-init reg\n");
        return -EINVAL;
    }
    clk.id = 0;
    ret = clk_set_rate(&clk, rates[clk.id]);
    if (ret < 0) {
        printf("Failed to set armclk\n");
        return ret;
    }
    return 0;
}
#endif
```

- Step 4: Refer to the CPU opp-table frequency-voltage table, and add `regulator-init-microvolt = <...>` in the ARM regulator node to specify the init voltage, ensuring the target frequency and voltage match

> SCMI: Refer to CH17-Appendix for details.

Category 3: CPU uses SCMI CLK

The difference from Category 2 is: only Step 4 needs to be executed. The CPU frequency will automatically increase as the voltage is raised.

5.4.5 Clock Tree

U-Boot does not provide full clock-tree management. RK platforms add `soc_clk_dump()` to print simplified clock information. For example:

```
CLK: (sync kernel. arm: enter 1200000 KHz, init 1200000 KHz, kernel 800000 KHz)
  apll 800000 KHz
  dpll 392000 KHz
  cpll 1000000 KHz
  gpll 1188000 KHz
  npll 24000 KHz
  ppll 100000 KHz
  hsclk_bus 297000 KHz
  msclk_bus 198000 KHz
  lsclk_bus 99000 KHz
  msclk_peri 198000 KHz
  lsclk_peri 99000 KHz
```

Meaning of the first line:

- `sync kernel`: the cru driver configured the bus frequencies specified in the kernel cru node via `clk_set_defaults()` (excluding ARM frequency); otherwise it shows as "sync uboot"
- `enter 1200000 KHz`: the ARM frequency when the previous-stage loader entered U-Boot
- `init 1200000 KHz`: the U-Boot ARM initialization frequency, i.e., the frequency defined by APLL_HZ
- `kernel 800000 KHz`: `set_armclk_rate()` is implemented and set the ARM frequency specified by `assigned-clocks` in the kernel cru node; otherwise it shows: "kernel 0N/A"

5.5 Crypto

The Crypto module is mainly used for hardware-level encryption and hashing algorithms.

5.5.1 Framework Support

Framework code:

```
./drivers/crypto/crypto_manager.c
./drivers/crypto/cipher/cipher-uclass.c
./drivers/crypto/hash/hash-uclass.c
./drivers/crypto/hmac/hmac-uclass.c
./drivers/crypto/rsa_mod_exp/mod_exp_uclass.c
```

Driver code:

```
./drivers/crypto/rockchip/
```

5.5.2 Related Interfaces

```
include/u-boot/hmac.h
include/u-boot/hash.h
include/u-boot/cipher.h
include/u-boot/rsa-mod-exp.h
```

For usage, refer to:

```
./cmd/crypto.c
```

5.6 Display

5.6.1 Framework Support

RK U-Boot currently supports display interfaces including RGB, LVDS, EDP, MIPI, HDMI, CVBS, and DP. The displayed logo images come from `logo.bmp` and `logo_kernel.bmp` in the kernel root directory and are packed into `resource.img`.

Image requirements:

- BMP in `BI_RGB 8bpp/16bpp/24bpp/32bpp` and `BI_RLE4/BI_RLE8`
- On chips based on VOP LITE architecture such as RK312X/PX30/RK3308/RV1126/RV1106, hardware mirror is not supported. If the logo appears mirrored in X/Y direction, preprocess the BMP using tools such as Photoshop or ffmpeg

Configuration:

```
CONFIG_DM_VIDEO
CONFIG_DISPLAY
CONFIG_DRM_ROCKCHIP
CONFIG_DRM_ROCKCHIP_PANEL
CONFIG_DRM_ROCKCHIP_DW_HDMI
CONFIG_DRM_ROCKCHIP_DW_HDMI_QP
CONFIG_DRM_ROCKCHIP_INNO_HDMI
CONFIG_ROCKCHIP_INNO_HDMI_PHY
CONFIG_DRM_ROCKCHIP_INNO_MIPI_PHY
CONFIG_DRM_ROCKCHIP_INNO_VIDEO_PHY
CONFIG_DRM_ROCKCHIP_INNO_VIDEO_COMBO_PHY
CONFIG_DRM_ROCKCHIP_DW_MIPI_DSI
CONFIG_DRM_ROCKCHIP_DW_MIPI_DSI2
CONFIG_DRM_ROCKCHIP_DW_DP
CONFIG_DRM_ROCKCHIP_ANALOGIX_DP
CONFIG_DRM_ROCKCHIP_LVDS
CONFIG_DRM_ROCKCHIP_RGB
CONFIG_DRM_ROCKCHIP_RK618
CONFIG_DRM_ROCKCHIP_RK628
CONFIG_DRM_ROCKCHIP_SAMSUNG_MIPI_DCPHY
CONFIG_PHY_ROCKCHIP_SAMSUNG_HDPTX
CONFIG_DRM_ROCKCHIP_TVE
CONFIG_DRM_SII902X
```

Framework code:

```
drivers/video/drm/rockchip_display.c
drivers/video/drm/rockchip_display.h
drivers/video/drm/rockchip_crtc.c
drivers/video/drm/rockchip_crtc.h
```

```
drivers/video/drm/rockchip_connector.c
drivers/video/drm/rockchip_connector.h
drivers/video/drm/rockchip_bridge.c
drivers/video/drm/rockchip_bridge.h
drivers/video/drm/rockchip_panel.c
drivers/video/drm/rockchip_panel.h
drivers/video/drm/inno_mipi_phy.c
drivers/video/drm/inno_video_phy.c
drivers/video/drm/inno_video_combo_phy.c
drivers/video/drm/rockchip-inno-hdmi-phy.c
drivers/video/drm/samsung_mipi_dcphy.c
```

Driver files:

```
vop:
 drivers/video/drm/rockchip_vop.c
 drivers/video/drm/rockchip_vop.h
 drivers/video/drm/rockchip_vop_reg.c
 drivers/video/drm/rockchip_vop_reg.h
 drivers/video/drm/rockchip_vop2.c

rgb:
 drivers/video/drm/rockchip_rgb.c
 drivers/video/drm/rockchip_rgb.h

lvds:
 drivers/video/drm/rockchip_lvds.c
 drivers/video/drm/rockchip_lvds.h

mipi:
 drivers/video/drm/drm_mipi_dsi.c
 drivers/video/drm/dw_mipi_dsi.c
 drivers/video/drm/dw_mipi_dsi2.c

edp:
 drivers/video/drm/analogix_dp.c
 drivers/video/drm/analogix_dp.h
 drivers/video/drm/analogix_dp_reg.c
 drivers/video/drm/analogix_dp_reg.h

hdmi:
 drivers/video/drm/dw_hdmi.c
 drivers/video/drm/dw_hdmi.h
 drivers/video/drm/rockchip_dw_hdmi.c
 drivers/video/drm/rockchip_dw_hdmi.h
 drivers/video/drm/dw_hdmi_qp.c
 drivers/video/drm/dw_hdmi_qp.h
 drivers/video/drm/rockchip_dw_hdmi_qp.c
 drivers/video/drm/rockchip_dw_hdmi_qp.h

cvbs:
 drivers/video/drm/rockchip_tve.c
 drivers/video/drm/rockchip_tve.h

```

```
dp:
 drivers/video/drm/dw-dp.c

bridge:
 drivers/video/drm/rk618.c
 drivers/video/drm/rk618.h
 drivers/video/drm/rk618_lvds.c
 drivers/video/drm/rk628/
 drivers/video/drm/sii902x.c
```

5.6.2 Related Interfaces

```
// Display U-Boot logo and kernel logo:
void rockchip_show_logo(void);

// Display BMP image, currently mainly used for charging image display:
void rockchip_show_bmp(const char *bmp);

// Pass some U-Boot variables to the kernel via DTB.
// Includes kernel logo size, address, format, bcsh/csc configuration,
// CRTC output scan timing and overscan configuration, etc.
void rockchip_display_fixup(void *blob);
```

5.6.3 DTS Configuration

```
reserved-memory {
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;

    drm_logo: drm-logo@00000000 {
        compatible = "rockchip,drm-logo";
        // Reserved buffer for kernel logo storage; the actual
        // address and size will be modified in U-Boot
        reg = <0x0 0x0 0x0 0x0>;
    };
};

&route-edp {
    status = "okay";                    // Enable U-Boot logo display
    logo,uboot = "logo.bmp";            // Specify U-Boot logo image
    logo,kernel = "logo_kernel.bmp";    // Specify kernel logo image
    logo,mode = "center";               // center: centered display; fullscreen: fullscreen display
    logo,rotate = <90>;                 // Rotation angle: 90/180/270
    charge_logo,mode = "center";        // center: centered display; fullscreen: fullscreen display
    connect = <&vopb_out_edp>;          // Determine display route: vopb->edp->panel
};

&edp {
    status = "okay"; // Enable edp
```

```
};

&vopb {
    status = "okay"; // Enable vopb
};

&panel {
    "simple-panel";
    ...
    status = "okay";

    disp_timings: display-timings {
        native-mode = <&timing0>;
        timing0: timing0 {
            ...
        };
    };
};
```

5.6.4 defconfig

Currently, except for platforms with strict boot-time or memory constraints, U-Boot defconfig supports display by default as long as DTS display information is configured.

For RK3308/RV1103/RV1106 and similar platforms, display is disabled by default for boot-time reasons and must be enabled explicitly with the following changes:

```
--- a/configs/evb-rk3308_defconfig
+++ b/configs/evb-rk3308_defconfig
@@ -4,7 +4,6 @@ CONFIG_SYS_MALLOC_F_LEN=0x2000
CONFIG_ROCKCHIP_RK3308=y
CONFIG_ROCKCHIP_SPL_RESERVE_IRAM=0x0
CONFIG_RKIMG_BOOTLOADER=y
-# CONFIG_DM_KERNEL_DTB is not set
CONFIG_TARGET_EVB_RK3308=y
CONFIG_DEFAULT_DEVICE_TREE="rk3308-evb"
CONFIG_DEBUG_UART=y
@@ -55,6 +54,11 @@ CONFIG_USB_GADGET_DOWNLOAD=y
CONFIG_G_DNL_MANUFACTURER="Rockchip"
CONFIG_G_DNL_VENDOR_NUM=0x2207
CONFIG_G_DNL_PRODUCT_NUM=0x330d
+CONFIG_DM_VIDEO=y
+CONFIG_DISPLAY=y
+CONFIG_DRM_ROCKCHIP=y
+CONFIG_DRM_ROCKCHIP_RGB=y
+CONFIG_LCD=y
CONFIG_USE_TINY_PRINTF=y
CONFIG_SPL_TINY_MEMSET=y
CONFIG_ERRNO_STR=y
```

Or enable the corresponding .config configuration:

```
// rk3308
make rk3308_defconfig rk3308-display.config
// rv1103/rv1106
make rv1106_defconfig rv1106-display.config
```

Regarding upstream defconfig configuration:

Upstream maintains a set of Rockchip U-Boot display drivers, currently mainly supporting RK3288 and RK3399:

```
./drivers/video/rockchip/
```

To use this driver set, enable CONFIG_VIDEO_ROCKCHIP and disable CONFIG_DRM_ROCKCHIP. Compared with the display driver currently used by the SDK, the advantages of the latter include:

- More comprehensive platform and display interface support
- HDMI, DP and other display interfaces can output specified resolutions, overscan effects, and display adjustment effects according to user settings
- U-Boot logo can transition smoothly to kernel logo until the system boots up

5.6.5 LOGO Partition

If users need to update the boot logo dynamically, for example from user space, a dedicated `LOGO` partition can be used.

Process:

- Add a dedicated `LOGO` partition to the partition table
- Users can dynamically update images in the `LOGO` partition as needed. When updating, directly write the raw images into the `LOGO` partition without any packaging. If the images in the `LOGO` partition are invalid, the default images from `resource` are used

LOGO Partition Support Status:

If the code only contains the following commit, the LOGO partition only supports 1 image, which can only replace the default `logo.bmp`:

```
1d30bcc rockchip: resource: support parse "logo" partition picture
```

The LOGO partition supports 2 images: image 1 replaces `logo.bmp`, image 2 replaces `logo_kernel.bmp`. The two images are adjacent, with 512-byte alignment between them, and the order must not be changed.

Assuming the images are `logo.bmp` and `logo_kernel.bmp`, the `logo.img` packaging command:

```
cat logo.bmp > logo.img && truncate -s %512 logo.img && cat logo_kernel.bmp >> logo.img
```

Flash the generated `logo.img` to the logo partition. After boot, you will see "LOGO: " prints:

```
U-Boot v5-g042c01531e-210512-dirty #cjh (May 14 2021 - 11:25:03 +0800)
```

```
Model: Rockchip RK3568 Evaluation Board
PreSerial: 2, raw, 0xfe660000
DRAM:  2 GiB
Sysmem: init
Relocation Offset: 7d34f000, fdt: 7b9f8758
Using default environment

dwmmc@fe2b0000: 1, dwmmc@fe2c0000: 2, sdhci@fe310000: 0
Bootdev(atags): mmc 0
MMC0: HS200, 200Mhz
PartType: EFI
boot mode: normal
FIT: No fdt blob
Android 11.0, Build 2021.4, v2
Found DTB in boot part
// The following print indicates that the images in the logo.img partition are correctly recognized.
LOGO: logo.bmp
LOGO: logo_kernel.bmp
DTB: rk-kernel.dtb
HASH(c): OK
......
```

5.6.6 Common Issue Analysis

Q1: If I want X/Y direction mirroring by default, is there a way?

A1: Default Y-direction mirroring is not supported. Default X-direction mirroring for VOP LITE architecture platforms is not supported. For VOP2 architecture platforms such as RK3568 and RK3588, X-direction mirroring can be enabled in DTS to ensure it takes effect from U-Boot:

```
&vp1 {
    xmirror-enable;
};
```

Q2: Can a 4K-resolution BMP logo be supported?

A2: Yes, but the default defconfig on each platform cannot support 4K logo display. The following changes are needed (using rk3576 as an example):

```
diff --git a/drivers/video/drm/rockchip_display.c b/drivers/video/drm/rockchip_display.c
index b1773ba6942..d1606db1ba5 100644
--- a/drivers/video/drm/rockchip_display.c
+++ b/drivers/video/drm/rockchip_display.c
@@ -52,7 +52,7 @@
 #define RK_BLK_SIZE 512
 #define BMP_PROCESSED_FLAG 8399
 #define BYTES_PER_PIXEL sizeof(uint32_t)
-#define MAX_IMAGE_BYTES (8 * 1024 * 1024)
+#define MAX_IMAGE_BYTES (32 * 1024 * 1024)

 DECLARE_GLOBAL_DATA_PTR;
```

```
 static LIST_HEAD(rockchip_display_list);
diff --git a/include/configs/rk3576_common.h b/include/configs/rk3576_common.h
index 16abba314c5..b4a8ec3b898 100644
--- a/include/configs/rk3576_common.h
+++ b/include/configs/rk3576_common.h
@@ -22,7 +22,7 @@
 #endif
 #define CONFIG_SPL_LOAD_FIT_ADDRESS    0x42000000

-#define CONFIG_SYS_MALLOC_LEN          (32 << 20)
+#define CONFIG_SYS_MALLOC_LEN          (32 << 21)
 #define CONFIG_SYS_CBSIZE              1024

 #ifdef CONFIG_SUPPORT_USBPLUG
```

Additionally, CONFIG_DRM_MEM_RESERVED_SIZE_MBYTES needs to be changed to 64 MB.

The reasons for the above changes:

- Due to memory usage and default parameter partition table configurations, using too large a BMP logo is not recommended, so MAX_IMAGE_BYTES is limited to 8 MB. For larger sizes, BI_RLE4/BI_RLE8 format BMP images are recommended
- A typical 4K BI_RGB 24bpp BMP image is approximately 24 MB, and the default malloc heap size on most platforms is 32 MB, which will cause BMP decode-related functions to fail to allocate memory
- To ensure smooth U-Boot to kernel logo transition, the two logo images typically have the same resolution and format, so the memory area reserved for the logo display function (determined by CONFIG_DRM_MEM_RESERVED_SIZE_MBYTES, default 32 MB) is insufficient

5.7 Efuse/Otp

5.7.1 Framework Support

efuse/otp drivers use `misc-uclass.c` and standard interfaces. Typically efuse/otp is divided into secure and non-secure parts:

- U-Boot provides non-secure access
- U-Boot SPL provides access to certain secure OTP regions

Non-secure configuration:

```
CONFIG_MISC
// Choose one; each platform's defconfig already enables the corresponding configuration by default
CONFIG_ROCKCHIP_EFUSE
CONFIG_ROCKCHIP_OTP
```

Secure configuration:

```
CONFIG_SPL_MISC=y
CONFIG_SPL_ROCKCHIP_SECURE_OTP=y
// Disable maskrom upgrade methods; only supported on some platforms
CONFIG_SPL_OTP_DISABLE_SD=y
CONFIG_SPL_OTP_DISABLE_USB=y
CONFIG_SPL_OTP_DISABLE_UART=y
CONFIG_SPL_OTP_DISABLE_SPI2APB=y
// Revoke the RSA public key hash already written to OTP; only supported on some platforms
CONFIG_SPL_REVOKE_PUB_KEY=y
```

To determine whether the above secure features are supported, take rk3572 as an example:

For example, `rk3572_common.h` contains the following definitions:

```
// The following definitions indicate support for disabling maskrom upgrade methods
#define OTP_DISABLE_UPGRADE_ADDR    0x194
#define OTP_DISABLE_USB_VAL     0x3
#define OTP_DISABLE_SD_VAL      0xc
#define OTP_DISABLE_UART_VAL        0x30
#define OTP_DISABLE_SPI2APB_VAL     0xc0
// The following definitions indicate support for revoking the RSA public key hash
// already written to OTP. By default only one revocation is supported.
// Contact RK engineers if multiple revocations are needed.
#define OTP_NEXT_RSA_HASH_ADDR      0x1e0
#define OTP_NEXT_RSA_HASH_SIZE      32
#define OTP_RSA_HASH_REVOKE_VAL     0x3
#define OTP_RSA_HASH_REVOKE_ADDR    0x23
#define OTP_RSA_HASH_REVOKE_SIZE    1
// The following definitions indicate support for reading UDS
// (used by the Android DICE feature)
#define OTP_DICE_UDS_ADDR       0x240
#define OTP_DICE_UDS_SIZE       32
```

Framework code:

```
./drivers/misc/misc-uclass.c
```

Driver code:

```
// non-secure:
./drivers/misc/rockchip-efuse.c
./drivers/misc/rockchip-otp.c
// secure, RK3588 as an example:
./drivers/misc/rk3588-secure-otp.S
```

5.7.2 Related Interfaces

```
int misc_read(struct udevice *dev, int offset, void *buf, int size)
int misc_write(struct udevice *dev, int offset, void *buf, int size)
int misc_ioctl(struct udevice *dev, unsigned long request, void *buf)
// Built-in read-back verification
int misc_otp_write_verify(struct udevice *dev, int offset, const uint8_t *write_buf, int size)
```

5.7.3 DTS Configuration

Take rk3572 as an example:

non-secure: by default the OTP node in the kernel DTB is used. If it is needed in SPL, define it as follows:

```
otp: otp@2a480000 {
    bootph-all;
    compatible = "rockchip,rk3308-otp";
    reg = <0x0 0x2a480000 0x0 0x400>;
};
```

secure: defined in u-boot.dtsi, not recommended to modify.

```
secure-otp@2a400000 {
    bootph-all;
    compatible = "rockchip,rk3572-secure-otp";
    mask_addr = <0x2a420000>;
    reg = <0x0 0x2a400000 0x0 0x4000>;
};
```

5.7.4 Usage Examples

non-secure example:

```
char data[10] = {0};
struct udevice *dev;

/* retrieve the device */
ret = uclass_get_device_by_driver(UCLASS_MISC,
                DM_DRIVER_GET(rockchip_otp), &dev);
if (ret) {
    printf("no OTP device found\n");
    return 0;
}

misc_read(dev, 0x10, &data, 10);
```

secure example:

```
char data[10] = {0};
struct udevice *dev;
int i;

dev = misc_otp_get_device(OTP_S);
if (!dev)
    return -ENODEV;

for (i = 0; i < 10; i++)
    data[i] = i;

misc_otp_write(dev, 0x10, &data, 10);
memset(data, 0, 10);
misc_otp_read(dev, 0x10, &data, 10);
// Recommended: automatically performs read-back verification
misc_otp_write_verify(dev, 0x10, &data, 10);
```

5.7.5 Others

Under U-Boot, secure OTP must be accessed through TEE. For platform support details, refer to `Rockchip OTP Developer Guide`.

5.8 Ethernet

5.8.1 Framework Support

Framework code:

```
./net/*
./drivers/net/*
./drivers/net/phy/*
```

Driver code:

```
./drivers/net/designware.c
./drivers/net/gmac_rockchip.c
./drivers/net/dwc_eth_qos.c
./drivers/net/dwc_eth_qos_rockchip.c
```

menuconfig configuration:

- Driver configuration
  Rockchip has two Ethernet driver solutions. If unsure which driver to use, refer to the corresponding SDK config.

```
// designware:
CONFIG_DM_ETH=y
CONFIG_ETH_DESIGNWARE=y
CONFIG_GMAC_ROCKCHIP=y
```

```
// dwc_eth_qos:
CONFIG_DM_ETH=y
CONFIG_DM_ETH_PHY=y
CONFIG_DWC_ETH_QOS=y
CONFIG_DWC_ETH_QOS_ROCKCHIP=y
```

VENDOR storage must be enabled so U-Boot can preserve the MAC address. This allows the same MAC address to be used on every boot and passed to the kernel, so the kernel uses the same MAC address as U-Boot.

```
-# CONFIG_ROCKCHIP_VENDOR_PARTITION is not set
```

In addition, GMAC uses the `clock-names` property, so remove it from `CONFIG_OF_REMOVE_PROPS`. If the GMAC DTS also uses `assigned-clocks`, remove those properties from `CONFIG_OF_REMOVE_PROPS` as well:

```
-CONFIG_OF_REMOVE_PROPS="clock-names interrupt-parent assigned-clocks assigned-clock-rates assigned-clock-parents"
+CONFIG_OF_REMOVE_PROPS="interrupt-parent"
```

- cmd configuration
  Enable the features as needed.

```
Command line interface  ---> Network commands  --->

[*] bootp, tftpboot
[ ] tftp put
[ ] tftp download and bootm
[ ] tftp download and flash
[ ] tftpsrv
[ ] rarpboot
-*- dhcp
-*- pxe
[ ] nfs
-*- mii
-*- ping
[ ] cdp
[ ] sntp
[ ] dns
[ ] linklocal
[ ] ethsw
```

5.8.2 Related Interfaces

- Data structure initialization interfaces

```
void net_init(void);
int eth_register(struct eth_device *dev);
int phy_init(void);
```

- Device registration interfaces

```
int eth_register(struct eth_device *dev);
int phy_register(struct phy_driver *drv);
```

- Network data read/write and phy read/write
  U-Boot data send/receive requires explicit invocation; neither interrupt nor polling is used. Refer to NetLoop() for the specific implementation.

```
int eth_send(void *packet, int length);
int eth_rx(void);

int phy_read(struct phy_device *phydev, int devad, int regnum);
int phy_write(struct phy_device *phydev, int devad, int regnum, u16 val);
```

5.8.3 DTS Configuration

The DTS node is the same as the kernel. Pay attention to the following board-level property configurations:

- PHY interface configuration (phy-mode)
- PHY reset GPIO and delays (snps,reset-gpio) (snps,reset-delays-us)
- Clock output direction from the SoC (clock_in_out)
- Clock source selection and frequency setting (assigned-clock-parents) (assigned-clock-rates)
- RGMII Delay lines, required for RGMII interfaces (tx_delay) (rx_delay)

```
&gmac {
    phy-mode = "rgmii";
    clock_in_out = "input";

    snps,reset-gpio = <&gpio3 RK_PA0 GPIO_ACTIVE_LOW>;
    snps,reset-active-low;
    /* Reset time is 20ms, 100ms for rtl8211f */
    snps,reset-delays-us = <0 20000 100000>;

    assigned-clocks = <&cru CLK_GMAC_SRC>, <&cru CLK_GMAC_TX_RX>, <&cru
CLK_GMAC_ETHERNET_OUT>;
    assigned-clock-parents = <&cru CLK_GMAC_SRC_M1>, <&cru RGMII_MODE_CLK>;
    assigned-clock-rates = <125000000>, <0>, <25000000>;

```

```
    pinctrl-names = "default";
    pinctrl-0 = <&rgmiim1_pins &clk_out_ethernetm1_pins>;

    tx_delay = <0x2a>;
    rx_delay = <0x1a>;

    phy-handle = <&phy>;
    status = "okay";
};
```

5.8.4 Usage Examples

Commonly used network commands:

- DHCP

```
Usage:
dhcp [loadAddress] [[hostIPaddr:]bootfilename]
```

Using this command eliminates the need to set serverip, ipaddr, and gateway. When DHCP successfully obtains an IP address from the DHCP server, it will fetch a file via TFTP from the hostIPaddr.

100M environment:

```
=> dhcp 0x20000000 192.168.0.100:kernel.img
ethernet@ffc40000 Waiting for PHY auto negotiation to complete. done
BOOTP broadcast 1
DHCP client bound to address 192.168.0.106 (2 ms)
Using ethernet@ffc40000 device
TFTP from server 192.168.0.100; our IP address is 192.168.0.106
Filename 'kernel.img'.
Load address: 0x20000000
Loading: ##################################################################
         ##################################################################
         ##################################################################
         
```

```
done
Bytes transferred = 19054084 (122be04 hex)
```

- PING

```
=> ping 192.168.0.1
ethernet@ffc40000 Waiting for PHY auto negotiation to complete. done
Using ethernet@ffc40000 device
host 192.168.0.1 is alive
```

- TFTP

1000M environment:

```
Usage:
tftp [loadAddress] [[hostIPaddr:]bootfilename]
```

You can also set the IP address manually:

```
=> setenv ipaddr 192.168.1.101
=> setenv serverip 192.168.1.100

=> tftp kernel.img 0x20000000
ethernet@ffc40000 Waiting for PHY auto negotiation to complete. done
Using ethernet@ffc40000 device
TFTP from server 192.168.1.100; our IP address is 192.168.1.101
Filename 'kernel.img'.
Load address: 0x20000000
Loading: #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################
         12.2 MiB/s
done
Bytes transferred = 20275220 (1356014 hex)
```

5.8.5 Network Troubleshooting

1. Network environment, common directions include:

- Is the PC firewall disabled?
- If crossing subnets, is the gateway set correctly?
- Is the TFTP server configured correctly?
- Is the router's TFTP service disabled?

2. Code issues, generally confirm the following 3 areas:

- pinctrl configuration correctness: Check whether the relevant pin's iomux and drive strength are correct. You can also dump relevant registers and compare with the kernel. In most cases, kernel networking is debugged first before U-Boot networking.
- clock configuration correctness: Clock configuration is relatively more difficult to check. Mainly check the frequency division ratio, clock source, and clock direction. Most registers are in the CRU, but some chips have some registers in the GRF. Similarly, dump relevant registers and compare with the kernel.
- PHY reset GPIO: Mainly verify that the reset GPIO is configured correctly and that the reset waveform meets the PHY requirements.

5.9 Gpio

5.9.1 Framework Support

GPIO drivers use the `gpio-uclass` framework and standard interfaces.

Configuration:

```
CONFIG_DM_GPIO
CONFIG_ROCKCHIP_GPIO
```

Framework code:

```
./drivers/gpio/gpio-uclass.c
```

Driver code:

```
./drivers/gpio/rk_gpio.c
```

5.9.2 DM Interfaces

DM standard interfaces. Users must access GPIOs through `struct gpio_desc`, which is the recommended type.

```
// Request/release GPIO
```

```
int gpio_request_by_name(struct udevice *dev, const char *list_name,
                         int index, struct gpio_desc *desc, int flags);
int gpio_request_by_name_nodev(ofnode node, const char *list_name, int index,
                               struct gpio_desc *desc, int flags);
int gpio_request_list_by_name(struct udevice *dev, const char *list_name,
                              struct gpio_desc *desc_list, int max_count, int flags);
int gpio_request_list_by_name_nodev(ofnode node, const char *list_name,
                                    struct gpio_desc *desc_list, int max_count,
                                    int flags);
int dm_gpio_free(struct udevice *dev, struct gpio_desc *desc)

// Configure GPIO direction. @flags: GPIOD_IS_OUT (output) and GPIOD_IS_IN (input)
int dm_gpio_set_dir_flags(struct gpio_desc *desc, ulong flags);

// Set/get GPIO level
int dm_gpio_get_value(const struct gpio_desc *desc)
int dm_gpio_set_value(const struct gpio_desc *desc, int value)
```

Note:

The return value of `dm_gpio_get_value()` indicates the active state, not the physical high or low level. For example:

- gpios = <&gpio2 RK_PD0 GPIO_ACTIVE_LOW>, returns 1 when the level is low, 0 when the level is high
- gpios = <&gpio2 RK_PD0 GPIO_ACTIVE_HIGH>, returns 0 when the level is low, 1 when the level is high

The same applies to the `value` parameter of `dm_gpio_set_value()`: 1 means active, 0 means inactive.

5.9.3 Legacy Interfaces

Legacy interface type. This interface type is mainly for backward compatibility with older U-Boot APIs. The internal implementation still uses DM, but it hides `struct gpio_desc` from the caller. Functionality is available but not recommended from the DM code standardization perspective.

```
int gpio_request(unsigned gpio, const char *label)
int gpio_free(unsigned gpio)
int gpio_direction_input(unsigned gpio)
int gpio_direction_output(unsigned gpio, int value)
int gpio_get_value(unsigned gpio)
int gpio_set_value(unsigned gpio, int value)
```

`@gpio` is calculated based on the pattern of 32 pins per GPIO group and 8 pins per bank. For example:

```
gpio0_a7 = (0 * 32) + (0 * 8) + 7 = 7
gpio1_b6 = (1 * 32) + (1 * 8) + 6 = 46
gpio3_c2 = (3 * 32) + (2 * 8) + 2 = 114
```

`@value`: the function behavior is consistent with the `dm_gpio_` interface type described above.

5.10 Interrupt

5.10.1 Framework Support

Native U-Boot has no interrupt framework. RK implements one for GICv2/v3 and enables it by default.

Current interrupt use cases:

- `pwrkey`: wake the CPU from low-power charging state
- `Timer`: used in charging and test cases
- `Debug`: `CONFIG_IRQ_TIMER_DUMP`

Configuration:

```
CONFIG_IRQ
CONFIG_GICV2
CONFIG_GICV3
```

Framework code:

```
./drivers/irq/irq-gpio-switch.c
./drivers/irq/irq-gpio.c
./drivers/irq/irq-generic.c
./drivers/irq/irq-gic.c
./drivers/irq/virq.c
./include/irq-generic.h
```

5.10.2 Related Interfaces

```
// CPU local interrupt enable/disable
void enable_interrupts(void);
int disable_interrupts(void);

// GPIO to interrupt number conversion
int gpio_to_irq(struct gpio_desc *gpio);
int phandle_gpio_to_irq(u32 gpio_phandle, u32 pin);
int hard_gpio_to_irq(unsigned gpio);

// Register/release interrupt callback
void irq_install_handler(int irq, interrupt_handler_t *handler, void *data);
void irq_free_handler(int irq);

// Enable/disable interrupt
int irq_handler_enable(int irq);
int irq_handler_disable(int irq);

// Interrupt trigger type
int irq_set_irq_type(int irq, unsigned int type);
```

Requesting IRQ

- Peripherals with independent hardware interrupt numbers do not need additional conversion, e.g.: pwm, timer, etc.
- GPIO pins do not have independent hardware interrupt numbers and need additional conversion.

There are 3 methods to request a GPIO pin interrupt number:

(1) Pass `struct gpio_desc`

```
// This method dynamically parses DTS configuration, is flexible and commonly used.
int gpio_to_irq(struct gpio_desc *gpio);
```

Example:

```
battery {
    compatible = "battery,rk817";
    ......
    dc_det_gpio = <&gpio2 7 GPIO_ACTIVE_LOW>;
    ......
};
```

```
struct gpio_desc dc_det;
int ret, irq;

ret = gpio_request_by_name_nodev(dev_ofnode(dev), "dc_det_gpio", 0,
                                 &dc_det, GPIOD_IS_IN);
// Return value checks omitted for simplicity
if (!ret) {
    irq = gpio_to_irq(&dc_det);
    irq_install_handler(irq, ...);
    irq_set_irq_type(irq, IRQ_TYPE_EDGE_FALLING);
    irq_handler_enable(irq);
}
```

(2) Pass GPIO phandle and pin

```
// This method dynamically parses DTS configuration, is flexible and commonly used.
int phandle_gpio_to_irq(u32 gpio_phandle, u32 pin);
```

Example (RK817 interrupt pin is GPIO0_A7):

```
rk817: pmic@20 {
    compatible = "rockchip,rk817";
    reg = <0x20>;
    ......
    interrupt-parent = <&gpio0>;          // "&gpio0": phandle pointing to the gpio0 node
    interrupts = <7 IRQ_TYPE_LEVEL_LOW>;    // "7": pin number
    ......
};
```

```
u32 interrupt[2], phandle;
```

```
int irq, ret;

phandle = dev_read_u32_default(dev->parent, "interrupt-parent", -1);
if (phandle < 0) {
    printf("failed get 'interrupt-parent', ret=%d\n", phandle);
    return phandle;
}

ret = dev_read_u32_array(dev->parent, "interrupts", interrupt, 2);
if (ret) {
    printf("failed get 'interrupt', ret=%d\n", ret);
    return ret;
}

// Return value checks omitted for simplicity
irq = phandle_gpio_to_irq(phandle, interrupt[0]);
irq_install_handler(irq, pwrkey_irq_handler, dev);
irq_set_irq_type(irq, IRQ_TYPE_EDGE_FALLING);
irq_handler_enable(irq);
```

(3) Force-specify GPIO

```
// This method directly force-specifies the GPIO. The passed GPIO must be declared
// using a special macro. It is inflexible and not recommended.
int hard_gpio_to_irq(unsigned gpio);
```

Example (requesting interrupt on GPIO0_A0):

```
int gpio0_a0, irq;

// Return value checks omitted for simplicity
gpio0_a0 = RK_IRQ_GPIO(RK_GPIO0, RK_PA0);
irq = hard_gpio_to_irq(gpio0_a0);
irq_install_handler(irq, ...);
irq_handler_enable(irq);
```

5.11 I2C

5.11.1 Framework Support

I2C drivers use the `i2c-uclass` framework and standard interfaces.

Configuration:

```
CONFIG_DM_I2C
CONFIG_SYS_I2C_ROCKCHIP
```

Framework code:

```
./drivers/i2c/i2c-uclass.c
```

Driver code:

```
./drivers/i2c/rk_i2c.c
./drivers/i2c/i2c-gpio.c    // GPIO-simulated I2C communication, currently not used
```

5.11.2 Related Interfaces

```
// I2C read/write
int dm_i2c_read(struct udevice *dev, uint offset, uint8_t *buffer, int len)
int dm_i2c_write(struct udevice *dev, uint offset, const uint8_t *buffer, int len)

// Wrapper for the above interfaces
int dm_i2c_reg_read(struct udevice *dev, uint offset)
int dm_i2c_reg_write(struct udevice *dev, uint offset, unsigned int val);
```

5.12 Led

5.12.1 Framework Support

LED drivers use `led-uclass.c` and standard interfaces.

Configuration:

```
CONFIG_LED_GPIO
```

Framework code:

```
drivers/led/led-uclass // compiled by default
```

Driver code:

```
drivers/led/led_gpio.c // supports compatible = "gpio-leds"
```

5.12.2 Related Interfaces

```
// Get LED device
int led_get_by_label(const char *label, struct udevice **devp);
// Set/get LED state
int led_set_state(struct udevice *dev, enum led_state_t state);
enum led_state_t led_get_state(struct udevice *dev);
// Ignored; no underlying driver implementation currently
int led_set_period(struct udevice *dev, int period_ms);
```

5.12.3 DTS Node

The U-Boot `led_gpio.c` is relatively simple in functionality and only parses 3 properties under the LED node:

- gpios: LED control pin and active state
- label: LED name
- default-state: default state, set during driver probe

```
leds {
    compatible = "gpio-leds";
    status = "okay";

    blue-led {
        gpios = <&gpio2 RK_PA1 GPIO_ACTIVE_LOW>;
        label = "battery_full";
        default-state = "off";
    };

    green-led {
        gpios = <&gpio2 RK_PA0 GPIO_ACTIVE_LOW>;
        label = "greenled";
        default-state = "off";
    };

    ......
};
```

5.13 Mtd

MTD (Memory Technology Device) supports parallel NAND, SPI NAND, and SPI NOR.

5.13.1 Framework Support

```
CONFIG_MTD=y
CONFIG_CMD_MTD=y
```

5.13.2 Related Interfaces

Common interfaces:

```
/* Get MTD device */
struct mtd_info *get_mtd_device_nm(const char *name);
void put_mtd_device(struct mtd_info *mtd);

/* Basic read/write/erase interfaces */
int mtd_read(struct mtd_info *mtd, loff_t from, size_t len, size_t *retlen, u_char *buf);
```

```
int mtd_write(struct mtd_info *mtd, loff_t to, size_t len, size_t *retlen, const u_char *buf);
int mtd_erase(struct mtd_info *mtd, struct erase_info *instr);

/* Bad block management interfaces */
int mtd_block_isbad(struct mtd_info *mtd, loff_t ofs);
int mtd_block_markbad(struct mtd_info *mtd, loff_t ofs);

/* OOB operation interfaces */
int mtd_read_oob(struct mtd_info *mtd, loff_t from, struct mtd_oob_ops *ops);
int mtd_write_oob(struct mtd_info *mtd, loff_t to, struct mtd_oob_ops *ops);

/* Bad-block-aware read/write interfaces (NAND only) */
int nand_read_skip_bad(struct mtd_info *mtd, loff_t offset, size_t *length,
                       size_t *actual, loff_t lim, u_char *buffer);
int nand_write_skip_bad(struct mtd_info *mtd, loff_t offset, size_t *length,
                        size_t *actual, loff_t lim, u_char *buffer, int flags);
```

5.13.3 Usage Examples

SPI NOR firmware loading example

Taking reading 0x800 bytes of data from flash offset 0x400000 to memory address 0x4000000:

```
#include <mtd.h>

#define MTD_SPINOR_NAME    "nor0"

static int mtd_demo(void)
{
   char *mtd_name = MTD_SPINOR_NAME;
   struct mtd_info *mtd;
   size_t retlen, off, size;
   u_char *des_buf;
   int ret;

   mtd = get_mtd_device_nm(mtd_name);
   if (IS_ERR_OR_NULL(mtd)) {
      printf("MTD device %s not found, ret %ld\n",
           mtd_name, PTR_ERR(mtd));
      return CMD_RET_FAILURE;
   }

   des_buf = (u_char *)0x4000000;

   off = 0x400000;
   size = 0x800;

   ret = mtd_read(mtd, off, size, &retlen, des_buf);
   if (ret || size != retlen) {
      pr_err("mtd read fail, ret=%d retlen=%ld size=%ld\n", ret, retlen, size);
   }

   put_mtd_device(mtd);
```

```
31
32      return ret;
33  }
```

NAND example

Recommendations:

- Refer to `drivers/mtd/nand/raw/nand_util.c` and use bad-block-aware read/write/erase interfaces, such as:
  - `nand_read_skip_bad()` - read with bad block skipping
  - `nand_write_skip_bad()` - write with bad block skipping (supports WITH_DROP_FFS, WITH_WR_VERIFY and other flags)
  - `nand_erase_opts()` - erase with options
- For small one-time write operations (typically less than 2KB per power cycle), consider using MTD_BLK interfaces. Frequent use of this interface will affect flash lifespan
- `nand_read_skip_bad` / `nand_write_skip_bad` parameter descriptions:
  - `mtd`: MTD device pointer
  - `offset`: flash offset address
  - `length`: read/write data length
  - `actual`: actual required length (accounting for bad blocks), can be NULL
  - `lim`: maximum allowed length limit
  - `buffer`: data buffer
  - `flags`: write operation flags (only for `nand_write_skip_bad`)

5.14 Mtd_blk

RK implements a custom MTD block layer supporting parallel NAND, SPI NAND, and SPI NOR, registering MTD block devices to expose block interfaces.

Characteristics:

- unit is sector (`512B`)
- any write request erases the corresponding flash block, so frequent fragmented writes shorten flash lifetime

5.14.1 Framework Support

U-Boot configuration:

```
// MTD driver
CONFIG_MTD=y
CONFIG_CMD_MTDPARTS=y

// MTD block device driver
CONFIG_CMD_MTD_BLK=y
CONFIG_MTD_BLK=y

// Other NAND device driver configs
......
```

SPL configuration:

```
CONFIG_MTD=y
CONFIG_CMD_MTDPARTS=y
CONFIG_SPL_MTD=y

// Other NAND device driver configs
......
```

Framework code:

```
drivers/mtd/mtd-uclass.c      // MTD uclass framework
drivers/mtd/mtdcore.c         // MTD core layer, provides get_mtd_device_nm/mtd_read/mtd_write etc.
drivers/mtd/mtd_uboot.c       // U-Boot MTD device probing and partition parsing
drivers/mtd/mtd_blk.c         // MTD block layer, provides BLK interface conversion
drivers/mtd/mtdpart.c         // MTD partition management
```

Drivers are individual controller drivers that hook read/write interfaces to the MTD layer.

5.14.2 Related Interfaces

MTD block devices are accessed through standard BLK interfaces:

```
/* Get block device by device type name */
struct blk_desc *blk_get_by_str(const char *ifname, int devnum);

/* Basic read/write interfaces */
unsigned long blk_dread(struct blk_desc *block_dev, lbaint_t start,
                        lbaint_t blkcnt, void *buffer);
unsigned long blk_dwrite(struct blk_desc *block_dev, lbaint_t start,
                         lbaint_t blkcnt, const void *buffer);

/* MTD device access interface (used internally by the MTD block layer) */
int mtd_blk_map_table_init(struct blk_desc *desc, loff_t offset, size_t length);
```

MTD block layer characteristics:

- Converts MTD device bad block mapping to linear LBA addresses
- Supports bad block management for NAND, SPI NAND and other devices

- Write operations automatically perform block erase

For SPL, use the same MTD-related configuration described in the "Framework Support" section above. In routine BLK-layer usage, the most commonly used interfaces are `blk_dread()` and `blk_dwrite()`.

5.15 Optee Client

U-Boot belongs to the Non-Secure World inside ARM TrustZone and must rely on the OPTEE Client to access secure resources.

5.15.1 Framework Support

Adapted from the default U-Boot OP-TEE driver.

Configuration:

```
// OP-TEE API interface enablement
CONFIG_OPTEE_LIB

// OP-TEE driver enablement
CONFIG_TEE
CONFIG_OPTEE

// Must be enabled when eMMC RPMB is not available, i.e., enable the security partition!
CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION
```

Framework and driver:

```
// OP-TEE API source path
lib/optee/

// OP-TEE driver source path
drivers/tee
```

5.15.2 Firmware Description

The `uboot.img` in use must enable TA support, otherwise OPTEE Client cannot interact with it.

```
// 32-bit platform
rkbin/bin/rk35/rk3506_tee_v1.26.bin // TA support not enabled
rkbin/bin/rk35/rk3506_tee_ta_v1.00.bin // TA support enabled

// 64-bit platform, TA support enabled by default
rkbin/bin/rk35/rk3588_bl32_v1.18.bin
```

5.15.3 Interface Description

Refer to `TEE_Client_API_Specification-V1.0_c.pdf`.
Download URL: https://globalplatform.org/specs-library/tee-client-api-specification/

RK wraps built-in TA functionality into a set of interfaces in files such as `lib/optee/optee_otp_api.c`. When using them, include the header `include/tee/optee.h`.

The following describes some of the APIs.

5.15.3.1 Applicability

Platform applicability for the following interfaces: **refer to the Platform Definition chapter**.

```
optee_base_write_security_data()
optee_base_read_security_data()
optee_read_vbootkey_hash()
optee_write_vbootkey_hash()
optee_read_vbootkey_enable_flag()
optee_read_attribute_hash()
optee_write_attribute_hash()
optee_write_oem_otp_key()
optee_oem_otp_key_is_written()
optee_set_oem_hr_otp_read_lock()
optee_oem_otp_key_cipher()
optee_read_oem_ns_otp()
optee_write_oem_ns_otp()
optee_write_ta_encryption_key()
optee_ta_encryption_key_is_written()
optee_write_oem_encrypt_data()
optee_oem_encrypt_data_is_written()
optee_check_security_level_flag()
optee_write_oem_huk()
```

5.15.3.2 Return Values

Unless otherwise specified, return values for the following APIs are described in the `Return Codes` chapter of `TEE_Client_API_Specification-V1.0_c.pdf`.

5.15.3.3 optee_base_write_security_data

```
uint32_t optee_base_write_security_data(char *filename,
                uint32_t filename_size,
                uint8_t *data,
                uint32_t data_size)
```

Function

U-Boot writes data to secure storage.

For eMMC/UFS devices, data is stored in the RPMB partition; for other devices such as NAND, data is stored in the security partition.

If CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION is defined, data is always stored in the security partition.

Parameters

- [in] filename - file name
- [in] filename_size - file name length
- [in] data - data to be written
- [in] data_size - data length to write

5.15.3.4 optee_base_read_security_data

```
uint32_t optee_base_read_security_data(char *filename,
                       uint32_t filename_size,
                       uint8_t *data,
                       uint32_t data_size)
```

Function

U-Boot reads data from secure storage.

For eMMC/UFS devices, data is read from the RPMB partition; for other devices such as NAND, data is read from the security partition.

If CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION is defined, data is always read from the security partition.

Parameters

- [in] filename - file name
- [in] filename_size - file name length
- [in] data - buffer for reading data
- [in] data_size - data length to read

5.15.3.5 optee_read_vbootkey_hash

```
uint32_t optee_read_vbootkey_hash(uint32_t *buf, uint32_t length);
```

Function

Reads the secure boot public key hash from OTP or eFuse.
For secure boot related information, refer to the `Rockchip_Developer_Guide_Secure_Boot_Application_Note_EN` document.

Parameters

- [out] buf - buffer for reading the hash
- [in] length - hash length; refer to the secure boot document for supported hash algorithm lengths.

Length is in words (32 bits).

5.15.3.6 optee_write_vbootkey_hash

```
uint32_t optee_write_vbootkey_hash(uint32_t *buf, uint32_t length);
```

Function

Writes the secure boot public key hash to OTP or eFuse, **simultaneously enables the secure boot flag and turns on secure boot**.
For secure boot related information, refer to the `Rockchip_Developer_Guide_Secure_Boot_Application_Note_EN` document.

Parameters

- [in] buf - buffer for writing the hash
- [in] length - hash length; refer to the secure boot document for supported hash algorithm lengths. Length is in words (32 bits).

5.15.3.7 optee_read_vbootkey_enable_flag

```
uint32_t optee_read_vbootkey_enable_flag(uint8_t *flag);
```

Function

Reads whether the secure boot flag is enabled.
For secure boot related information, refer to the `Rockchip_Developer_Guide_Secure_Boot_Application_Note_EN` document.

Parameters

- [in] flag - 1 Byte, 1 indicates secure boot is enabled, 0 indicates disabled.

5.15.3.8 optee_read_attribute_hash

```
uint32_t optee_read_attribute_hash(uint32_t *buf, uint32_t length)
```

Function

U-Boot reads the hash of `atx_permanent_attributes.bin`. This hash is used to verify the legitimacy of `atx_permanent_attributes.bin`.

Parameters

- [out] buf - hash data
- [in] length - length

5.15.3.9 optee_write_attribute_hash

```
uint32_t optee_write_attribute_hash(uint32_t *buf, uint32_t length)
```

Function

U-Boot writes the hash of `atx_permanent_attributes.bin`.

Parameters

- [in] buf - hash data
- [in] length - length

5.15.3.10 optee_write_oem_otp_key

```
uint32_t optee_write_oem_otp_key(enum RK_OEM_OTP_KEYID key_id,
                                 uint8_t *byte_buf,
                                 uint32_t byte_len);
```

Function

Writes a plaintext key to the specified OEM OTP region.
For OEM OTP related feature descriptions, refer to the `Rockchip_Developer_Guide_OTP_CN` document.

Parameters

- [in] key_id - key ID to write. By default supports `RK_OEM_OTP_KEY0 - 3` (4 keys total). For rv1126/rv1109, additionally supports key_id `RK_OEM_OTP_KEY_FW`
  `RK_OEM_OTP_KEY_FW`: key used by Boot ROM to decrypt loader. The `optee_oem_otp_key_cipher` interface supports using this key for business data encryption/decryption or decrypting kernel images.
- [in] byte_buf - plaintext key
- [in] byte_len - plaintext key length. For `RK_OEM_OTP_KEY_FW`, byte_len only supports 16. For other keys, byte_len supports 16, 24, and 32.

5.15.3.11 optee_oem_otp_key_is_written

```
uint32_t optee_oem_otp_key_is_written(enum RK_OEM_OTP_KEYID key_id, uint8_t *value);
```

Function

Determines whether a key has already been written to the specified OEM OTP region.
For OEM OTP related feature descriptions, refer to the `Rockchip_Developer_Guide_OTP_CN` document.

Parameters

- [in] key_id - key region index to check. By default supports `RK_OEM_OTP_KEY0 - 3` (4 keys total). For rv1126/rv1109, additionally supports key_id `RK_OEM_OTP_KEY_FW`.
- [out] value - whether the key has been written: 1 means written, 0 means not written.

Return Value

The `value` is only meaningful when the return value is `#define TEEC_SUCCESS 0x00000000`.

On the RK3588 platform, it will also check whether key_id is locked. If the corresponding key_id is locked, it returns `#define TEEC_ERROR_ACCESS_DENIED 0xFFFF0001`.

5.15.3.12 optee_set_oem_hr_otp_read_lock

```
uint32_t optee_set_oem_hr_otp_read_lock(enum RK_OEM_OTP_KEYID key_id);
```

Function

Sets the read lock flag for the specified OEM OTP region. After successful setting, this region prohibits data writes, and existing data in this region becomes unreadable by CPU software. The key can still be used through the `optee_oem_otp_key_cipher` interface.
For OEM OTP related feature descriptions, refer to the `Rockchip_Developer_Guide_OTP_CN` document.

Note: When the key_id being set is `RK_OEM_OTP_KEY0` or `RK_OEM_OTP_KEY1` or `RK_OEM_OTP_KEY2`, after successful setting, it will affect attributes of other OTP regions, such as some OTP regions becoming unwritable. See the `Rockchip_Developer_Guide_OTP_CN` document for details.

Parameters

- [in] key_id - key ID to set, supports `RK_OEM_OTP_KEY0 - 3`

5.15.3.13 optee_oem_otp_key_cipher

```
uint32_t optee_oem_otp_key_cipher(enum RK_OEM_OTP_KEYID key_id,
                                  rk_cipher_config *config,
                                  uint32_t src_phys_addr,
                                  uint32_t dst_phys_addr,
                                  uint32_t len);
```

Function

Selects a key from the OEM OTP region and performs a single cipher computation.

Parameters

- [in] key_id - key ID to use. By default supports `RK_OEM_OTP_KEY0 - 3`. For rv1126/rv1109, additionally supports `RK_OEM_OTP_KEY_FW`
- [in] config - algorithm, mode, key, IV, etc.

Supported algorithms: AES, SM4

Supported modes: ECB/CBC/CTS/CTR/CFB/OFB

Supported key lengths: 16, 24, 32 Bytes. For rv1109/rv1126 platforms, key length only supports 16 and 32. When key_id is `RK_OEM_OTP_KEY_FW`, key length only supports 16.

- [in] src_phys_addr - buffer address of data to be computed. Supports being the same as dst_phys_addr, i.e., supports in-place encryption/decryption.

- [out] dst_phys_addr - buffer address of computation result. Supports being the same as src_phys_addr.
- [in] len - byte length of input and output data buffers. Must be aligned to the block size of the algorithm used.

5.15.3.14 optee_read_oem_ns_otp

```
uint32_t optee_read_oem_ns_otp(uint32_t byte_off, uint8_t *byte_buf, uint32_t byte_len)
```

Function

U-Boot reads Non-Protected OEM Zone data from secure OTP.

Parameters

- [in] byte_off - position offset in the OTP region to read
- [out] byte_buf - returned OTP data. The byte_buf memory space must not be smaller than byte_len.
- [in] byte_len - length to read

5.15.3.15 optee_write_oem_ns_otp

```
uint32_t optee_write_oem_ns_otp(uint32_t byte_off, uint8_t *byte_buf, uint32_t byte_len)
```

Function

U-Boot writes data to the Non-Protected OEM Zone in secure OTP.

Parameters

- [in] byte_off - position offset in the OTP region to write
- [in] byte_buf - data to write to OTP
- [in] byte_len - length

5.15.3.16 optee_write_ta_encryption_key

```
uint32_t optee_write_ta_encryption_key(uint32_t *buf, uint32_t length)
```

Function

U-Boot writes the TA encryption key.

Parameters

- [in] buf - key
- [in] length - length

5.15.3.17 optee_ta_encryption_key_is_written

```
uint32_t optee_ta_encryption_key_is_written(uint8_t *value)
```

Function

U-Boot checks whether the TA encryption key has been written.

Parameters

- [out] value - 0 not written, 1 written

5.15.3.18 optee_write_oem_encrypt_data

```
uint32_t optee_write_oem_encrypt_data(uint32_t *buf, uint32_t length)
```

Function

U-Boot writes anti-cloning private data.

Parameters

- [in] buf - data
- [in] length - length

5.15.3.19 optee_oem_encrypt_data_is_written

```
uint32_t optee_oem_encrypt_data_is_written(uint8_t *value)
```

Function

U-Boot checks whether the anti-cloning private data has been written.

Parameters

- [out] value - 0 not written, 1 written

5.15.3.20 optee_check_security_level_flag

```
uint32_t optee_check_security_level_flag(uint8_t flag)
```

Function

U-Boot sets the TEE strong/weak security level.

Parameters

- [in] flag - security level

5.15.3.21 optee_write_oem_huk

```
uint32_t optee_write_oem_huk(uint32_t *buf, uint32_t length)
```

Function

U-Boot writes the OEM HUK.

Parameters

- [in] buf - data
- [in] length - length

5.15.4 Shared Memory

When U-Boot communicates with OPTEE, the data must be placed in shared memory. Users can allocate it with `TEEC_AllocateSharedMemory()`. It is recommended to keep the size under 1 MB. If larger, split the data into multiple transfers. After use, release it with `TEEC_ReleaseSharedMemory()`.

5.15.5 Test Command

Purpose: test secure storage functionality. U-Boot command line:

```
=> mmc testsecurestorage
```

This test case will loop-test secure storage read/write functionality. When hardware uses eMMC, both RPMB and security partition secure storage methods will be tested; when hardware uses NAND, only the security partition secure storage is tested.

5.15.6 Common Error Logs

- No eMMC or NAND device found. Check if U-Boot is missing configuration or if the hardware is damaged.

  ```
  "TEEC: Could not find device"
  ```

- No security partition found. When RPMB is not available, the security partition must be defined in parameter.txt.

  ```
  "TEEC: Could not find security partition"
  ```

- This message appears when using the security partition for secure storage for the first time, or when security partition data has been illegally tampered with.

  ```
  "TEEC: verify [%d] fail, cleanning ...."
  ```

- Insufficient secure storage space. Check if the stored data is too large, or if a large amount of data was previously stored but not deleted.

```
"TEEC: Not enough space available in secure storage !"
```

5.16 PCIe

5.16.1 Development Notes

Determine at which U-Boot stage PCIe is needed and configure DTS accordingly:

1. NVMe as the boot device must be initialized very early, since all subsequent firmware is on that NVMe, so PCIe can only use U-Boot DTS
2. If PCIe is not a boot device, such as for network cards, later initialization is allowed. Since the U-Boot framework supports using kernel DTB, the kernel DTB in boot.img can be used
3. In the U-Boot stage, PCIe RC registers only a 32-bit memory range, not a 64-bit prefetchable range

5.16.2 Framework Support

Framework code:

```
./drivers/pci/*
./drivers/phy/*
```

Driver code:

```
drivers/pci/pcie_dw_rockchip.c
drivers/phy/rockchip/phy-rockchip-snps-pcie3.c
drivers/phy/rockchip/phy-rockchip-naneng-combphy.c
```

menuconfig configuration:

- Driver configuration

For currently supported Rockchip PCIe driver platforms, check the compatible property in `pcie_dw_rockchip.c`. If unsure which driver to use, refer to the corresponding SDK config.

```
CONFIG_DM_REGULATOR_GPIO=y
CONFIG_DM_REGULATOR_FIXED=y
CONFIG_PCI=y
CONFIG_DM_PCI=y
CONFIG_DM_PCI_COMPAT=y
CONFIG_PCI_PNP=y
CONFIG_PCIE_DW_ROCKCHIP=y
CONFIG_PHY_ROCKCHIP_SNPS_PCIE3=y
CONFIG_PHY_ROCKCHIP_NANENG_COMBOPHY=y
CONFIG_PHY=y
CONFIG_CMD_PCI=y
```

```
// Add NVMe support
CONFIG_NVME=y
CONFIG_CMD_NVME=y
// Add PCIe-to-USB support
CONFIG_USB_XHCI_PCI=y
// Add Embedded DTB support. Enabling Embedded DTB will increase image size.
CONFIG_EMBED_KERNEL_DTB_ALWAYS=y
CONFIG_SPL_FIT_IMAGE_KB=2560
```

5.16.3 DTS Configuration

Recommended loading strategies:

- flash + PCIe NVMe dual storage: use PCIe before loading the kernel DTB
- AMP with PCIe: use PCIe after loading the embedded DTB
- general case: use PCIe after loading the kernel DTB

Using PCIe before loading the kernel DTB

It is recommended to configure the U-Boot DTSI nodes based on the kernel DTB node configuration and add the `bootph-all` attribute:

- phy power supply, can be omitted if already enabled by default
- vcc 3v3 power supply
- phy node
- controller node

Example for RK3588 PCIe3x4:

```
diff --git a/arch/arm/dts/rk3588-u-boot.dtsi b/arch/arm/dts/rk3588-u-boot.dtsi
index 3fe8054aac..a8e2defbad 100644
--- a/arch/arm/dts/rk3588-u-boot.dtsi
+++ b/arch/arm/dts/rk3588-u-boot.dtsi
@@ -22,6 +22,28 @@
                compatible = "rockchip,rk3588-secure-otp";
                reg = <0x0 0xfe3a0000 0x0 0x4000>;
        };
+
+       vcc12v_dcin: vcc12v-dcin {
+               bootph-all;
+               compatible = "regulator-fixed";
+               regulator-name = "vcc12v_dcin";
+               regulator-always-on;
+               regulator-boot-on;
+               regulator-min-microvolt = <12000000>;
+               regulator-max-microvolt = <12000000>;
+       };
+
+       vcc3v3_pcie30: vcc3v3-pcie30 {
+               bootph-all;
+               compatible = "regulator-fixed";
+               regulator-name = "vcc3v3_pcie30";
```

```
+            regulator-min-microvolt = <3300000>;
+            regulator-max-microvolt = <3300000>;
+            enable-active-high;
+            gpio = <&gpio3 RK_PC3 GPIO_ACTIVE_HIGH>;
+            startup-delay-us = <5000>;
+            vin-supply = <&vcc12v_dcin>;
+    };
 };

 &firmware {
@@ -117,6 +139,19 @@
     status = "okay";
 };

+&pcie30phy {
+    bootph-all;
+    rockchip,pcie30-phymode = <PHY_MODE_PCIE_AGGREGATION>;
+    status = "okay";
+};
+
+&pcie3x4 {
+    bootph-all;
+    reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
+    vpcie3v3-supply = <&vcc3v3_pcie30>;
+    status = "okay";
+};
+
 &uart2 {
     bootph-all;
     status = "okay";
```

Example for RK3566:

```
From b58a47956bbd03de0fcef572fa06cdeea974e2a9 Mon Sep 17 00:00:00 2001
From: Jon Lin <jon.lin@rock-chips.com>
Date: Thu, 9 Mar 2023 15:29:37 +0800
Subject: [PATCH] TEST: uboot: rk3566_evb2_v11: nvme

Change-Id: I87b3786a433691f3c385460fa8636291bce8ed9a
Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
---
 arch/arm/dts/rk3568-u-boot.dtsi | 43 +++++++++++++++++++++++++++++++++++++++
 configs/rk3568_defconfig        | 13 ++++++++++
 2 files changed, 56 insertions(+)

diff --git a/arch/arm/dts/rk3568-u-boot.dtsi b/arch/arm/dts/rk3568-u-boot.dtsi
index a0678e35db..1ab8ea4436 100644
--- a/arch/arm/dts/rk3568-u-boot.dtsi
+++ b/arch/arm/dts/rk3568-u-boot.dtsi
@@ -26,6 +26,27 @@
     cru_rst_addr = <0xfdd20470>;
     bootph-all;
   };
+
```

```
+  dc_12v: dc-12v {
+      compatible = "regulator-fixed";
+      regulator-name = "dc_12v";
+      regulator-always-on;
+      regulator-boot-on;
+      regulator-min-microvolt = <12000000>;
+      regulator-max-microvolt = <12000000>;
+  };
+
+  vcc3v3_pcie: gpio-regulator {
+      bootph-all;
+      compatible = "regulator-fixed";
+      regulator-name = "vcc3v3_pcie";
+      regulator-min-microvolt = <3300000>;
+      regulator-max-microvolt = <3300000>;
+      enable-active-high;
+      gpio = <&gpio0 RK_PC2 GPIO_ACTIVE_HIGH>;
+      startup-delay-us = <5000>;
+      vin-supply = <&dc_12v>;
+  };
 };

 &psci {
@@ -386,6 +407,28 @@
    status = "okay";
 };

+&pipegrf {
+  bootph-all;
+  status = "okay";
+};
+
+&pipe_phy_grf2 {
+  bootph-all;
+  status = "okay";
+};
+
+&combphy2_psq {
+  bootph-all;
+  status = "okay";
+};
+
+&pcie2x1 {
+  bootph-all;
+  reset-gpios = <&gpio1 RK_PB2 GPIO_ACTIVE_HIGH>;
+  vpcie3v3-supply = <&vcc3v3_pcie>;
+  status = "okay";
+};
+
 &pinctrl {
    bootph-all;
    status = "okay";
```

Using PCIe after loading the Embedded DTB

U-Boot supports the embedded DTB approach, which avoids the impact of kernel DTB changes, including for product solutions without kernel support. Typically, the embedded DTB is sourced from the standard kernel DTB file. Main steps:

- Build the kernel firmware to generate the target DTB file
- Enable Embedded DTB configuration in U-Boot

```
CONFIG_EMBED_KERNEL_DTB=y
CONFIG_EMBED_KERNEL_DTB_ALWAYS=y
CONFIG_EMBED_KERNEL_DTB_PATH="./dts/rk<target-chip>-<target-device>.dtb"  # e.g.,
CONFIG_EMBED_KERNEL_DTB_PATH="dts/rk3588-evb1.dtb"
```

- Build to generate the U-Boot image

Suggestions:

- For some AMP solutions that have neither PCIe early init requirements nor a kernel, and need to complete PCIe enumeration before AMP loading, refer to the following patch to enumerate PCIe in the boot flow:

```
diff --git a/arch/arm/mach-rockchip/board.c b/arch/arm/mach-rockchip/board.c
index 979598ff7b..87d131e118 100644
--- a/arch/arm/mach-rockchip/board.c
+++ b/arch/arm/mach-rockchip/board.c
@@ -537,6 +537,11 @@ int board_init(void)
        io_domain_init();
 #endif
        set_armclk_rate();
+
+#ifdef CONFIG_PCI
+       pci_init();
+#endif
+
 #ifdef CONFIG_DM_DVFS
        dvfs_init(true);
 #endif
```

Using PCIe after loading the kernel DTB

Consider placing the relevant call after the "RK u-boot using kernel DTB" stage to directly reuse the kernel DTB. For related documentation, refer to the kernel PCIe configuration instructions.

5.16.4 Usage Examples

Commonly used commands:

5.16.4.1 PCIe CMD

```
## Chapter-5 PCI enumeration, where the CFG-mapped memory address is 0x00000000f0000000
=> pci enum
pcie@fe150000: PCIe Linking... LTSSM is 0x1
pcie@fe150000: PCIe Linking... LTSSM is 0x6
pcie@fe150000: PCIe Linking... LTSSM is 0x4
pcie@fe150000: PCIe Linking... LTSSM is 0x210023
pcie@fe150000: PCIe Link up, LTSSM is 0x230011
pcie@fe150000: PCIE-0: Link up (Gen3-x2, Bus0)
pcie@fe150000: invalid flags type!
pcie@fe150000: Config space: [0x00000000f0000000 - 0x00000000f0100000, size 0x100000]

## Chapter-5 Scan all devices
=> pci
BusDevFun  VendorId   DeviceId   Device Class       Sub-Class
_____________________________________________________________
00.00.00   0x1d87     0x3588     Bridge device          0x04
01.00.00   0x144d     0xa809     Mass storage controller 0x08

## Chapter-5 Display detailed info for bus 01 device
=> pci 01 long
Scanning PCI devices on bus 1

Found PCI device 01.00.00:
  vendor ID =                0x144d
  device ID =                0xa809
  command register ID =      0x0006
  status register =          0x0010
  revision ID =              0x00
  class code =               0x01 (Mass storage controller)
  sub class code =           0x08
  programming interface =    0x02
  cache line =               0x08
  latency time =             0x00
  header type =              0x00
  BIST =                     0x00
  base address 0 =           0xf0300004
  base address 1 =           0x00000000
  base address 2 =           0x00000000
  base address 3 =           0x00000000
  base address 4 =           0x00000000
  base address 5 =           0x00000000
  cardBus CIS pointer =      0x00000000
  sub system vendor ID =     0x144d
  sub system ID =            0xa801
  expansion ROM base address =  0x00000000
  interrupt line =           0xff
  interrupt pin =            0x01
  min Grant =                0x00
  max Latency =              0x00

## Chapter-5 Display BAR-mapped addresses for bdf 01.00.00 device, example shows bar0 mapped to
memory address 0xf0300000, size 0x4000
```

```
=> pci bar 01.00.00
ID   Base             Size             Width  Type
------------------------------------------------------------
 0   0x00000000f0300000  0x0000000000004000  64     MEM

## Chapter-5 Read CFG space info for bdf 01.00.00 device
=> pci d.w 01.00.00 0
00000000: 144d a809 0006 0010 0200 0108 0008 0000
00000010: 0004 f030 0000 0000 0000 0000 0000 0000
00000020: 0000 0000 0000 0000 0000 0000 144d a801
00000030: 0000 0000 0040 0000 0000 0000 01ff 0000

## Chapter-5 AER function, reads AER register info for the specified device, output format consistent with Linux lspci tool
=> pci aer 01.00.0
   AER Capability found at offset 0x40
   UESta: DLP-- SDES-- TLP-- FCP-- CmpltTO-- CmpltAbrt-- UnxCmplt-- RxOF-- MalfTLP-- ECRC-- UnsupReq-- ACSViol--
   UEMsk: DLP-- SDES-- TLP-- FCP-- CmpltTO-- CmpltAbrt-- UnxCmplt-- RxOF-- MalfTLP-- ECRC-- UnsupReq-- ACSViol--
   UESvrt: DLP+ SDES- TLP+ FCP- CmpltTO- CmpltAbrt- UnxCmplt- RxOF- MalfTLP- ECRC- UnsupReq- ACSViol-
   CESta: RxErr-- BadTLP-- BadDLLP-- Rollover-- Timeout-- NonFatalErr--
   CEMsk: RxErr-- BadTLP-- BadDLLP-- Rollover-- Timeout-- NonFatalErr--
   AERCap: First Error Pointer: 00, GenCap- CGenEn- ChkCap- ChkEn-

## Chapter-5 retrain link function, for link re-establishment
=> pci x 01.00.0
   Retrain triggered for device 0001:0000
   Link Status for device 0001:0000: 0x1022
   Speed: Gen3
   Width: x2
   Link Up: Yes

## Chapter-5 FLR (Function Level Reset) function, for resetting peripheral functions
=> pci flr 01.00.0
   FLR completed and state restored for device 01:00.0
```

5.16.4.2 NVMe

```
## Chapter-5 Initiate NVMe scan
=> nvme scan

## Chapter-5 List NVMe device details
=> nvme details
   Blk device 0: Optional Admin Command Support:
        Namespace Management/Attachment: no
        Firmware Commit/Image download: yes
        Format NVM: yes
        Security Send/Receive: no
   Blk device 0: Optional NVM Command Support:
        Reservation: yes
        Save/Select field in the Set/Get features: yes
```

```
        Write Zeroes: yes
        Dataset Management: yes
        Write Uncorrectable: yes
    Blk device 0: Format NVM Attributes:
        Support Cryptographic Erase: No
        Support erase a particular namespace: Yes
        Support format a particular namespace: Yes
    Blk device 0: LBA Format Support:
    Blk device 0: End-to-End DataProtect Capabilities:
        As last eight bytes: No
        As first eight bytes: No
        Support Type3: No
        Support Type2: No
        Support Type1: No
    Blk device 0: Metadata capabilities:
        As part of a separate buffer: No
        As part of an extended data LBA: No

## Chapter-5 See a 256GB NVMe. If capacity is not visible, power off the device completely and retry.
=> nvme info
    Device 0: Vendor: 0x144d Rev: EXD7201Q Prod: S444NA0M384608
            Type: Hard Disk
            Capacity: 244198.3 MB = 238.4 GB (500118192 x 512)

## Chapter-5 Select NVMe device with ID 0
=> nvme device 0

Device 0: Vendor: 0x144d Rev: EXD7201Q Prod: S444NA0M384608
        Type: Hard Disk
        Capacity: 244198.3 MB = 238.4 GB (500118192 x 512)
... is now current device

## Chapter-5 Set 0x40000000 memory to 0x55aa55aa
=> md.l 0x40000000 1
    40000000: d08ec033                  3...
=> mw.l 0x40000000 0x55aa55aa
=> md.l 0x40000000 1
    40000000: 55aa55aa                  .U.U

## Chapter-5 Write 1 block from 0x40000000 memory to NVMe LBA address 0
=> nvme write 0x40000000 0x0 0x1

    nvme write: device 0 block # 0, count 1 ... 1 blocks written: OK

## Chapter-5 Check 0x44000000 memory to confirm original data
=> md.l 0x44000000 1
    44000000: ffffffff                  ....

## Chapter-5 Read 1 block from NVMe LBA address 0 to memory 0x44000000
=> nvme read 0x44000000 0x0 0x1

    nvme read: device 0 block # 0, count 1 ... 1 blocks read: OK

## Chapter-5 Confirm 0x44000000 memory data was read back from NVMe
=> md.l 0x44000000 1
```

```
   44000000: 55aa55aa

## Chapter-5 Erase 1 block starting from address 0
=> nvme erase 0x0 0x1
   nvme erase: device 0 block # 0, count 1 ... 1 blocks erased: OK

## Chapter-5 Read 1 block from NVMe LBA address 0 to memory 0x44000000
=> nvme read 0x44000000 0x0 0x1
   nvme read: device 0 block # 0, count 1 ... 1 blocks read: OK

## Chapter-5 Confirm 0x44000000 memory data was read back from NVMe
=> md.l 0x44000000 0x200       # NVMe first block data has been cleared
   44000000: 00000000 00000000 00000000 00000000    ................
   44000010: 00000000 00000000 00000000 00000000    ................
   44000020: 00000000 00000000 00000000 00000000    ................
   ...

## Chapter-5 Use write_zeroes to clear 1 block of data starting from address 0x100
=> nvme write_zeroes 0x100 0x1
   nvme write_zeroes: device 0 block # 0x100, count 1 ... 1 blocks write_zeroed: OK
```

5.16.4.3 RK3588 RC DMA

```
=> pci
BusDevFun  VendorId   DeviceId   Device Class       Sub-Class
_____________________________________________________________
00.00.00   0x1d87     0x3588     Bridge device          0x04
01.00.00   0x1d87     0x356a     ???                    0x00
=> pci 1 long
Scanning PCI devices on bus 1

Found PCI device 01.00.00:
  vendor ID =                 0x1d87
  device ID =                 0x356a
  command register ID =       0x0006
  status register =           0x0010
  revision ID =               0x01
  class code =                0x12 (???)
  sub class code =            0x00
  programming interface =     0x00
  cache line =                0x08
  latency time =              0x00
  header type =               0x00
  BIST =                      0x00
  base address 0 =            0xf0400000    # BAR0 mapped CPU address; since RK PCIe uses CPU-BUS one-to-one mapping, bus addr is the same value
  base address 1 =            0x00000000
  base address 2 =            0x0400000c
  base address 3 =            0x00000000
  base address 4 =            0xf0800000
  base address 5 =            0x00000000
  cardBus CIS pointer =       0x00000000
  sub system vendor ID =      0x0000
```

```
  sub system ID =             0x0000
  expansion ROM base address =  0x00000000
  interrupt line =            0xff
  interrupt pin =             0x01
  min Grant =                 0x00
  max Latency =               0x00
=>

## Chapter-5 BAR CPU access, PIO access
md.l 0xf0400000 0x40

## Chapter-5 DMA read
mw.l 0x3c000000 0xffffffff
dcache flush 0x3c000000 0x100     # flush
mw.l 0xf538002c 0x1
mw.l 0xf5380300 0x4000008
mw.l 0xf5380304 0x0
mw.l 0xf5380308 0x100
mw.l 0xf538030c 0xf0400000
mw.l 0xf5380310 0x0
mw.l 0xf5380314 0x3c000000
mw.l 0xf5380318 0x0
mw.l 0xf5380030 0x0
dcache invalidate 0x3c000000 0x100  # invalidate
md.l 0x3c000000

## Chapter-5 DMA write
mw.l 0x3c000000 0xffffffff
dcache flush 0x3c000000 0x100 # flush
mw.l 0xf538000c 0x1
mw.l 0xf5380200 0x4000008
mw.l 0xf5380204 0x0
mw.l 0xf5380208 0x100
mw.l 0xf538020c 0x3c000000
mw.l 0xf5380210 0x0
mw.l 0xf5380214 0xf0400000
mw.l 0xf5380218 0x0
mw.l 0xf5380010 0x0
md.l 0x3c000000 0x40
```

Note:

- dcache flush/clean requires CONFIG_CMD_CACHE and the following support patch:

```
commit b46a81a12dd4a1514a6522e33a1d16194f242d62
Author: Joseph Chen <chenjh@rock-chips.com>
Date:   Wed Sep 28 01:36:45 2022 +0000

    cmd: cache: Add flush/invalidate dcache range support

    Signed-off-by: Joseph Chen <chenjh@rock-chips.com>
    Change-Id: Id0e0cd9019072e8c557ebd2987b439057cb4ae3b
```

5.16.4.4 RK3568 RC DMA

```
=> pci
BusDevFun  VendorId   DeviceId   Device Class      Sub-Class
_____________________________________________________________
00.00.00   0x1d87     0x356a     Bridge device         0x04
01.00.00   0x1d87     0x356a     ???                   0x00
=> pci 1 long
Scanning PCI devices on bus 1

Found PCI device 01.00.00:
  vendor ID =                 0x1d87
  device ID =                 0x356a
  command register ID =       0x0006
  status register =           0x0010
  revision ID =               0x01
  class code =                0x12 (???)
  sub class code =            0x00
  programming interface =     0x00
  cache line =                0x08
  latency time =              0x00
  header type =               0x00
  BIST =                      0x00
  base address 0 =            0xf0400000    # BAR0 mapped CPU address; since RK PCIe uses CPU-BUS one-to-one mapping, bus addr is the same value
  base address 1 =            0x00000000
  base address 2 =            0x0400000c
  base address 3 =            0x00000000
  base address 4 =            0xf0800000
  base address 5 =            0x00000000
  cardBus CIS pointer =       0x00000000
  sub system vendor ID =      0x0000
  sub system ID =             0x0000
  expansion ROM base address =  0x00000000
  interrupt line =            0xff
  interrupt pin =             0x01
  min Grant =                 0x00
  max Latency =               0x00
=>

## Chapter-5 BAR CPU access, PIO access
md.l 0xf0400000 0x40

## Chapter-5 DMA read
mw.l 0x3c000000 0xffffffff
dcache flush 0x3c000000 0x100     # flush
mw.l 0xf638002c 0x1
mw.l 0xf6380300 0x4000008
mw.l 0xf6380304 0x0
mw.l 0xf6380308 0x100
mw.l 0xf638030c 0xf0400000
mw.l 0xf6380310 0x0
mw.l 0xf6380314 0x3c000000
mw.l 0xf6380318 0x0
```

```
mw.l 0xf6380030 0x0
dcache invalidate 0x3c000000 0x100  # invalidate
md.l 0x3c000000

## Chapter-5 DMA write
mw.l 0x3c000000 0xffffffff
dcache flush 0x3c000000 0x100 # flush
mw.l 0xf638000c 0x1
mw.l 0xf6380200 0x4000008
mw.l 0xf6380204 0x0
mw.l 0xf6380208 0x100
mw.l 0xf638020c 0x3c000000
mw.l 0xf6380210 0x0
mw.l 0xf6380214 0xf0400000
mw.l 0xf6380218 0x0
mw.l 0xf6380010 0x0
md.l 0x3c000000 0x40
```

Note:

- dcache flush/clean requires CONFIG_CMD_CACHE and the following support patch:

```
commit b46a81a12dd4a1514a6522e33a1d16194f242d62
Author: Joseph Chen <chenjh@rock-chips.com>
Date:   Wed Sep 28 01:36:45 2022 +0000

    cmd: cache: Add flush/invalidate dcache range support

    Signed-off-by: Joseph Chen <chenjh@rock-chips.com>
    Change-Id: Id0e0cd9019072e8c557ebd2987b439057cb4ae3b
```

5.16.5 Common Issue Analysis

RK3568 Linux5.10 U-Boot disable PCIe ASPM power saving

The controller already has ASPM disabled by default. To confirm PCIe3x2 ASPM status in U-Boot shell:

```
pci display.l 20.00.00 0x80 1
```

Where BITS[1:0] PCIE_CAP_ACTIVE_STATE_LINK_PM_CONTROL:

```
Values:
0x0 (DISABLED): Disabled
0x1 (L0S_ENTRY_EN): L0s Entry Enabled
0x2 (L1_ENTRY_En): L1 Entry Enabled
0x3 (L0S_L1_ENTRY_EN): L0s and L1 Entry Enabled
```

RC DMA access to FPGA limitations

FPGA and most peripherals have varying BAR access behaviors:

- Some devices support memory read/write TLP packets of different lengths

- Some devices only support 4-byte memory read/write TLP packets

Therefore, supporting only 4-byte memory read/write TLP packets causes:

- CPU access to BAR space works normally
- RC DMA requests exceeding 4-byte address transfer length result in different errors, such as CA abort

```
## Chapter-5 RK3568 DMA read CA abort info
=> md.l 0xf63800b8 1
f63800b8: 00000100                          ....
```

Other notes:

- To capture such issues, the PCIe protocol analyzer should use TLP trigger, not memory trigger; otherwise after the error there is no CPL to trigger
- It is suspected that after the peripheral receives TLP requests exceeding 4B length, PCIe bus access is restricted, causing CA abort, but specific analysis needs to be done with the peripheral manufacturer's assistance

5.17 Pinctrl

5.17.1 Framework Support

Pinctrl drivers use the `pinctrl-uclass` framework and standard interfaces.

Configuration:

```
CONFIG_PINCTRL_GENERIC
CONFIG_PINCTRL_ROCKCHIP
```

Framework code:

```
./drivers/pinctrl/pinctrl-uclass.c
```

Driver code:

```
./drivers/pinctrl/rockchip/pinctrl-rockchip-core.c
```

5.17.2 Related Interfaces

```
int pinctrl_select_state(struct udevice *dev, const char *statename)   // Set state
int pinctrl_get_gpio_mux(struct udevice *dev, int banknum, int index)   // Get state
```

The pinctrl framework automatically sets the "default" state for each driver during probe, so users typically do not need to call pinctrl interfaces explicitly.

5.18 Pmic/Regulator

5.18.1 Framework Support

PMIC/Regulator drivers use `pmic-uclass` and `regulator-uclass` together with standard interfaces.

Supported PMICs:

```
rk805/rk808/rk809/rk816/rk817/rk818/...
```

Supported regulators:

```
rk805/rk808/rk809/rk816/rk817/rk818/syr82x/tcs452x/fan53555/pwm/gpio/fixed/...
```

Configuration:

```
CONFIG_DM_PMIC
CONFIG_PMIC_CHILDREN
CONFIG_PMIC_RK8XX       // Applicable to all current RK8XX series chips
CONFIG_DM_REGULATOR
CONFIG_REGULATOR_PWM
CONFIG_REGULATOR_RK8XX   // Applicable to all current RK8XX series chips
CONFIG_DM_REGULATOR_FAN53555
```

Framework code:

```
./drivers/power/pmic/pmic-uclass.c
./drivers/power/regulator/regulator-uclass.c
```

Driver files:

```
./drivers/power/pmic/rk8xx.c
./drivers/power/regulator/rk8xx.c
./drivers/power/regulator/fixed.c
./drivers/power/regulator/gpio-regulator.c
./drivers/power/regulator/pwm_regulator.c
./drivers/power/regulator/fan53555.c
......
```

5.18.2 Related Interfaces

```
// Get regulator. @platname: name specified by "regulator-name", e.g.: vdd_arm, vdd_logic
int regulator_get_by_platname(const char *platname, struct udevice **devp);

// Enable/disable
int regulator_get_enable(struct udevice *dev);
int regulator_set_enable(struct udevice *dev, bool enable);
int regulator_set_suspend_enable(struct udevice *dev, bool enable);
```

```
int regulator_get_suspend_enable(struct udevice *dev);

// Configure/get voltage
int regulator_get_value(struct udevice *dev);
int regulator_set_value(struct udevice *dev, int uV);
int regulator_set_suspend_value(struct udevice *dev, int uV);
int regulator_get_suspend_value(struct udevice *dev);
```

5.18.3 Initial Voltage

Two methods are currently supported to set regulator initialization voltage, but both require `regulator-boot-on`:

- set `regulator-min-microvolt` and `regulator-max-microvolt` to the same value
- set `regulator-init-microvolt = <...>`

```
vdd_arm: DCDC_REG1 {
    regulator-name = "vdd_arm";
    regulator-boot-on;                  // Required
    regulator-min-microvolt = <712500>;
    regulator-max-microvolt = <1450000>;
    regulator-init-microvolt = <1100000>;   // Set initialization voltage to 1.1v
    ......
};
```

5.18.4 Skip Initialization

To skip initialization of a regulator, add `regulator-loader-ignore`.

```
vdd_arm: DCDC_REG1 {
    regulator-name = "vdd_arm";
    regulator-loader-ignore; // Only effective for U-Boot stage regulator initialization, not effective for kernel
    ......
};
```

5.19 Reset

5.19.1 Framework Support

Reset drivers use `reset-uclass.c` and standard interfaces. On RK platforms, reset is essentially a CRU soft reset.

Configuration:

```
CONFIG_DM_RESET
CONFIG_RESET_ROCKCHIP
```

Framework code:

```
./drivers/reset/reset-uclass.c
```

Driver code:

```
./drivers/reset/reset-rockchip.c
```

5.19.2 Related Interfaces

```
// Get reset handle
int reset_get_by_index(struct udevice *dev, int index, struct reset_ctl *reset_ctl);
int reset_get_by_name(struct udevice *dev, const char *name,
                      struct reset_ctl *reset_ctl);
// Release reset
int reset_free(struct reset_ctl *reset_ctl);
// Request reset
int reset_request(struct reset_ctl *reset_ctl);
// Assert/deassert reset
int reset_assert(struct reset_ctl *reset_ctl);
int reset_deassert(struct reset_ctl *reset_ctl);
```

Example:

```
struct reset_ctl reset_ctl;

ret = reset_get_by_name(dev, "mac-phy", &reset_ctl);
if (ret) {
    debug("reset_get_by_name() failed: %d\n", ret);
    return ret;
}

ret = reset_request(&reset_ctl);
if (ret)
    return ret;

ret = reset_assert(&reset_ctl);
if (ret)
    return ret;

......

ret = reset_deassert(&reset_ctl);
if (ret)
    return ret;

......

ret = reset_free(&reset_ctl);
if (ret)
    return ret;
```

5.19.3 DTS Configuration

U-Boot enables reset functionality by default. Users only need to specify the reset target in the peripheral node:

```
// Format:
reset-names = <name-string-list>
resets = <cru-phandle-list>
```

For example, gmac2phy:

```
gmac2phy: ethernet@ff550000 {
    compatible = "rockchip,rk3328-gmac";
    ......

    // Specify reset properties
    reset-names = "stmmaceth", "mac-phy";
    resets = <&cru SRST_GMAC2PHY_A>, <&cru SRST_MACPHY>;
};
```

5.20 Rng

5.20.1 Framework Support

RNG provides hardware random-number functionality.

Framework code:

```
./drivers/rng/rng-uclass.c
```

Driver code:

```
./drivers/rng/rockchip_rng.c
```

Configuration:

```
CONFIG_DM_RNG=y
CONFIG_RNG_ROCKCHIP=y
```

5.20.2 Related Interfaces

```
// @buffer: save random number output
// @size: random number length in bytes
int dm_rng_read(struct udevice *dev, void *buffer, size_t size)
```

5.20.3 DTS Configuration

RNG is one function of the Crypto hardware block, so RNG nodes are also divided into V1/V2. The RNG node compatible field has two forms:

```
compatible = "rockchip,cryptov1-rng";
compatible = "rockchip,cryptov2-rng";
```

For complete node configuration, refer to `rv1126.dtsi`, `rk3568.dtsi`, `rk3399.dtsi` and other files.

5.21 Spi

5.21.1 Framework Support

Framework code:

```
./drivers/spi/spi-uclass.c
```

Driver code:

```
./drivers/spi/rk_spi.c
```

menuconfig configuration:

```
CONFIG_ROCKCHIP_SPI=y
CONFIG_CMD_SPI=y
```

5.21.2 Related Interfaces

`./include/spi.h`

```
// Initialize the corresponding SPI bus
struct spi_slave *spi_setup_slave(unsigned int bus, unsigned int cs, unsigned int max_hz, unsigned int mode);

// Claim/release bus
int spi_claim_bus(struct spi_slave *slave);
void spi_release_bus(struct spi_slave *slave);

// Common read/write interfaces
int spi_xfer(struct spi_slave *slave, unsigned int bitlen, const void *dout,
             void *din, unsigned long flags);
int spi_write_then_read(struct spi_slave *slave, const u8 *opcode,
                        size_t n_opcode, const u8 *txbuf, u8 *rxbuf,
                        size_t n_buf);
```

5.21.3 DTS Configuration

```
&spi0 {
	bootph-all;
	status = "okay";
};
```

5.21.4 Usage Example

You can refer to `drivers/power/power_spi.c`.

You can also refer to the following demo:

```
static u32 spi_bus_test(int bus, int cs)
{
    struct spi_slave *spi_slave;
    u32 tx_data, rx_data;
    int ret;

#ifdef CONFIG_DM_SPI
    struct udevice *dev;
    char name[30], *str;

    snprintf(name, sizeof(name), "generic_%d:%d", bus, cs);
    str = strdup(name);
    if (!str)
        return -ENOMEM;
    ret = spi_get_bus_and_cs(bus, cs, 50000000, SPI_MODE_0, "spi_generic_drv", str, &dev, &spi_slave);
    if (ret)
        return ret;
#else
    spi_slave = spi_setup_slave(bus, cs, 50000000, SPI_MODE_0);
    if (!spi_slave) {
        /*
         * Invalid bus 1 (err=-19) means that spi1 is disabled in dts
         * Invalid chip select 1:0 (err=-19) means that there is no dev under spi1 bus in dts
         * check it in uboot dtb or kernel dtb(if is enabled)
         *
         * btw, spi_get_bus_and_cs support no sub dev operation but spi_setup_slave can't
         */
        return -ENODEV;
    }
#endif

    if (spi_claim_bus(spi_slave))
        return -ENODEV;

    tx_data = 0x12345678;
    ret = spi_xfer(spi_slave, 32, &tx_data, &rx_data, SPI_XFER_BEGIN | SPI_XFER_END);

    spi_release_bus(spi_slave);

```

```
    pr_err("%s succuss\n", __func__);

    return ret;
}
```

Note:

- SPI peripherals support specifying the rate through the DTS child node's `spi-max-frequency` property, and also support setting the rate when calling `spi_get_bus_and_cs` / `spi_setup_slave`

5.21.5 Test Command

Use cmd_spi related commands:

```
sspi 3:0.0 @1000000 24 AAA        # bus3:cs0:mode0, interface speed 1 MHz, transfer length 24 bits, transfer data "AAA"
```

5.21.6 Common Issue Analysis

Q1: No signal?

A1: Confirm that the corresponding iomux and clk are configured correctly.

Q2: cmd spi abnormal?

A2: The cmd stage uses the kernel DTB by default. Confirm that the kernel DTSI aliases specify the corresponding SPI.

Q3: Why does spi_setup_slave fail?

A3: The SPI bus node must have a device attached for `spi_setup_slave` to work. However, U-Boot provides a standard device with driver `spi_generic_drv` for the bus, with device name `generic_1:0`. Declare this device when calling `spi_setup_slave`.

Q4: How to confirm the SPI frequency in U-Boot?

A4: Enable the debug switch:

```
diff --git a/drivers/spi/rk_spi.c b/drivers/spi/rk_spi.c
index 962cc596591..3ca47f0cfd3 100644
--- a/drivers/spi/rk_spi.c
+++ b/drivers/spi/rk_spi.c
@@ -26,7 +26,8 @@
 #include "rk_spi.h"

 /* Change to 1 to output registers at the start of each transaction */
-#define DEBUG_RK_SPI   0
+#define DEBUG
+#define DEBUG_RK_SPI   1
```

Take testing with `cmd/spi.c` as an example. Key debug log description:

```
=> sspi 0:0.0 @1000000 24 AAA
rockchip_spi_of_to_plat: base=2c0e0000, max-frequency=50000000, deactivate_delay=0, rsd=0
rockchip_spi_probe: probe
rockchip_spi_probe: rate = 198000000
spi speed 1000000, div 198
rockchip_spi_xfer: dout=00000000fddeab50, din=00000000fddeab30, len=0, flags=3
ctrl0:        0x00002401
ctrl1:        0x00000000
ssienr:             0x00000000
ser:          0x00000000
baudr:        0x000000c6
txftlr:             0x00000000
rxftlr:             0x00000000
txflr:        0x00000000
rxflr:        0x00000000
sr:           0x0000004c
imr:          0x00000000
isr:          0x00000000
dmacr:        0x00000000
dmatdlr:      0x00000000
dmardlr:      0x00000000
activate cs0
deactivate cs0
```

5.22 Storage

Storage drivers use the standard storage framework and are connected to the BLK layer to support file systems. Supported storage includes eMMC, NAND flash, SPI NAND flash, and SPI NOR flash. The flash-related framework comparison is as follows:

Abbreviation	Mainly supported flash types	Controller driver	Flash framework	Registered device type	Mainly supported file systems
rknand solution	MLC TLC Nand	drivers/rkand	drivers/rkand	block device	FAT, EXT, SquashFS
rkflash solution	SLC Nand, SPI Nand	drivers/rkflash	drivers/rkflash	block device	FAT, EXT, SquashFS
rkflash solution (SPI Nor support)	SPI Nor	drivers/rkflash	drivers/rkflash	block or mtd device	SquashFS, JFFS2
SLC Nand open-source solution	SLC Nand	drivers/mtd/nand/raw	drivers/mtd/nand/raw	mtd device	UBI
SPI Nand open-source solution	SPI Nand	drivers/spi/rockchip_sfc.c	drivers/mtd/nand/spi	mtd device	UBI
SPI Nor open-source solution	SPI Nor	drivers/spi/rockchip_sfc.c	drivers/mtd/spi	mtd or mtd block device	SquashFS, JFFS2

> Note:
>
> 1. The main difference between rkflash and the open-source solution for NAND flash is: rkflash integrates RK FTL (Flash Transfer Layer) in the storage driver, while the open-source solution relies on the file system's own flash management for the FTL part, such as the UBI file system which supports bad block management, wear leveling and other file system features suitable for NAND flash.

5.22.1 Framework Support

MMC & SD

MMC is MultiMediaCard, such as eMMC; SD is a new-generation memory device based on semiconductor flash memory. On Rockchip platforms, they share the `dw_mmc` controller (except RK3399 and RK3399Pro).

Configuration:

```
CONFIG_MMC_DW=y
CONFIG_MMC_DW_ROCKCHIP=y
CONFIG_CMD_MMC=y
```

Driver files:

```
./drivers/mmc/
```

SLC NAND & SPI NAND & SPI NOR Open-Source Solution

With the continuous improvement of the open-source community and the feasibility of the UBI file system, RK has also improved the flash solution with more open-source code. Since the open-source solution defaults to using SPL as the pre-loader, most configurations are combined with SPL-related configurations.

Configuration:

```
// MTD driver support
CONFIG_MTD=y
CONFIG_CMD_MTD_BLK=y
CONFIG_SPL_MTD=y
CONFIG_MTD_BLK=y

// SPI NAND driver support
CONFIG_MTD_SPI_NAND=y
CONFIG_ROCKCHIP_SFC=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
CONFIG_SPL_SPI_SUPPORT=y

// NAND driver support
CONFIG_NAND=y
CONFIG_CMD_NAND=y
CONFIG_NAND_ROCKCHIP=y /* NandC v6, confirm via TRM NANDC->NANDC_NANDC_VER register, 0x00000801 */
//CONFIG_NAND_ROCKCHIP_V9=y  /* NandC v9, confirm via TRM NANDC->NANDC_NANDC_VER register, 0x56393030, e.g.: RK3326/PX30 is this version */
CONFIG_SPL_NAND_SUPPORT=y
CONFIG_SYS_NAND_U_BOOT_LOCATIONS=y
// NAND page size must be defined according to the actual size. If using NAND with capacity >= 512MB, generally configure as 4096
#define CONFIG_SYS_NAND_PAGE_SIZE   2048

// SPI NOR driver support
```

```
CONFIG_CMD_SF=y
CONFIG_CMD_SPI=y
CONFIG_SPI_FLASH=y
CONFIG_SF_DEFAULT_MODE=0x1
CONFIG_SF_DEFAULT_SPEED=50000000
CONFIG_SPI_FLASH_GIGADEVICE=y
CONFIG_SPI_FLASH_MACRONIX=y
CONFIG_SPI_FLASH_WINBOND=y
CONFIG_SPI_FLASH_MTD=y
CONFIG_ROCKCHIP_SFC=y
CONFIG_SPL_SPI_SUPPORT=y
CONFIG_SPL_MTD=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
```

Remove rkflash/rknand macro configuration:

Driver files:

```
./drivers/mtd/nand/raw        // SLC NAND controller driver and protocol layer
./drivers/mtd/nand/spi        // SPI NAND protocol layer
./drivers/spi/rockchip_sfc.c  // SPI Flash controller driver
./drivers/mtd/spi             // SPI NOR protocol layer
./drivers/mtd/mtd_blk.c       // MTD block layer
```

5.22.2 Related Interfaces

Storage driver access interfaces are all connected to the BLK layer, so regardless of storage type, access is unified through:

```
// Get storage handle
struct blk_desc *plat_bootdev(void)

// Access interfaces
unsigned long blk_dread(struct blk_desc *block_dev, lbaint_t start,
                        lbaint_t blkcnt, void *buffer)
unsigned long blk_dwrite(struct blk_desc *block_dev, lbaint_t start,
                         lbaint_t blkcnt, const void *buffer)
unsigned long blk_derase(struct blk_desc *block_dev, lbaint_t start,
                         lbaint_t blkcnt)
```

5.22.3 Type Distinction

U-Boot distinguishes current boot storage type in two ways:

- by environment variables `devtype` and `devnum`
- by `if_type` and `devnum` in `struct blk_desc`

devtype	if_type	devnum	Storage Type	Description
mmc	IF_TYPE_MMC	0	eMMC	-

devtype	if_type	devnum	Storage Type	Description
mmc	IF_TYPE_MMC	1	SD card	-
mtd	IF_TYPE_MTD	0	NAND	MTD open-source solution
mtd	IF_TYPE_MTD	1	SPI NAND	MTD open-source solution
mtd	IF_TYPE_MTD	2	SPI NOR	MTD open-source solution
rknand	IF_TYPE_RKNAND	0	NAND	rkflash solution
spinand	IF_TYPE_SPINAND	0	SPI NAND	rkflash solution
spinor	IF_TYPE_SPINOR	1	SPI NOR	rkflash solution
nvme	IF_TYPE_NVME	0	SSD	-
scsi	IF_TYPE_SCSI	0	SATA	-

5.22.4 DTS Configuration

eMMC configuration:

```
// rkxxxx.dtsi configuration
emmc: dwmmc@ff390000 {
    compatible = "rockchip,px30-dw-mshc", "rockchip,rk3288-dw-mshc";
    reg = <0x0 0xff390000 0x0 0x4000>;     // Controller register base address and length
    max-frequency = <150000000>;          // eMMC normal mode clock is 50MHz; when configured as eMMC
                                          // HS200 mode, this max-frequency takes effect
    clocks = <&cru HCLK_EMMC>, <&cru SCLK_EMMC>,
        <&cru SCLK_EMMC_DRV>, <&cru SCLK_EMMC_SAMPLE>;   // Controller clock numbers
    clock-names = "biu", "ciu", "ciu-drv", "ciu-sample";  // Controller clock names
    fifo-depth = <0x100>;                          // FIFO depth, default configuration
    interrupts = <GIC_SPI 53 IRQ_TYPE_LEVEL_HIGH>;       // Interrupt configuration
    status = "disabled";
};

// rkxxxx-u-boot.dtsi
&emmc {
    bootph-all;
    status = "okay";
}

// rkxxxx.dts
&emmc {
    bus-width = <8>;               // Device bus width
    cap-mmc-highspeed;             // Indicates this slot supports high-speed MMC
    mmc-hs200-1_8v;                // Supports HS200
    supports-emmc;                 // Indicates this slot is eMMC functionality; must be added, otherwise
peripheral initialization will fail
    disable-wp;                    // For slots without a physical WP pin, this must be configured
    non-removable;                 // Indicates this slot is a non-removable device. This is a required item.
    num-slots = <1>;               // Indicates slot number
```

```
    status = "okay";
};
```

Nandc configuration:

```
&nandc0 {
    bootph-all;
    status = "okay";
    #address-cells = <1>;
    #size-cells = <0>;

    nand@0 {
        bootph-all;
        reg = <0>;
        nand-ecc-mode = "hw_syndrome";
        nand-ecc-strength = <16>;
        nand-ecc-step-size = <1024>;
    };
}
```

SFC configuration:

```
&sfc {
    bootph-all;
    status = "okay";
    spi_nand: flash@0 {
        bootph-all;
        compatible = "spi-nand";
        reg = <0>;
        spi-tx-bus-width = <1>;
        spi-rx-bus-width = <4>;
        spi-max-frequency = <96000000>;
    };
    spi_nor: flash@1 {
        bootph-all;
        compatible = "jedec,spi-nor";
        reg = <0>;
        spi-tx-bus-width = <1>;
        spi-rx-bus-width = <4>;
        spi-max-frequency = <96000000>;
    };
};
```

> Note:
>
> 1. For software compatibility, U-Boot only supports SPI flash transfer with `spi-tx-bus-width = <1>` (1-line mode);

5.22.5 Dual Storage Extension

Refer to `Rockchip_Developer_Guide_Dual_Storage_CN.pdf`.

5.22.6 Common Issue Analysis

Q1: How to adjust and confirm the FSPI/SFC controller output clock frequency in the open-source solution?

A1: Set the `spi-max-frequency` property value in the device child node under the `sfc` node in `rkxxxx-u-boot.dtsi`, disable inactive child devices, and then enable debug information in the driver:

```
diff --git a/drivers/spi/rockchip_sfc.c b/drivers/spi/rockchip_sfc.c
index 939b48e377c..62a425a29f4 100644
--- a/drivers/spi/rockchip_sfc.c
+++ b/drivers/spi/rockchip_sfc.c
@@ -790,7 +790,7 @@ static int rockchip_sfc_set_speed(struct udevice *bus, uint speed)
        sfc->cur_speed = speed;
        sfc->cur_real_speed = clk_get_rate(&sfc->clk);

-       dev_dbg(sfc->dev, "set_freq=%dHz real_freq=%dHz\n",
+       dev_err(sfc->dev, "set_freq=%dHz real_freq=%dHz\n",
                sfc->cur_speed, sfc->cur_real_speed);
 #else
        dev_dbg(sfc->dev, "sfc failed, CLK not support\n");
```

5.23 Thermal

5.23.1 Framework Support

Thermal is used to obtain chip temperature sampled by TSADC, currently usually the CPU temperature.

Framework code:

```
./drivers/thermal/thermal-uclass.c
```

Driver code:

```
./drivers/thermal/rockchip_thermal.c
```

Configuration:

```
CONFIG_DM_THERMAL=y
CONFIG_ROCKCHIP_THERMAL=y
```

5.23.2 Related Interfaces

```
// @temp: saves the obtained temperature
int thermal_get_temp(struct udevice *dev, int *temp)
```

5.23.3 DTS Configuration

Kernel DTS usually already contains complete and enabled thermal configuration by default.

5.24 Uart

UART uses `serial-uclass.c` and standard interfaces. It is currently mainly used for UART debug.

Configuration:

```
// Enable configuration
CONFIG_DEBUG_UART
CONFIG_SYS_NS16550

// Parameter configuration
CONFIG_DEBUG_UART_BASE
CONFIG_DEBUG_UART_CLOCK
CONFIG_BAUDRATE
```

Framework code:

```
./drivers/serial/serial-uclass.c
```

Driver code:

```
./drivers/serial/ns16550.c
```

5.24.1 Change Only U-Boot UART

To change UART debug only in the U-Boot stage (using uart2 as an example):

- Disable `CONFIG_ROCKCHIP_PRELOADER_SERIAL`
- Configure UART iomux in `board_debug_uart_init()` (note: some platforms have m0, m1... modes that need configuration)
- Configure UART clock in `board_debug_uart_init()`, ensuring the clock source is 24MHz
- Update `CONFIG_BAUDRATE` in defconfig
- Update `CONFIG_DEBUG_UART_BASE` in defconfig

5.24.2 Global Replacement

Pre-loader serial is the mechanism that enables earlier-stage firmware to share UART debug configuration. These firmware images include: DDR, miniloader, BL31, OP-TEE, and U-Boot. The principle: the DDR bin at the earliest stage configures UART debug and passes it down level by level through the ATAGS parameter mechanism. Each firmware stage obtains the UART debug configuration for use (excluding the kernel).

Users can achieve global UART debug replacement by modifying the serial configuration in the DDR bin. Steps:

DDR bin configuration

The rkbin repository provides tools for users to configure various parameters, including serial port replacement:

```
tools/ddrbin_tool
tools/ddrbin_param.txt
tools/ddrbin_tool_user_guide.txt
```

U-Boot configuration

1. Enable configuration: CONFIG_ROCKCHIP_PRELOADER_SERIAL // Already enabled by default
2. Add the "bootph-all;" attribute to the UART node being used in `rkxx-u-boot.dtsi`
3. Create a serial alias in aliases, because U-Boot finds the target node via aliases and initializes it.

For example, `./arch/arm/dts/rk1808-u-boot.dtsi` creates aliases for all UARTs for convenience:

```
aliases {
    mmc0 = &emmc;
    mmc1 = &sdmmc;

// Must create alias
    serial0 = &uart0;
    serial1 = &uart1;
    serial2 = &uart2;
    serial3 = &uart3;
    serial4 = &uart4;
    serial5 = &uart5;
    serial6 = &uart6;
    serial7 = &uart7;
};

// Must add bootph-all attribute
&uart0 {
    bootph-all;
};
&uart1 {
    bootph-all;
};
&uart2 {
    bootph-all;
    clock-frequency = <24000000>;
```

```
    status = "okay";
};
&uart3 {
    bootph-all;
};
&uart4 {
    bootph-all;
};
```

5.24.3 Disable Console

```
CONFIG_DISABLE_CONSOLE=y
```

5.24.4 Related Interfaces

```
// UART debug interfaces
void putc(const char c);
void puts(const char *s);
int printf(const char *fmt, ...);
void flushc(void);

// Regular UART interfaces for peripheral communication
int serial_dev_getc(struct udevice *dev);
int serial_dev_tstc(struct udevice *dev);
void serial_dev_putc(struct udevice *dev, char ch);
void serial_dev_puts(struct udevice *dev, const char *str);
void serial_dev_setbrg(struct udevice *dev, int baudrate);
void serial_dev_clear(struct udevice *dev);
```

5.25 USB

U-Boot USB mainly includes Device, Host, PHY, and USB peripheral drivers. This chapter provides detailed descriptions of each part's framework configuration, board-level configuration, and related command usage.

5.25.1 Framework Support

Device

Device mode is based on the Gadget framework (not using DM_USB), typically configured as rockusb or fastboot mode for firmware download and verification. The rockusb protocol is based on the UMS protocol, and its state machine is embedded into the UMS framework via hooks. Refer to the rockusb driver implementation for details.

Currently, there are two USB Device controllers: DWC2 and DWC3. Typically, a chip integrates only one OTG controller, so only one controller-related configuration needs to be enabled in CONFIG. To determine which controller is used, refer to the chip TRM or Rockchip USB Developer Guide.

Configuration:

```
CONFIG_USB=y

// Gadget configuration
CONFIG_USB_GADGET=y
CONFIG_USB_GADGET_MANUFACTURER="Rockchip"
CONFIG_USB_GADGET_VENDOR_NUM=0x2207
CONFIG_USB_GADGET_PRODUCT_NUM=0x330a // Configure according to chip ID
CONFIG_USB_GADGET_VBUS_DRAW=2
CONFIG_USB_GADGET_DUALSPEED=y

// rockusb configuration
#define CONFIG_USB_FUNCTION_MASS_STORAGE // Located in include/configs/rkxxx_common.h
CONFIG_USB_GADGET_DOWNLOAD=y
CONFIG_CMD_ROCKUSB=y

// DWC3 controller configuration
CONFIG_USB_DWC3=y
CONFIG_USB_DWC3_GADGET=y

// DWC2 controller configuration
CONFIG_USB_GADGET_DWC2_OTG=y
```

Framework code:

```
// gadget framework
drivers/usb/gadget/g_dnl.c
drivers/usb/gadget/g_dnl.c
drivers/usb/gadget/config.c
drivers/usb/gadget/epautoconf.c
drivers/usb/gadget/usbstring.c
drivers/usb/gadget/f_mass_storage.c
```

Driver code:

```
// rockusb
cmd/rockusb.c
drivers/usb/gadget/f_rockusb.c

// controller
drivers/usb/gadget/dwc2_udc_otg* // DWC2 OTG controller
drivers/usb/dwc3 // DWC3 OTG controller
```

Host

Host controllers include OHCI, EHCI, and xHCI. xHCI can support USB3 devices, but not all chips integrate xHCI controllers. Refer to the chip TRM or Rockchip USB Developer Guide for specific integration details. All Host controller drivers use UPSTREAM code with DM_USB enabled, so Host development mainly involves controller DTS adaptation.

Configuration:

```
CONFIG_USB=y
CONFIG_DM_USB=y

// xHCI
CONFIG_USB_HOST=y
CONFIG_USB_XHCI_HCD=y
CONFIG_USB_XHCI_DWC3=y
CONFIG_USB_DWC3_GENERIC=y

// EHCI
CONFIG_USB_EHCI_HCD=y
CONFIG_USB_EHCI_GENERIC=y

// OHCI
#define CONFIG_USB_OHCI_NEW // Located in include/configs/rkxxx_common.h
#define CONFIG_SYS_USB_OHCI_MAX_ROOT_PORTS      1
CONFIG_USB_OHCI_HCD=y
CONFIG_USB_OHCI_GENERIC=y
```

Framework code:

```
// Framework code
cmd/usb.c
drivers/usb/host/usb-uclass.c

// EHCI
drivers/usb/host/ehci-generic.c
drivers/usb/host/ehci-hcd.c

// OHCI
drivers/usb/host/ohci-generic.c
drivers/usb/host/ohci-hcd.c

// xHCI
drivers/usb/host/xhci.c
drivers/usb/host/xhci-dwc3.c
drivers/usb/host/xhci-mem.c
drivers/usb/host/xhci-ring.c
```

PHY

U-Boot USB PHY mainly includes USB2 and USB3 PHY drivers, using DM_USB configuration, compatible with Linux kernel DTB. For specific PHY IP integration details, refer to the chip TRM or Rockchip USB PHY Developer Guide.

Configuration:

```
CONFIG_PHY=y
CONFIG_PHY_ROCKCHIP_INNO_USB2=y
CONFIG_PHY_ROCKCHIP_USBDP=y
CONFIG_PHY_ROCKCHIP_NANENG_COMBOPHY=y
CONFIG_PHY_ROCKCHIP_TYPEC=y
```

Framework code:

```
drivers/phy/phy-uclass.c
```

Driver code:

```
drivers/phy/rockchip/phy-rockchip-inno-usb2.c
drivers/phy/rockchip/phy-rockchip-usbdp.c
drivers/phy/rockchip/phy-rockchip-naneng-combphy.c
drivers/phy/rockchip/phy-rockchip-typec.c
```

Other peripherals

U-Boot USB peripheral support mainly includes USB HUB, USB keyboard, and UMS devices.

Configuration:

```
// USB keyboard
CONFIG_USB_KEYBOARD=y
CONFIG_USB_KEYBOARD_FN_KEYS=y // Supports F1-F12, INS, HOME and other shortcut keys.

// USB storage device
CONFIG_USB_STORAGE=y
```

Framework code:

```
// Framework code
common/usb.c
drivers/usb/host/usb-uclass.c

// USB keyboard
drivers/input/usb_kbd.c
drivers/input/keyboard-uclass.c

// USB storage device
common/usb_storage.c
```

5.25.2 Board-Level Configuration

Device

Because USB Device does not use DM_USB, board files must provide controller properties such as the controller base address and TX FIFO sizes.

```
// DWC3 controller configuration
```

```
// board/rockchip/evb_rk3588/evb_rk3588.c
#ifdef CONFIG_USB_DWC3
#define CRU_BASE          0xfd7c0000
#define CRU_SOFTRST_CON42   0x0aa8
#define U3PHY_BASE         0xfed80000

static struct dwc3_device dwc3_device_data = {
    .maximum_speed = USB_SPEED_SUPER,
    .base = 0xfc000000,
    .dr_mode = USB_DR_MODE_PERIPHERAL,
    .hsphy_mode = USBPHY_INTERFACE_MODE_UTMIW,
    .index = 0,
    .dis_u2_susphy_quirk = 1,
    .dis_u1u2_quirk = 1,
};

.....
#endif

// DWC2 controller configuration is already implemented in the Rockchip common board.c file; generally no modification is needed.
// arch/arm/mach-rockchip/board.c
```

USB keyboard

If a USB keyboard is used as the U-Boot standard input device, add `usbkbd` to the stdin environment variable. Reference code:

```
// Environment variable configuration is in each board-level header file
// include/configs/evb_rk3568.h

#define ROCKCHIP_DEVICE_SETTINGS \
        "stdin=serial,usbkbd\0" \
```

5.25.3 DTS Configuration

As described above, USB Device does not use DM_USB, so no DT node configuration is required. USB Host and USB PHY drivers are compatible with the Linux kernel DTB and can usually reuse it directly. If configuring to use the U-Boot DTB, refer to the Linux kernel's corresponding node implementations.

5.25.4 Related Commands

rockusb

```
rockusb - Use the rockusb Protocol

Usage:
rockusb <USB_controller> <devtype> <dev[:part]>  e.g. rockusb 0 mmc 0
```

Three ways to enter U-Boot loader upgrade mode:

- Reset + recovery key method
- Enter U-Boot command line and execute the above command to start rockusb, entering upgrade mode
- From the OS, execute "reboot loader" to soft-reboot into upgrade mode

usb

```
usb - USB sub-system

Usage:
usb start - start (scan) USB controller
usb reset - reset (rescan) USB controller
usb stop [f] - stop USB [f]=force stop
usb tree - show USB device tree
usb info [dev] - show available USB devices
usb test [dev] [port] [mode] - set USB 2.0 test mode
   (specify port 0 to indicate the device's upstream port)
   Available modes: J, K, S[E0_NAK], P[acket], F[orce_Enable]
usb storage - show details of USB storage devices
usb dev [dev] - show or set current USB storage device
usb part [dev] - print partition table of one or all USB storage    devices
usb read addr blk# cnt - read `cnt' blocks starting at block `blk#'
   to memory address `addr'
usb write addr blk# cnt - write `cnt' blocks starting at block `blk#'
   from memory address `addr'
```

U-Boot USB does not support hot plug detection, so enumeration and disconnect must be done explicitly with commands.

- Use "usb start" or "usb reset" command to parse controller nodes and scan all devices connected to ports
- Use "usb stop" command to disconnect all devices and destruct controller devices
- "usb info" and "usb tree" commands can view controller information and currently connected device information
- "usb storage" and its sub-commands are for UMS functionality; refer to the command description for specific usage

fastboot

For fastboot configuration and usage, refer to the Fastboot section in `CH04-System_Modules`.

5.26 Vendor Storage

Vendor Storage is used to store small data such as SN and MAC that does not need encryption. The data is stored in reserved NVM partitions such as eMMC or NAND, with multiple backups for reliability.

For detailed information, refer to the `appnote rk vendor storage` document.

5.26.1 Principle Overview

Vendor Storage is split into four partitions: `vendor0`, `vendor1`, `vendor2`, and `vendor3`. Each `vendorX` (X=0, 1, 2, 3) has a monotonically increasing `version` field in its header to indicate when `vendorX` was last updated. Each read operation only reads the newest `vendorX` (i.e., the one with the highest version). On write, the version is incremented and the entire existing data plus new data is moved to the `vendorX+1` partition. For example, if information is read from `vendor2`, after modification it is written to `vendor3`. This provides simple protection against data loss.

5.26.2 Framework Support

U-Boot does not natively support Vendor Storage. Rockchip implements its own Vendor Storage driver.

Configuration:

```
CONFIG_ROCKCHIP_VENDOR_PARTITION
```

Driver files:

```
./arch/arm/mach-rockchip/vendor.c
./arch/arm/include/asm/arch-rockchip/vendor.h
```

5.26.3 Related Interfaces

```
int vendor_storage_read(u16 id, void *pbuf, u16 size)
int vendor_storage_write(u16 id, void *pbuf, u16 size)
```

For the definition and usage of `id`, refer to the `appnote rk vendor storage` document.

5.26.4 Self-Test

Use `rktest vendor` in the U-Boot serial command line to test Vendor Storage functionality.

5.27 Watchdog

5.27.1 Framework Support

Watchdog drivers use `wdt-uclass.c` and standard interfaces.

Configuration:

```
CONFIG_WDT
CONFIG_WDT_ROCKCHIP
```

Framework code:

```
./drivers/watchdog/wdt-uclass.c
```

Driver code:

```
./drivers/watchdog/rockchip_wdt.c
```

5.27.2 Related Interfaces

```
// Set watchdog timeout and start wdt (@flags defaults to 0)
int wdt_start(struct udevice *dev, u64 timeout_ms, ulong flags);
// Stop wdt
int wdt_stop(struct udevice *dev);
int wdt_stop_all(void);
// Feed the watchdog
int wdt_reset(struct udevice *dev);
// Ignored; no underlying driver implementation currently
int wdt_expire_now(struct udevice *dev, ulong flags)
```

By default, U-Boot does not enable or use watchdog functionality in its normal flow. Users may enable it according to product requirements.

6. Chapter-6 Advanced Principles

6.1 kernel-DTB

6.1.1 Design Background

The native U-Boot architecture requires one U-Boot DTS per board, and the DTB generated from the U-Boot DTS is packaged into the U-Boot image itself. This means that on a given SoC platform, N boards require N U-Boot images.

In practice, the main differences between different boards on the same SoC platform are peripherals, while the SoC core part is consistent. To make one SoC platform require only one U-Boot image, RK platforms introduce the kernel DTB mechanism. The essence is to switch to the kernel DTB at an early stage and use its configuration information to initialize peripherals.

Therefore, by supporting the kernel DTB, RK platforms can accommodate board differences such as display, pmic/regulator, pinctrl, clk, and so on.

Enabling the kernel DTB depends on OF_LIVE (live device tree, abbreviated as live-dt).

```
config USING_KERNEL_DTB
	bool "Using dtb from Kernel/resource for U-Boot"
	depends on RKIMG_BOOTLOADER && OF_LIVE
	default y
	help
	  This enable support to read dtb from resource and use it for U-Boot,
	  the uart and emmc will still using U-Boot dtb, but other devices like
	  regulator/pmic, display, usb will use dts node from kernel.
```

6.1.2 Live device tree

Background and principle:

After introducing the kernel DTB, two DTBs exist in the U-Boot stage. Storage, UART, Crypto, and similar modules are associated with the U-Boot DTB, while the remaining modules are associated with the kernel DTB. At different times in the U-Boot stage, these two groups of modules may need to be accessed, and each module may also need to access its own DTB node information.

However, these two groups belong to different DTBs, while `gd->fdt_blob` can only point to one of them and is not convenient to switch freely. Also, because the kernel DTS must eventually be passed to the kernel, certain nodes in the U-Boot DTS cannot simply be overlaid onto the kernel DTS to form a merged DTS.

Live dt solves this problem. Its principle is that during initialization, U-Boot scans the whole DTB directly, converts all DTB nodes into a linked list of `struct device_node`, and binds them to specific device drivers. Afterwards, when a device driver needs to access DTB node information, it directly accesses its own `device_node` and no longer needs to access the original DTB.

This means that both the U-Boot DTB and the kernel DTB are bound to their own groups of device drivers and no longer need direct access to the DTB files.

That resolves the conflicts caused by accessing two DTBs.

More reference:

```
./doc/driver-model/livetree.txt
```

Conversion between fdt and live dt:

The `ofnode` type (`include/dm/ofnode.h`) is an encapsulation format supported by both DT types. When using live dt, `device_node` is used to access DT nodes. When using fdt, offsets are used to access DT nodes. If a driver needs to support both types, use `ofnode` .

`ofnode` structure:

```
/*
 * @np: Pointer to device node, used for live tree
 * @of_offset: Pointer into flat device tree, used for flat tree. Note that this
 *     is not a really a pointer to a node: it is an offset value. See above.
 */
typedef union ofnode_union {
        const struct device_node *np;   /* will be used for future live tree */
        long of_offset;
} ofnode;
```

- Functions starting with `dev_` and `ofnode_` support both DT access methods.
- Functions starting with `of_` support only live dt.
- Functions starting with `fdtdec_` and `fdt_` support only fdt.

6.1.3 Mechanism Implementation

Kernel DTB switching is implemented in `init_kernel_dtb()` inside `./arch/arm/mach-rockchip/board.c` . At this point the U-Boot DTS scan has already completed, and storage drivers such as mmc/nand/nor are working normally.

The kernel DTB is then read from the firmware, a live dt table is built, all device drivers are bound, and finally the `gd->fdt_blob` pointer is updated to point to the kernel DTB.

6.1.4 U-Boot

- After compilation, U-Boot generates two dtb files in `./dts/` :
  - `dt.dtb` : compiled from the DTS specified by `CONFIG_DEFAULT_DEVICE_TREE` in defconfig
  - `dt-spl.dtb` : generated by extracting all nodes in `dt.dtb` with the `bootph-all` property, then removing the properties specified by `CONFIG_OF_REMOVE_PROPS` in defconfig. It usually contains only the nodes required by drivers such as UART, DDR, storage, DMC, UART, MMC, NAND, GRF, CRU, and so on.
- When `CONFIG_DM_KERNEL_DTB` is not enabled, the system uses `dt.dtb` .

- When `CONFIG_DM_KERNEL_DTB` is enabled, the system uses `dt-spl.dtb`.
- After compilation, either `dt.dtb` or `dt-spl.dtb` is renamed to `u-boot.dtb` and appended to the end of `u-boot.bin`. Users can inspect `u-boot.dtb` with `fdtdump`.

6.2 Kernel Parameters Passed by U-Boot

This section introduces how U-Boot passes parameters to the kernel.

6.2.1 cmdline

U-Boot reads `/chosen/bootargs` from the kernel DTB, modifies or appends new content, and then writes it back to `/chosen/bootargs`, thus passing the cmdline.

6.2.2 Memory Size

U-Boot modifies the `/memory` node in the kernel DTB and fills in the available memory size information. Relevant boot logs include:

```
......
## Chapter-6 Booting Android Image at 0x0027f800 ...
Kernel load addr 0x00280000 size 23387 KiB
RAM disk load addr 0x0a200000 size 782 KiB
## Chapter-6 Flattened Device Tree blob at 08300000
   Booting using the fdt blob at 0x8300000
   XIP Kernel Image ... OK
  'reserved-memory' ramoops@110000: addr=110000 size=f0000
   Using Device Tree in place at 0000000008300000, end 0000000008314648

// Memory visible to the kernel
Adding bank: 0x00200000 - 0x08400000 (size: 0x08200000)
Adding bank: 0x0a200000 - 0x80000000 (size: 0x75e00000)
Total: 473.217 ms

Starting kernel ...
```

6.2.3 Other Methods

Other parameter-passing methods are essentially also modifications to the kernel DTB:

Node/Property	Operation	Purpose
/serial-number	Create	Serial number
/memory	Modify	Kernel-visible memory
/display-subsystem/route/route-edp/	Append	Display-related params (edp example)

Node/Property	Operation	Purpose
/chosen/linux,initrd-start	Create	Ramdisk start address
/chosen/linux,initrd-end	Create	Ramdisk end address
/bootargs	Modify	Kernel-visible cmdline
mac-address or local-mac-address in the GMAC node	Modify	MAC address
arch/arm/mach-rockchip/board.c: board_fdt_fixup()	Modify	Board-level fdt fixup

6.3 AB System

6.3.1 AB Data Format

The A/B data structure is located at offset 2 KB in the `misc` partition.

```
/* Magic for the A/B struct when serialized. */
#define AVB_AB_MAGIC "\0AB0"
#define AVB_AB_MAGIC_LEN 4

/* Versioning for the on-disk A/B metadata - keep in sync with avbtool. */
#define AVB_AB_MAJOR_VERSION 1
#define AVB_AB_MINOR_VERSION 0

/* Size of AvbABData struct. */
#define AVB_AB_DATA_SIZE 32

/* Maximum values for slot data */
#define AVB_AB_MAX_PRIORITY 15
#define AVB_AB_MAX_TRIES_REMAINING 7

typedef struct AvbABSlotData {
  /* Slot priority. Valid values range from 0 to AVB_AB_MAX_PRIORITY,
   * both inclusive with 1 being the lowest and AVB_AB_MAX_PRIORITY
   * being the highest. The special value 0 is used to indicate the
   * slot is unbootable.
   */
  uint8_t priority;

  /* Number of times left attempting to boot this slot ranging from 0
   * to AVB_AB_MAX_TRIES_REMAINING.
   */
  uint8_t tries_remaining;

  /* Non-zero if this slot has booted successfully, 0 otherwise. */
  uint8_t successful_boot;

  /* Reserved for future use. */
```

```
  uint8_t reserved[1];
} AVB_ATTR_PACKED AvbABSlotData;

/* Struct used for recording A/B metadata.
 *
 * When serialized, data is stored in network byte-order.
 */
typedef struct AvbABData {
  /* Magic number used for identification - see AVB_AB_MAGIC. */
  uint8_t magic[AVB_AB_MAGIC_LEN];

  /* Version of on-disk struct - see AVB_AB_{MAJOR,MINOR}_VERSION. */
  uint8_t version_major;
  uint8_t version_minor;

  /* Padding to ensure |slots| field start eight bytes in. */
  uint8_t reserved1[2];

  /* Per-slot metadata. */
  AvbABSlotData slots[2];

  uint8_t last_boot;
  /* Reserved for future use. */
  uint8_t reserved2[11];

  /* CRC32 of all 28 bytes preceding this field. */
  uint32_t crc32;
} AVB_ATTR_PACKED AvbABData;
```

For small-capacity storage that has no `misc` partition but does have a `vendor` partition, the data can be stored in `vendor`.

On this basis, a `last_boot` field is added to mark the last bootable firmware. It is mainly used when low-battery conditions or factory production tests consume all retry counts before the system boots fully and calls the `boot_ctrl` service. Refer to the following:

```
typedef struct AvbABSlotData {
  /* Slot priority. Valid values range from 0 to AVB_AB_MAX_PRIORITY,
   * both inclusive with 1 being the lowest and AVB_AB_MAX_PRIORITY
   * being the highest. The special value 0 is used to indicate the
   * slot is unbootable.
   */
  uint8_t priority;

  /* Number of times left attempting to boot this slot ranging from 0
   * to AVB_AB_MAX_TRIES_REMAINING.
   */
  uint8_t tries_remaining;

  /* Non-zero if this slot has booted successfully, 0 otherwise. */
  uint8_t successful_boot;

  /* Reserved for future use. */
  uint8_t reserved[1];
```

```
} AVB_ATTR_PACKED AvbABSlotData;

typedef struct AvbABData {
  /* Magic number used for identification - see AVB_AB_MAGIC. */
  uint8_t magic[AVB_AB_MAGIC_LEN];

  /* Version of on-disk struct - see AVB_AB_{MAJOR,MINOR}_VERSION. */
  uint8_t version_major;
  uint8_t version_minor;

  /* Padding to ensure |slots| field start eight bytes in. */
  uint8_t reserved1[2];

  /* Per-slot metadata. */
  AvbABSlotData slots[2];

  /* mark last boot slot */
  uint8_t last_boot;
  /* Reserved for future use. */
  uint8_t reserved2[11];

  /* CRC32 of all 28 bytes preceding this field. */
  uint32_t crc32;
} AVB_ATTR_PACKED AvbABData;
```

Finally, the meaning of each parameter is summarized as follows.

`AvbABSlotData`:

Parameter	Meaning
priority	Slot priority, where 0 means unbootable and 15 is the highest priority
tries_remaining	Remaining boot attempts, set to 7
successful_boot	Set after the system boots successfully. `1`: slot booted successfully, `0`: slot has not booted successfully

`AvbABData`:

Parameter	Meaning
magic	Struct header: `\0AB0`
version_major	Major version
version_minor	Minor version
slots	Slot boot information, see `AvbABSlotData`
last_boot	Last successfully booted slot. `0`: slot A, `1`: slot B
crc32	Data checksum

6.3.2 AB Boot Modes

The current `system bootctrl` design uses two control modes, and the bootloader supports booting in both modes.

6.3.2.1 successful-boot

After normal entry into the system, `boot_ctrl` sets variables for the current slot according to `androidboot.slot_suffix`:

```
successful_boot = 1;
priority = 15;
tries_remaining = 0;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix
```

During system upgrade, `boot_ctrl` sets:

```
Upgraded slot:
successful_boot = 0;
priority = 14;
tries_remaining = 7;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix

Current slot:
successful_boot = 1;
priority = 15;
tries_remaining = 0;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix
```

After the upgrade completes:

```
Upgraded slot:
successful_boot = 0;
priority = 15;
tries_remaining = 7;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix

Current slot:
successful_boot = 1;
priority = 14;
tries_remaining = 0;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix
```

6.3.2.2 reset-retry

After normal entry into the system, `boot_ctrl` sets variables for the current slot according to `androidboot.slot_suffix`:

```
successful_boot = 0;
priority = 15;
tries_remaining = 7;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix
```

During system upgrade:

```
Upgraded slot:
successful_boot = 0;
priority = 14;
tries_remaining = 7;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix

Current slot:
successful_boot = 0;
priority = 15;
tries_remaining = 7;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix
```

After the upgrade completes:

```
Upgraded slot:
successful_boot = 0;
priority = 15;
tries_remaining = 7;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix

Current slot:
successful_boot = 0;
priority = 14;
tries_remaining = 7;
last_boot = 0 or 1;    :refer to androidboot.slot_suffix
```

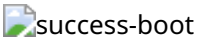
6.3.2.3 Mode Comparison

- `successful_boot` mode
  - Advantage: once the system boots normally, it does not roll back to the old firmware unless `system bootctrl` explicitly configures it
  - Disadvantage: after long-term operation, abnormal storage blocks may cause the system to keep rebooting
- `reset-retry` mode
  - Advantage: always keeps the retry mechanism, which can handle storage anomalies
  - Disadvantage: may roll back to old firmware

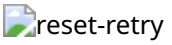
6.3.3 Boot Flow



AB successful_boot mode data flow:



AB reset retry mode data flow:



6.3.4 Upgrade and Exceptions

- System upgrade: refer to the *Rockchip Linux Upgrade Solution Developer Guide*
- Recovery upgrade: the AB system does not consider supporting recovery upgrade

6.3.5 Verification Method

6.3.5.1 successful-boot

1. Flash only slot A. The system boots from slot A. Set booting from slot B, and the system still boots from slot A. After the test, clear the `misc` partition.
2. Flash both slot A and slot B. Boot the system, which is currently slot A. Set the system to boot from slot B and reboot. The current system becomes slot B. After the test, clear the `misc` partition.
3. Flash both slot A and slot B. Quickly reset the system 14 times. After the retry counter is exhausted, the system can still boot from the slot specified by `last_boot`, i.e. it can still boot normally from slot A. After the test, clear the `misc` partition.
4. Flash both slot A and slot B. Boot into slot A, switch to slot B and reboot, then switch back to slot A and reboot. After the test, clear the `misc` partition.

6.3.5.2 reset-retry

1. Flash only slot A. The system boots from slot A. Set booting from slot B, and the system still boots from slot A. After the test, clear the `misc` partition.
2. Flash both slot A and slot B. Boot the system, which is currently slot A. Set the system to boot from slot B and reboot. The current system becomes slot B. After the test, clear the `misc` partition.
3. Flash both slot A and slot B. Quickly reset the system 14 times. After the retry counter is exhausted, the system can still boot from the slot specified by `last_boot`, i.e. it can still boot normally from slot A. After the test, clear the `misc` partition.
4. Flash both slot A and slot B, but corrupt `boot.img` in slot B. Boot into slot A, then set the system to boot from slot B and reboot. The system reboots 7 times and then falls back to slot A successfully. After the test, clear the `misc` partition.
5. Flash both slot A and slot B. Boot into slot A, switch to slot B and reboot, then switch back to slot A and reboot. After the test, clear the `misc` partition.

6.3.6 References

- `Rockchip-Secure-Boot2.0.md`
- `Rockchip-Secure-Boot-Application-Note.md`
- `Android Verified Boot 2.0`

6.4 AVB Secure Boot

6.4.1 References

- `Rockchip-Secure-Boot-Application-Note.md`
- `Android Verified Boot 2.0`
- `Rockchip_Developer_Guide_Linux4.4_SecureBoot_CN.pdf`

6.4.2 Terminology

- AVB: Android Verified Boot
- OTP & efuse: One Time Programmable
- Product RootKey (PRK): the AVB root key, used to verify the root key that signs the loader and U-Boot
- Product Intermediate Key (PIK): intermediate key
- Product Signing Key (PSK): used to sign firmware
- Product Unlock Key (PUK): used to unlock devices

Separating the various keys and making responsibilities explicit reduces the risk of key leakage.

6.4.3 Introduction

This section introduces the Rockchip secure verified boot flow. The secure verified boot flow consists of security verification and integrity verification.

- Security verification verifies the encrypted public key. The process reads the public key hash from secure storage (OTP & efuse), compares it with the computed public key hash, and if they match, uses the public key to decrypt the firmware hash.
- Integrity verification checks firmware integrity. The process loads the firmware from storage, computes its hash, and compares it with the decrypted hash.

6.4.4 Encryption Example

The secure boot verification flow is similar to digital encryption and signature verification in communication. This analogy helps understand AVB verification more quickly.

Suppose Alice sends digital information to Bob. To ensure confidentiality, authenticity, integrity, and non-repudiation, the transmission process is:

1. Alice prepares the plaintext.
2. Alice hashes the plaintext to obtain a digest.
3. Alice signs the digest with her private key and attaches the signature to the information.
4. Alice generates a random encryption key and encrypts the information with it.
5. Alice encrypts that random key with Bob's public key and sends both the encrypted key and the ciphertext to Bob.
6. Bob uses his private key to decrypt the random key.
7. Bob uses the random key to decrypt the ciphertext and recover the plaintext.
8. Bob uses Alice's public key to decrypt Alice's signature and obtains the digest.
9. Bob hashes the plaintext again with the same hash algorithm.
10. Bob compares the two digests. If they match, the information has not been modified.

The DES algorithm above can be replaced with another algorithm such as AES, and the public/private key algorithm can use RSA. The process is shown below:



6.4.5 AVB

AVB stands for Android Verified Boot. It is a firmware verification flow designed by Google, mainly used to verify firmware such as boot and system. Rockchip Secure Boot references the communication-style verification method and AVB to implement a complete Secure Boot solution.

6.4.5.1 AVB Features

- Security verification
- Integrity verification
- Anti-rollback protection
- Support for persistent partitions
- Support for chained partitions, which may use the same signing private key as boot/system or a

separate OEM private key, but must still be signed by the PRK

6.4.5.2 Generate key, signature, and certificate

```
#!/bin/sh
touch temp.bin
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out testkey_prk.pem
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out testkey_psk.pem
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out testkey_pik.pem
python avbtool make_atx_certificate --output=pik_certificate.bin --subject=temp.bin --subject_key=testkey_pik.pem --subject_is_intermediate_authority --subject_key_version 42 --authority_key=testkey_prk.pem
python avbtool make_atx_certificate --output=psk_certificate.bin --subject=product_id.bin --subject_key=testkey_psk.pem --subject_key_version 42 --authority_key=testkey_pik.pem
python avbtool make_atx_metadata --output=metadata.bin --intermediate_key_certificate=pik_certificate.bin --product_key_certificate=psk_certificate.bin
```

Generating `permanent_attributes.bin`:

```
python avbtool make_atx_permanent_attributes --output=permanent_attributes.bin --product_id=product_id.bin --root_authority_key=testkey_prk.pem
```

`product_id.bin` must be user-defined, occupies 16 bytes, and can be used as a product ID.

Example signing of `boot.img`:

```
avbtool add_hash_footer --image boot.img --partition_size 33554432 --partition_name boot --key testkey_psk.pem --algorithm SHA256_RSA4096
```

Note: the partition size must be at least 64 KB larger than the original firmware, must be 4 KB aligned, and must not exceed the partition size defined in `parameter.txt`.

Example signing of `system.img`:

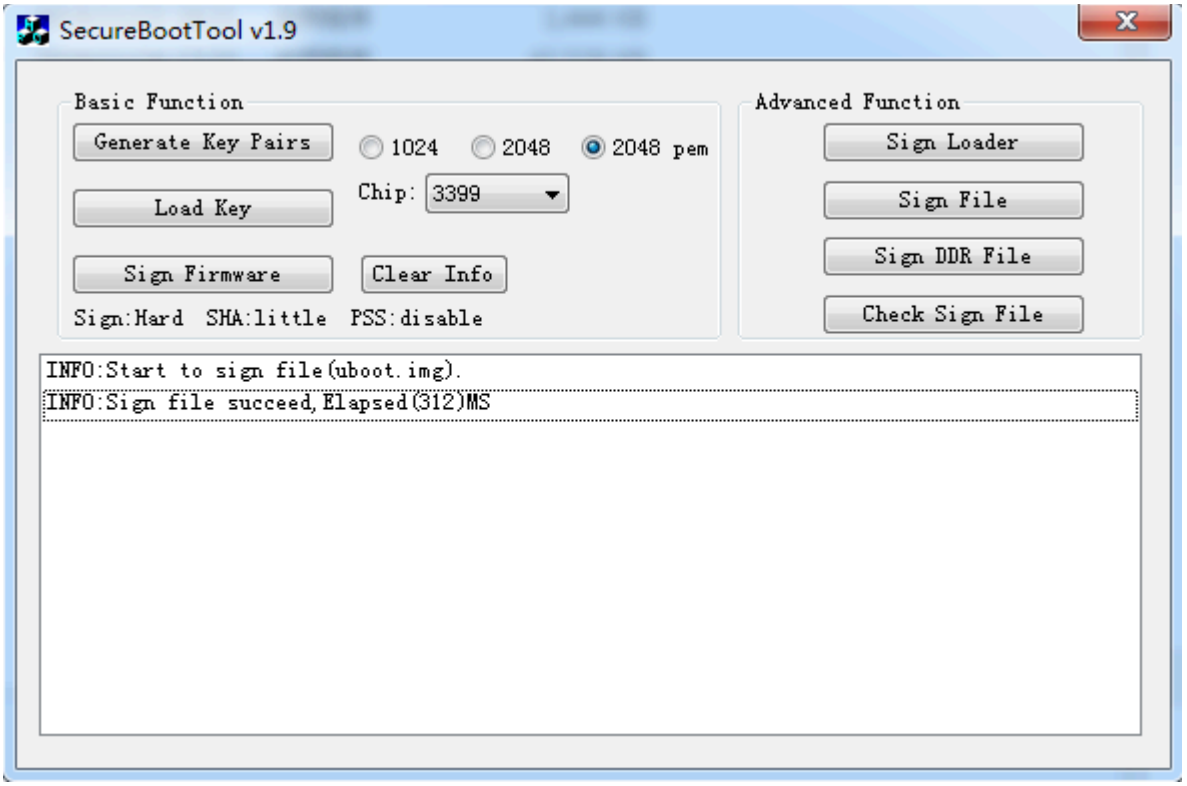
```
avbtool add_hashtree_footer --partition_size 536870912 --partition_name system --image system.img --algorithm SHA256_RSA4096 --key testkey_psk.pem
```

Generate `vbmeta` containing `metadata.bin`:

```
python avbtool make_vbmeta_image --public_key_metadata metadata.bin --include_descriptors_from_image boot.img --include_descriptors_from_image system.img --generate_dm_verity_cmdline_from_hashtree system.img --algorithm SHA256_RSA4096 --key testkey_psk.pem --output vbmeta.img
```

Finally flash the generated `vbmeta.img` into the corresponding partition, such as the `vbmeta` partition.

Use `SecureBootTool` to generate `PrivateKey.pem` and `PublicKey.pem`.



Sign `permanent_attributes.bin`:

```
openssl dgst -sha256 -out permanent_attributes_cer.bin -sign PrivateKey.pem permanent_attributes.bin
```

`permanent_attributes.bin` is the security authentication data for the entire system. Its hash must be burned into efuse or OTP, or the data must be authenticated by the previous secure stage.

- If the platform uses efuse, the planned efuse space is usually not enough to store the certificate hash, so `permanent_attributes.bin` is verified by the previous-stage public key together with the certificate of `permanent_attributes.bin`.
- If the platform uses OTP, the secure data space is sufficient and the data can be burned directly.

For efuse and OTP support status on each platform: **please refer to the document Rockchip OTP Developer Guide.**

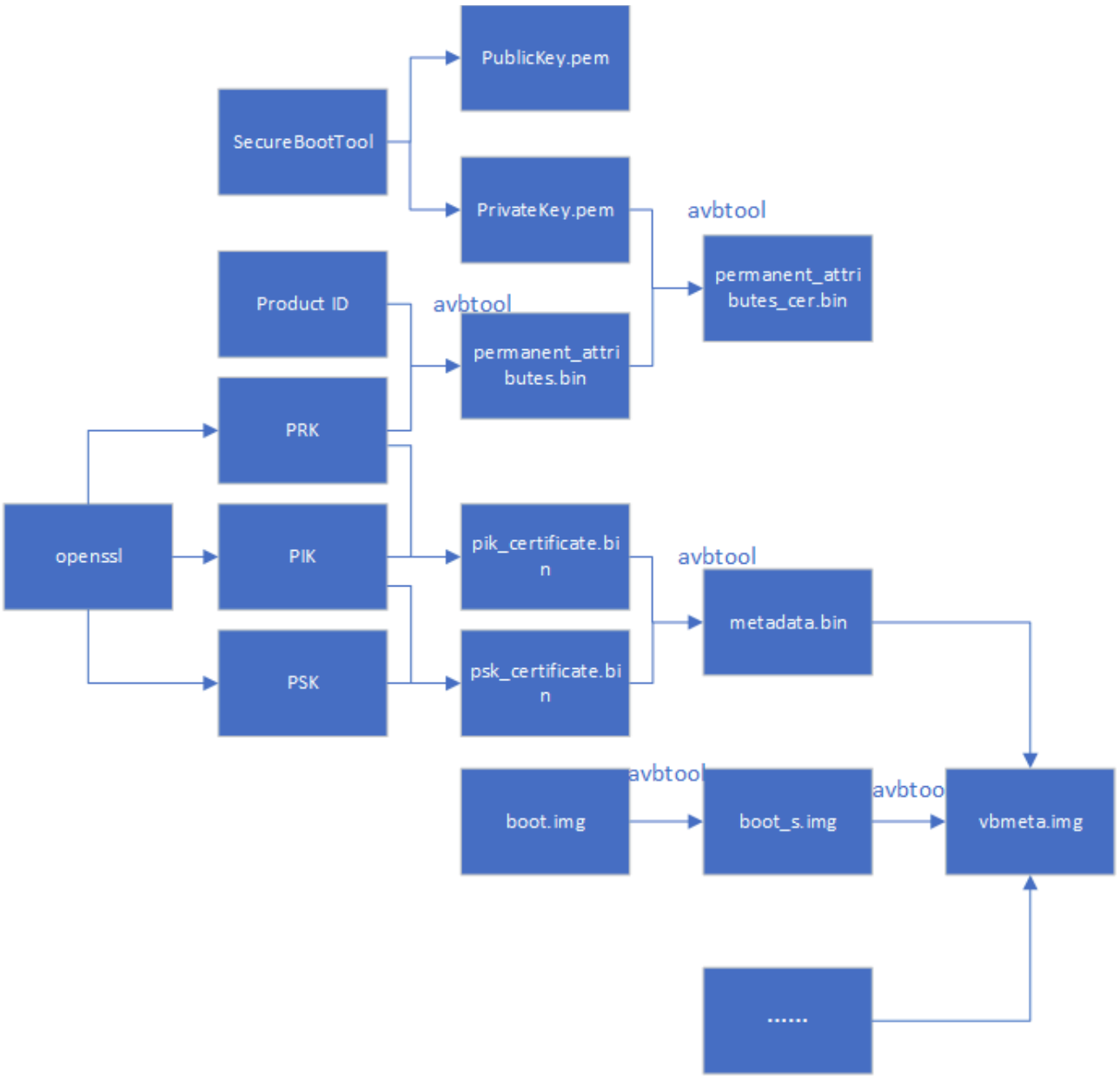
Burn the public key on efuse-based platforms:

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
fastboot stage permanent_attributes_cer.bin
fastboot oem board:fuse at-rsa-perm-attr
```

Burn the public key on OTP-based platforms:

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
```

Overall signing flow:



6.4.5.3 AVB lock

```
fastboot oem board:lock
```

See the fastboot command support section for how to enter fastboot.

6.4.5.4 AVB unlock

Rockchip currently uses strict security verification. Add the following in defconfig:

```
CONFIG_LIBAVB_ATH_UNLOCK_SUPPORT=y
```

Otherwise entering `fastboot oem board:unlock` can unlock the device, and the device may still boot even if verification of `vbmeta.img` or `boot.img` fails.

First generate the PUK:

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out testkey_puk.pem
```

First generate `puk_certificate.bin`, which is used when generating `unlock_credential.bin`:

```
python avbtool make_atx_certificate --output=puk_certificate.bin --subject=product_id.bin --subject_key=testkey_puk.pem --usage=com.google.android.things.vboot.unlock --subject_key_version 42 --authority_key=testkey_pik.pem
```

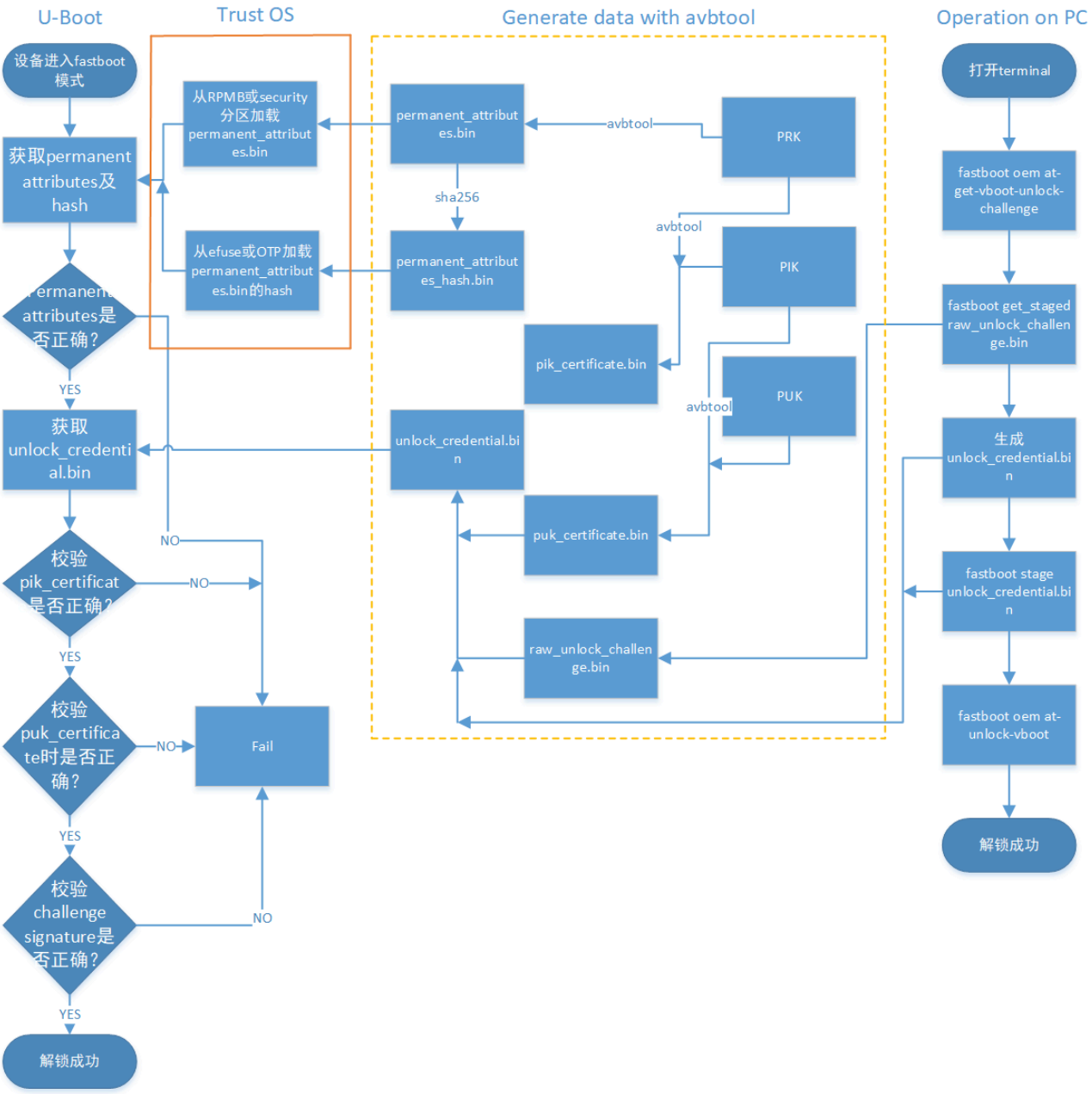
Then generate `unlock_credential.bin` as follows:

```
python avbtool make_atx_unlock_credential --output=unlock_credential.bin --intermediate_key_certificate=pik_certificate.bin --unlock_key_certificate=puk_certificate.bin --challenge=unlock_challenge.bin --unlock_key=testkey_puk.pem
```

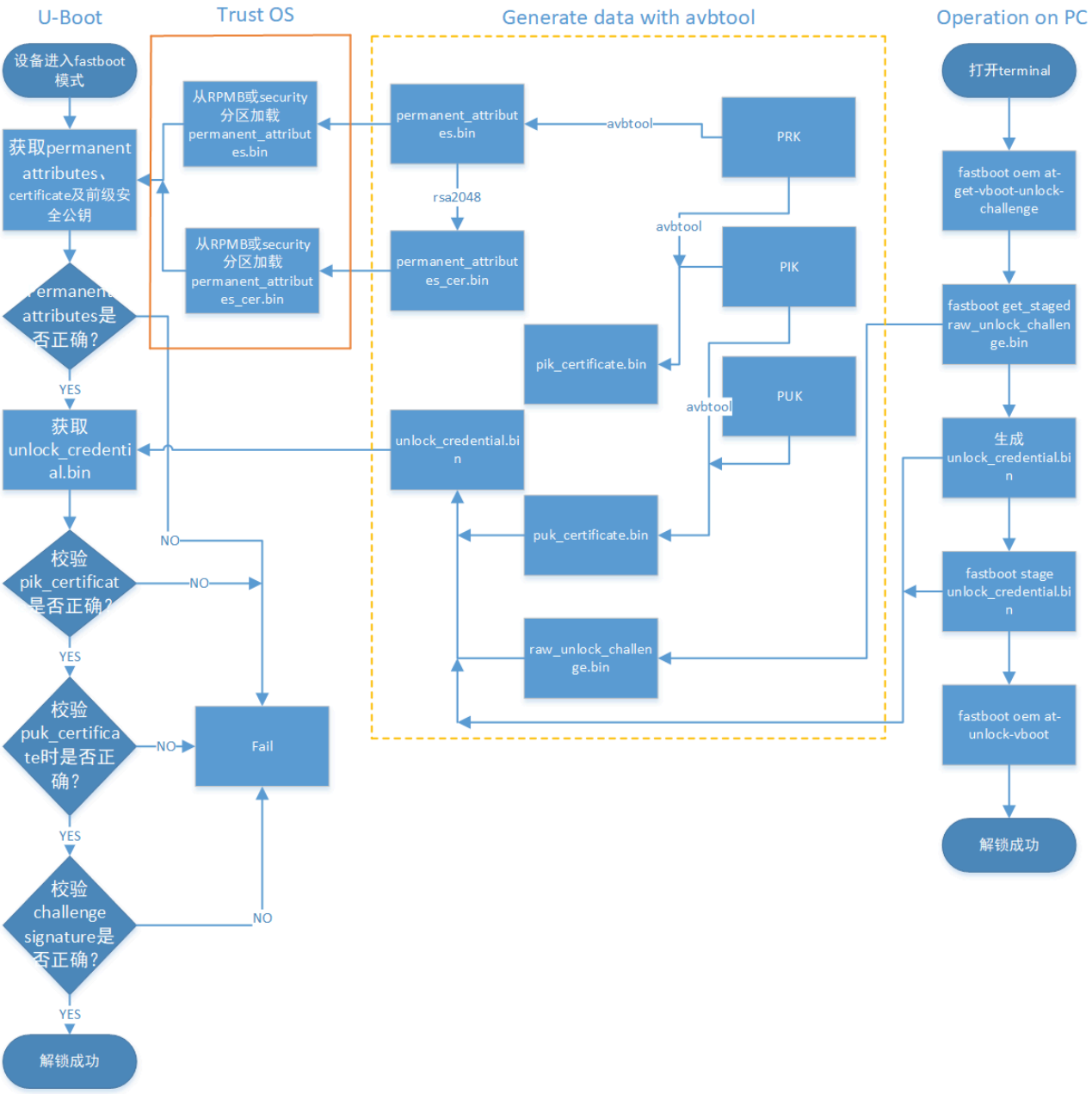
Finally download the certificate to the device with fastboot and unlock it:

```
fastboot stage unlock_credential.bin
fastboot oem board:unlock
```

OTP device unlock flow:



efuse device unlock flow:



The final operation flow is:

1. Put the device into fastboot mode and run:

```
fastboot oem board:at-get-vboot-unlock-challenge
fastboot get_staged raw_unlock_challenge.bin
```

This obtains data containing the version, Product ID, and a 16-byte random number. Extract the random number as `unlock_challenge.bin`.

2. Use `avbtool` to generate `unlock_credential.bin`, referring to `make_unlock.sh`.
3. On the PC:

```
fastboot stage unlock_credential.bin
fastboot oem board:unlock
```

Note: during this process, the device must remain in the same first fastboot session. It must not lose power, shut down, or reboot. After step 1, the device stores the generated random number. If power is lost or the device reboots, the random number is lost, and subsequent challenge signature verification fails due to mismatch.

If the following are enabled:

```
CONFIG_MISC=y
CONFIG_ROCKCHIP_EFUSE=y
CONFIG_ROCKCHIP_OTP=y
```

then CPUID is used as the challenge number. Since CPUID matches the device and is not lost after power-off, the generated `unlock_credential.bin` can be reused. The repeated generation of `unlock_challenge.bin` and `unlock_credential.bin` is no longer needed. The unlock steps become:

```
fastboot oem board:at-get-vboot-unlock-challenge
fastboot stage unlock_credential.bin
fastboot oem board:unlock
```

4. The device enters the unlocked state.

`make_unlock.sh` reference:

```
#!/bin/sh
python avb-challenge-verify.py raw_unlock_challenge.bin product_id.bin
python avbtool make_atx_unlock_credential --output=unlock_credential.bin --
intermediate_key_certificate=pik_certificate.bin --unlock_key_certificate=puk_certificate.bin --
challenge=unlock_challenge.bin --unlock_key=testkey_puk.pem
```

Source of `avb-challenge-verify.py`:

```
#!/user/bin/env python
"This is a test module for getting unlock_challenge.bin"
import sys
import os
from hashlib import sha256

def challenge_verify():
    if (len(sys.argv) != 3) :
        print "Usage: rkpublickey.py [challenge_file] [product_id_file]"
        return
    if ((sys.argv[1] == "-h") or (sys.argv[1] == "--h")):
        print "Usage: rkpublickey.py [challenge_file] [product_id_file]"
        return
    try:
        challenge_file = open(sys.argv[1], 'rb')
        product_id_file = open(sys.argv[2], 'rb')
        challenge_random_file = open('unlock_challenge.bin', 'wb')
        challenge_data = challenge_file.read(52)
        product_id_data = product_id_file.read(16)
        product_id_hash = sha256(product_id_data).digest()
        print("The challege version is %d" %ord(challenge_data[0]))
        if (product_id_hash != challenge_data[4:36]) :
```

```
            print("Product id verify error!")
            return
        challenge_random_file.write(challenge_data[36:52])
        print("Success!")

    finally:
        if challenge_file:
            challenge_file.close()
        if product_id_file:
            product_id_file.close()
        if challenge_random_file:
            challenge_random_file.close()

if __name__ == '__main__':
    challenge_verify()
```

6.4.5.5 kernel Configuration

Verification of partitions such as system, vendor, and oem is performed by the kernel's dm-verity module, so that module must be enabled.

To enable AVB, configure the `avb` parameters in the kernel DTS:

```
&firmware_android {
    compatible = "android,firmware";
    boot_devices = "fe330000.sdhci";
    vbmeta {
        compatible = "android,vbmeta";
        parts = "vbmeta,boot,system,vendor,dtbo";
    };
    fstab {
        compatible = "android,fstab";
        vendor {
            compatible = "android,vendor";
            dev = "/dev/block/by-name/vendor";
            type = "ext4";
            mnt_flags = "ro,barrier=1,inode_readahead_blks=8";
            fsmgr_flags = "wait,avb";
        };
    };
};
```

To enable the A/B system, configure the `slotselect` parameter:

```
firmware {
    android {
        compatible = "android,firmware";
        fstab {
            compatible = "android,fstab";
            system {
                compatible = "android,system";
                dev = "/dev/block/by-name/system";
```

```
            type = "ext4";
            mnt_flags = "ro,barrier=1,inode_readahead_blks=8";
            fsmgr_flags = "wait,verify,slotselect";
        };
        vendor {
            compatible = "android,vendor";
            dev = "/dev/block/by-name/vendor";
            type = "ext4";
            mnt_flags = "ro,barrier=1,inode_readahead_blks=8";
            fsmgr_flags = "wait,verify,slotselect";
        };
    };
  };
};
```

6.4.5.6 Android SDK

Below are some relevant Android SDK configurations.

AVB Enable

Enable `BOARD_AVB_ENABLE`.

6.4.5.7 New cmdline content

```
Kernel command line: androidboot.verifiedbootstate=green androidboot.slot_suffix=_a dm="1 vroot none ro 1,0 1031864 verity 1 PARTUUID=b2110000-0000-455a-8000-44780000706f PARTUUID=b2110000-0000-455a-8000-44780000706f 4096 4096 128983 128983 sha1 90d1d406caac04b7e3fbf48b9a4dcd6992cc628e 4172683f0d6b6085c09f6ce165cf152fe3523c89 10 restart_on_corruption ignore_zero_blocks use_fec_from_device PARTUUID=b2110000-0000-455a-8000-44780000706f fec_roots 2 fec_blocks 130000 fec_start 130000" root=/dev/dm-0 androidboot.vbmeta.device=PARTUUID=f24f0000-0000-4e1b-8000-791700006a98 androidboot.vbmeta.avb_version=1.1 androidboot.vbmeta.device_state=unlocked androidboot.vbmeta.hash_alg=sha512 androidboot.vbmeta.size=6528 androidboot.vbmeta.digest=41991c02c82ea1191545c645e2ac9cc7ca08b3da0a2e3115aff479d2df61feaccdd35b6360cfa936f6f4381e4557ef18e381f4b236000e6ecc9ada401eda4cae androidboot.vbmeta.invalidate_on_error=yes androidboot.veritymode=enforcing
```

A few key points:

1. `vbmeta` PARTUUID is passed so that the kernel can verify `vbmeta` again later. The digest is `androidboot.vbmeta.digest`.
2. `skip_initramfs` is needed when the boot ramdisk is not packed into `boot.img`. In an A/B system, cmdline needs to pass this parameter.
3. `root=/dev/dm-0` enables dm-verity and points to system.
4. `androidboot.vbmeta.device_state` indicates Android verify state.
5. `androidboot.verifiedbootstate` indicates the verification result.

Verification state meanings:

- `green`: locked state, and the verification key was not set by the end user

- `yellow`: locked state, and the verification key was set by the end user
- `orange`: unlocked state

A special note on the generation of `dm="1 vroot none ro....."` parameters:

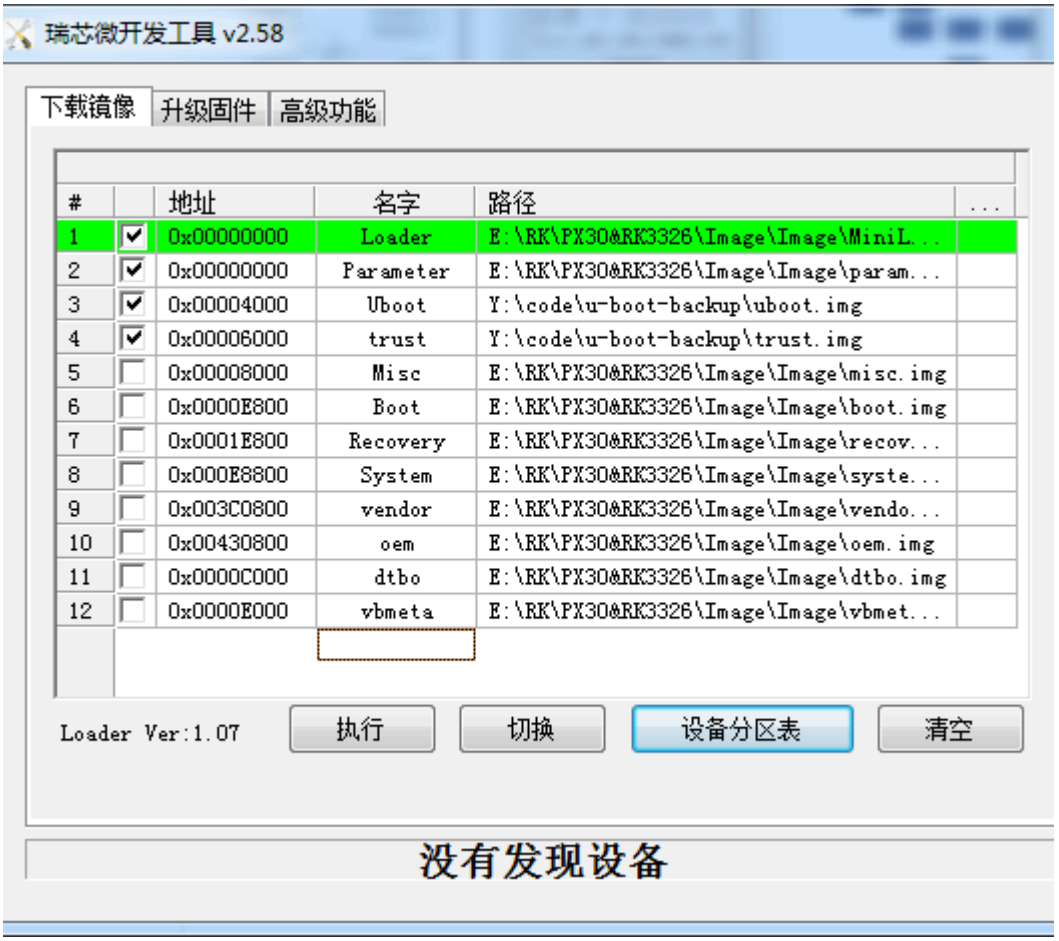
```
avbtool make_vbmeta_image --include_descriptors_from_image boot.img --include_descriptors_from_image system.img --generate_dm_verity_cmdline_from_hashtree system.img --include_descriptors_from_image vendor.img --algorithm SHA512_RSA4096 --key testkey_psk.pem --public_key_metadata metadata.bin --output vbmeta.img
```

When generating vbmeta with avbtool, add `--generate_dm_verity_cmdline_from_hashtree` for the system firmware. The `dm="1 vroot none ro....."` information will be saved to vbmeta. This part is Android-specific; if verification only goes up to `boot.img`, there is no need to add this parameter.

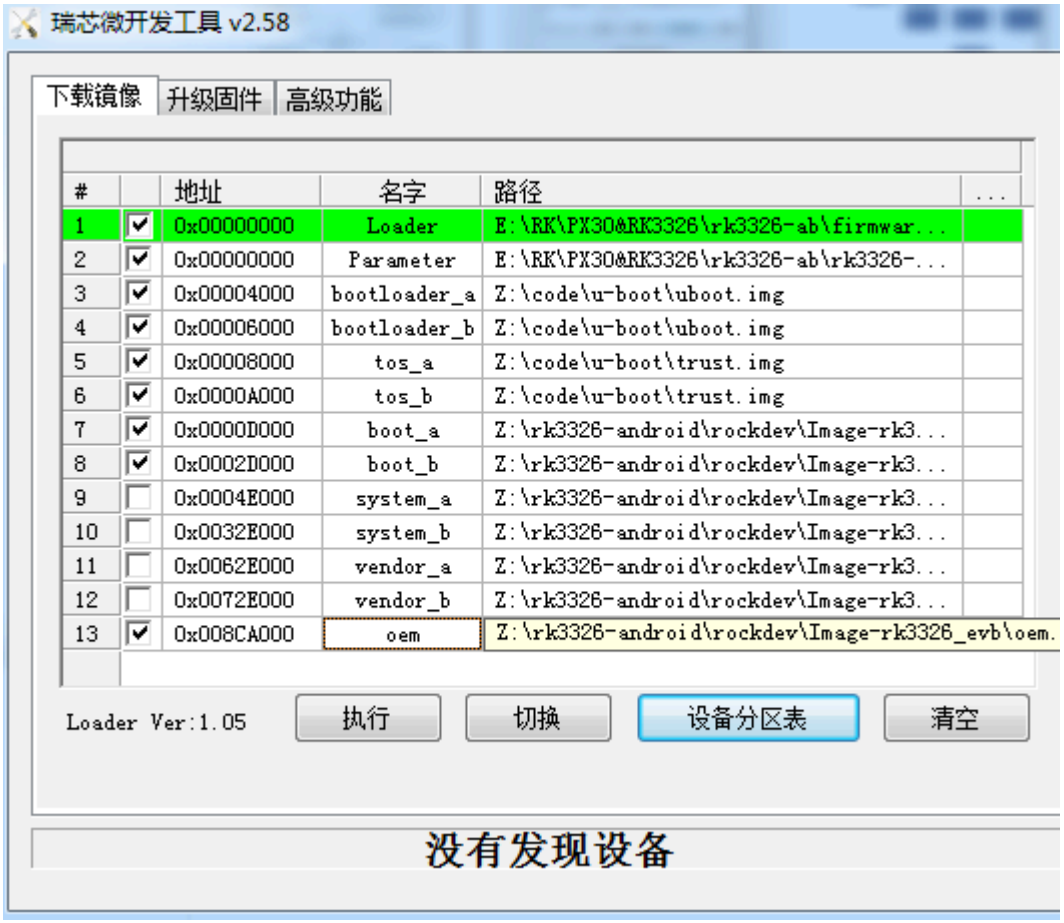
Enabling Android SDK AVB-related configuration automatically includes this information in `vbmeta`.

6.4.6 Firmware Flashing

The Windows firmware flashing tool is shown below:

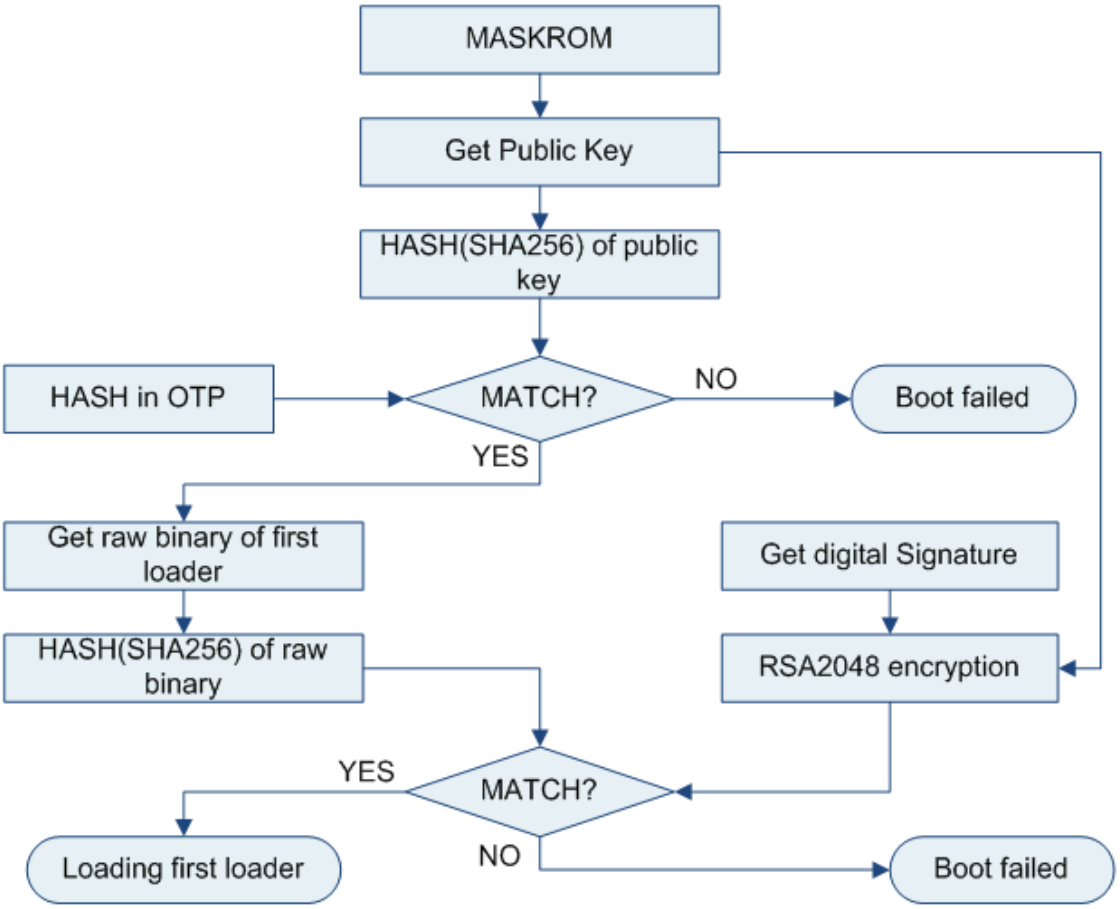


A/B system flashing:



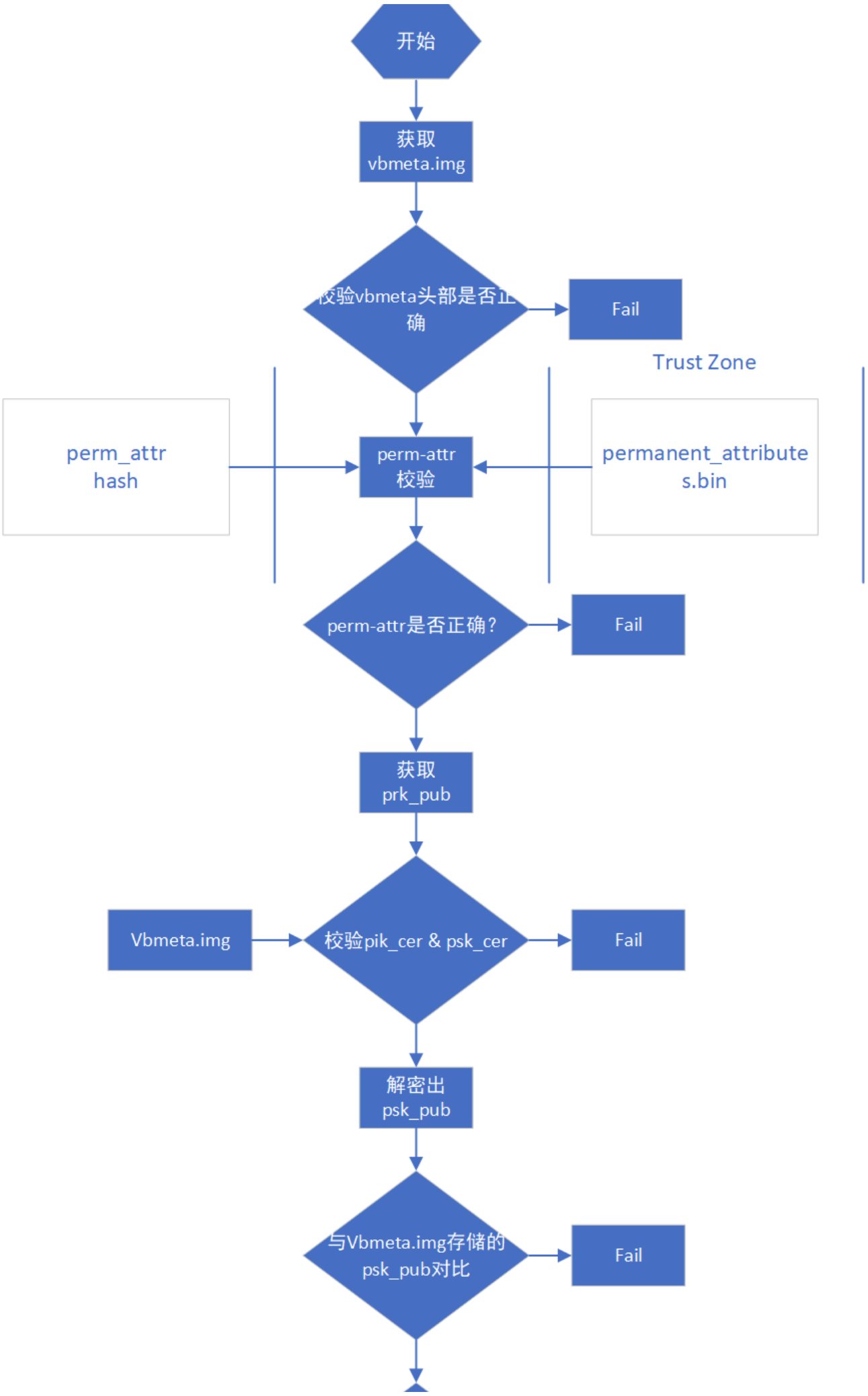
6.4.7 Pre-loader verified

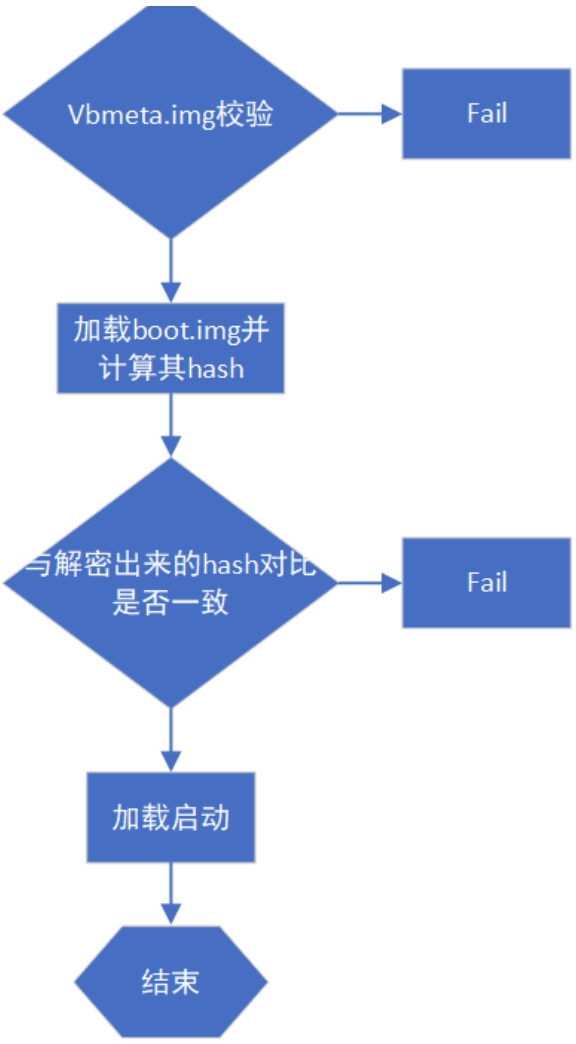
Refer to `Rockchip-Secure-Boot-Application-Note.md`.



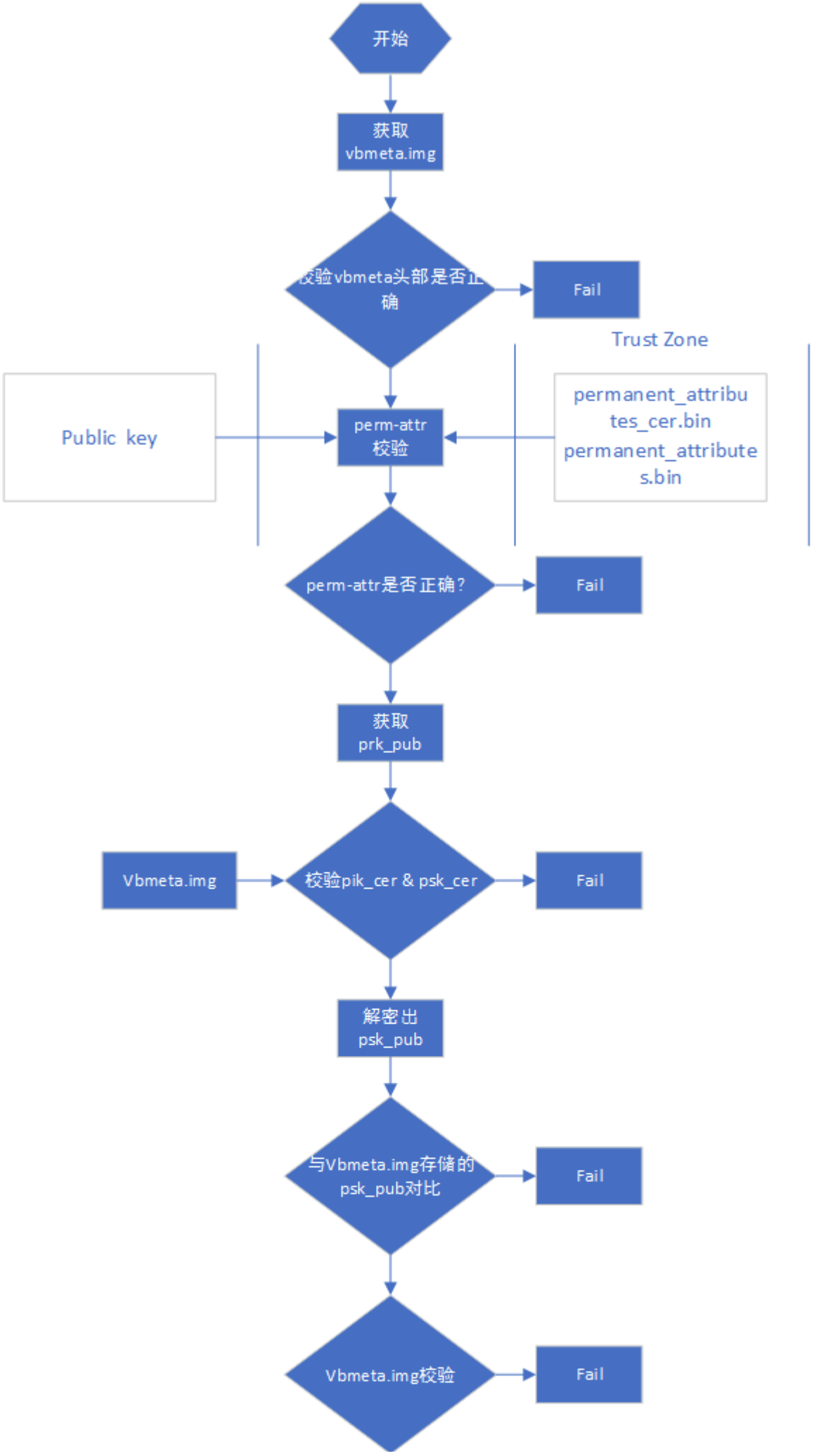
6.4.8 U-Boot verified

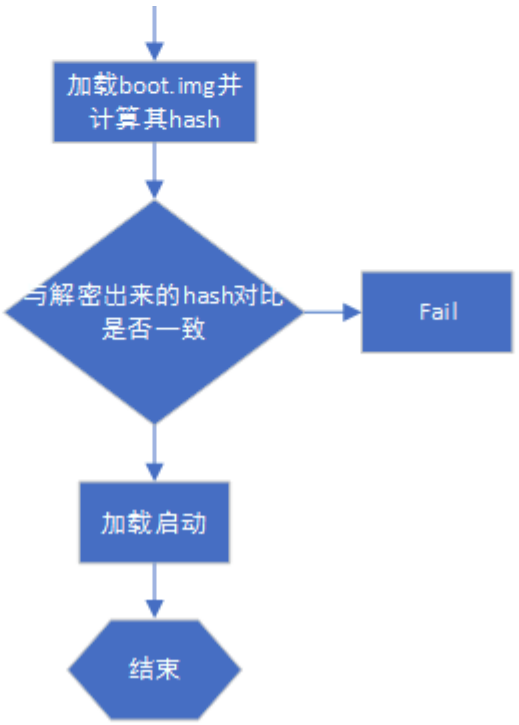
OTP device verification flow:


开始
获取
vbmeta.img
校验vbmeta头部是否正确
Fail
Trust Zone
perm_attr
hash
perm-attr
校验
permanent_attributes.bin
perm-attr是否正确?
Fail
获取
prk_pub
Vbmeta.img
校验pik_cer & psk_cer
Fail
解密出
psk_pub
与Vbmeta.img存储的
psk_pub对比
Fail

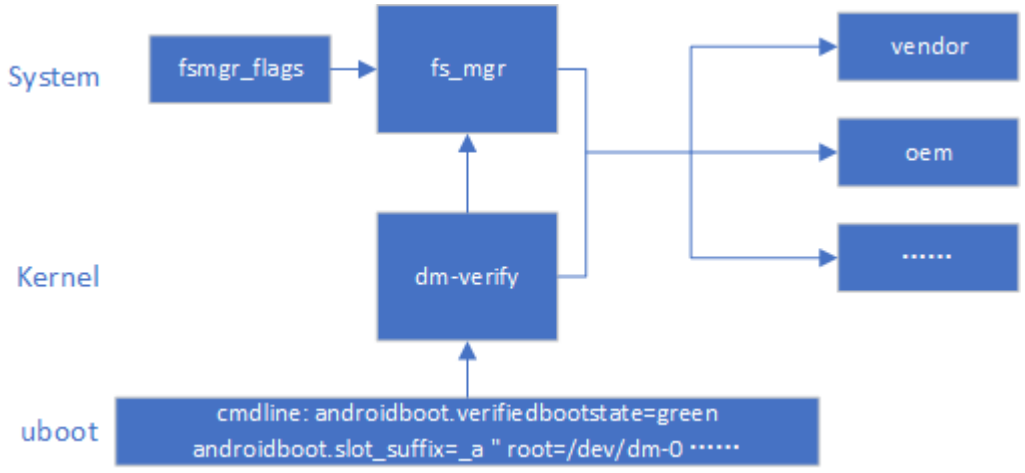


efuse device verification flow:


开始
获取 vbmeta.img
校验vbmeta头部是否正确
Fail
Trust Zone
Public key
perm-attr 校验
permanent_attributes_cer.bin
permanent_attributes.bin
perm-attr是否正确?
Fail
获取 prk_pub
Vbmeta.img
校验pik_cer & psk_cer
Fail
解密出 psk_pub
与Vbmeta.img存储的psk_pub对比
Fail
Vbmeta.img校验
Fail



6.4.9 System Verification Boot



After the system boots to the kernel, the kernel first parses the cmdline parameters passed by U-Boot to determine whether dm-verity is used. It then loads the `fs_mgr` service for system startup. `fs_mgr` verifies and loads firmware according to `fsmgr_flags`, with firmware hash and hash tree stored in `vbmeta.img`.

Main parameters:

- `avb`: use AVB mode to load and verify partitions
- `slotselect`: the partition is part of A/B, so loading uses the cmdline parameter `androidboot.slot_suffix=_a`

6.4.10 Linux AVB

This section introduces AVB operation and verification flow in a Linux environment.

6.4.10.1 Operation Flow

1. Generate the complete firmware set.
2. Use `SecureBootConsole` / `rk_sign_tool` to generate `PrivateKey.pem` and `PublicKey.pem`. The commands are as follows:

```
rk_sign_tool cc --chip 3399
rk_sign_tool kk --out .
```

3. Load the key.

```
rk_sign_tool lk --key privateKey.pem --pubkey publicKey.pem
```

4. Sign the loader.

```
rk_sign_tool sl --loader loader.bin
```

5. Sign `uboot.img` and `trust.img`.

```
rk_sign_tool si --img uboot.img
rk_sign_tool si --img trust.img
```

6. Prepare AVB-signed firmware: prepare an empty `temp.bin`, 16-byte `product_id.bin`, and the `boot.img` to be signed, then run the following script:

```
#!/bin/bash
touch temp.bin
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out testkey_prk.pem
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out testkey_psk.pem
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out testkey_pik.pem
python avbtool make_atx_certificate --output=pik_certificate.bin --subject=temp.bin --subject_key=testkey_pik.pem --subject_is_intermediate_authority --subject_key_version 42 --authority_key=testkey_prk.pem
python avbtool make_atx_certificate --output=psk_certificate.bin --subject=product_id.bin --subject_key=testkey_psk.pem --subject_key_version 42 --authority_key=testkey_pik.pem
python avbtool make_atx_metadata --output=metadata.bin --intermediate_key_certificate=pik_certificate.bin --product_key_certificate=psk_certificate.bin
python avbtool make_atx_permanent_attributes --output=permanent_attributes.bin --product_id=product_id.bin --root_authority_key=testkey_prk.pem
python avbtool add_hash_footer --image boot.img --partition_size 33554432 --partition_name boot --key testkey_psk.pem --algorithm SHA256_RSA4096
python avbtool make_vbmeta_image --public_key_metadata metadata.bin --include_descriptors_from_image boot.img --algorithm SHA256_RSA4096 --key testkey_psk.pem  --output vbmeta.img
openssl dgst -sha256 -out permanent_attributes_cer.bin -sign PrivateKey.pem permanent_attributes.bin
```

This generates `vbmeta.img`, `permanent_attributes_cer.bin`, and `permanent_attributes.bin`.

This step also signs `boot.img`...

7. Flash the firmware.

```
rkdeveloptool db loader.bin
rkdeveloptool ul loader.bin
rkdeveloptool gpt parameter.txt
rkdeveloptool wlx uboot uboot.img
rkdeveloptool wlx trust trust.img
rkdeveloptool wlx boot boot.img
rkdeveloptool wlx system system.img
```

`rkdeveloptool` reference: https://github.com/rockchip-linux/rkdeveloptool

8. Flash `permanent_attributes_cer.bin` and `permanent_attributes.bin`.

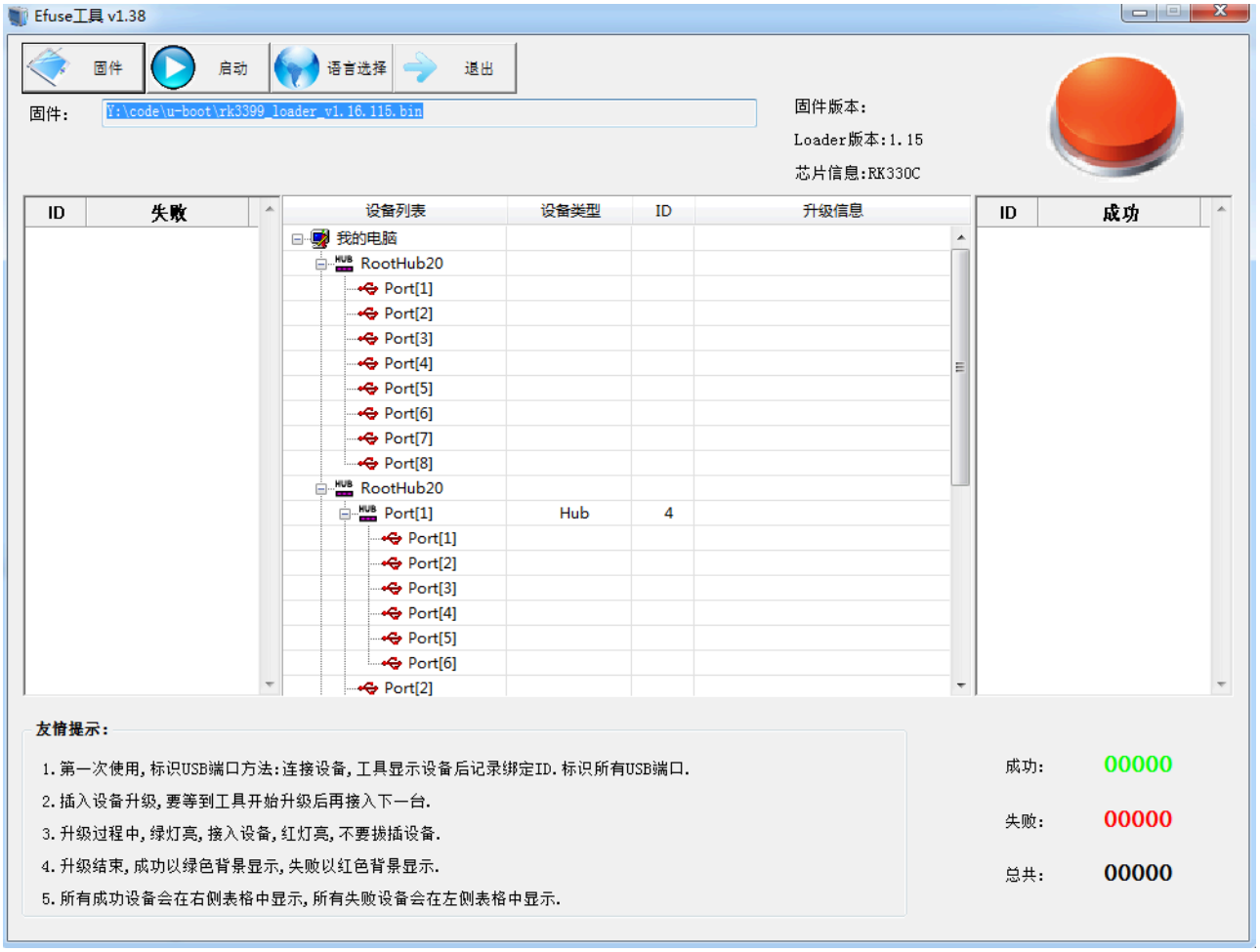
For OTP platforms:

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
```

For efuse platforms:

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
fastboot stage permanent_attributes_cer.bin
fastboot oem board:fuse at-rsa-perm-attr
```

9. Burn efuse (the efuse tool is currently only available for Windows). Select the specific loader, select the corresponding device, and click start to burn.



10. Burn the loader public key on OTP platforms if required.

Refer to `Rockchip-Secure-Boot-Application-Note.md`.

6.4.10.2 Verification Flow

The verification flow should be completed according to the production environment. Add full validation steps when the platform-specific secure boot procedure is finalized.

6.5 SD Boot and Upgrade

6.5.1 Introduction

Rockchip currently divides SD cards into regular SD cards, SD upgrade cards, SD boot cards, and SD repair cards. The Rockchip upgrade disk creation tool can write `update.img` into the SD card and create different card types.

Card Type	Function
Regular SD card	Ordinary storage device
SD upgrade card	Device boots from SD into recovery, and recovery updates firmware from SD to local storage
SD boot card	Device boots directly from SD
SD repair card	Copies firmware from the SD card to local storage starting from the pre-loader stage

6.5.2 Categories

6.5.2.1 Regular Card

A regular SD card behaves exactly like an SD card used on a PC. It can be used as normal storage in both U-Boot and the kernel without any special preparation.

6.5.2.2 Upgrade Card

An SD upgrade card is created using RK tools and upgrades firmware in local storage such as eMMC or NAND flash from the SD card. It is a firmware upgrade method independent of a PC or network.

The tool writes SD boot code into the reserved area of the SD card, then copies the firmware into the visible partition of the SD card. When the SoC boots from the SD card, the SD startup code and upgrade code flash the firmware into local storage.

An SD upgrade card also supports PCBA testing and copying demo files. This makes it possible to upgrade firmware without a PC, improving production efficiency.

To update only firmware and demo files on an already created SD upgrade card:

1. Copy the firmware to the root directory of the SD card and rename it to `sdupdate.img`
2. Copy demo files into the `Demo` directory under the SD card root directory

SD upgrade card format (non-GPT):

Offset	Data Segment
Sector 0	MBR
Sector 64-4M	IDBLOCK (startup flag set to 0)
4M-8M	Parameter
12M-16M	uboot
16M-20M	trust
...	misc
...	resource
...	kernel
...	recovery
Remaining	Fat32 containing update.img

SD upgrade card format (GPT):

Offset	Data Segment
Sector 0	MBR
Sector 1-34	GPT partition table
Sector 64-4M	IDBLOCK (startup flag set to 0)
4M-8M	Parameter
...	uboot
...	trust
...	misc
...	resource
...	kernel
...	recovery
Remaining	Fat32 containing update.img

6.5.2.3 Boot Card

An SD boot card is created using RK tools and allows direct boot from the SD card, making it very convenient to test new firmware without reflashing local device storage. Firmware and temporary files are all stored on the SD card, and the system can work normally with or without local storage. It is mainly used to boot the device system from the SD card or for PCBA testing.

Note: PCBA testing is only one function item under recovery, and it can be used on both upgrade cards and boot cards.

SD boot card format (non-GPT):

Offset	Data Segment
Sector 0	MBR
Sector 64-4M	IDBLOCK (startup flag set to 1)
4M-8M	Parameter
8M-12M	uboot
12M-16M	trust
...	misc
...	resource
...	boot
...	kernel
...	recovery
...	system
...	user

SD boot card format (GPT):

Offset	Data Segment
Sector 0	MBR
Sector 1-34	GPT partition table
Sector 64-4M	IDBLOCK (startup flag set to 1)
...	uboot
...	Boot
...	trust
...	resource
...	kernel

Offset	Data Segment
...	recovery
...	system
...	vendor
...	oem
...	user
Last 33 sectors	Backup GPT

6.5.2.4 Repair Card

An SD repair card is similar to the SD upgrade function, but the firmware upgrade is performed by the miniloader. It is mainly used when device firmware is corrupted and the SD card is used to repair the device. The tool writes the startup code into the reserved area of the SD card, then copies the firmware into the visible partition of the SD card. When the SoC boots from the SD card, the SD upgrade code flashes the firmware into local storage.

SD repair card format (non-GPT):

Offset	Data Segment
Sector 0	MBR
Sector 64-4M	IDBLOCK (startup flag set to 2)
4M-8M	Parameter
8M-12M	uboot
12M-16M	trust
...	misc
...	resource
...	boot
...	kernel
...	recovery
...	system
...	user

SD repair card format (GPT):

Offset	Data Segment
Sector 0	MBR

Offset	Data Segment
Sector 1-34	GPT partition table
Sector 64-4M	IDBLOCK (startup flag set to 2)
...	uboot
...	Boot
...	trust
...	resource
...	kernel
...	recovery
...	system
...	vendor
...	oem
...	user
Last 33 sectors	Backup GPT

6.5.3 Firmware Flags

Different special card types are distinguished by flags stored in the SD card.

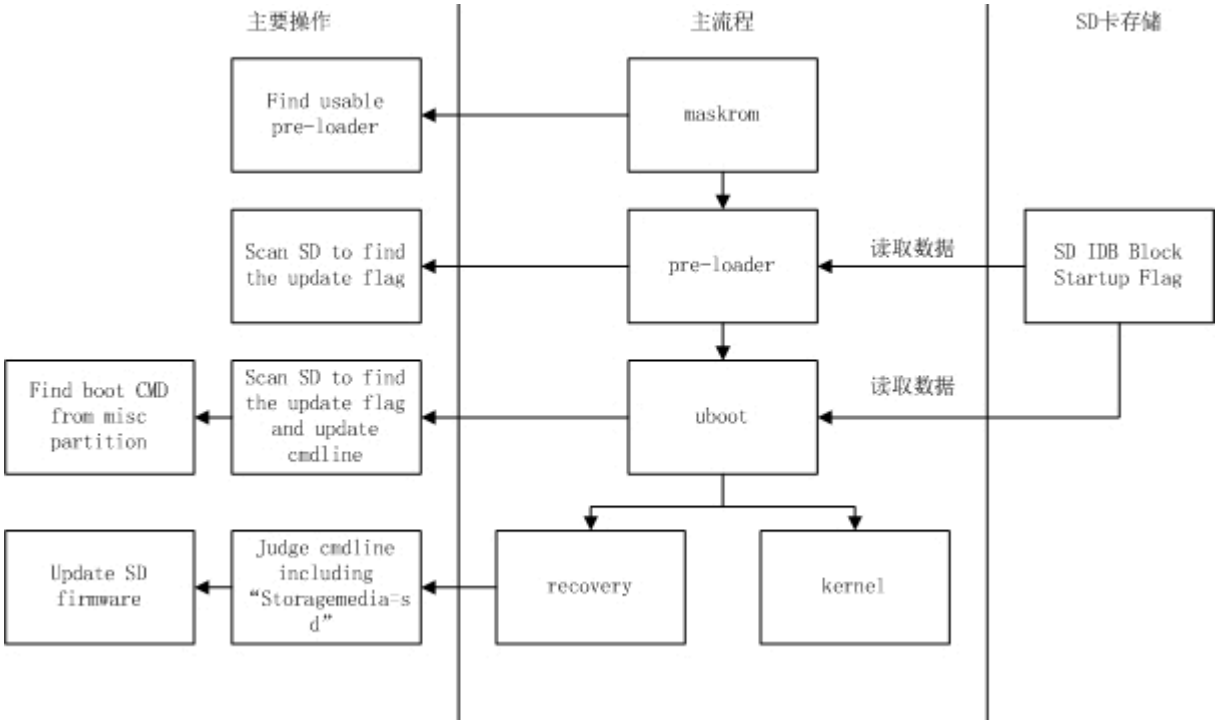
At sector 64 of the SD card, if the starting magic number is `0xFCDC8C3B`, the card is treated as a special card and firmware is loaded from the SD card to boot the device. Otherwise it is treated as a normal SD card.

At offset `(sector 64 + 616 bytes)`, the card-type flag is stored. Currently there are three types:

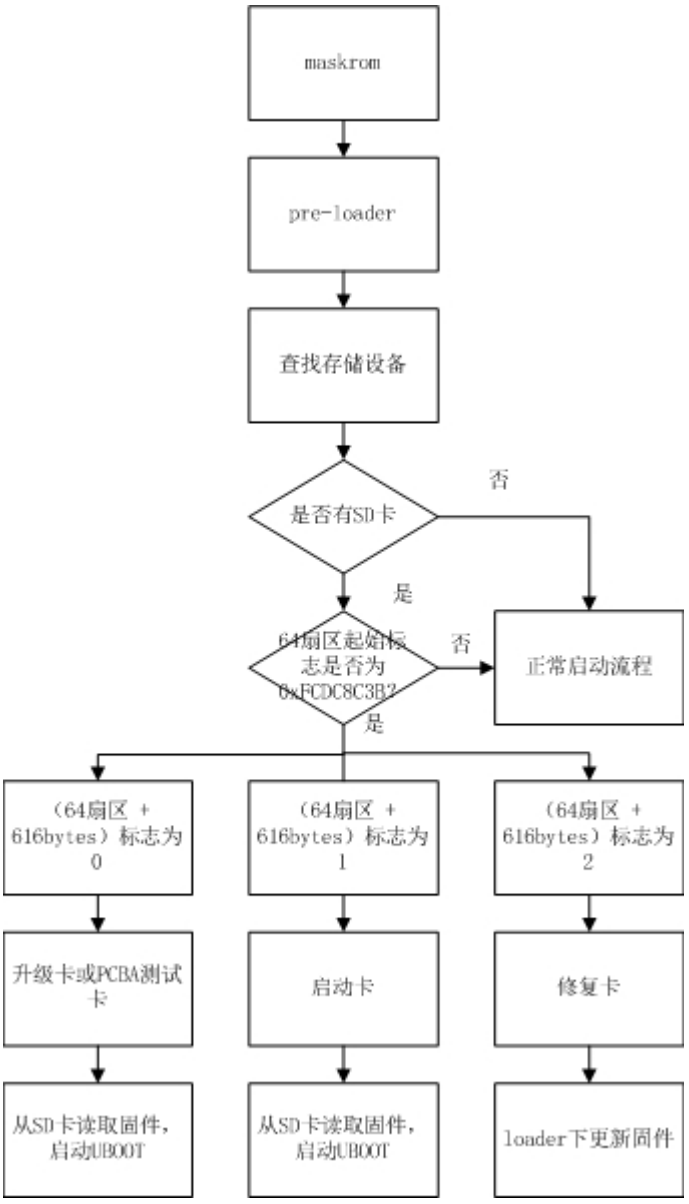
Flag	Card Type
0	Upgrade card or PCBA test card
1	Boot card
2	Repair card

6.5.4 Boot Flow

The SD card boot flow can be divided into the pre-loader boot flow and the U-Boot boot flow. Both flows need to detect the SD card and the Startup Flag in the IDB block, and they execute different functions according to those flags.



6.5.4.1 pre-loader boot

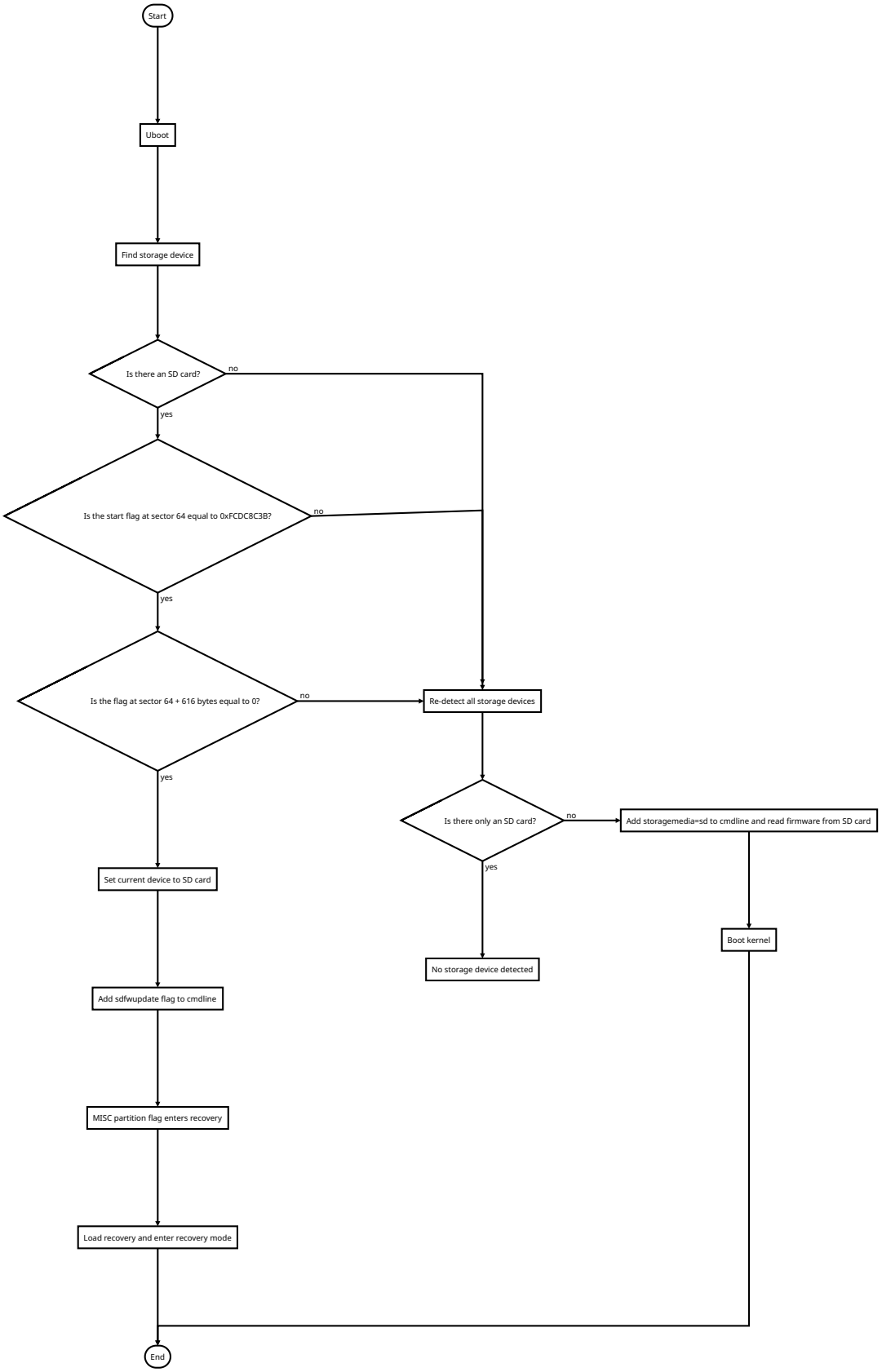


Maskrom first finds a usable miniloader firmware by scanning supported boot storage devices in priority order, then jumps to miniloader. Miniloader scans storage devices again. If it detects an SD card and the SD card contains IDB-format firmware, it checks the card flag:

- If the SD card is valid and the flag is `0` or `1`, it reads the U-Boot firmware from the SD card and boots U-Boot.
- If the flag is `2`, it enters the repair-card flow and updates firmware at the loader stage.
- In the normal boot flow, it scans other storage devices and loads the next-stage loader.

6.5.4.2 U-Boot boot

The original flowchart is kept below:



Upgrade card: U-Boot rescans storage devices. If it detects an SD card containing IDB-format firmware and the card flag is `0`, it appends `sdfwupdate` to the cmdline passed to the kernel. It then reads the `misc` partition on the SD card, reads the card boot mode, and if it is recovery mode, loads and boots recovery.

Boot card: U-Boot rescans storage devices. If it detects an SD card containing IDB-format firmware and the card flag is `1`, it reads the `misc` partition and checks the boot mode. If it is recovery, recovery is loaded. If it is normal mode, the kernel is loaded and booted.

6.5.4.3 Recovery and PCBA

Refer to `Rockchip Recovery User Operation Guide V1.03.pdf`.

6.5.5 Notes

- When creating non-GPT firmware, U-Boot needs `CONFIG_RKPARM_PARTITION`.
- When creating an SD upgrade card, `update.img` must include `MiniloaderAll.bin`, `parameter.txt`, `uboot.img`, `trust.img`, `misc.img`, `resource.img`, and `recovery.img`. Otherwise flashing `update.img` will report a failure when writing the MBR.

7. Chapter-7 Configuration Trimming

TODO

8. Chapter-8 Debug Methods

This chapter introduces some commonly used debugging methods in the U-Boot stage, including commands, scripts, configuration options, boot logs, and so on.

8.1 DEBUG

Function: enable global `debug()` output.

You can enable it by adding the following macro definition to each platform's `rkxxx_common.h`:

```
#define DEBUG
```

8.2 Initcall

Function: print the boot flow.

The U-Boot boot process is essentially a series of initcall invocations. Change `debug()` to `printf()` inside `initcall_run_list()`. Example:

```
U-Boot v5-01725-g03b8d3b-dirty (Jul 06 2018 - 10:08:27 +0800)

initcall: 0000000000214388
initcall: 0000000000214724
Model: Rockchip RK3399 Evaluation Board
initcall: 0000000000214300
DRAM:  initcall: 0000000000203f68
initcall: 0000000000214410  // Use disassembly to identify the function corresponding to the address
initcall: 00000000002140dc
....
3.8 GiB
initcall: 00000000002143b8
....
Relocation Offset is: f5c03000
initcall: 00000000f5e176bc
initcall: 00000000002146a4 (relocated to 00000000f5e176a4)
initcall: 0000000000214668 (relocated to 00000000f5e17668)

...
```

8.3 io command

Function: read and write memory.

```
// Read
md - memory display
Usage: md [.b, .w, .l, .q] address [# of objects]

// Write
mw - memory write (fill)
Usage: mw [.b, .w, .l, .q] address value [count]
```

Read example: display 0x10 consecutive data items starting from address `0x76000000`.

```
=> md.l 0x76000000 0x10
76000000: fffffffe ffffffff ffffffff ffffffff    ................
76000010: fffffdf ffffffff feffffff ffffffff    ................
76000020: ffffffff ffffffff ffffffff ffffffff    ................
76000030: ffffffff ffffffff ffffffff ffffffff    ................
```

Write example: write `0x1234` to address `0x76000000`.

```
=> mw.l 0x76000000 0xffff1234 // High 16 bits contain the mask
=> md.l 0x76000000 0x10       // Read back
76000000: ffff1234 ffffffff ffffffff ffffffff    ................
76000010: fffffdf ffffffff feffffff ffffffff    ................
76000020: ffffffff ffffffff ffffffff ffffffff    ................
76000030: ffffffff ffffffff ffffffff ffffffff    ................
```

8.4 iomem command

Function: read memory. It is more flexible than `md` because it automatically parses DTS nodes to get base address information.

```
=> iomem
iomem - Show iomem data by device compatible

Usage:
// @<compatible>: keyword match against the compatible property of a node
iomem <compatible> <start offset>  <end offset>
eg: iomem -grf 0x0 0x200
```

Example: on RK3228, read data from `0x00 ~ 0x20` in GRF:

```
// Use "-grf" to distinguish from "rockchip, rk3288-pmugrf"
=> iomem -grf 0x0 0x20
rockchip,rk3228-grf:
11000000:  00000000 00000000 00004000 00002000
11000010:  00000000 00005028 0000a5a5 0000aaaa
11000020:  00009955
```

8.5 i2c command

Function: read and write I2C devices.

```
=> i2c
i2c - I2C sub-system

Usage:
i2c dev [dev] - show or set current I2C bus
i2c md chip address[.0, .1, .2] [# of objects] - read from I2C device
i2c mw chip address[.0, .1, .2] value [count] - write to I2C device (fill)
......
```

Read example:

```
=> i2c dev 0                // Switch to i2c0 (only needs to be specified once)
Setting bus to 0

=> i2c md 0x1b 0x2e 0x20        // Device address 1b (7-bit), read 0x20 consecutive registers starting from 0x2e
002e: 11 0f 00 00 11 0f 00 00 01 00 00 00 09 00 00 0c    ................
003e: 00 0a 0a 0c 0c 0c 00 07 07 0a 00 0c 0c 00 00 00    ................
```

Write example:

```
=> i2c dev 0                // Switch to i2c0 (only needs to be specified once)
Setting bus to 0

=> i2c mw 0x1b 0x2e 0x10        // Device address 1b (7-bit), write 0x10 to register 0x2e
=> i2c md 0x1b 0x2e 0x20        // Read back
002e: 10 0f 00 00 11 0f 00 00 01 00 00 00 09 00 00 0c    ................
003e: 00 0a 0a 0c 0c 0c 00 07 07 0a 00 0c 0c 00 00 00    ................
```

8.6 gpio command

Function: read and write pin input/output states.

```
=> gpio
gpio - query and control gpio pins

Usage:
gpio <input|set|clear|toggle> <pin>
   - input/set/clear/toggle the specified pin
gpio status [-a] [<bank> | <pin>]  - show [all/claimed] GPIOs
```

View pin status, for example on RV1126:

```
=> gpio status -a
Bank A:
A0: input: 0 [ ]
A1: output: 1 [ ]
A2: input: 1 [ ]
...
A29: unused: 1 [ ]
A30: unknown
A31: unused: 0 [ ]
...
D6: input: 0 [ ]
D7: output: 1 [x] vcc18-lcd-n.gpio
...
D31: input: 0 [ ]

Bank E:
E0: input: 0 [ ]
E1: input: 0 [ ]
```

Pin input:

```
=> gpio input A7
```

Pin output INACTIVE:

```
=> gpio clear A7
```

Pin output ACTIVE:

```
=> gpio set A7
```

Toggle pin state, for example change `A7: input: 0` to `A7: output: 1`:

```
=> gpio toggle A7
```

8.7 fdt command

Function: print DTB contents.

```
=> fdt
fdt - flattened device tree utility commands

Usage:
fdt addr [-c]  <addr> [<length>]    - Set the [control] fdt location to <addr>
fdt print  <path> [<prop>]          - Recursive print starting at <path>
fdt list   <path> [<prop>]          - Print one level starting at <path>
......
NOTE: Dereference aliases by omitting the leading '/', e.g. fdt print ethernet0.
```

The following two commands are commonly used together to dump the complete device tree:

```
=> fdt addr $fdt_addr_r   // Specify the fdt address
=> fdt print              // Print the entire fdt content
```

8.8 mmc command

Function: MMC device read/write and switching.

View MMC device information:

```
=> mmc info
Device: dwmmc@ff0f0000              // Device node
Manufacturer ID: 15
OEM: 100
Name: 8GME4
Timing Interface: High Speed        // Speed mode
Tran Speed: 52000000                // Current speed
Rd Block Len: 512
MMC version 5.1
High Capacity: Yes
Capacity: 7.3 GiB                   // Storage capacity
Bus Width: 8-bit                    // Bus width
Erase Group Size: 512 KiB
HC WP Group Size: 8 MiB
User Capacity: 7.3 GiB WRREL
Boot Capacity: 4 MiB ENH
RPMB Capacity: 512 KiB ENH
```

Switch MMC device:

```
=> mmc dev 0                        // Switch to eMMC
=> mmc dev 1                        // Switch to SD card
```

Read/write MMC device:

```
mmc read addr blk# cnt
mmc write addr blk# cnt
mmc erase blk# cnt
Example:
=> mmc read 0x70000000 0 1    // Read 1 sector from block 0 of the MMC device into memory 0x70000000
=> mmc write 0x70000000 0 1   // Write 1 sector from memory 0x70000000 to block 0 of the storage device
=> mmc erase 0 1              // Erase 1 sector starting from block 0
```

If MMC read/write behaves abnormally, quickly locate the issue with the following steps:

Change `debug()` to `printf()` in `drivers/mmc/dw_mmc.c`, then rebuild and flash. Check the MMC device logs:

- If the last log is `Sending CMD0`, check hardware power, pin connections, and whether the software IOMUX has been switched away by another IP.
- If the last log is `Sending CMD8`, in secure software configuration make sure the MMC device is allowed to access secure storage.
- If all initialization commands have passed and the last log is `Sending CMD18`, check MMC hardware power and whether the capacitor near the MMC power rail is sufficient. You can try a larger capacitor, lower the clock frequency in software, or switch the MMC speed mode.

8.9 Timestamp

Function: add timestamps (relative time) to U-Boot logs.

```
CONFIG_BOOTSTAGE_PRINTF_TIMESTAMP
```

Example:

```
[    0.259266] U-Boot v5-01739-g856f373-dirty (Jul 10 2018 - 20:26:05 +0800)
[    0.260596] Model: Rockchip RK3399 Evaluation Board
[    0.261332] DRAM:  3.8 GiB
Relocation Offset is: f5bfd000
Using default environment

[    0.354038] dwmmc@fe320000: 1, sdhci@fe330000: 0
[    0.521125] Card did not respond to voltage select!
[    0.521188] mmc_init: -95, time 9
[    0.671451] switch to partitions #0, OK
[    0.671500] mmc0(part 0) is current device
[    0.675507] boot mode: None
[    0.683738] DTB: rk-kernel.dtb
[    0.706940] Using kernel dtb
......
```

The timestamp is simply the current system timer reading, not a timer that starts from zero. So it represents relative time, not absolute time.

8.10 dm tree

Function: view the binding and probe status of all device drivers.

```
=> dm tree

 Class     Probed       Driver             Name
-----------------------------------------------------------
 root      [ + ]   root_driver              root_driver
 syscon    [   ]   rk322x_syscon             |-- syscon@11000000
 serial    [ + ]   ns16550_serial            |-- serial@11030000 *
 clk       [ + ]   clk_rk322x                |-- clock-controller@110e0000
 sysreset  [   ]   rockchip_sysreset         |   |-- sysreset
 reset     [   ]   rockchip_reset            |   `-- reset
 mmc       [ + ]   rockchip_rk3288_dw_mshc   |-- dwmmc@30020000 *
 blk       [ + ]   mmc_blk                   |   `-- dwmmc@30020000.blk *
 ram       [   ]   rockchip_rk322x_dmc       |-- dmc@11200000
 serial    [ + ]   ns16550_serial            |-- serial@11020000
 i2c       [ + ]   i2c_rockchip              |-- i2c@11050000
 ......
```

Meaning of the output:

- Lists all device drivers that have completed binding
- Lists the hierarchy among uclass, device, and driver
- `[ + ]` means the current driver has completed probing
- `*` means the current device driver comes from the U-Boot DTB; otherwise it comes from the kernel DTB

8.11 dm uclass

Function: view all devices under a certain uclass.

```
=> dm uclass

uclass 0: root
- * root_driver @ 7be54c88, seq 0, (req -1)

uclass 11: adc
- * saradc@ff100000 @ 7be56220, seq 0, (req -1)
......
uclass 40: backlight
- * backlight @ 7be81178, seq 0, (req -1)

uclass 77: key
-   rockchip-key @ 7be811f0
......
```

8.12 stacktrace.sh

Use the call stack backtrace mechanism to analyze the context of `abort` and `dump_stack()`. See the RK Architecture chapter.

8.13 System Hang

Function: print the current CPU context and call stack, suitable for system hang debugging. The UART dumps information similar to an abort every 5 seconds.

```
CONFIG_IRQ_TIMER_DUMP
```

After obtaining the call stack information, use the `stacktrace.sh` script to decode it. See the RK Architecture chapter.

8.14 Modify DDR Size

During boot, the DDR initialization code passes the DDR size to U-Boot. U-Boot removes some secure memory and then passes the remaining size to the kernel. Users can modify the DDR size passed to the kernel in the U-Boot stage.

Example:

```
......
// Usable memory blocks passed to the kernel (secure memory blocks already removed).
Adding bank: 0x00200000 - 0x08400000 (size: 0x08200000)
Adding bank: 0x0a200000 - 0x40000000 (size: 0x35e00000)
Total: 895.411 ms

Starting kernel ...
[    0.000000] Booting Linux on physical CPU 0x0
```

Code location:

```
./arch/arm/mach-rockchip/param.c
```

Modification point:

```
struct memblock *param_parse_ddr_mem(int *out_count)
{
    ......

    // This is the DDR capacity information passed to U-Boot.
    // Because addresses may be discontinuous, the data may be passed in multiple blocks,
    // each specifying the start address and size of a memory block.
    // PS: In most cases memory is continuous and split blocks are unnecessary.
    for (i = 0, n = 0; i < count; i++, n++) {
        // For example, for 2 GB of continuous memory:
```

```
        // count=1, base = 0, size = 0x80000000.
        // Users can modify this as needed for debugging.
        base = t->u.ddr_mem.bank[i];
        size = t->u.ddr_mem.bank[i + count];

        /* 0~4GB */
        if (base < SZ_4GB) {
            mem[n].base = base;
            mem[n].size = ddr_mem_get_usable_size(base, size);
            if (base + size > SZ_4GB) {
                n++;
                mem[n].base_u64 = SZ_4GB;
                mem[n].size_u64 = base + size - SZ_4GB;
            }
        } else {
            /* 4GB+ */
            mem[n].base_u64 = base;
            mem[n].size_u64 = size;
        }

        assert(n < count + MEM_RESV_COUNT);
    }
    ......
}
```

8.15 Jump Information

Function: confirm firmware version and flow. In some cases boot logs can also help users locate certain crash issues.

1. System hangs immediately after Trust

Possible reason: firmware packaging or flashing is wrong, causing Trust to jump to the wrong U-Boot entry address. In this case, users can check the U-Boot entry address printed by Trust during boot.

On 64-bit platforms, the U-Boot entry address is usually at offset `0x200000` from the DDR base address `0x0`:

```
NOTICE:  BL31: v1.3(debug):d98d16e
NOTICE:  BL31: Built : 15:03:07, May 10 2018
NOTICE:  BL31: Rockchip release version: v1.1
INFO:    GICv3 with legacy support detected. ARM GICV3 driver initialized in EL3
INFO:    Using opteed sec cpu_context!
INFO:    boot cpu mask: 0
INFO:    plat_rockchip_pmu_init(1151): pd status 3e
INFO:    BL31: Initializing runtime services
INFO:    BL31: Initializing BL32
INFO:    BL31: Preparing for EL3 exit to normal world
INFO:    Entry point address = 0x200000  // U-Boot address
INFO:    SPSR = 0x3c9
```

On 32-bit platforms, the U-Boot entry address is usually at offset `0x0` from the DDR base address `0x60000000`:

```
INF [0x0] TEE-CORE:init_primary_helper:378: Release version: 1.9
INF [0x0] TEE-CORE:init_primary_helper:379: Next entry point address: 0x60000000  // U-Boot address
INF [0x0] TEE-CORE:init_teecore:83: teecore inits done
```

2. U-Boot version traceability

The U-Boot boot log can be used to trace the build version. The example below corresponds to commit `b34f08b`:

```
U-Boot v5-01730-gb34f08b (Jul 06 2018 - 17:47:52 +0800)
```

If `dirty` appears in the boot log, it means there were local uncommitted changes when building, so the build point is not clean.

```
U-Boot v5-01730-gb34f08b-dirty (Jul 06 2018 - 17:35:04 +0800)
```

8.16 Boot Information

Through the U-Boot boot logs, users can learn the current U-Boot flow and the status of various peripherals, which helps them quickly locate abnormalities.

Currently U-Boot supports booting three firmware types: `Android format > FIT format > DISTRO format`. RK SDK releases mainly use the first two, while DISTRO is generally used by open-source users.

Note: if the codebase is not new enough, some logs may not appear. This does not affect understanding of the overall U-Boot boot flow.

Android firmware:

```
// First standard U-Boot log line, containing commit version, build time, etc.
// Note: this is only a relatively early formal U-Boot log line,
// not the earliest debug output U-Boot can produce.
// Enable debug logs to see earlier debug output.
U-Boot v5-03033-g81b79f7-dirty (Jul 04 2019 - 15:04:00 +0800)

// The "model" field in the U-Boot DTS, indicating which U-Boot DTS is being used
Model: Rockchip RK3399 Evaluation Board
// preloader-serial is enabled, reusing the UART configuration from the previous-stage loader; UART2 is used for logging
PreSerial: 2
// Total board memory is 2 GB
DRAM: 2 GiB
// The current version supports the sysmem memory block management mechanism
Sysmem: init
// U-Boot relocates its own code from an earlier DDR address to a later DDR address
// (see the U-Boot development guide boot flow for details)
// The relocated code starts at 0x7dbe2000, which may be useful in disassembly debugging
Relocation Offset is: 7dbe2000
// ENV is stored in DDR by default. If ENV is stored in eMMC, NAND, or another storage device, this log will not appear
Using default environment
```

```
21
22 // The current storage medium is mmc0, i.e. eMMC (mmc1 would mean SD card)
23 dwmmc@fe320000: 1, sdhci@fe330000: 0
24 // The storage type is passed from the previous-stage miniloader through ATAGS
25 Bootdev(atags): mmc 0
26 // The current eMMC is running in HS400 mode at 150 MHz
27 MMC0: HS400, 150Mhz
28 // GPT partition table is in use (if RK parameter table is used, it prints RKPARM)
29 PartType: EFI
30 // Currently in recovery mode
31 // The "reboot xxx" command executed in the kernel is ultimately reflected by this log
32 boot mode: recovery
33 // Kernel DTB comes from recovery.img and is loaded normally
34 Load FDT from recovery part
35 DTB: rk-kernel.dtb
36 HASH: OK(c)
37
38 // ==> Note: from this point on, U-Boot has switched to the kernel dtb.
39 // All subsequent peripheral drivers use information from the kernel dtb.
40
41 // DTBO executed successfully
42 ANDROID: fdt overlay OK
43 // I2C speed. This affects U-Boot boot time, especially on PMICs with many DCDC and LDO rails.
44 // If boot time matters, pay attention to this log
45 I2c speed: 400000Hz
46 // The current PMIC is RK818
47 // on corresponds to the ON_SOURCE register and indicates the cause of the current power-on
48 // off corresponds to the OFF_SOURCE register and indicates the cause of the previous shutdown or
   power loss
49 // on/off information is valuable when abnormal reboot or shutdown occurs
50 PMIC: RK818 (on=0x20 off=0x40)
51 // Current voltages of adjustable regulators, usually DCDC rails such as arm, logic, center
52 // This is valuable when startup is abnormal or unstable
53 vdd_center 900000 uV
54 vdd_cpu_l 900000 uV
55 vdd_log 900000 uV
56 // The "model" field in the kernel DTS, indicating which kernel DTS is being used
57 Model: Rockchip RK3399 Excavator Board edp avb (Android)
58 enter Recovery mode!
59 // Display driver-related information
60 Rockchip UBOOT DRM driver version: v1.0.1
61 Using display timing dts
62 Detailed mode clock 200000 kHz, flags[a]
63   H: 1536 1548 1564 1612
64   V: 2048 2056 2060 2068
65 bus_format: 100e
66 // clk-tree information. See the CLK chapter in the U-Boot development guide for details
67 CLK: (uboot. arml: enter 816000 KHz, init 816000 KHz, kernel 0N/A)
68 CLK: (uboot. armb: enter 24000 KHz, init 24000 KHz, kernel 0N/A)
69  aplll 816000 KHz
70  apllb 24000 KHz
71  dpll 800000 KHz
72  cpll 200000 KHz
73  gpll 800000 KHz
74  npll 600000 KHz
```

```
 vpll 24000 KHz
 aclk_perihp 133333 KHz
 hclk_perihp 66666 KHz
 pclk_perihp 33333 KHz
 aclk_perilp0 266666 KHz
 hclk_perilp0 88888 KHz
 pclk_perilp0 44444 KHz
 hclk_perilp1 100000 KHz
 pclk_perilp1 50000 KHz
// GMAC driver enabled
Net:  eth0: ethernet@fe300000
// Hold ctrl+c during boot to enter the U-Boot command line after the following log
Hit key to stop autoboot('CTRL+C'): 0
// Confirms again that the current mode is recovery
ANDROID: reboot reason: "recovery"
// vboot=0 means secure boot is not enabled. Since the current firmware is AVB, normal AVB verification is performed.
Vboot=0, AVB images, AVB verify
// Whether the device is unlocked
read_is_device_unlocked() ops returned that device is UNLOCKED
// Native U-Boot loads the entire boot.img/recovery.img first, then relocates ramdisk/fdt/kernel
// to user-defined addresses, which is time-consuming, especially when ramdisk is large.
// RK modified this so ramdisk/fdt/kernel are loaded directly from storage to target addresses in one step.
// The following log indicates this time-saving optimization is enabled
Fdt Ramdisk skip relocation

// Load Android-format firmware. Kernel is loaded to 0x00280000, fdt to 0x8300000
// For an LZ4-compressed kernel, this may instead print:
// Booting LZ4 kernel at 0x00680000(Uncompress to 0x00280000) with fdt at 0x8300000...
Booting IMAGE kernel at 0x00280000 with fdt at 0x8300000...

// Can be ignored if not needed
### Booting Android Image at 0x0027f800 ...
// Kernel and ramdisk load addresses and sizes
Kernel load addr 0x00280000 size 19081 KiB
RAM disk load addr 0x0a200000 size 9627 KiB
// FDT load address
### Flattened Device Tree blob at 08300000
   Booting using the fdt blob at 0x8300000
// Can be ignored if not needed
   XIP Kernel Image ... OK
// Only prints the reserved-memory specified by the kernel dts, useful for analyzing kernel startup problems
   'reserved-memory' secure-memory@20000000: addr=20000000 size=10000000
// Start and end addresses of the FDT
   Using Device Tree in place at 0000000008300000, end 000000000831c6f7
// Memory ranges passed to the kernel, excluding ATF, OP-TEE, and so on
Adding bank: 0x00200000 - 0x08400000 (size: 0x08200000)
Adding bank: 0x0a200000 - 0x80000000 (size: 0x75e00000)
// U-Boot stage boot time
Total: 367.128 ms

// Printed by U-Boot. After this, U-Boot performs some ARM architecture-related cleanup
// such as cache cleaning, interrupt disabling, CPU state switching, and DM device teardown.
```

```
// The probability of failure here is extremely low.
// Once this is complete, execution jumps to the kernel, so this log effectively marks entry to the kernel stage.
Starting kernel ...

// Kernel-stage logs
[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Initializing cgroup subsys cpuset
[    0.000000] Initializing cgroup subsys cpu
[    0.000000] Initializing cgroup subsys cpuacct
[    0.000000] Initializing cgroup subsys schedtune
[    0.000000] Linux version 4.4.167 (hgc@ubuntu) (gcc version 6.3.1 20170404 (Linaro GCC 6.3-2017.05) ) #83 SMP PREEMPT Thu Mar 21 09:31:08 CST 2019
[    0.000000] Boot CPU: AArch64 Processor [410fd034]
[    0.000000] earlycon: Early serial console at MMIO32 0xff1a0000 (options '')
[    0.000000] bootconsole [uart0] enabled
[    0.000000] Reserved memory: failed to reserve memory for node 'stb-devinfo@00000000': base 0x0000000000000000, size 0 MiB
[    0.000000] cma: Reserved 16 MiB at 0x000000007f000000
......
```

9. Chapter-9 Test Cases

TODO

10. Chapter-10 SPL

10.1 Firmware Boot

The role of SPL is to replace the miniloader and complete the loading and booting of `trust.img` and `uboot.img`. SPL currently supports booting the following two firmware types:

- FIT firmware: enabled by default
- RKFW firmware: disabled by default and must be configured and enabled separately by the user

10.1.1 FIT Firmware

FIT (flattened image tree) is a relatively new firmware format supported by SPL. It supports packaging and verifying multiple images. FIT uses DTS syntax to describe the packaged images. The description file is `u-boot.its`, and the final generated FIT image is `u-boot.itb`.

The advantage of FIT is that it reuses DTS syntax and compilation rules, making the description more flexible, and firmware parsing can directly use the `libfdt` library.

`u-boot.its` file:

- `/images`: statically defines all available resource configurations, similar in role to a dtsi
- `/configurations`: each config node describes a bootable configuration, similar to a board-level dts
- `default =`: specifies the default configuration currently selected

Example:

```
/dts-v1/;

/ {
    description = "Configuration to load ATF before U-Boot";
    #address-cells = <1>;

    images {
        uboot@1 {
            description = "U-Boot (64-bit)";
            data = /incbin/("u-boot-nodtb.bin");
            type = "standalone";
            os = "U-Boot";
            arch = "arm64";
            compression = "none";
            load = <0x00200000>;
        };

        atf@1 {
            description = "ARM Trusted Firmware";
            data = /incbin/("bl31_0x00010000.bin");
```

```
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0x00010000>;
            entry = <0x00010000>;
        };

        atf@2 {
            description = "ARM Trusted Firmware";
            data = /incbin/("bl31_0xff091000.bin");
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0xff091000>;
        };

        optee@1 {
            description = "OP-TEE";
            data = /incbin/("bl32.bin");
            type = "firmware";
            arch = "arm64";
            os = "op-tee";
            compression = "none";
            load = <0x08400000>;
        };

        fdt@1 {
            description = "rk3328-evb.dtb";
            data = /incbin/("arch/arm/dts/rk3328-evb.dtb");
            type = "flat_dt";
            compression = "none";
        };
    };

    configurations {
        default = "config@1";
        config@1 {
            description = "rk3328-evb.dtb";
            firmware = "atf@1";
            loadables = "uboot@1", "atf@2", "optee@1" ;
            fdt = "fdt@1";
        };
    };
};
```

`u-boot.itb` file:

```
                    mkimage + dtc
[u-boot.its] + [images]      ==>       [u-boot.itb]
```

The diagram above shows how the itb file is generated. FIT firmware can be understood as a special DTB file whose contents are image data. Users can inspect an itb file using `fdtdump`:

```
cjh@ubuntu:~/uboot-nextdev/u-boot$ fdtdump u-boot.itb | less

/dts-v1/;
// magic:               0xd00dfeed
// totalsize:           0x497 (1175)
// off_dt_struct:       0x38
// off_dt_strings:      0x414
// off_mem_rsvmap:      0x28
// version:             17
// last_comp_version:   16
// boot_cpuid_phys:     0x0
// size_dt_strings:     0x83
// size_dt_struct:      0x3dc

/ {
    timestamp = <0x5d099c85>;
    description = "Configuration to load ATF before U-Boot";
    #address-cells = <0x00000001>;
    images {
        uboot@1 {
            data-size = <0x0009f8a8>;
            data-offset = <0x00000000>;
            description = "U-Boot (64-bit)";
            type = "standalone";
            os = "U-Boot";
            arch = "arm64";
            compression = "none";
            load = <0x00600000>;
        };
        atf@1 {
            data-size = <0x0000c048>;   // Added automatically during build, indicates the size of atf@1
            data-offset = <0x0009f8a8>; // Added automatically during build, indicates the offset of atf@1
            description = "ARM Trusted Firmware";
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0x00010000>;
            entry = <0x00010000>;
        };
        atf@2 {
            data-size = <0x00002000>;
            data-offset = <0x000ab8f0>;
            description = "ARM Trusted Firmware";
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0xfff82000>;
        };
        fdt@1 {
            data-size = <0x00005793>;
            data-offset = <0x000ad8f0>;
            description = "rk3308-evb.dtb";
```

```
            type = "flat_dt";
            ......
        };
        ......
    };
};
```

For more FIT information, see:

```
./doc/uImage.FIT/
```

10.1.2 RKFW Firmware

To replace the miniloader more directly while avoiding changes to the partitions and packaging format of downstream firmware, RK platforms add support for booting RKFW format firmware, that is, firmware stored in separate partitions: `trust.img` and `uboot.img`.

Configuration:

```
CONFIG_SPL_LOAD_RKFW          // Enable option
CONFIG_RKFW_TRUST_SECTOR      // trust.img partition address, must match the partition table definition
CONFIG_RKFW_U_BOOT_SECTOR     // uboot.img partition address, must match the partition table definition
```

Code:

```
./include/spl_rkfw.h
./common/spl/spl_rkfw.c
```

10.1.3 Storage Priority

In the U-Boot DTS, `u-boot,spl-boot-order` specifies the boot priority of storage devices.

```
/ {
    aliases {
        mmc0 = &emmc;
        mmc1 = &sdmmc;
    };

    chosen {
        u-boot,spl-boot-order = &sdmmc, &nandc, &emmc;
        stdout-path = &uart2;
    };
    ......
};
```

10.2 Build and Packaging

10.2.1 Code Build

U-Boot builds SPL firmware from the same codebase using **a different build path**. When building SPL, the macro `CONFIG_SPL_BUILD` is defined automatically. After compiling `u-boot.bin`, U-Boot continues compiling SPL and creates a separate output directory `./spl/`.

```
// Build u-boot
......
DTC     arch/arm/dts/rk3399-puma-ddr1866.dtb
DTC     arch/arm/dts/rv1108-evb.dtb
make[2]: `arch/arm/dts/rk3328-evb.dtb' is up to date.
SHIPPED dts/dt.dtb
FDTGREP dts/dt-spl.dtb
CAT     u-boot-dtb.bin
MKIMAGE u-boot.img
COPY    u-boot.dtb
MKIMAGE u-boot-dtb.img
COPY    u-boot.bin

// Build spl, with a separate spl/ directory
LD      spl/arch/arm/cpu/built-in.o
CC      spl/board/rockchip/evb_rk3328/evb-rk3328.o
LD      spl/dts/built-in.o
CC      spl/common/init/board_init.o
COPY    tpl/u-boot-tpl.dtb
CC      spl/cmd/nvedit.o
CC      spl/env/common.o
CC      spl/env/env.o
.....
LD      spl/drivers/block/built-in.o

......
```

After the build finishes:

```
./spl/u-boot-spl.bin
```

10.3 System Modules

10.3.1 Partition Table

SPL uses the GPT partition table.

Configuration:

```
CONFIG_SPL_LIBDISK_SUPPORT=y
CONFIG_SPL_EFI_PARTITION=y
CONFIG_PARTITION_TYPE_GUID=y
```

Driver:

```
./disk/part.c
./disk/part_efi.c
```

Interfaces:

```
int part_get_info(struct blk_desc *dev_desc, int part, disk_partition_t *info);
int part_get_info_by_name(struct blk_desc *dev_desc,
                          const char *name, disk_partition_t *info);
```

10.3.2 A/B system

SPL supports A/B system boot.

Configuration:

```
CONFIG_SPL_AB=y
```

Driver:

```
./common/spl/spl_ab.c
```

Interfaces:

```
int spl_get_current_slot(struct blk_desc *dev_desc, char *partition, char *slot);
int spl_get_partitions_sector(struct blk_desc *dev_desc, char *partition,u32 *sectors);
```

10.3.3 Boot Priority

- SPL uses the boot order defined by `u-boot,spl-boot-order` in `rkxxxx-u-boot.dtsi`:

  ```
  chosen {
  stdout-path = &uart2;
  u-boot,spl-boot-order = &sdmmc, &sfc, &nandc, &emmc;
  };
  ```

- Maskrom boot priority:

  ```
   spi nor > spi nand > emmc > sd
  ```

- Pre-loader (SPL) boot priority:

```
sd > spi nor > spi nand > emmc
```

Raising the SD card to the highest priority makes it easier to boot the system from SD.

10.3.4 ATAGS

SPL and U-Boot pass parameters through the ATAGS mechanism. The transmitted information includes the boot storage device, debug UART, and so on.

Configuration:

```
CONFIG_ROCKCHIP_PRELOADER_ATAGS=y
```

Driver:

```
./arch/arm/include/asm/arch-rockchip/atags.h
./arch/arm/mach-rockchip/atags.c
```

Interfaces:

```
int atags_set_tag(u32 magic, void *tagdata);
struct tag *atags_get_tag(u32 magic);
```

10.3.5 kernel boot

Normally the kernel is loaded and booted by U-Boot, but SPL can also support loading the kernel. At present, it supports loading `boot.img` with Android boot image header version 2 and also supports RK-format firmware.

Boot flow:

```
Maskrom -> ddr -> SPL -> Trust -> Kernel
```

10.3.6 pinctrl

Configuration:

```
CONFIG_SPL_PINCTRL_GENERIC=y
CONFIG_SPL_PINCTRL=y
```

Driver:

```
./drivers/pinctrl/pinctrl-uclass.c
./drivers/pinctrl/pinctrl-generic.c
./drivers/pinctrl/pinctrl-rockchip.c
```

DTS configuration:

Take `sdmmc` as an example:

```
&pinctrl {
    bootph-all;
};

&pcfg_pull_none_4ma {
    bootph-all;
};

&pcfg_pull_up_4ma {
    bootph-all;
};

&sdmmc {
    bootph-all;
};

&sdmmc_pin {
    bootph-all;
};

&sdmmc_clk {
    bootph-all;
};

&sdmmc_cmd {
    bootph-all;
};

&sdmmc_bus4 {
    bootph-all;
};

&sdmmc_pwren {
    bootph-all;
};
```

Notes:

When enabling pinctrl in SPL, modify the `CONFIG_OF_SPL_REMOVE_PROPS` definition in defconfig and remove the `pinctrl-0 pinctrl-names` fields from it.

10.4 Driver Modules

10.4.1 MMC

Configuration:

```
CONFIG_SPL_MMC_SUPPORT=y  // Enabled by default
```

Driver:

```
./common/spl/spl_mmc.c
```

Interface:

```
int spl_mmc_load_image(struct spl_image_info *spl_image,
                       struct spl_boot_device *bootdev);
```

10.4.2 MTD block

SPL unifies NAND, SPI NAND, and SPI NOR interfaces under the block layer.

Configuration:

```
// MTD driver support
CONFIG_MTD=y
CONFIG_CMD_MTD_BLK=y
CONFIG_SPL_MTD_SUPPORT=y
CONFIG_MTD_BLK=y
CONFIG_MTD_DEVICE=y

// spi nand driver support
CONFIG_MTD_SPI_NAND=y
CONFIG_ROCKCHIP_SFC=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
CONFIG_SPL_SPI_SUPPORT=y

// nand driver support
CONFIG_NAND=y
CONFIG_CMD_NAND=y
CONFIG_NAND_ROCKCHIP=y /* NandC v6 can be identified by the TRM register NANDC->NANDC_NANDC_VER, value 0x00000801 */
//CONFIG_NAND_ROCKCHIP_V9=y  /* NandC v9 can be identified by the TRM register NANDC->NANDC_NANDC_VER, value 0x56393030, e.g. RK3326/PX30 */
CONFIG_SPL_NAND_SUPPORT=y
CONFIG_SYS_NAND_U_BOOT_LOCATIONS=y
CONFIG_SYS_NAND_U_BOOT_OFFS=0x8000
CONFIG_SYS_NAND_U_BOOT_OFFS_REDUND=0x10000
// The nand page size must match the actual size. For NAND capacity >= 512MB, 4096 is commonly required
```

```
#define CONFIG_SYS_NAND_PAGE_SIZE   2048

// spi nor driver support
CONFIG_CMD_SF=y
CONFIG_CMD_SPI=y
CONFIG_SPI_FLASH=y
CONFIG_SF_DEFAULT_MODE=0x1
CONFIG_SF_DEFAULT_SPEED=50000000
CONFIG_SPI_FLASH_GIGADEVICE=y
CONFIG_SPI_FLASH_MACRONIX=y
CONFIG_SPI_FLASH_WINBOND=y
CONFIG_SPI_FLASH_MTD=y
CONFIG_ROCKCHIP_SFC=y
CONFIG_SPL_SPI_SUPPORT=y
CONFIG_SPL_MTD_SUPPORT=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
```

Driver:

```
./common/spl/spl_mtd_blk.c
```

Interface:

```
int spl_mtd_load_image(struct spl_image_info *spl_image,
                       struct spl_boot_device *bootdev);
```

10.4.3 OTP

Used to store immutable data and is involved in secure boot.

Configuration:

```
CONFIG_SPL_MISC=y
CONFIG_SPL_ROCKCHIP_SECURE_OTP=y
```

Driver:

```
./drivers/misc/misc-uclass.c
./drivers/misc/rockchip-secure-otp.S
```

Interfaces:

```
int misc_read(struct udevice *dev, int offset, void *buf, int size);
int misc_write(struct udevice *dev, int offset, void *buf, int size);
```

11. Chapter-11 TPL

TODO

12. Chapter-12 FIT

12.1 Preface

This chapter introduces the FIT format and the details of secure and non-secure boot solutions based on FIT. For convenience, this chapter mainly uses `boot.img` as the example object for explanation and operation, but the same applies to `recovery.img`.

12.2 Introduction

12.2.1 Basic Introduction

FIT (flattened image tree) is a new firmware boot format supported by U-Boot. It supports packaging and verification of an arbitrary number of images. FIT uses an `its` (image source file) to describe image information, and finally uses the `mkimage` tool to generate an `itb` (flattened image tree blob) image. The `its` file uses DTS syntax, which is highly flexible and can directly use the `libfdt` library and related tools.

FIT is the default and recommended firmware format in U-Boot. Both SPL and the U-Boot stage support booting FIT-format firmware. For more information, see:

```
./doc/usage/fit/
```

Because the official FIT feature cannot fully satisfy actual product requirements, RK platforms adapt and optimize FIT. Therefore, the FIT solution must use the `mkimage` built from RK U-Boot, rather than the `mkimage` that comes with the PC.

12.2.2 Example Introduction

The following uses `u-boot.its` and `u-boot.itb` as examples.

- `/images`: statically defines all resources, equivalent to a dtsi file
- `/configurations`: each `conf` node describes a bootable configuration, equivalent to a board-level dts file
- `default =`: specifies the config enabled by default

```
/dts-v1/;

/ {
    description = "Simple image with OP-TEE support";
    #address-cells = <1>;

    images {
        uboot {
```

```
            description = "U-Boot";
            data = /incbin/("./u-boot-nodtb.bin");
            type = "standalone";
            os = "U-Boot";
            arch = "arm";
            compression = "none";
            load = <0x00400000>;
            hash {
                algo = "sha256";
            };
        };
        optee {
            description = "OP-TEE";
            data = /incbin/("./tee.bin");
            type = "firmware";
            arch = "arm";
            os = "op-tee";
            compression = "none";
            load = <0x8400000>;
            entry = <0x8400000>;
            hash {
                algo = "sha256";
            };
        };
        fdt {
            description = "U-Boot dtb";
            data = /incbin/("./u-boot.dtb");
            type = "flat_dt";
            compression = "none";
            hash {
                algo = "sha256";
            };
        };
    };

    // Any number of conf nodes can be defined under configurations,
    // but actual product designs usually need only one.
    configurations {
        default = "conf";
        conf {
            description = "Rockchip armv7 with OP-TEE";
            rollback-index = <0x0>;
            firmware = "optee";
            loadables = "uboot";
            fdt = "fdt";
            signature {
                algo = "sha256,rsa2048";
                padding = "pss";
                key-name-hint = "dev";
                sign-images = "fdt", "firmware", "loadables";
            };
        };
    };
};
```

An itb file can be generated using `mkimage` and the its file:

```
                    mkimage + dtc
[u-boot.its] + [images]    ==========>     [u-boot.itb]
```

The contents of an itb file can be viewed with `fdtdump`:

```
cjh@ubuntu:~/uboot-nextdev/u-boot$ fdtdump fit/u-boot.itb | less

/dts-v1/;
// magic:       0xd00dfeed
// totalsize:       0x600 (1536)
// off_dt_struct:   0x48
// off_dt_strings:  0x48c
// off_mem_rsvmap:  0x28
// version:     17
// last_comp_version:   16
// boot_cpuid_phys: 0x0
// size_dt_strings: 0xc3
// size_dt_struct:  0x444

/memreserve/ 7f34d3411000 600;
/ {
    version = <0x00000001>;             // Newly added firmware version number
    totalsize = <0x000bb600>;           // Newly added field describing the size of the whole itb
    timestamp = <0x5ecb3553>;           // Newly added timestamp for the current firmware build time
    description = "Simple image with OP-TEE support";
    #address-cells = <0x00000001>;
    images {
        uboot {
            data-size = <0x0007ed54>;     // Newly added field describing firmware size
            data-position = <0x00000a00>; // Newly added field describing firmware offset
            description = "U-Boot";
            type = "standalone";
            os = "U-Boot";
            arch = "arm";
            compression = "none";
            load = <0x00400000>;
            hash {
                // Newly added sha256 checksum of the firmware
                value = <0xeda8cd52 0x8f058118 0x00000003 0x35360000 0x6f707465 0x0000009f 0x00000091
0x00000000>;
                algo = "sha256";
            };
        };
        optee {
            data-size = <0x0003a058>;
            data-position = <0x0007f800>;
            description = "OP-TEE";
            type = "firmware";
            arch = "arm";
            os = "op-tee";
            compression = "none";
            load = <0x08400000>;
```

```
        entry = <0x08400000>;
        hash {
          value = <0xa569b7fc 0x2450ed39 0x00000003 0x35360000 0x66647400 0x00001686 0x000b9a00
0x552d426f>;
          algo = "sha256";
        };
      };
      fdt {
        data-size = <0x00001686>;
        data-position = <0x000b9a00>;
        description = "U-Boot dtb";
        type = "flat_dt";
        compression = "none";
        hash {
          value = <0x0f718794 0x78ece7b2 0x00000003 0x35360000 0x00000001 0x6e730000 0x636f6e66
0x00000000>;
          algo = "sha256";
        };
      };
    };
    configurations {
      default = "conf";
      conf {
        description = "Rockchip armv7 with OP-TEE";
        rollback-index = <0x00000001>; // Firmware anti-rollback version, defaults to 0 if not specified
        firmware = "optee";
        loadables = "uboot";
        fdt = "fdt";
        signature {
          algo = "sha256,rsa2048";
          padding = "pss";
          key-name-hint = "dev";
          sign-images = "fdt", "firmware", "loadables";
        };
      };
    };
};
```

12.2.3 itb Structure

An itb is essentially a file collection of `fdt_blob + images`. There are two packaging methods below, and RK platforms use structure 2.

```
          fdt blob
|-----------------------------------|
|   |------|  |------|  |------|    |
|   | img0 |  | img1 |  | img2 |    | Structure 1: image is inside fdt_blob, i.e. itb = fdt_blob (including img)
|   |------|  |------|  |------|    |
|-----------------------------------|

|--------------|------|------|------|
|              |      |      |      |
|   fdt blob   | img0 | img1 | img2 | Structure 2: image is outside fdt_blob, i.e. itb = fdt_blob + img
|              |      |      |      |
|--------------|------|------|------|
```

12.3 Platform Configuration

12.3.1 Chip Support

For platforms officially released in the SDK as a feature, **please refer to the chip feature support status on the home page**.

12.3.2 Code Configuration

Code:

```
// Framework code
./boot/image.c
./boot/image-fit.c
./common/spl/spl_fit.c

// Platform code
./arch/arm/mach-rockchip/fit.c
./cmd/bootfit.c

// Tool code
./tools/mkimage.c
./tools/fit_image.c
```

Configuration:

```
// FIT support in the U-Boot stage
CONFIG_ROCKCHIP_FIT_IMAGE=y

// U-Boot stage: secure boot, anti-rollback, hardware crypto
CONFIG_FIT_SIGNATURE=y
CONFIG_FIT_ROLLBACK_PROTECT=y
CONFIG_DM_CRYPTO=y
CONFIG_FIT_HW_CRYPTO=y

// SPL stage: secure boot, anti-rollback, hardware crypto
```

```
CONFIG_SPL_FIT_SIGNATURE=y
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y
CONFIG_SPL_DM_CRYPTO=y
CONFIG_SPL_FIT_HW_CRYPTO=y

// Number and size of uboot.itb copies included in uboot.img
CONFIG_SPL_FIT_IMAGE_KB=3072
CONFIG_SPL_FIT_IMAGE_MULTIPLE=2

// After building uboot, output fit-format uboot.img by default;
// otherwise output traditional RK-format uboot.img and trust.img
CONFIG_ROCKCHIP_FIT_IMAGE_PACK=y
```

Because crypto capabilities vary by platform, RSA configuration parameters also differ. Refer to the common defconfig of the current platform.

```
CONFIG_RSA_N_SIZE
CONFIG_RSA_E_SIZE
CONFIG_RSA_C_SIZE
```

> Common defconfig: `[chip]_defconfig`, for example `rv1126_defconfig`, `rk3568_defconfig`.

If the FIT solution is an officially released SDK feature, most basic options are already enabled. The options users need to configure themselves are:

```
// U-Boot secure boot and anti-rollback mechanism
CONFIG_FIT_SIGNATURE=y
CONFIG_FIT_ROLLBACK_PROTECT=y

// SPL secure boot and anti-rollback mechanism
CONFIG_SPL_FIT_SIGNATURE=y
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y
```

- If `CONFIG_FIT_SIGNATURE` is not enabled, U-Boot can support booting three firmware formats simultaneously: android, uimage, and fit. The released SDK enables them according to platform needs.
- If `CONFIG_FIT_SIGNATURE` is enabled, U-Boot supports only FIT firmware.

12.3.3 Image Files

In the FIT solution, two FIT-format firmware images are finally output for flashing: `uboot.img` and `boot.img`, plus an SPL file used for packaging into the loader.

- `uboot.img`

  `uboot.itb = trust + u-boot.bin + mcu.bin(optional)`

  `uboot.img = uboot.itb * N copies (N is usually 2)`

  > `trust` and `mcu` come from the `rkbin` project. The build script automatically indexes and retrieves them.

- `boot.img`

`boot.itb = kernel + fdt + resource + ramdisk(optional)`

`boot.img = boot.itb * M copies (M is usually 1)`

- MCU configuration

  Some platforms currently may include MCU firmware. Whether it is enabled can be determined by the corresponding TRUST ini configuration for different products. For example:

```
// File: RKTRUST/RV1126TOS_TB.ini, for fast boot products, MCU enabled.
[TOS]
TOSTA=bin/rv11/rv1126_tee_ta_tb_v1.04.bin
ADDR=0x00040000

// MCU configuration format: firmware path, start address, status (okay or disabled).
// If disabled, the MCU will not be packaged into uboot.img.
[MCU]
MCU=bin/rv11/rv1126_mcu_v1.02.bin,0x108000,okay
```

- Firmware compression

  Some platforms can currently support compression of sub-images inside `uboot.img`, as follows:

Platform	Compression Format	Firmware
RV1126	gzip, none	u-boot.bin, trust, mcu(optional)

  Users can enable it by adding attributes in the corresponding TRUST ini in the `rkbin` project. For example:

```
// File: RKTRUST/RV1126TOS_SPI_NOR_TINY.ini, for small-capacity SPI NOR products.
[TOS]
TOS=bin/rv11/rv1126_tee_v1.02.bin
ADDR=0x08400000
[MCU]
MCU=bin/rv11/rv1126_mcu_v1.00.bin,0x208000,disabled

// Compression format: gzip or none. If this field does not exist, uncompressed is used by default.
[COMPRESSION]
COMPRESSION=gzip
```

- SPL file

  The SPL file refers to the generated `spl/u-boot-spl.bin`, which is responsible for booting FIT-format `uboot.img`. Users need to use it to replace the closed-source miniloader on RK platforms and package the final loader.

- `./fit` directory

  After U-Boot compilation, a `./fit` folder is generated containing some intermediate files, which are introduced in later sections.

`boot.img` and `uboot.img` are built respectively in the SDK project and the U-Boot project. However, for secure boot, the `boot.img` generated by the SDK must be re-packaged and signed inside the U-Boot project, as described later.

12.3.4 its File

- The its file for uboot is `./fit/u-boot.its`, dynamically created by the script specified by `CONFIG_SPL_FIT_GENERATOR` in defconfig, and visible after a successful build.
- The its file for boot is located in the SDK project:

```
device/rockchip/[platform]/xxx.its  // [platform] is the platform directory
```

12.3.5 Related Tools

```
// Core packaging tool, generated automatically after compilation.
// Exists in both the U-Boot and rkbin repositories.
./tools/mkimage
// Firmware packaging script
./make.sh
// Firmware re-signing script
scripts/fit-resign.sh
// Firmware unpacking script
scripts/fit-unpack.sh
// Firmware replacement script
./scripts/fit-repack.sh
```

The use of these scripts is introduced in later sections. Here we first focus on the parameters of `make.sh`.

Optional arguments (users choose as needed):

- `--spl-new`: package the loader using the currently built SPL file; otherwise use the SPL file in `rkbin`.
- `--version-uboot [n]`: specify the firmware version number of `uboot.img`, where `n` must be a positive decimal integer.
- `--version-boot [n]`: specify the firmware version number of `boot.img`, where `n` must be a positive decimal integer.
- `--version-recovery [n]`: specify the firmware version number of `recovery.img`, where `n` must be a positive decimal integer.

Mandatory arguments (when secure boot is enabled):

- `--rollback-index-uboot [n]`: specify the anti-rollback version number of `uboot.img`, where `n` must be a positive decimal integer.
- `--rollback-index-boot [n]`: specify the anti-rollback version number of `boot.img`, where `n` must be a positive decimal integer.
- `--rollback-index-recovery [n]`: specify the anti-rollback version number of `recovery.img`, where `n` must be a positive decimal integer.
- `--no-check`: used when packaging secure firmware, to skip the self-check of the secure firmware packaging script.

> Notes:
>
> 1. Firmware anti-rollback version numbers are valid only when secure boot is enabled. The

> version is stored in OTP or other secure storage. Its purpose is to prevent vulnerability attacks caused by downgrading firmware versions.
> 2. Firmware version numbers are optional and default to 0 if not specified. They are used only as firmware version identifiers for version management.

12.4 Non-Secure Boot

12.4.1 uboot.img

Build command:

```
./make.sh rv1126 --spl-new --uboot-version 10  // --spl-new and --uboot-version are optional
```

Build result:

```
......
CC      spl/common/spl/spl.o
CC      spl/lib/display_options.o
LD      spl/common/spl/built-in.o
LD      spl/lib/built-in.o
LD      spl/u-boot-spl
OBJCOPY spl/u-boot-spl-nodtb.bin
CAT     spl/u-boot-spl-dtb.bin
COPY    spl/u-boot-spl.bin
CFGCHK  u-boot.cfg

out:rv1126_spl_loader_v1.00.100.bin
fix opt:rv1126_spl_loader_v1.00.100.bin
merge success(rv1126_spl_loader_v1.00.100.bin)
/home4/cjh/uboot-nextdev

// Generate rv1126_spl_loader_v1.00.100.bin (using SPL to replace the traditional RK miniloader)
// Source of the loader ini file
pack loader(SPL) okay! Input: /home4/cjh/rkbin/RKBOOT/RV1126MINIALL.ini
// Prompt from --spl-new; users may choose not to use this parameter.
pack loader with new: spl/u-boot-spl.bin

// Generate uboot.img (containing trust and uboot), with version number 10
Image(no-signed, version=10):  uboot.img (FIT with uboot, trust...) is ready
// Source of the trust ini file
pack uboot.img okay! Input: /home4/cjh/rkbin/RKTRUST/RV1126TOS.ini

Platform RV1126 is build OK, with exist .config
```

Backup packaging: configure multiple copies of `uboot.img` through defconfig:

```
CONFIG_SPL_FIT_IMAGE_KB=3072    // size of a single itb
CONFIG_SPL_FIT_IMAGE_MULTIPLE=2 // number of packaged copies
```

SPL uses this configuration to detect and boot U-Boot and trust. It mainly addresses the problem of firmware corruption caused by unexpected power loss during OTA upgrade, which could otherwise prevent booting.

12.4.2 boot.img

If the FIT solution is released as an official SDK feature, the SDK generates FIT-format `boot.img` after compilation.

To generate `boot.img` for secure boot, the SDK-generated `boot.img` must be placed in the U-Boot project for repackaging and signing, because the signing tool, configuration, and parameters for secure firmware packaging all come from the U-Boot project.

12.5 Secure Boot

The FIT solution supports secure boot, with the following features:

- `sha256 + rsa2048 + pkcs-v2.1(pss) padding`
- firmware anti-rollback
- firmware re-signing (remote signing)
- hardware crypto acceleration

12.5.1 Principle

12.5.1.1 Verification Flow

- Maskrom verifies the loader (including SPL, ddr, usbplug)
- SPL verifies `uboot.img` (including trust, U-Boot, ...)
- U-Boot verifies `boot.img` (including kernel, fdt, ramdisk, ...)

Currently only the secure verification mode `sha256+rsa2048+pkcs-v2.1(pss) padding` is supported by default.

12.5.1.2 Key Storage

The RSA key is packaged by `mkimage` into `u-boot.dtb` and `u-boot-spl.dtb`, and then they are packaged into `u-boot.bin` and `u-boot-spl.bin`.

The RSA key format in `u-boot.dtb` is as follows (same for `u-boot-spl.dtb`):

```
cjh@ubuntu:~/uboot-nextdev$ fdtdump u-boot.dtb | less
/dts-v1/;
....

/ {
    #address-cells = <0x00000001>;
```

```
    #size-cells = <0x00000001>;
    compatible = "rockchip,rv1126-evb", "rockchip,rv1126";
    model = "Rockchip RV1126 Evaluation Board";

    // The signature node is automatically inserted by mkimage and stores
    // the RSA-SHA algorithm type, RSA core factor parameters, and so on.
    signature {
        key-dev {
            required = "conf";
            algo = "sha256,rsa2048";
            rsa,np = <0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x1327f633
0x00000003 0x00000003 0x00000003 0xc7aead6a 0xb4c79f40 0xa82bdf76 0xfb2f8387 0xa1e06dce
0xd451a706 0xc7f865e3 0x3e2d7ca8 0x6a71762e 0x125f1828 0x36ab1a41 0xb7e9e852 0x7bd0011a
0x7279e0b8 0xf37e189c 0x8cf00963 0x00000100 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000377 0x00000004 0x00000004 0x00000004
0x00000002 0x00000003 0x69616c40 0x00000003 0x6d634066 0x00000010 0x66633630 0x73797363>;
            rsa,c = <0x00000000>;
            rsa,r-squared = <0x00000000>;
            rsa,modulus = <0xc25ae693 0xc359f2a4 0xa866c89d 0xb7b1994f 0xf9f9f690 0x518d54a7
0xda0b83e8 0x06606e12 0x6ad1cbf9 0x92438edd 0x81e039c0 0x5d7322cc 0x124cdc80 0xa0c3288a
0x9265c3ae 0x6ac47a4b 0x00000003 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000008 0x00000003 0x00000003 0x00000003 0x00000002
0x73657300 0x2f736572 0x00000000 0x2f64776d 0x00000003 0x6d634066 0x00000001 0x30303000
0x726f636b 0x67726600 0x00000008 0x00000003 0x00000004 0x00000001 0x30303000 0x726f636b
0x706d7567 0x00000003 0x00001000 0x00000003 0x00000002 0x6e616765 0x30000000 0x726f636b
0x706d7500 0x00000008>;
            rsa,exponent-BN = <0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000003 0x00010001 0xe95771c5 0x00000800 0x64657600 0x616c6961
0x0000002c 0x30303030 0x00000034 0x30303000 0x2f64776d 0x00000002 0x65303030 0x0000001b
0x3132362d 0x00000003 0x00020000 0x00000003 0x00000002 0x65303230 0x0000001b 0x3132362d
0x6e000000 0xfe020000 0x00000042 0x0000006d 0x722d6d61 0x65303030 0x0000001b 0x3132362d
0x00000003 0x00001000 0x00000002 0x6e74726f 0x30000000 0x726f636b 0x706d7563 0x0000003e
0x00000004 0x00000004 0x00000004 0x00000000 0x00000050 0x636c6f63 0x40666634 0x00000014
0x2c727631 0x00000008>;
            rsa,exponent = <0x00000000 0x00000368>;
            rsa,n0-inverse = <0xe95771c5>;
            rsa,num-bits = <0x00000800>;
            key-name-hint = "dev";
        };
    };
```

SPL supports burning the key hash. In that case `u-boot-spl.dtb:key-dev` contains an extra field: `burn-key-hash = <0x00000001>;` .

12.5.1.3 Key Usage

From Maskrom to kernel, the secure boot chain uses a single RSA public key for verification:

- Maskrom verifies the loader.

  The RSA public key must be written into the loader header using the PC tool `rk_sign_tool`. During secure boot, Maskrom first obtains the RSA public key from the loader header and verifies its validity, then uses it to verify the loader signature.

  > `rk_sign_tool` can be obtained from the `rkbin` repository. U-Boot automatically signs the loader.

- SPL verifies U-Boot and trust.

  SPL stores the RSA public key in `u-boot-spl.dtb`, which is packaged into `u-boot-spl.bin` and finally into the loader. During secure boot, SPL reads the RSA public key from its own dtb and uses it to verify `uboot.img`.

- U-Boot verifies boot.

  U-Boot stores the RSA public key in `u-boot.dtb`, which is packaged into `u-boot.bin` and finally into `uboot.img`. During secure boot, U-Boot reads the RSA public key from its own dtb and uses it to verify `boot.img`.

So the RSA key at the current stage is already part of the current firmware and has been securely verified by the previous-stage loader, ensuring key security.

12.5.1.4 Signature Storage

The RSA signature result is stored in the itb file. The signed content is specified by `hashed-nodes`, including all properties of the `conf` node and the nodes of the packaged firmware.

The following shows the signature information in `u-boot.itb`, similarly for `boot.itb`:

```
cjh@ubuntu:~/uboot-nextdev$ fdtdump uboot.img | less
/dts-v1/;
......

   configurations {
      default = "conf";
      conf {
         description = "Rockchip armv7 with OP-TEE";
         // Current firmware version number
         rollback-index = <0x0000001c>;
         firmware = "optee";
         loadables = "uboot";
         fdt = "fdt";

         // Signed content and signature result, inserted automatically by mkimage
         signature {
            hashed-strings = <0x00000000 0x000000da>;
            // Specifies the signed content
            hashed-nodes = "/", "/configurations", "/configurations/conf", "/images/fdt", "/images/fdt/hash",
"/images/optee", "/images/optee/hash", "/images/uboot", "/images/uboot/hash";
```

```
            // Time, signer, and version of signing
            timestamp = <0x5e9427b4>;
            signer-version = "v5-g8bb63db-200413-dirty #cjh";
            signer-name = "mkimage";
            // Signature result!! (sha256+rsa2048)
            value = <0x78397d5d 0xb9219a0b 0xa7cb91a7 0xe1f32867 0x62719d9b 0x8901200c 0xfcbac03a
0x1295ccc8 0x1cff9608 0xdf5f69d2 0x21391225 0x7af10ca7 0x5527864f 0xb13f527e 0xddf9ee62
0xea50199d 0x00000003 0x35362c72 0x00000004 0x00000017 0x77617265 0x00000002 0x00000009
0x23616464 0x6d616765 0x73006172 0x6f6e006c 0x72790064 0x61636b2d 0x7265006c 0x006b6579
0x69676e2d 0x706f7369 0x7a650074 0x75650073 0x69676e65 0x73686564 0x642d7374 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000>;
            algo = "sha256,rsa2048";
            key-name-hint = "dev";
            sign-images = "fdt", "firmware", "loadables";
        };
    };
};
```

12.5.1.5 Anti-Rollback

- Secure boot supports separately specifying the current firmware version number for `boot.img` and `uboot.img`. If the current version is lower than the minimum version on the device, booting is not allowed.
- Minimum version update: after secure verification completes and the system is confirmed to boot normally, the minimum version is updated into OTP or secure storage.

12.5.2 Preparation

12.5.2.1 Key

Run the following three commands in the U-Boot project to generate the RSA key pair used for signing. Normally this only needs to be done once. Afterwards the same key pair is used to sign and verify firmware, so keep it safe.

```
// 1. Directory for keys: keys
mkdir -p keys

// 2. Use RK's "rk_sign_tool" to generate an RSA2048 private key privateKey.pem
//    and public key publicKey.pem (see the rk_sign_tool user guide),
//    then rename and store them as keys/dev.key and keys/dev.pubkey.

// 3. Use -x509 and the private key to generate a self-signed certificate: keys/dev.crt
//    (functionally equivalent to the public key for this use case)
openssl req -batch -new -x509 -key keys/dev.key -out keys/dev.crt
```

> If the following error occurs because `~/.rnd` does not exist:

> Can't load /home4/cjh//.rnd into RNG
> 140522933268928:error:2406F079:random number generator:RAND_load_file:Cannot open file:../crypto/rand/randfile.c:88:Filename=/home4/cjh//.rnd
>
> Create it manually first: `touch ~/.rnd`

`ls keys/` result:

```
dev.crt  dev.key  dev.pubkey
```

> Note: the names `keys`, `dev.key`, `dev.crt`, and `dev.pubkey` must not be changed. They are statically defined in the its files, and packaging will fail if they are changed.

12.5.2.2 Configuration

Enable the following options in the U-Boot defconfig:

```
// Required.
CONFIG_FIT_SIGNATURE=y
CONFIG_SPL_FIT_SIGNATURE=y

// Optional.
CONFIG_FIT_ROLLBACK_PROTECT=y      // Anti-rollback for boot.img
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y  // Anti-rollback for uboot.img
```

> It is recommended to select configuration items through `make menuconfig`, then update the original defconfig with `make savedefconfig`. This avoids build failures caused by invalid dependency relationships from forcibly editing defconfig.

12.5.2.3 Firmware

Copy the `boot.img` generated in the SDK project into the U-Boot root directory.

12.5.3 Build and Packaging

(1) Basic command (without anti-rollback):

```
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img
```

Build result:

```
......

// After build, signed uboot.img and boot.img are generated.
start to sign rv1126_spl_loader_v1.00.100.bin
......
sign loader ok.
......
Image(signed, version=0):  uboot.img (FIT with uboot, trust...) is ready
```

```
Image(signed, version=0):  recovery.img (FIT with kernel, fdt, resource...) is ready
Image(signed, version=0):  boot.img (FIT with kernel, fdt, resource...) is ready
Image(signed):  rv1126_spl_loader_v1.05.106.bin (with spl, ddr, usbplug) is ready
pack uboot.img okay! Input: /home4/cjh/rkbin/RKTRUST/RV1126TOS.ini

Platform RV1126 is build OK, with new .config(make rv1126-secure_defconfig)
```

(2) Extended command 1:

If anti-rollback is enabled, append rollback parameters to command (1). For example:

```
// Set the minimum version numbers of uboot.img and boot.img to 10 and 12 respectively.
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img --rollback-index-uboot 10 --rollback-index-boot 12 --rollback-index-recovery 12
```

Build result:

```
......

// After build, signed uboot.img and boot.img are generated with anti-rollback version numbers.
start to sign rv1126_spl_loader_v1.00.100.bin
......
sign loader ok.
......
Image(signed, version=0, rollback-index=10):  uboot.img (FIT with uboot, trust) is ready
Image(signed, version=0, rollback-index=12):  recovery.img (FIT with kernel, fdt, resource...) is ready
Image(signed, version=0, rollback-index=12):  boot.img (FIT with kernel, fdt, resource...) is ready
Image(signed):  rv1126_spl_loader_v1.00.100.bin (with spl, ddr, usbplug) is ready
```

(3) Extended command 2:

If the public key hash is to be burned into OTP/eFUSE, append `--burn-key-hash` to command (1) or (2). For example:

```
// Set the minimum version numbers of uboot.img and boot.img to 10 and 12 respectively.
// Require the SPL stage to burn the public key hash into OTP/eFUSE.
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img --rollback-index-uboot 10 --rollback-index-boot 12 --rollback-index-recovery 12 --burn-key-hash
```

Build result:

```
......

// Enable burn-key-hash
### spl/u-boot-spl.dtb: burn-key-hash=1

// After build, signed uboot.img and boot.img are generated with anti-rollback version numbers.
start to sign rv1126_spl_loader_v1.00.100.bin
......
sign loader ok.
......
Image(signed, version=0, rollback-index=10):  uboot.img (FIT with uboot, trust) is ready
Image(signed, version=0, rollback-index=12):  recovery.img (FIT with kernel, fdt, resource...) is ready
```

```
Image(signed, version=0, rollback-index=12):  boot.img (FIT with kernel, fdt, resource...) is ready
Image(signed):  rv1126_spl_loader_v1.00.100.bin (with spl, ddr, usbplug) is ready
```

> On power-on, SPL prints: `RSA: Write key hash successfully`.

(4) Notes:

- `--boot_img`: optional. Specifies the `boot.img` to be signed.
- `--recovery_img`: optional. Specifies the `recovery.img` to be signed.
- `--rollback-index-uboot`, `--rollback-index-boot`, `--rollback-index-recovery`: optional. Specify anti-rollback version numbers.
- `--spl-new`: if this parameter is not used, the SPL file in `rkbin` is used to package the loader. Otherwise the currently built SPL file is used.

  Because `u-boot-spl.dtb` needs to include the RSA public key from the user, RK-released SDKs do not submit secure-boot-capable SPL files to the `rkbin` repository. Therefore, users must specify this parameter when building. However, users can also submit their own SPL version into the `rkbin` project; after that, the parameter is no longer required and that stable SPL file can be reused.
- Three firmware images are generated after compilation: `loader`, `uboot.img`, and `boot.img`. As long as the RSA key has not changed, any one of them can be updated independently.

12.5.4 Verification Rules

(1) Maskrom verifies SPL

If OTP does not contain the key: Maskrom follows the non-secure boot flow.

If OTP contains the key: Maskrom checks the key in the Loader and requires it to match the key in OTP before secure verification begins. If they do not match, booting is not allowed.

(2) SPL verifies U-Boot

If `CONFIG_SPL_FIT_SIGNATURE=y`: SPL always performs secure verification on `uboot.img` and boots only on success. If `uboot.img` is unsigned or verification fails, it does not boot.

If `CONFIG_SPL_FIT_SIGNATURE=n`: SPL itself does not include secure boot code and therefore never verifies `uboot.img`, regardless of whether it is signed.

(3) U-Boot verifies boot/recovery

If `CONFIG_FIT_SIGNATURE=y`: U-Boot always performs secure verification on `boot.img` / `recovery.img` and boots only on success. If `boot.img` / `recovery.img` is unsigned or verification fails, it does not boot.

If `CONFIG_FIT_SIGNATURE=n`: U-Boot itself does not include secure boot code and therefore never verifies `boot.img` / `recovery.img`, regardless of whether it is signed.

> Note: whether the current stage verifies the next stage has nothing to do with whether the current stage firmware itself is signed. It depends only on whether the current stage includes secure boot code, i.e. whether the above configuration is `y`.

12.5.5 Boot Logs

Below is an example of secure boot logs:

```
BW=32 Col=10 Bk=8 CS0 Row=15 CS=1 Die BW=16 Size=1024MB
out
U-Boot SPL board init
U-Boot SPL v5-gacb99c5-200408-dirty #cjh (Apr 09 2020 - 20:51:21)
unrecognized JEDEC id bytes: 00, 00, 00

Trying to boot from MMC1
// SPL completed signature verification
sha256,rsa2048:dev+
// Anti-rollback check: current uboot.img version is 10, minimum version on the device is 9
rollback index: 10 >= 9, OK
// SPL completed hash verification of sub-images
### Checking optee ... sha256+ OK
### Checking uboot ... sha256+ OK
### Checking fdt ... sha256+ OK

Jumping to U-Boot via OP-TEE
I/TC:
E/TC:0 0 plat_rockchip_pmu_init:2003 0
E/TC:0 0 plat_rockchip_pmu_init:2006 cpu off
E/TC:0 0 plat_rockchip_pmusram_prepare:1945 pmu sram prepare 0x14b10000 0x8400880 0x1c
E/TC:0 0 plat_rockchip_pmu_init:2020 pmu sram prepare
E/TC:0 0 plat_rockchip_pmu_init:2056 remap
I/TC: OP-TEE version: 3.6.0-233-g35ecf936 #1 Tue Mar 31 08:46:13 UTC 2020 arm
I/TC: Next entry point address: 0x00400000
I/TC: Initialized

U-Boot v5-gacb99c5-200408-dirty #cjh (Apr 09 2020 - 20:51:21 +0800)

Model: Rockchip RV1126 Evaluation Board
PreSerial: 2
DRAM:  1023.5 MiB
Sysmem: init
Relocation Offset: 00000000, fdt: 3df404e0
Using default environment

dwmmc@ffc50000: 0
Bootdev(atags): mmc 0
MMC0: HS200, 200Mhz
PartType: EFI
boot mode: normal
conf: sha256,rsa2048:dev+
resource: sha256+
DTB: rk-kernel.dtb
FIT: signed, conf required
HASH(c): OK

I2c0 speed: 400000Hz
PMIC:  RK8090 (on=0x10, off=0x00)
vdd_logic 800000 uV
```

```
vdd_arm 800000 uV
vdd_npu init 800000 uV
vdd_vepu init 800000 uV
......

Hit key to stop autoboot('CTRL+C'):  0
### Booting FIT Image at 0x3d8122c0 with size 0x0052b200
Fdt Ramdisk skip relocation
### Loading kernel from FIT Image at 3d8122c0 ...
   Using 'conf' configuration
   // U-Boot completed signature verification
   Verifying Hash Integrity ... sha256,rsa2048:dev+ OK
   // Anti-rollback check: current boot.img version is 22, minimum version on the device is 21
   Verifying Rollback-index ... 22 >= 21, OK
   Trying 'kernel' kernel subimage
     Description:  Kernel for arm
     Type:         Kernel Image
     Compression:  uncompressed
     Data Start:   0x3d8234c0
     Data Size:    5349248 Bytes = 5.1 MiB
     Architecture: ARM
     OS:           Linux
     Load Address: 0x02008000
     Entry Point:  0x02008000
     Hash algo:    sha256
     Hash value:   64b4a0333f7862967be052a67ee3858884fcefebf4565db5c3828a941a15f34a
   Verifying Hash Integrity ... sha256+ OK  // Kernel hash verification completed
### Loading ramdisk from FIT Image at 3d8122c0 ...
   Using 'conf' configuration
   Trying 'ramdisk' ramdisk subimage
     Description:  Ramdisk for arm
     Type:         RAMDisk Image
     Compression:  uncompressed
     Data Start:   0x3dd3d4c0
     Data Size:    0 Bytes = 0 Bytes
     Architecture: ARM
     OS:           Linux
     Load Address: 0x0a200000
     Entry Point:  unavailable
     Hash algo:    sha256
     Hash value:   e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
   Verifying Hash Integrity ... sha256+ OK  // Ramdisk hash verification completed
   Loading ramdisk from 0x3dd3d4c0 to 0x0a200000
### Loading fdt from FIT Image at 3d8122c0 ...
   Using 'conf' configuration
   Trying 'fdt' fdt subimage
     Description:  Device tree blob for arm
     Type:         Flat Device Tree
     Compression:  uncompressed
     Data Start:   0x3d812ec0
     Data Size:    66974 Bytes = 65.4 KiB
     Architecture: ARM
     Load Address: 0x08300000
     Hash algo:    sha256
     Hash value:   8fb1f170766270ed4f37cce4b082a51614cb346c223f96ddfe3526fafc5729d7
```

```
  Verifying Hash Integrity ... sha256+ OK // FDT hash verification completed
  Loading fdt from 0x3d812ec0 to 0x08300000
  Booting using the fdt blob at 0x8300000
  Loading Kernel Image from 0x3d8234c0 to 0x02008000 ... OK
  Using Device Tree in place at 08300000, end 0831359d
Adding bank: 0x00000000 - 0x08400000 (size: 0x08400000)
Adding bank: 0x0848a000 - 0x40000000 (size: 0x37b76000)
Total: 236.327 ms

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0xf00
[    0.000000] Linux version 4.19.111 (cjh@ubuntu) (gcc version 6.3.1 20170404 (Linaro GCC 6.3-2017.05)) #28 SMP PREEMPT Wed Mar 25 16:03:27 CST 2020
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
```

12.6 Standalone Signing

Based on the characteristics of the FIT format, standalone signing unpacks unsigned firmware and then repackages and signs it. This section first introduces the procedure and then the related tools. Standalone signing was supported only starting in June 2026, so make sure the `uboot` and `rkbin` repositories include the relevant patches:

`uboot` repository:

```
de0830c397f scripts: fit-core: Don't copy dtc for fit_signcfg_export
5f0a499b4f5 tools: fit_check_sign/mkimage: Support build static version
```

`rkbin` repository:

```
a4c58c05622 tools: fit: Add tools build from U-Boot v5
```

12.6.1 Signing Flow

People building firmware:

1. Refer to the secure boot section above and enable the relevant options.
2. Build:

   ```
   ./make.sh rk3588 --spl-new --no-sign   # --spl-new --no-sign must be added
   ```

   This generates:

   ```
   uboot.img
   rk3588_spl_loader_v1.18.113.bin
   fit/fit_signcfg/
   ```

3. Send these three generated files plus any other FIT files to be signed, such as `boot.img` and

`recovery.img`, to the signing personnel or server.

Note: the generated `loader`, `uboot`, and `fit_signcfg` are strictly matched to each other and cannot be mixed with firmware of the same name built at another time, otherwise the signing script will fail.

Signing personnel or server:

1. Create a working directory (any name is fine):

```
$ mkdir workspace
$ cd workspace
$ mkdir keys src output
```

2. Download the `rkbin` repository, then copy or symlink the `tools` directory in `rkbin` into the workspace:

```
$ cp ~/rkbin/tools/ ./ -R
```

3. Following the secure boot section above, place the RSA key pair in `keys`, and place the firmware to be signed plus the `fit_signcfg` directory in `src`.

   `loader`, `uboot`, and `fit_signcfg` are mandatory. Other FIT images to be signed, such as `boot.img` and `recovery.img`, may also be included. For example:

```
cjh@sys2:/home4/cjh/rkbin/workspace$ tree
├── keys
│   ├── dev.crt
│   ├── dev.key
│   └── dev.pubkey
├── output
├── src
│   ├── fit_signcfg
│   │   ├── MINIALL.ini
│   │   ├── sign.readonly_config
│   │   └── test_images.tar.gz
│   ├── boot.img
│   ├── recovery.img
│   ├── rk3588_spl_loader_v1.18.113.bin
│   └── uboot.img
└── tools
    ├── bmp2gray16
    ├── boot_merger
    ├── burner_image_kits
    │   ├── make.sh
    │   ├── make_slc_nand.sh
    ......
```

4. Run signing:

```
$ ./tools/fit-sign.sh --key-dir keys/ --src-dir src/ --out-dir output/
```

   The signing process does not modify the source files in `src`. After success, all signed firmware is

generated under `output`:

```
$ tree output/
output/
├── boot.img
├── recovery.img
├── rk3588_spl_loader_v1.18.113.bin
└── uboot.img
```

12.6.2 Signing Script

All tools related to standalone signing have been committed to the `rkbin` repository. The developer-facing script is `./tools/fit-sign.sh`.

```
$ ./tools/fit-sign.sh -h

Usage:
   ./tools/fit-sign.sh [args]

Args:
   --key-dir            <dir>                  | Mandatory
   --src-dir           <dir>                  | Mandatory
   --out-dir            <dir>                  | Mandatory
   --burn-key-hash                               | Optional
   --rollback-index        <image1 n1> <image2 n2> ...   | Optional
   --version            <image1 n1> <image2 n2> ...   | Optional
```

- `--key-dir`: required. Points to the RSA key directory.
- `--src-dir`: required. Points to the directory containing the firmware to be signed. This directory must contain `loader`, `uboot`, and `fit_signcfg`.
- `--out-dir`: required. Points to the output directory for signed firmware.
- `--burn-key-hash`: optional. Requires no additional parameters. Enables SPL to burn the RSA public key hash into OTP/eFUSE.
- `--rollback-index`: optional. Assign anti-rollback version numbers to corresponding images, in decimal.
- `--version`: optional. Assign regular file version numbers to corresponding images, in decimal.

Example:

```
./tools/fit-sign.sh --key-dir keys/ --src-dir src/ --out-dir output/ --version uboot.img 1 boot.img 3 --rollback-index uboot.img 3 boot.img 5
```

12.6.3 Signing Configuration

To strictly ensure that the features specified during signing are actually enabled in SPL and U-Boot code, extra configuration record files are generated when building the firmware. The signing script uses these files.

```
$ tree fit_signcfg/
fit_signcfg/
 ├──── MINIALL.ini          # ini file used to package the loader
 ├──── sign.readonly_config  # record file of current build configuration and uboot/loader versions
 └──── test_images.tar.gz    # currently unused, ignore
```

12.7 Remote Signing

Standalone signing requires uploading complete firmware to the signing personnel or server, which affects transfer size and speed. The remote signing mechanism is a lighter solution. Each approach has its pros and cons; choose according to your needs.

12.7.1 Implementation Idea

- Because only the server's public key is available, the user first uses a temporary private key plus the server's public key to package and sign the firmware locally on the PC, generating secure firmware containing a temporary signature and the data to be signed.

  > The public key is used to package the public key into the dtb for the secure boot flow; the private key is used to create a temporary signature.

- The user sends only the data to be signed to the server, not the whole firmware, which saves time. The server uses the private key to sign the data and returns the signature to the user.
- The user replaces the temporary signature in the secure firmware with this signature to obtain the final secure firmware to be flashed.

12.7.2 Data to Be Signed

The data to be signed mentioned above includes: `fdt blob configuration + collection of sub-image hashes`.

- `fdt blob` node configuration

  `hashed-nodes` specifies a set of nodes, and the content of these nodes is included in the signed data.

```
cjh@ubuntu:~/uboot-nextdev$ fdtdump uboot.img | less
/dts-v1/;
......

configurations {
        default = "conf";
        conf {
            description = "Rockchip armv7 with OP-TEE";
            rollback-index = <0x0000001c>;
            firmware = "optee";
            loadables = "uboot";
            fdt = "fdt";

            signature {
```

```
            hashed-strings = <0x00000000 0x000000da>;
            // The content of these nodes is all included in the signed data
            hashed-nodes = "/", "/configurations/conf", "/images/fdt", "/images/fdt/hash",
"/images/optee", "/images/optee/hash", "/images/uboot", "/images/uboot/hash";
            ......
```

- Collection of sub-image hashes

  `mkimage` automatically generates hash values for each sub-image and appends them into hash nodes. All sub-image hashes specified by `sign-images` are included in the signed data (essentially the hash nodes are specified via `hashed-nodes`). For example:

```
cjh@ubuntu:~/uboot-nextdev/u-boot$ fdtdump fit/u-boot.itb | less

/dts-v1/;
......

/ {
   totalsize = <0x000bb600>;
   timestamp = <0x5ecb3553>;
   description = "Simple image with OP-TEE support";
   #address-cells = <0x00000001>;
   images {
      uboot {
         data-size = <0x0007ed54>;
         data-position = <0x00000a00>;
         description = "U-Boot";
         type = "standalone";
         os = "U-Boot";
         arch = "arm";
         compression = "none";
         load = <0x00400000>;
         hash {
            // Hash of the uboot image, automatically calculated by mkimage
            value = <0xeda8cd52 0x8f058118 0x00000003 0x35360000 0x6f707465 0x0000009f
0x00000091 0x00000000>;
            algo = "sha256";
         };
      };
      ......
```

12.7.3 Detailed Steps

The RSA key pair used for signing firmware is `dev.key`, `dev.pubkey`, and `dev.crt`. The remote server holds `dev.key` as the private key, while the user only has `dev.pubkey` and `dev.crt`.

Step 1:

In the local U-Boot project environment, the user places `dev.crt` in the `keys` directory, then uses RK's `rk_sign_tool` to generate a random temporary private key named `dev.key` in `keys`. Following the secure boot steps above, generate signed firmware `uboot.img` and `boot.img`, but add `--no-check` to the build parameters. The final signed output is not actually used; the user needs the intermediate files.

Note: **the build command must include `--no-check`**, otherwise the packaging script self-check fails because `dev.key` and `dev.crt` do not match. Example:

```
./make.sh rv1126 --spl-new --boot_img boot.img --rollback-index-uboot 10 --rollback-index-boot 12 --no-check
```

Besides generating signed `uboot.img` and `boot.img`, the following intermediate files are also produced under `fit/`:

```
// Signed content (data2sign means: data to sign)
fit/uboot.data2sign
fit/boot.data2sign

// Signed itb files (using the temporary private key); the img files are generated from them with multiple backups.
fit/uboot.itb
fit/boot.itb
```

Step 2:

The user sends `uboot.data2sign` to the remote server. Suppose the server holds the private key `dev.key`; it signs with:

```
openssl dgst -sha256 -sign dev.key -sigopt rsa_padding_mode:pss -out uboot.sig uboot.data2sign
```

The server returns the signature result file `uboot.sig` to the user. The user then replaces the temporary signature in `uboot.itb` with:

```
./scripts/fit-resign.sh -f fit/uboot.itb -s uboot.sig // generates a new uboot.img for flashing
```

The same applies to `boot.itb`. This gives the final valid signed firmware `uboot.img` and `boot.img`.

Notes:

- In `fit-resign.sh`, the file specified by `-f` is an `itb` file, not an `img` file. The script re-signs the `itb` and then generates the `img`.
- The `itb` file used with `fit-resign.sh` must be the one generated in step 1. The `itb` file and `data2sign` are one-to-one because `data2sign` contains the build timestamp from the `itb`, i.e. `/timestamp = <...>`. Therefore, even if there are no code changes, rebuilding a new `uboot.itb` and replacing its signature with an old `uboot.sig` will still cause secure boot to fail.
- Since the private key is unavailable, the loader must be sent to the server separately for signing.

12.8 Firmware Unpacking

Users can use a script to unpack firmware, for example `boot.img`:

```
cjh@ubuntu:~/uboot-nextdev$ ./scripts/fit-unpack.sh -f boot.img -o out
Unpack to directory out:
  fdt          : 82813 bytes... sha256+
  kernel       : 5844640 bytes... sha256+
  ramdisk      : 0 bytes... sha256+
  resource     : 120832 bytes... sha256+
```

> If the img contains multiple backups, the script unpacks only the first itb. `sha256+` means the firmware is intact; otherwise it shows `sha256-`.

12.9 Firmware Replacement

Users can use a script to batch replace sub-images. For example, to replace someone else's `uboot_legacy.img` with your own `bl31.elf`:

1. Build your own `uboot.img` using your own `bl31.elf`.
2. Use `fit-unpack.sh` to unpack `uboot_legacy.img` into the `out/` directory.

```
cjh@ubuntu:~/uboot-nextdev$ ./scripts/fit-unpack.sh -f uboot_legacy.img -o out/

uboot_legacy.img: Device Tree Blob version 17, size=2560, boot CPU=0, string block size=197, DT structure block size=2204
Unpack to directory out:
  uboot        : 576352 bytes... sha256+
  atf-1        : 69089 bytes... sha256+
  atf-2        : 36864 bytes... sha256+
  atf-3        : 24576 bytes... sha256+
  optee        : 228134 bytes... sha256+
  fdt          : 8867 bytes... sha256+
```

3. Delete all `atf-xxx` files under `out/`.
4. Use `fit-repack.sh` to replace all sub-images in your own `uboot.img` with the files in `out/`. The new `uboot.img` then contains your own `bl31` together with the other sub-images from `uboot_legacy.img`, achieving the intended replacement.

```
cjh@ubuntu:~/uboot-nextdev$ ./scripts/fit-repack.sh -f uboot.img -d out/

uboot.img: Device Tree Blob version 17, size=2560, boot CPU=0, string block size=197, DT structure block size=2204
Unpack to directory out/repack/:
  uboot        : 576352 bytes... sha256+
  atf-1        : 69089 bytes... sha256+
  atf-2        : 36864 bytes... sha256+
  atf-3        : 24576 bytes... sha256+
  optee        : 228134 bytes... sha256+
  fdt          : 8867 bytes... sha256+
  ......
Image(repack):  uboot.img is ready
```

Principle:

The replacement strategy is not "replace his sub-images into my `uboot.img`", but rather "replace his sub-images into my base `uboot.img`".

The reason is that the `atf-xxx` files in `uboot.img` come from `bl31.elf`, and different versions of `bl31.elf` may contain different numbers of `atf-xxx` entries, making direct one-for-one replacement impossible. Although sub-images such as `u-boot`, `bl32`, and `mcu` have fixed counts, this reverse replacement strategy is used in order to support replacing `bl31.elf`.

The same strategy applies to replacing other sub-images.

12.10 Secure Verification Step-by-Step

1. Enter the `u-boot` directory, open the corresponding platform `configs/rxxxxx_defconfig`, and select:

```
// Required.
CONFIG_FIT_SIGNATURE=y
CONFIG_SPL_FIT_SIGNATURE=y

// Optional.
CONFIG_FIT_ROLLBACK_PROTECT=y      // Anti-rollback for boot.img
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y  // Anti-rollback for uboot.img
```

2. Generate keys:

```
mkdir -p keys
../rkbin/tools/rk_sign_tool kk --bits 2048 --out .
cp privateKey.pem keys/dev.key && cp publicKey.pem keys/dev.pubkey
openssl req -batch -new -x509 -key keys/dev.key -out keys/dev.crt
```

Note: this step only needs to be done once, then keep these keys safely.

3. Build and sign, taking `rv1126` as an example (if signing firmware for other chips, such as rk3566, simply replace `rv1126` with `rk3566` in the commands below):

```
// Linux: copy boot.img and recovery.img into the u-boot directory, then sign loader, uboot, boot, recovery,
// and set anti-rollback version numbers for uboot, boot, and recovery as needed.
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img --rollback-index-uboot 1 --rollback-index-boot 2 --rollback-index-recovery 2

// Android: sign loader and uboot, and set the anti-rollback version number for uboot as needed.
./make.sh rv1126 --spl-new --rollback-index-uboot 1
```

If the following appears during build:

```
Can't load XXXXXX//.rnd into RNG
```

Run:

```
touch ~/.rnd
```

4. Burn the public key hash:

```
// Linux: copy boot.img and recovery.img into the u-boot directory, then sign loader, uboot, boot, recovery,
// set anti-rollback version numbers, and enable key hash burning.
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img --rollback-index-uboot 1 --rollback-index-boot 2 --rollback-index-recovery 2 --burn-key-hash

// Android: sign loader and uboot, set anti-rollback version number, and enable key hash burning.
./make.sh rv1126 --spl-new --rollback-index-uboot 1 --burn-key-hash
```

Note: only add `--burn-key-hash` after the entire product development and validation process is complete. Otherwise secure boot is turned on and only signed firmware can be updated during development.

5. Signing other Android firmware:

Refer to `Rockchip_Developer_Guide_Secure_Boot_for_UBoot_Next_Dev_CN.md`.

13. Chapter-13 Fast Boot

TODO

14. Chapter-14 Platform Definition

14.1 ATF/OP-TEE

Minimum ATF/OP-TEE versions required for U-Boot charging standby:

Chip	Minimum Version
RK3576	rk3576_bl31_v1.04.elf
RK3588	rk3588_bl31_v1.24.elf
RK3538	N/A
RK3572	N/A

14.2 Clock

CPU frequency boost support matrix:

Chip	Support Status	Boost Handling Item
RK3576	N/A	N/A
RK3588	N/A	N/A
RK3538	N/A	N/A
RK3572	N/A	N/A

14.3 Defconfig

1. Defconfig support by platform (based on the SDK release version):

> `[chip]_defconfig` or `[chip].config` is usually the full-feature build, while the others are feature-specific variants. Refer to the actual repository for the exact file list.

Chip	defconfig	Supports Kernel DTB	Description
RK3576	rk3576_defconfig rk3576-usbplug.config rk3576-car.config rk3576-ab-car.config rk3576-eink.config	Y	General-purpose build Open-source USB plug build In-vehicle build In-vehicle build with A/B system support E-reader build
RK3588	rk3588_defconfig rk3588-ramboot.config rk3588-sata.config rk3588-aarch32.config rk3588-ipc.config	Y	General-purpose build No storage device (RAM boot) Dual-storage configuration with SATA boot support Supports AArch32 mode Used in the IPC SDK
RK3538	rk3538_defconfig rk3538-usbplug.config	Y	General-purpose build Open-source USB plug build
RK3572	rk3572_defconfig rk3572-usbplug.config rk3572-car.config rk3572-ab-car.config rk3572-eink.config	Y	General-purpose build Open-source USB plug build In-vehicle build In-vehicle build with A/B system support E-reader build

14.4 DFU

TODO

14.5 OP-TEE

Availability of OP-TEE client APIs on each platform:

API	RK3588	RK3576	RK3538	RK3572
optee_read_vbootkey_hash	√	√	√	√
optee_write_vbootkey_hash	√	√	√	√
optee_read_vbootkey_enable_flag	√	√	√	√
optee_write_oem_otp_key	√	√	√	√
optee_oem_otp_key_is_written	√	√	√	√
optee_set_oem_hr_otp_read_lock	√	√	√	√

API	RK3588	RK3576	RK3538	RK3572
optee_oem_otp_key_cipher	√	√	√	√

15. Chapter-15 Remarks

15.1 Configuration Changes

- Recheck boot, flashing, and upgrade flows after modifying defconfig or fragments.
- Revalidate packaging results after changing SPL, TPL, memory layout, or partition sizes.

15.2 Documentation Usage

- Use the current repository as the primary source for paths, commands, and configuration names.
- If a platform has product-specific differences, follow the corresponding defconfig and code comments.

15.3 Debug And Release

- Disable unnecessary debug output in release builds.
- Keep essential recovery and flashing paths available to avoid unrecoverable boards.

16. Chapter-16 Tools

Development tool paths related to this chapter, relative to the U-Boot root directory:

```
./scripts/mkbootimg
./scripts/unpack_bootimg
./scripts/repack-bootimg
./scripts/pack_resource.sh
./scripts/stacktrace.sh
../rkbin/tools/resource_tool
../rkbin/tools/boot_merger
```

16.1 boot_merger

Purpose: package `miniloader + ddr + usb plug` according to the `.ini` configuration file and generate loader firmware.

INI file:

Take `RK3288MINIALL.ini` as an example:

```
[CHIP_NAME]
NAME=RK320A             ----Chip name: "RK" plus the 4-byte chip model agreed with MaskROM
[VERSION]
MAJOR=2                 ----Major version number
MINOR=36                ----Minor version number
[CODE471_OPTION]        ----code471, currently set to ddr bin
NUM=1
Path1=tools/rk_tools/bin/rk32/rk3288_ddr_400MHz_v1.06.bin
[CODE472_OPTION]        ----code472, currently set to usbplug bin
NUM=1
Path1=tools/rk_tools/bin/rk32/rk3288_usbplug_v2.36.bin
[LOADER_OPTION]
NUM=2
LOADER1=FlashData       ----flash data, currently set to ddr bin
LOADER2=FlashBoot       ----flash boot, currently set to miniloader bin
FlashData=tools/rk_tools/bin/rk32/rk3288_ddr_400MHz_v1.06.bin
FlashBoot=tools/rk_tools/bin/rk32/rk3288_miniloader_v2.36.bin
[OUTPUT]                ----Output file name
PATH=rk3288_loader_v1.06.236.bin
```

Packaging command:

```
// @[ini file]: required. INI file

./tools/boot_merger [ini file]
```

Example:

```
./tools/boot_merger ./RKBOOT/RK3399MINIALL.ini
out:rk3399_loader_v1.17.115.bin
fix opt:rk3399_loader_v1.17.115.bin
merge success(rk3399_loader_v1.17.115.bin)
```

Unpacking command:

```
// @[input image]: required. Firmware image to unpack, usually a loader file

./tools/boot_merger --unpack [input image]
```

Example:

```
./tools/boot_merger --unpack rk3399_loader_v1.17.115.bin
unpack entry(rk3399_ddr_800MHz_v1.17)
unpack entry(rk3399_usbplug_v1.15)
unpack entry(FlashData)
unpack entry(FlashBoot)
unpack success
```

16.2 resource_tool

Purpose: package arbitrary resource files into `resource.img`.

Packaging command:

```
../rkbin/tools/resource_tool [--pack] [--image=<resource.img>] <file list>
```

Example:

```
../rkbin/tools/resource_tool ./arch/arm/boot/dts/rk3126-evb.dtb logo.bmp logo_kernel.bmp
Pack to resource.img succeeded!
```

Unpacking command:

```
../rkbin/tools/resource_tool --unpack --image=<resource.img> [output dir]
```

Example:

```
../rkbin/tools/resource_tool --unpack --image=resource.img ./out/

Dump header:
partition version:0.0
header size:1
index tbl:
        offset:1        entry size:1    entry num:3
Dump Index table:
```

```
entry(0):
        path:rk-kernel.dtb
        offset:4        size:33728
entry(1):
        path:logo.bmp
        offset:70       size:170326
entry(2):
        path:logo_kernel.bmp
        offset:403      size:19160
Unpack resource.img to ./out succeeded!
```

16.3 mkimage

Purpose: generate loader firmware for SPL mode.

For example, the following command generates the IDBLOCK format required by Rockchip BootROM for `rk3328`. It also modifies the first 4 bytes of `u-boot-tpl.bin` to the ID checked by BootROM:

```
./tools/mkimage -n rk3328 -T rksd -d tpl/u-boot-tpl.bin idbloader.img
```

For details, refer to:

```
./doc/mkimage.1
```

16.4 stacktrace.sh

Purpose: parse call stack information. See the RK Architecture chapter.

16.5 mkbootimg

Purpose: package firmware into `boot.img` and `recovery.img`. The source files come from the Android project.

Example:

```
./scripts/mkbootimg --kernel zImage --second resource.img --ramdisk ramdisk.img --out boot.img
```

16.6 unpack_bootimg

Purpose: unpack `boot.img` and `recovery.img`. The source files come from the Android project.

Example:

```
./scripts/unpack_bootimg --boot_img boot.img --out out/
```

16.7 repack-bootimg

Purpose: replace firmware inside `boot.img` and `recovery.img`.

Example:

```
// Example: replace only the kernel
./scripts/repack-bootimg --boot_img boot.img --kernel zImage -o boot_repack.img

// Example: replace only the resource
./scripts/repack-bootimg --boot_img boot.img --second resource.img -o boot_repack.img
```

16.8 pack_resource.sh

Purpose: package the charging images under `./tools/images/` into `resource.img`.

Example:

```
./scripts/pack_resource.sh resource.img

Pack ./tools/images/ & resource.img to resource.img ...
Unpacking old image(resource.img):
rk-kernel.dtb 1
Pack to resource.img successed!
Packed resources:
rk-kernel.dtb battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp battery_5.bmp battery_fail.bmp
battery_0.bmp

The trailing number in the example indicates the number of resources packed in this run.

resource.img is packed ready
```

17. Chapter-17 Appendix

17.1 Terminology

- U-Boot: Universal Boot Loader
- AOSP: Android Open-Source Project
- AVB: Android Verified Boot
- DTB: Device Tree Binary
- DTS: Device Tree Source
- Fastboot: Originally an Android firmware update method, now widely used in embedded systems
- GPT: GUID Partition Table
- MMC: MultiMediaCard, including eMMC, SD cards, etc.
- SPL: Secondary Program Loader
- TPL: Tertiary Program Loader
- DTB: noun, device tree blob
- DTBO: noun, device tree blob used for overlays
- DTC: noun, device tree compiler
- DTO: verb, device tree overlay operation
- DTS: noun, device tree source file
- FDT: noun, flattened device tree
- SCMI: System Control and Management Interface